

龙芯教学实验平台 LS-CPU-EXB-002

CPU 设计与体系结构实验指导手册

V1.0

自主决定命运, 创新成就未来



北京市海淀区中关村环保科技示范园龙芯产业园2号楼
Building No.2, Loongson Industrial Park, Zhongguancun Environmental
Protection Park, Haidian District Beijing 100095, P. R. China

www.loongson.cn

目录

一、LS-CPU-EXB-002 CPU 设计与体系结构教学实验系统介绍	5
二、实验一 数据运算：定点加法	7
三、实验二 数据运算：定点乘法	37
四、实验三 寄存器堆实现	47
五、实验四 ALU 模块实现	61
六、实验五 存储器实现	74
七、实验六 单周期 CPU 实现	103
八、实验七 多周期 CPU 实现	124
九、课程设计 静态 5 级流水线 CPU 实现	160
十、课程设计拓展 优化 CPU 系统	207
附录 A 实现的 MIPS 指令集	209
附 A1 单周期 CPU 实现指令	209
附 A2 多周期 CPU 新增实现指令	211
附 A3 静态 5 级流水 CPU 新增实现指令	215
附录 B 实现的 cp0 寄存器	217

一、LS-CPU-EXB-002 CPU 设计与体系结构教学实验系统介绍

LS-CPU-EXB-002 CPU 设计与体系结构教学实验系统是根据高等院校计算机专业本科生及研究生相关专业开设的《数字逻辑》、《计算机组成原理》、《计算机体系结构》等计算机基础软硬件课程的实验教学需要，自主研发，具有自主知识产权的集成实验设计、开发与教学平台。该实验系统围绕计算机系统的基础软硬件专业能力的培养，可以满足不同层次院校开设上述课程实验的验证型、综合型、创新型实验教学要求。

实验平台的基本结构和软硬件结构图如下所示。

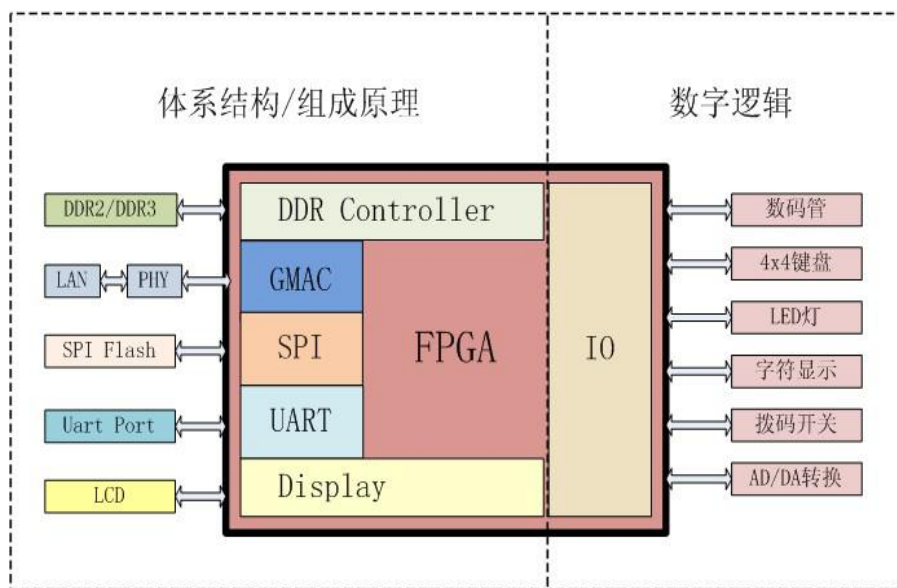


图 1.1 FPGA 实验平台框架图

实验箱里配套资源有

- FPGA 实验板；
- Xilinx 的下载线；
- 串口线（含 USB 串口转化器）；
- 电源线（含适配器）。

为了使在实验板上演示实验成为可能，LS-CPU-EXB-002 试验箱实现了 LCD 触摸屏的硬件驱动，使得不需要处理器核就能使用触摸屏的显示和输入功能，且设计了简单清晰的调用接口方便学生使用。

为降低学生在 FPGA 硬件平台上的实际操作难度，方便教师指导，特针对组成原理和体系结构课程编写了本实验指导手册。指导手册主要分为两部分，cpu 设计部分和 cpu 优化部分：前者主要为组成原理需要，包含实验一到实验七，以及课程设计；后者主要为体系结构需要，包含课程设计和课程设计拓展。

具体实验内容如下：

实验一 数据运算：定点加法(软硬件平台入门)

实验二 数据运算：定点乘法

实验三 寄存器堆实现

实验四 ALU 模块实现

实验五 存储器实现

实验六 单周期 CPU 实现

实验七 多周期 CPU 实现

课程设计 静态 5 级流水线 CPU 实现

课程设计拓展 优化 CPU 系统

二、 实验一 数据运算：定点加法

1 实验目的

1. 熟悉 LS-CPU-EXB-002 实验箱和软件平台。
2. 掌握利用该实验箱各项功能开发组成原理和体系结构实验的方法。
3. 理解并掌握加法器的原理和设计。
4. 熟悉并运用 verilog 语言进行电路设计。
5. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 阅读 LS-CPU-EXB-002 实验箱相关文档，熟悉硬件平台, 特别需要掌握利用显示屏观察特定信号的方法。学习软件平台和设计流程。
2. 熟悉计算机中加法器的原理。
3. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，本次实验的加法器可以使用全加器自己搭建加法模块，也可以在 verilog 中直接使用“+”（系统是自动调用库里加法 IP，且面积时序更优），依据教师要求选择一种方法实现。
4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 2.1。外围模块中需调用封装好的触摸屏模块，显示两个加数和加法结果，且需要利用触摸功能输入两个加数。

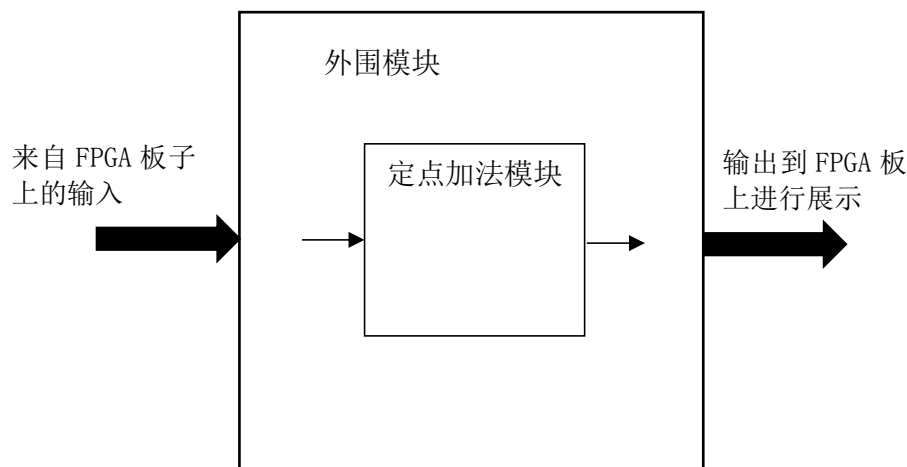


图 2.1 定点加法设计实验的顶层模块大致框图

7. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板上进行演示。

4 实验要求

1. 做好预习：
 - 1) 了解软硬件平台；
 - 2) 掌握定点加法的工作原理；
 - 3) 确定定点加法的输入输出端口设计；
 - 4) 在课前画好设计框图或实验原理图；
 - 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 2.1。
2. 实验实施：
 - 1) 确认定点加法的设计框图的正确性；
 - 2) 编写 verilog 代码；
 - 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
 - 4) 完成调用定点加法模块的外围模块的设计，并编写代码；
 - 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。
3. 实验检查：
 - 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，可对演示结果进行拍照作为实验报告结果一项的材料。
4. 实验报告的撰写：
 - 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考实现

本部分给出了定点加法实验的参考设计方案，详细阐述了软硬件平台的使用方法，特别是 FPGA 实验板上的 LCD 触摸屏的调用模块的使用。

5.1 实验步骤

1) 新建工程

启动 vivado 软件，在菜单栏点击“File”->“New Project”，出现新建工程向导，选择“Next”，输入工程名称，选择工程的文件位置，然后选择“Next”：

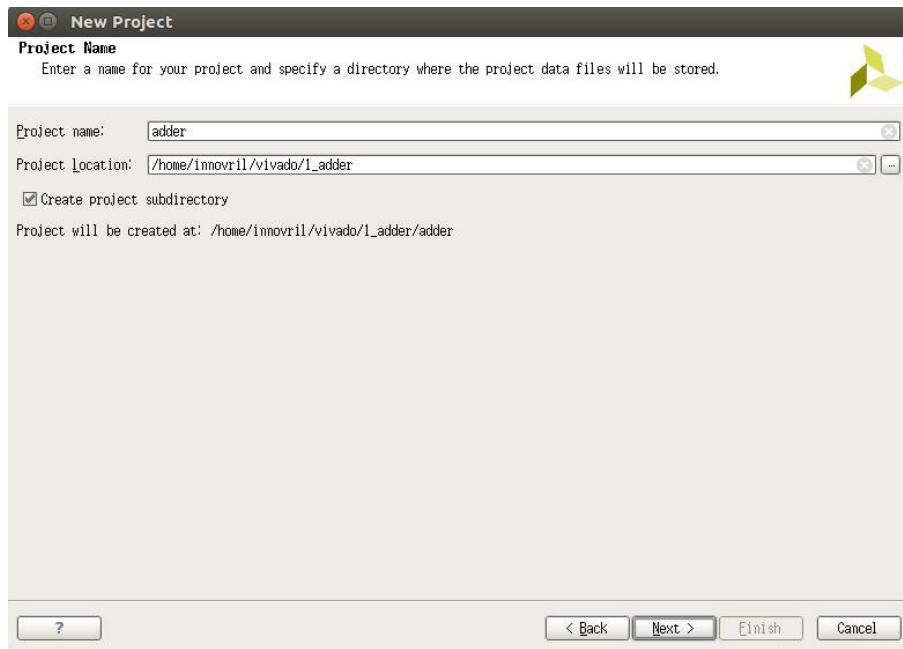


图 2.2 设置工程名称和位置

选择“RTL Project”，勾选“Do not specify sources at this time”，点击“Next”：

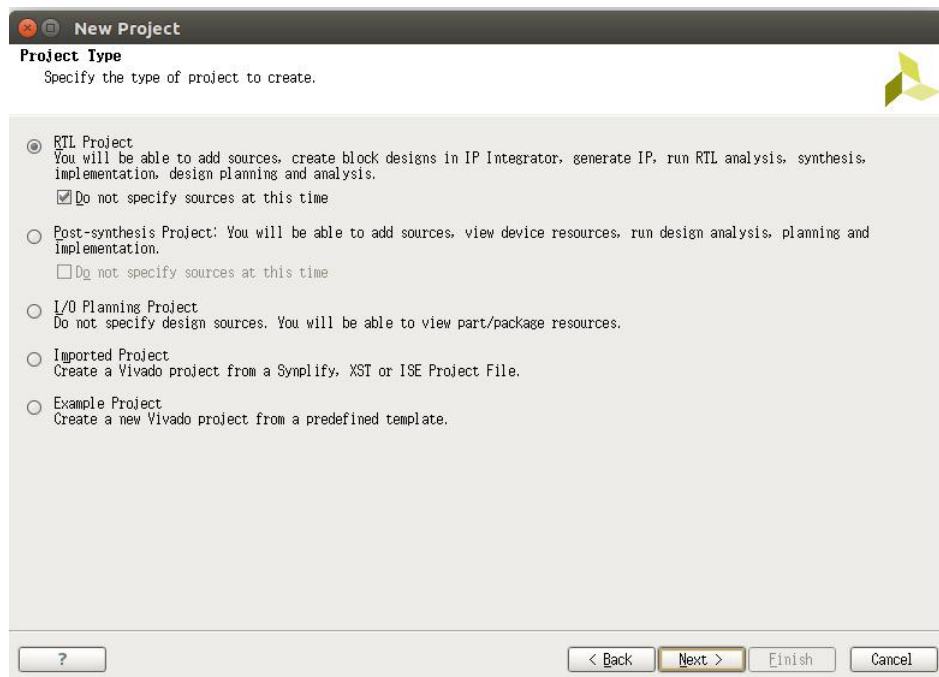


图 2.3 工程设置

在筛选器的“family”选择“Artix 7”，“package”选择“fbg676”，在筛选得到的型号里面选择“xc7a200tfbg676-2”：

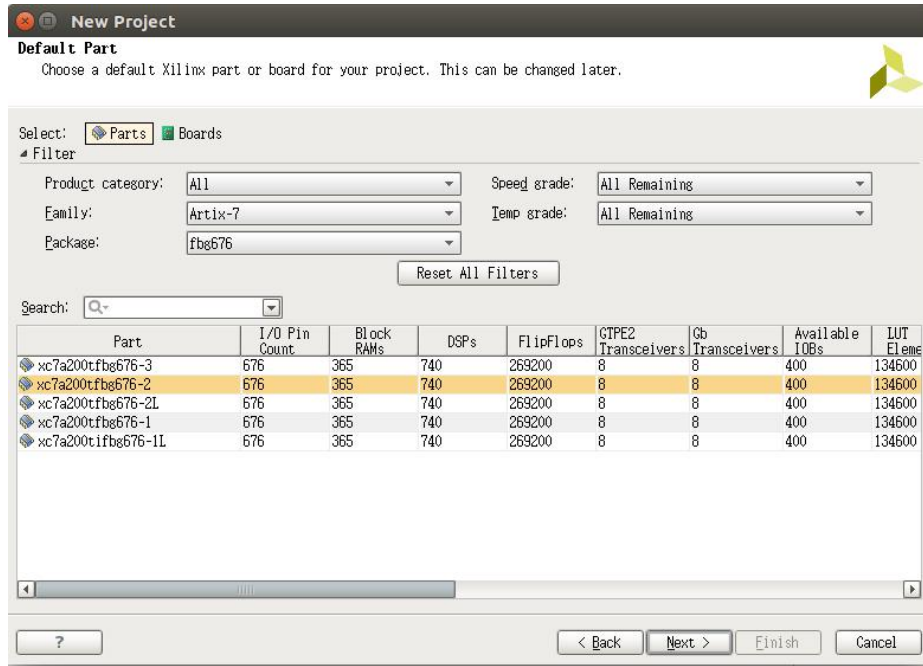


图 2.3 工程设置

然后选择“Next”，出现下图界面，点击“Finish”。



图 2.4 工程信息

2) 添加源文件

verilog 代码都是以“.v”为后缀名的文件，可以在其他文件编辑器里写好，再添加到新建的工程中，也可以在工程中新建一个再编辑。

添加已有 verilog 文件的方法如下：在”Project Manager”下点击”Add sources”，选择”Add or create design sources”



图 2.5 添加源文件

点击”Next”，可以看到如下的界面，可以按文件添加或者按文件夹添加，也可以自己创建：

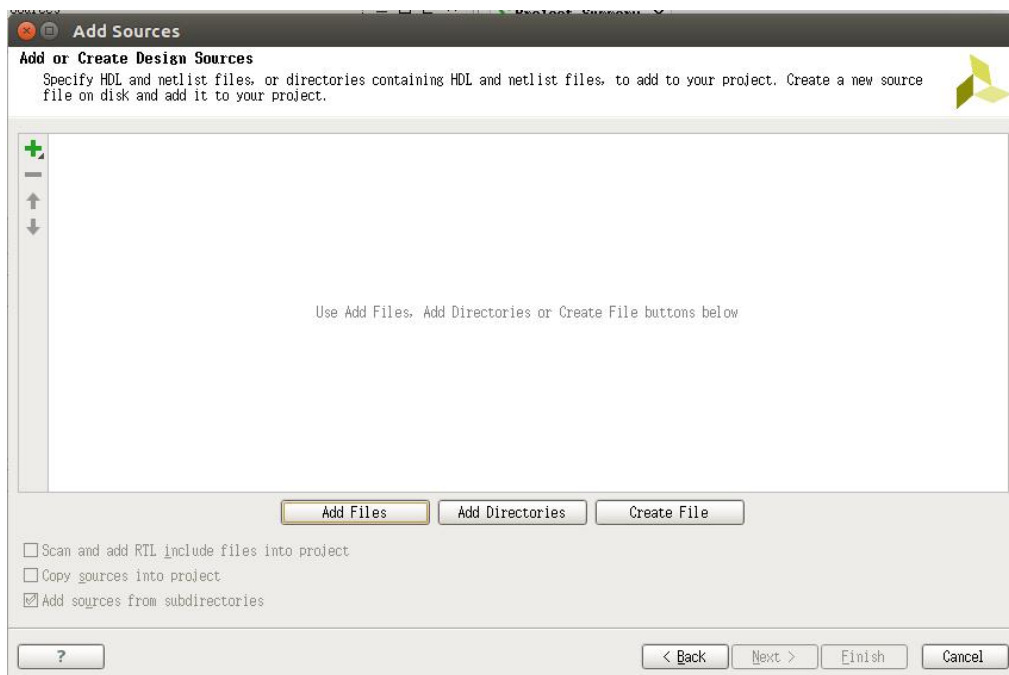


图 2.6 添加源文件

点击”Add Files”，选择 adder.v，点击”OK”：

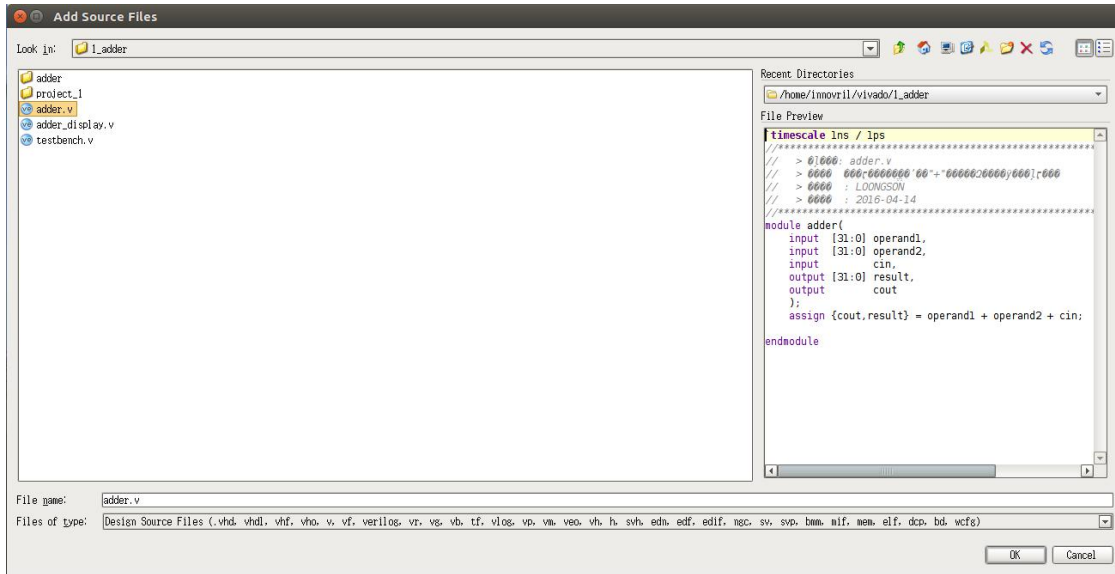


图 2.6 添加源文件

adder.v 的代码如下:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: adder.v
4  //    > 描述   : 加法器, 直接使用"+", 会自动调用库里的加法器
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module adder(
9      input  [31:0] operand1,
10     input  [31:0] operand2,
11     input          cin,
12     output [31:0] result,
13     output          cout
14 );
15     assign {cout,result} = operand1 + operand2 + cin;
16
17 endmodule

```

代码比较简单, 有 2 个 32 位数的输入和 1 个进位输入, 产生 1 个 32 位的加法和结果和 1 个向高位的进位。本实验提供的参考设计是直接写“+”号实现加法功能的, 这样的写法综合工具会调用内部的模块库的加法器来实现, 往往会比自行设计的加法模块更高效和省资源。

本例中, adder.v 为定点加法实验的主体代码, 由于实验较简单, 故只有这一个.v 文件, 对于以后的 CPU 实验, 则会有多个.v 文件, 形成一定的调用层次。

3) 添加展示外围模块

在本例中, 按照实验要求还需要一个外围模块, 该外围模块调用 adder.v, 且要调用触摸屏模块的以便在板上实验结果, 本例中将该模块命名为 adder_display.v, 添加该模块到工程里, 结果如下图:

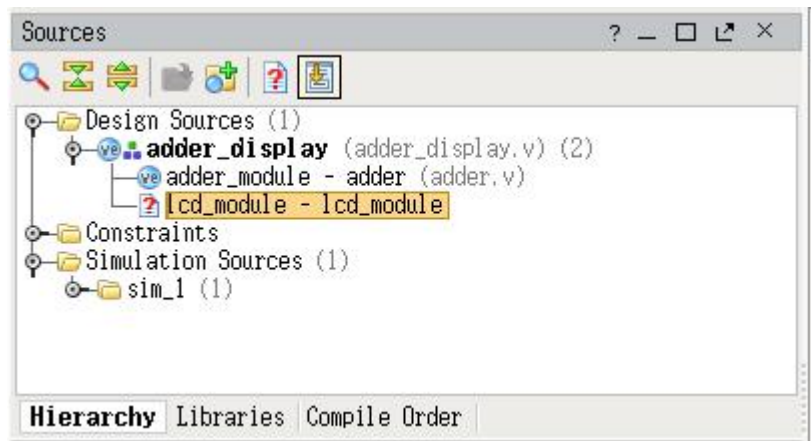


图 2.7 工程管理区

在工程管理区可以看到各模块间的层次关系，本例中顶层模块即为 `adder_display`，里面调用了两个子模块：一个为 `adder`，即定点加法主体代码；一个为 `lcd_module`，即 LCD 触摸屏的模块。外围展示模块 `adder_display.v` 具体代码解析见 5.2 小节，以后所有实验的外围展示模块都可以仿照 `adder_display.v` 编辑。

在图 2.7 中，lcd_module 前面打了问号，表示该模块还未添加。LS-CPU-EXB-002 配套资源设计时，将 lcd_module 模块封装为一个黑盒的网表文件，使得学生不会过分关注而导致迷失在 LCD 触摸屏实现中，只需要调用即可。继续选择“Add Source”，添加 lcd_module.dcp，使用 vivado2012.3 版本软件工具时，需先解压 dcp 文件后再添加，如下图：

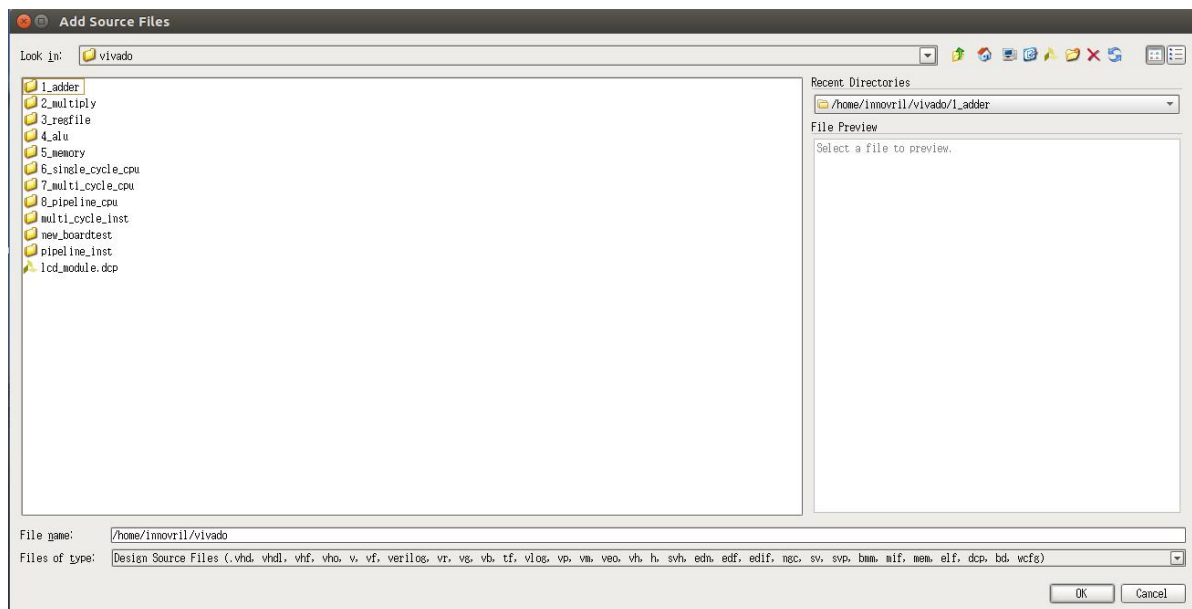


图 2.8 添加 lcd module 文件

添加成功后，结果如下图：

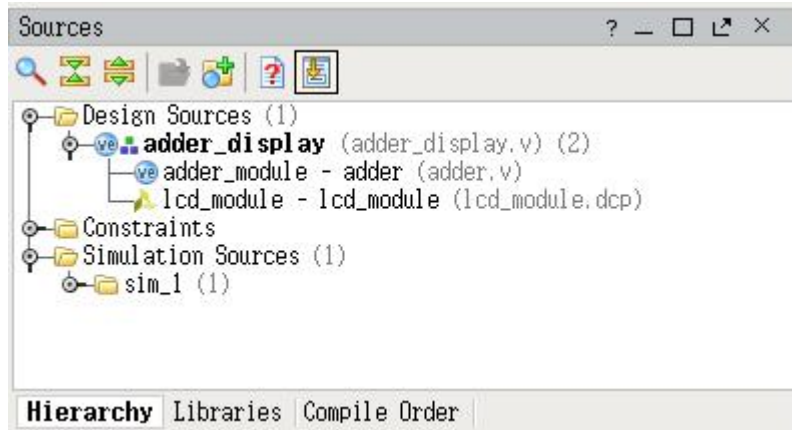


图 2.9 添加 lcd_module 文件成功

至此，代码实现都已经完成，下面需要对代码功能进行仿真验证功能的正确性。

4) 功能仿真

在进行功能仿真时，需要先建立一个 testbench(测试平台)。一个比较完备的 testbench 就是产生输入激励，送入到要测试的功能模块里，然后读出功能模块的执行结果，与预期的结果进行比较，以此验证功能模块的正确性。但在目前的实验设计上，只需要一个最简单的 testbench——只产生输入激励即可。

在本例中，需要产生的输入激励就是 2 个加数和 1 个低位进位信号，在该激励输入到加法功能模块中后，会输出加法结果和向高位的进位信号。仿真的过程中会产生波形文件，可以通过观察波形文件确定功能的正确性，在出错的情况下可以定位错误位置。

点击 “Add Source”，选择”Add or create simulation sources”，添加“testbench.v”。

testbench.v 的代码如下：

```
1 `timescale 1ns / 1ps //仿真单位时间为 1ns，精度为 1ps
2 module testbench;
3
4     // Inputs
5     reg [31:0] operand1;
6     reg [31:0] operand2;
7     reg cin;
8
9     // Outputs
10    wire [31:0] result;
11    wire cout;
12    // Instantiate the Unit Under Test (UUT)
13    adder uut (
14        .operand1(operand1),
15        .operand2(operand2),
```

```

16     .cin(cin),
17     .result(result),
18     .cout(cout)
19 );
20 initial begin
21     // Initialize Inputs
22     operand1 = 0;
23     operand2 = 0;
24     cin = 0;
25     // Wait 100 ns for global reset to finish
26     #100;
27     // Add stimulus here
28 end
29 always #10 operand1 = $random; //$random 为系统任务，产生一个随机的 32 位数
30
31 always #10 operand2 = $random; //#10 表示等待 10 个单位时间(10ns)，即每过 10ns，赋
    值一个随机的 32 位数
32 always #10 cin = {$random} % 2; //加了拼接符，{$random}产生一个非负数，除 2 取余
    得到 0 或 1
33 endmodule

```

添加 testbench.v 后，工程管理区如图所示，如果 testbench.v 前面没有三个方形的 top 标志，则右键点击 testbench.v，选择”Set as Top”：

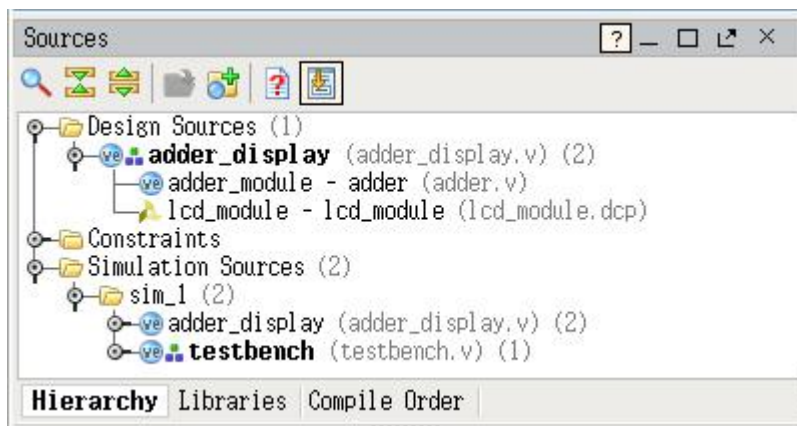


图 2.10 添加 testbench.v 成功

在左侧的导航栏中点击”Run Simulation”，选择”Run Behavioral Simulation”：

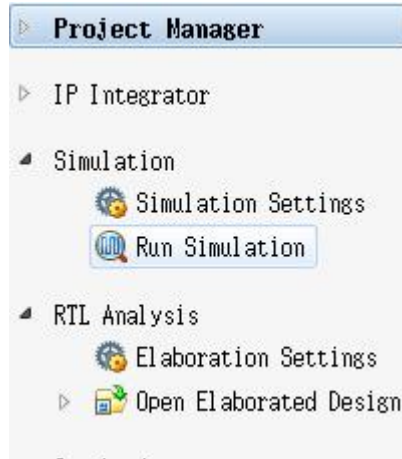


图 2.11 运行仿真

如果没有语法错误，会弹出如下的界面：

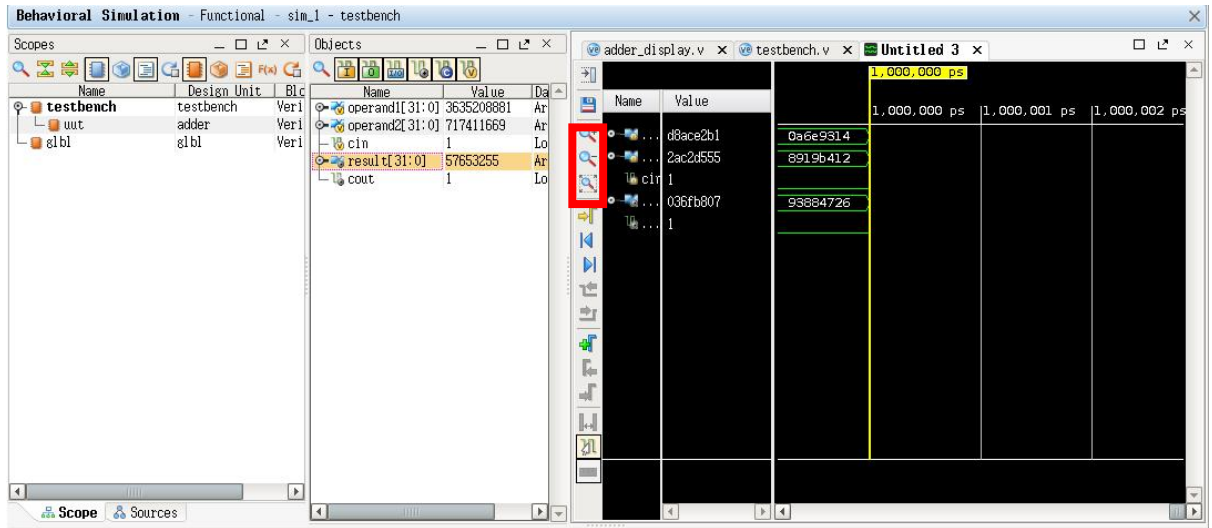


图 2.12 仿真界面

可以通过图 2.12 中红色圈出的放大、缩小、缩放到全屏三个按钮以及鼠标左键点击波形界面(出现图中黄线，缩放会以黄线为中心)观察特定波形区域的信息。

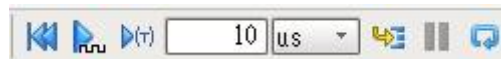


图 2.13 仿真工具栏

上图是仿真工具栏，从左到右依次是”从 0 时刻开始仿真”、”运行仿真”、”运行特定时间的仿真”、”仿真时长”、”时间单位”、”单步仿真”、”暂停”、”重新载入仿真”。

在仿真界面左侧的 “Scopes”(图 2.12 的最左列)选择要观察信号所在源文件对象，然后在“Objects”窗口(图 2.12 的第二列)可以看到该对象里所有的信号，可以左键选中要观察的信号，然后右键会出现很多可选项，选择“Add To Wave Window”添加该信号到波形窗口，如下图。

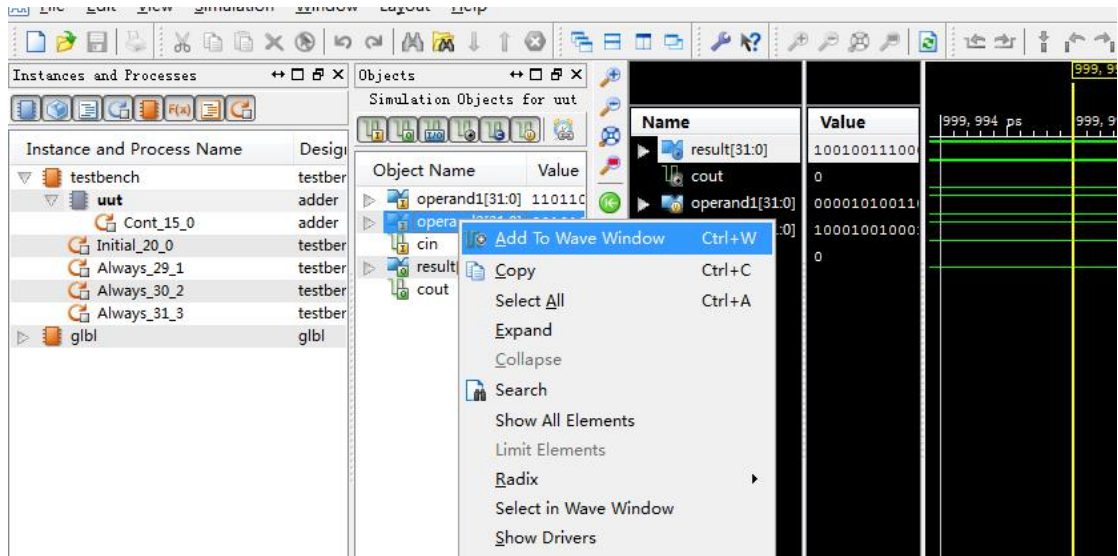


图 2.14 添加要观察的信号到波形界面

在添加新的信号到波形窗口后，需要点击 **Relaunch** 才能看到该信号的数据。由于定点加法模块内部所有信号均为输入输出信号，故此处没有新的信号可以添加。但在以后的实验中，会经常需要添加新的信号到波形窗口。

另外，当前波形窗口显示的数据均为 2 进制数，对于 32 位的数不便于观察，可以在波形窗口的 **Name** 列左键选择信号，然后右键会出现很多可选项，选择“Radix”可以选择要数据显示的进制。

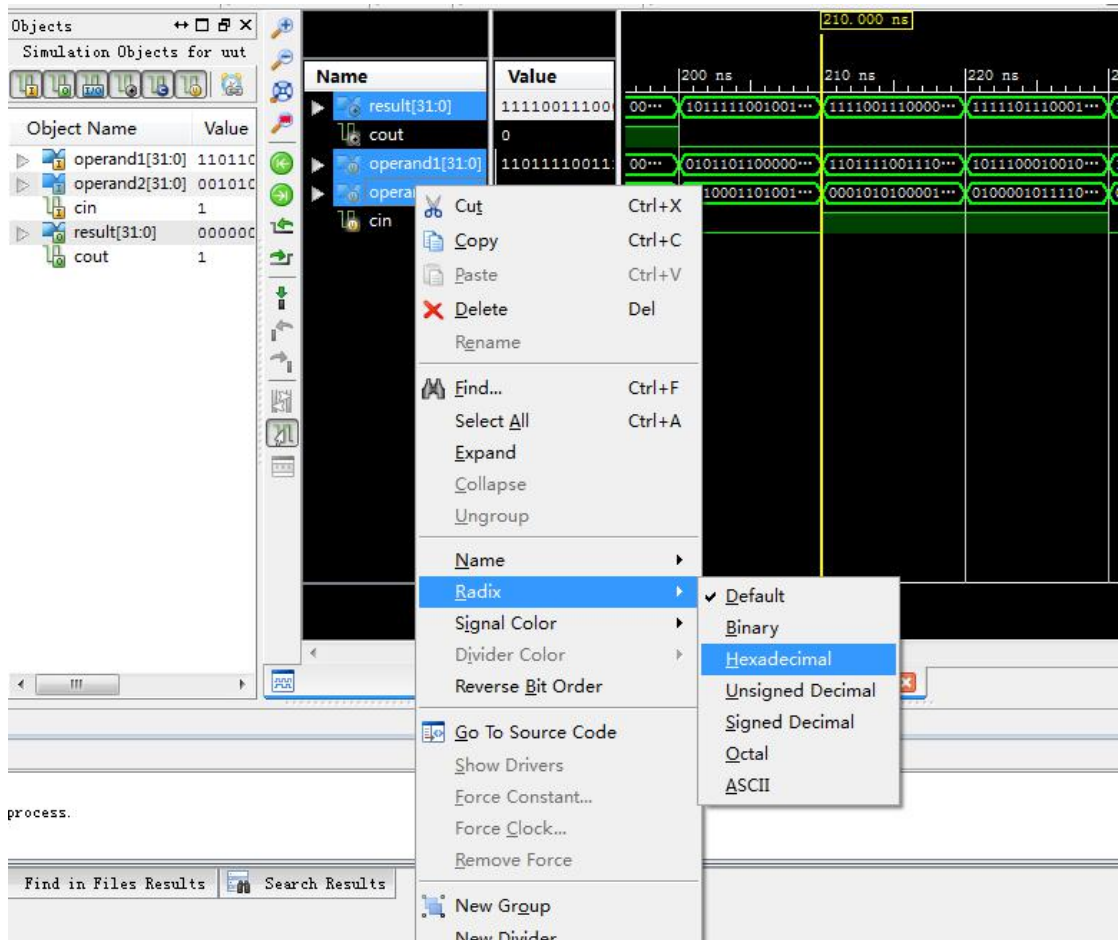


图 2.15 更改波形窗口信号的显示进制

此处选择 16 进制，结果如下图：

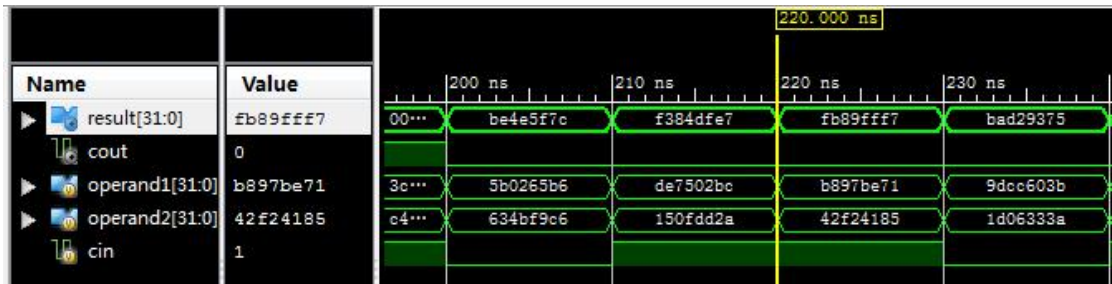


图 2.16 数据以 16 进制形式显示

可以检查几组数据，比如： $b897be71 + 42f24186 + 1 = fb89ff7$ ，正确。类似的，通过观察波形可以看到加法功能模块随机测试并没有出错，我们认为功能趋于稳定，可以认为是正确的。

至此，代码编辑和功能仿真都已完成，认为功能基本正确，后续流程就是上板验证了。

5) 上板验证

所谓上板验证是指将功能代码进行综合和布局布线后下载到 FPGA 板上运行，在板上检查运行的正确性。

因此，需要设定一套在板上检查结果的机制。比如，对于本例中的定点加法，可以设定使用拨码开关作为加数的输入，使用 led 灯作为加法结果的输出，这样就能在 FPGA 实验板上观察加法模块的运行结果了。但遗憾的是，对于 32 位加法，需要 64 个拨码开关和 32 个 led 灯，但显然实验板上提供不了如此之多的 IO 接口，当然也可以通过精巧的设计使用实验板上有限的资源来完成加数输入和结果显示的，但使用起来不够方便。

对于 LS-CPU-EXB-002 试验箱并没有上述的问题，因为可以通过 LCD 触摸屏输入 32 位加数，并能显示加法结果。之前提到的外围展示模块 `adder_display.v` 就是调用 LCD 触摸屏来完成上板验证的机制的设计的，具体验证机制的解析见本章的 5.2 小节。

在有了板上验证机制后，需要添加引脚绑定的约束文件。所谓约束文件就是将顶层模块(本例中为 `adder_display`)的输入输出端口与 FPGA 板上的 IO 接口引脚绑定，以完成在板上的输入输出。

约束文件后缀名为 `.xdc`，有两种方法可以添加约束文件，其一是用“Add or create constraints”添加或创建约束文件，如下图：



图 2.17 添加约束文件

之后选择添加 `adder.xdc`。

`adder.xdc` 内容如下：

```
1 set_property PACKAGE_PIN AC19 [get_ports clk]
2 set_property PACKAGE_PIN A3 [get_ports led_cout]
```

```
3 set_property PACKAGE_PIN Y3 [get_ports resetn]
4 set_property PACKAGE_PIN AC21 [get_ports input_sel]
5 set_property PACKAGE_PIN Y6 [get_ports sw_cin]
6
7 set_property IOSTANDARD LVCMOS33 [get_ports clk]
8 set_property IOSTANDARD LVCMOS33 [get_ports led_cout]
9 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
10 set_property IOSTANDARD LVCMOS33 [get_ports input_sel]
11 set_property IOSTANDARD LVCMOS33 [get_ports sw_cin]
12
13 #lcd
14 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
15 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
16 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
17 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
18 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
19 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
20 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
21 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
22 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
23 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
24 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
25 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
26 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
27 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
28 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
29 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
30 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
31 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
32 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
33 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
34 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
35 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
36 set_property PACKAGE_PIN L19 [get_ports ct_int]
37 set_property PACKAGE_PIN J24 [get_ports ct_sda]
38 set_property PACKAGE_PIN H21 [get_ports ct_scl]
39 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
40
41 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
```

```
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
47 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

可以看到主要有时钟与复位信号的引脚连接，led 灯和拨码开关的引脚连接，以及 LCD 触摸屏引脚的连接。此处时钟与复位信号和 LCD 触摸屏引脚都是 lcd_module 需要用到的，可以不用管。

对于 led 灯的连接（led_cout 绑定到 A3 引脚上），可以看到是用来显示加法向高位的进位的。

对于拨码开关的连接，“sw_cin”是用来输入低位的进位的，而“input_sel”是用来选择通过触摸屏输入的 32 位数据为加数 1 还是加数 2。

由于以后的实验都需要用到 LCD 触摸屏，故 LCD 触摸屏相关引脚的绑定是固定不变的，故建议以后实验都通过添加已有 xdc 文件，然后再根据需求修改 led 和拨码开关等引脚的绑定。

也可以通过 Vivado 中的 I/O Planning 功能来产生约束文件，具体过程如下：
点击 Flow Navigator 中 Synthesis 下 Run Synthesis，进行综合：

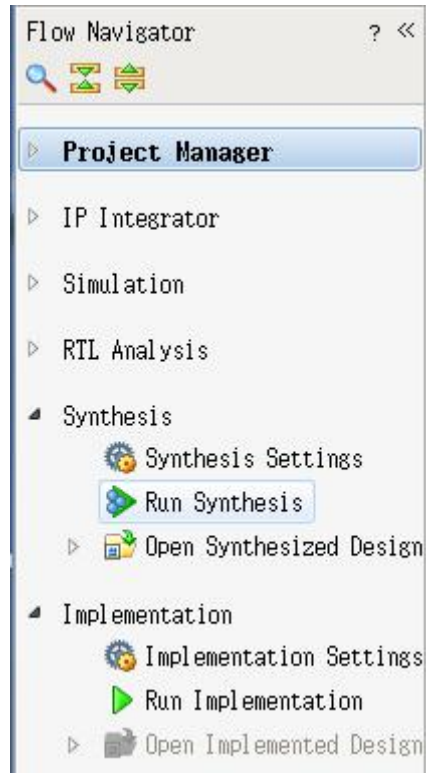


图 2.18 运行综合

综合完成后，选择”Open Synthesized Design”，查看综合结果：



图 2.19 综合完成

得到如下界面：

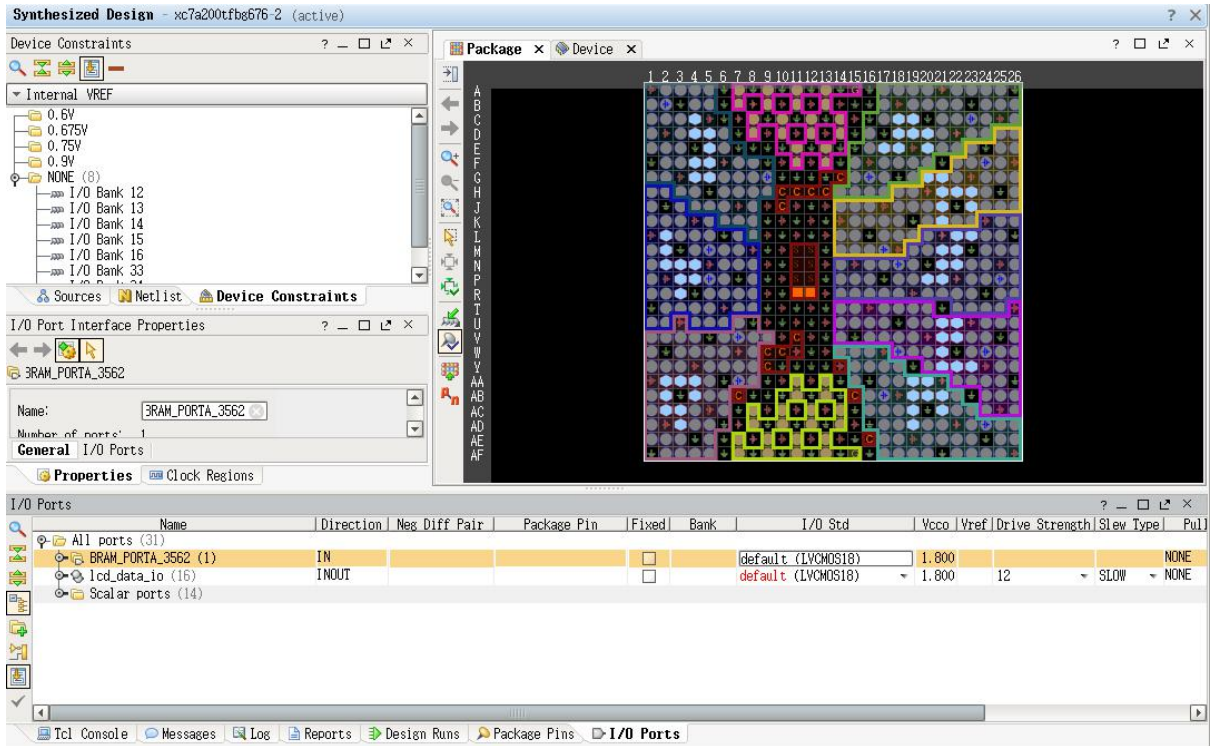


图 2.20 综合结果显示

之后参照实验板原理图和引脚对应关系表，在 I/O Ports 栏下填入正确的 Package Pin，针对本实验板，I/O Std 要统一设置为 LVCMOS33。

所有约束条件填写完成之后效果如下：

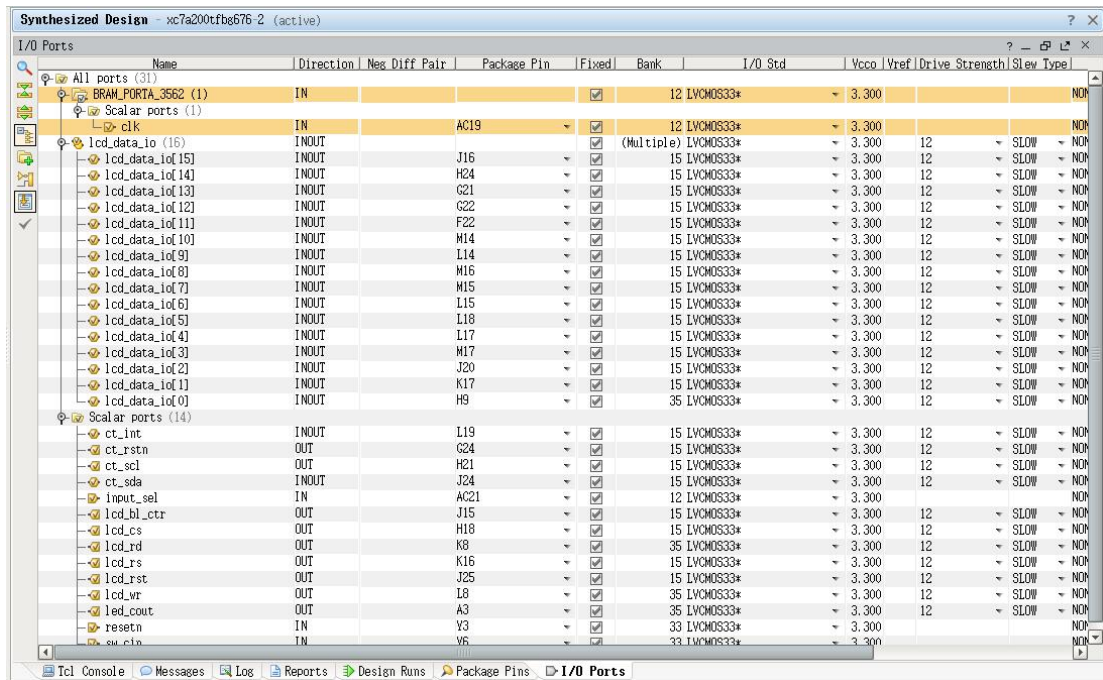


图 2.21 填写完成的 I/O Planning

点击右上角的“保存”键即可保存为 xdc 文件。

后续的流程就是综合、布局布线和产生可烧写文件，可以依次双击运行，也可以只双击“Generate Bitstream”会自动运行这三步，运行结果如下，可以选择 Open Implemented Design 来查看实现结果：

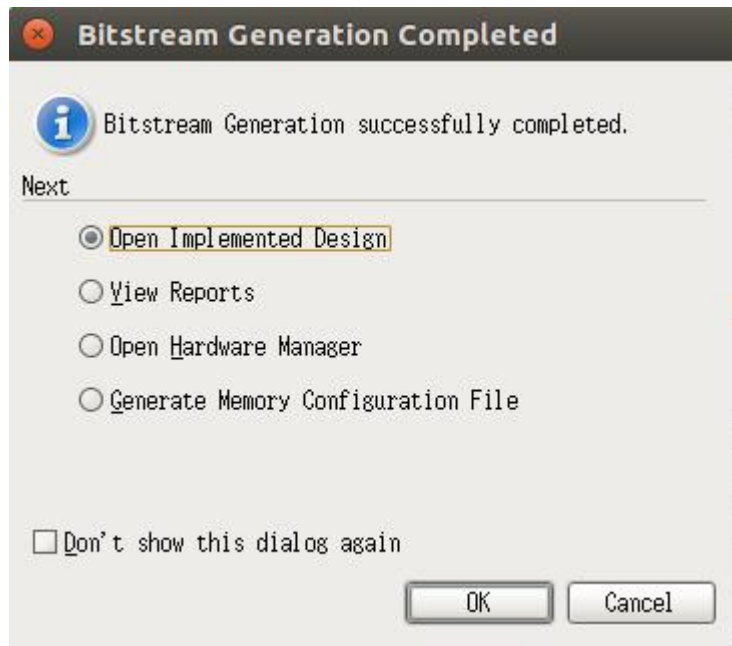


图 2.22 综合、布局布线和产生可烧写文件运行结果

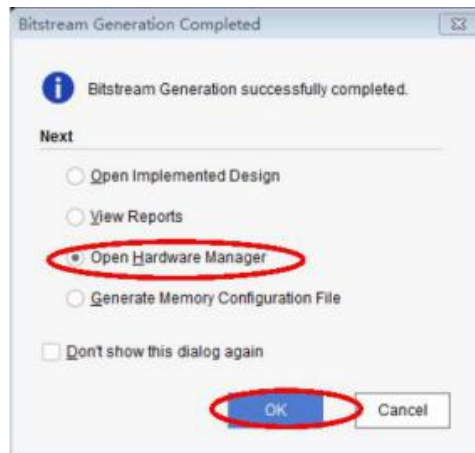
这时可烧写的文件已经产生成功了，后缀名为.bit。

在打开 FPGA 实验板，上电，并将下载线与电脑相连后，打开电源，FPGA 板如下图所示：



图 2.23 上电后初始的 FPGA 板

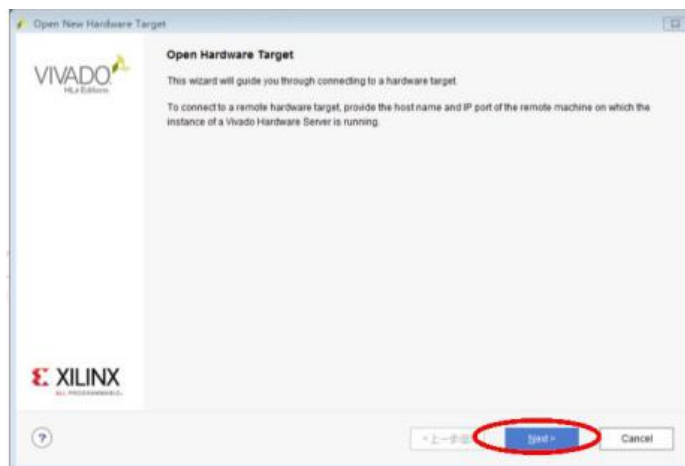
在比特流文件生成完成的窗口选择“Open Hardware Manager”，进入硬件管理界面。连接 FPGA 开发板的电源线和与电脑的下载线，打开 FPGA 电源。如下图：



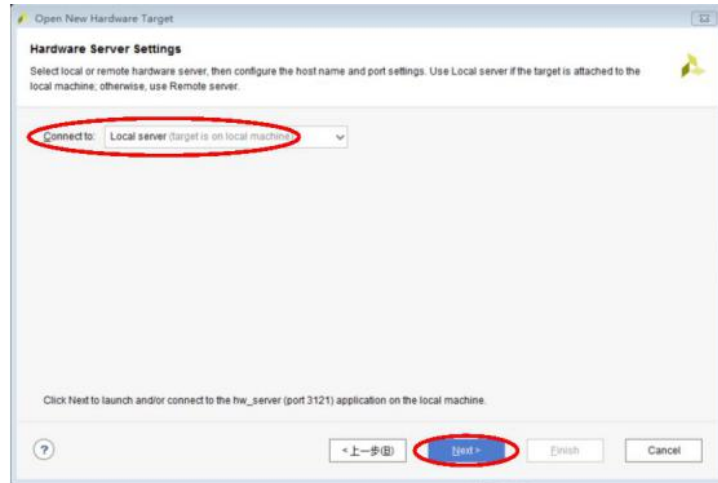
在“Hardware Manager”窗口的提示信息中，点击“Open target”的下拉菜单的“Open New Target”（或在“Flow Navigator”下“Program and Debug”中展开“Open Hardware Manager”，点击“Open Target”->“Open NewTarget”）。也可以选择“Auto Connect”自动连接器件。



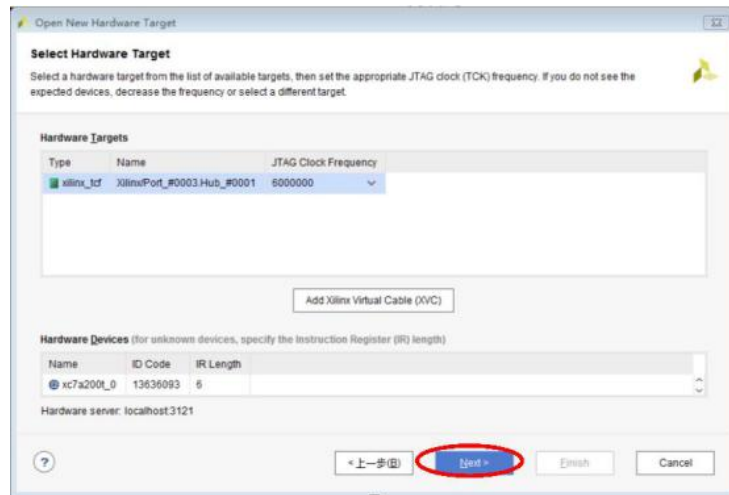
在“Open Hardware Target”向导中，先点击“Next”，进入 Server 选择向导。如下图：



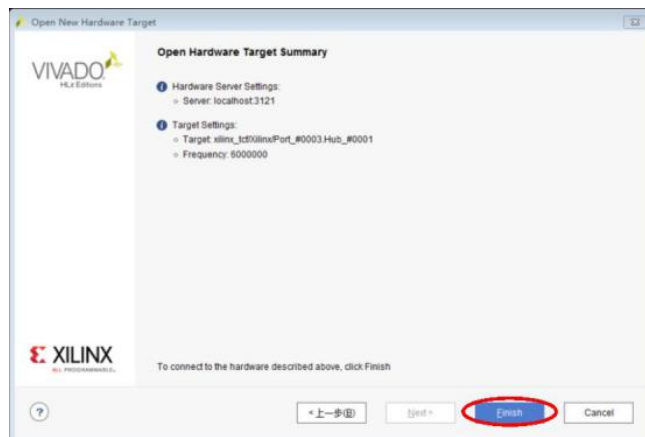
选择连接到“Local server”，点击“Next”。出现以下图：



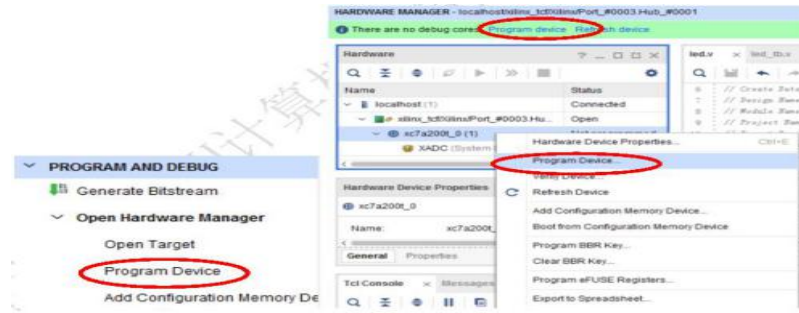
选择目标硬件，点击“Next”。



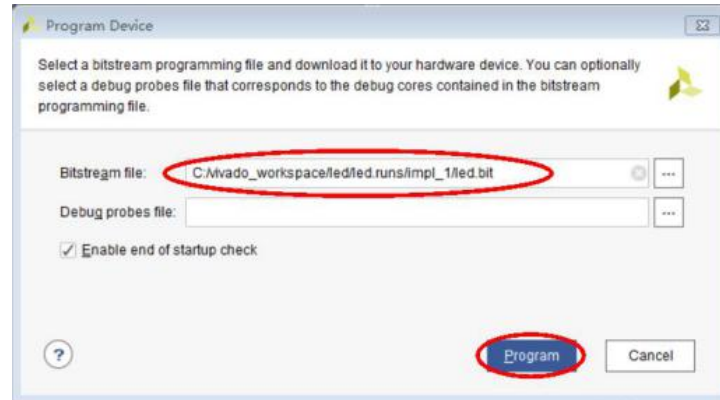
完成目标硬件打开。点击“Finish”。



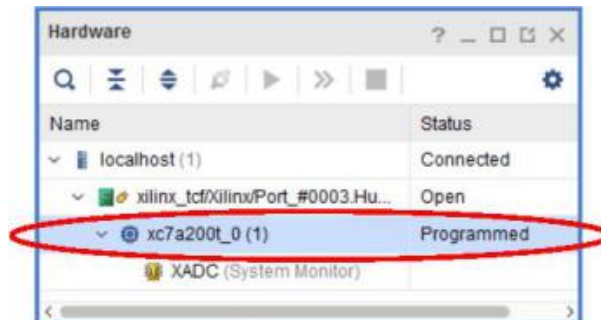
对目标硬件编程。在“Hardware”窗口右键单击目标器件“xc7a200t_0”，选择“Program Device...”。或者“Flow Navigator”窗口中“Program and Debug”->“Hardware Manager”->“Program Device”。



选择下载的 bit 流文件，点击“Program”。



完成下载后，“Hardware”窗口下的“xc7a200t_0”的状态变成“Programmed”。



烧写 bit 文件，烧写完成后的 FPGA 板显示如下：



图 2.24 加法模块下载成功后的 FPGA 板

从图 2.24 中可以看到 LCD 触摸屏上分别显示了 2 个加数和加法结果，最右侧的 led 灯为向高位的进位，该 led 灯为共阴极的，即输入为 0 是亮，为 1 是不亮，由于当前进位为 0，故 led 灯为亮。拨码开关最右侧的开关用来选择触摸屏输入的数据为加数 1 还是加数 2，当前该开关置为 1，说明触摸屏输入的 32 位数位加数 2。

在需要使用触摸屏的输入功能时，手指点击屏上最后一列的“START INPUTING”即可进入输入模式，如下图：



图 2.25 LCD 屏进入输入模式

从图 2.25 中可看到，触摸屏的输入模式为小键盘，包含 0~F，回退键(BACK)和确认键(OK)。输入的数据为 32 位的 16 进制数，当输出错误的时候，可以按 BACK 键回退一格，当输入完成时按 OK 键完成输入，同时会退出输入模式。当按 OK 时，输入未满 32 位，则会高位补 0，比如：只输入了 123，则最终屏调用模块输入时，输入数据为 0x00000123。

在本例中，给加数 2 输入 0x11111111 后，可看到 FPGA 板上结果如下图：



图 2.26 加数 2 输入 0x11111111

可看到加法结果页变为 0x11111111，正确。可以将拨码开关最右侧的置为 0，此时表示输入数据为加数 1。因此，通过触摸屏可以给加数 1 和加数 2 输入任意 32 位数，并能实时看到加法结果，完成上板验证。

5.2 LCD 触摸屏调用方法

正如前面所述，adder_display.v 作为顶层模块，里面实例化了 adder.v，同时也调用了 lcd_module 用来上板演示。此小节着重讲解 adder_display.v 里调用 LCD 触摸屏的方法。

1) LCD 触摸屏调用接口

先来看 lcd_module.v 的代码：

```
1 //*****
2 //    > 文件名：lcd_module.v
3 //    > 描述   ：lcd 触摸屏模块，为黑盒文件
```

```
4 // > 作者 : LOONGSON
5 // > 日期 : 2016-04-14
6 //*****
7 //synthesis attribute box_type <lcd_module> "black_box"
8 module lcd_module(
9     input  clk,          //10Mhz
10    input  resetn,       //低使能
11
12    //调用触摸屏的接口
13    input          display_valid,
14    input  [39:0] display_name,
15    input  [31:0] display_value,
16    output [ 5:0] display_number,
17    output          input_valid,
18    output [31:0] input_value,
19
20    //lcd 触摸屏相关接口, 不需要更改
21    output reg      lcd_rst,
22    output          lcd_cs,
23    output          lcd_rs,
24    output          lcd_wr,
25    output          lcd_rd,
26    inout [15:0] lcd_data_io,
27    output          lcd_bl_ctr,
28    inout          ct_int,
29    inout          ct_sda,
30    output          ct_scl,
31    output          ct_rstn
32 );
33 endmodule
```

可以看到该模块为黑盒文件, 其中时钟和复位信号以及 LCD 触摸屏引脚接口不需要关注, 下面介绍调用触摸屏的 6 个接口。

2) 调用 LCD 触摸屏显示

```
input          display_valid,
input  [39:0] display_name,
input  [31:0] display_value,
output [ 5:0] display_number,
```

display 的 4 个接口用于在屏上显示数据。从图 2.24 FPGA 板的实物图中可以看到, LCD 屏用于显示的区域块共有 2 列, 22 行, 故共可显示 44 组数据。

display_number 就是输出到外部说明当前需要显示的区域块为第几块, 有效编号从 1~44, 指示 44 块显示区域块。

每块显示区域块可显示 14 个字符, 其中头五个字符为块名, 指示当前块显示的数据的意义, 由 display_name 输入指定。display_name 输入的为要显示字符的 ASCII 码, 5 个 ASCII 码共 40 位。块名可显示字符为大写的 26 个字母, 下划线“_”, 0~9 数字以及空格。

每块显示区域块的第 6 个字符为冒号，用于区分块名和块数据段。后 8 个字符显示该块的数值，显示的为 32 位 16 进制数，故占用 8 个字符。该段由 display_value 输入指定，display_value 输入的为 32 位 2 进制数，lcd_module 内部会自动转换为 8 个字符显示。

最后还有 display_valid 输入，用于指示是否需要在当前显示区域块(由 display_number)显示数据，为 1 有效。

在本例中，顶层模块 adder_display.v 调用 LCD 触摸屏显示功能的代码如下：

```
117 //-----{输出到触摸屏显示}begin
118 //根据需要显示的数修改此小节，
119 //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
120 //44 块显示区域从 1 开始编号，编号为 1~44，
121     always @(posedge clk)
122     begin
123         case(display_number)
124             6'd1 :
125                 begin
126                     display_valid <= 1'b1;
127                     display_name  <= "ADD_1";
128                     display_value <= adder_operand1;
129                 end
130             6'd2 :
131                 begin
132                     display_valid <= 1'b1;
133                     display_name  <= "ADD_2";
134                     display_value <= adder_operand2;
135                 end
136             6'd3 :
137                 begin
138                     display_valid <= 1'b1;
139                     display_name  <= "RESUL";
140                     display_value <= adder_result;
141                 end
142             default :
143                 begin
144                     display_valid <= 1'b0;
145                     display_name  <= 40'd0;
146                     display_value <= 32'd0;
147                 end
148             endcase
149         end
150 //-----{输出到触摸屏显示}end
151 //-----{调用触摸屏模块}end-----//
```

可以看到加法模块只需要区域块 1~3 用于显示，其他块(default 分支)的 display_valid 为 0。另外，display_name 接口赋值 5 位字符组成的字符串即可，字符串里应当只出现大写的 26 个字母，下划线“_”，0~9 数字或空格，其他字符在 LCD 触摸屏上暂不支持显示。而 display_value 直接赋值 32 位的数据即可。

3) 调用 LCD 触摸屏输入

```
output          input_valid,  
output [31:0]   input_value,
```

input 的两个接口用于使用触摸屏的输入功能。从图 2.24 可以看到，当需要使用输入功能时，触摸屏底部的“START INPUTING”栏即可进入输入模式，见图 2.25。点击屏小键盘上的 OK 键完成输入，会退出输入模式，同时 lcd_module 会拉高 input_valid 信号一拍，表示有数据要输出，而输出数据 input_value 会依据之前的输入确定，当输入不足 32 位时，会自动高位补 0，比如：只输入了 123，就按 OK 键，则最终 input_valid 的值为 0x00000123。当输入有误时，可以按 BACK 键回退一格。

在本例中，顶层模块 adder_display.v 调用 LCD 触摸屏输入功能的代码如下：

```
87 //-----{从触摸屏获取输入}begin  
88 //根据实际需要输入的数修改此小节，  
89 //建议对每一个数的输入，编写单独一个 always 块  
90 //当 input_sel 为 0 时，表示输入数为加数 1，即 operand1  
91 always @(posedge clk)  
92 begin  
93     if (!resetn)  
94     begin  
95         adder_operand1 <= 32'd0;  
96     end  
97     else if (input_valid && !input_sel)  
98     begin  
99         adder_operand1 <= input_value;  
100    end  
101 end  
102  
103 //当 input_sel 为 1 时，表示输入数为加数 2，即 operand2  
104 always @(posedge clk)  
105 begin  
106     if (!resetn)  
107     begin  
108         adder_operand2 <= 32'd0;  
109     end  
110     else if (input_valid && input_sel)  
111     begin  
112         adder_operand2 <= input_value;  
113     end  
114 end  
115 //-----{从触摸屏获取输入}end
```

可以看到加法模块需要使用触摸屏输入 2 个加数，故需要一个拨码开关指示输入的数据为加数 1 还是加数 2。故理论上，触摸屏加上外部的选择信号，可以给任意信号输入 32 位的数，比如可以输入一条 32 位的指令，可以输入内存的 32 位地址等等，这些在后续的 CPU 实验中可能会经常用到。当需要使用触摸屏输入多个数据时，比如本例中输入 2 个加数，建议每个数据单独用一个 always 块采集输入，这样的写法更接近电路实际情况，示例中的代码就是如此写的。

4) adder_display.v 完整代码

```
1  //*****
2  //   > 文件名: adder_display.v
3  //   > 描述   : 加法器显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
4  //   > 作者   : LOONGSON
5  //   > 日期   : 2016-04-14
6  //*****
7  module adder_display(
8      //时钟与复位信号
9      input clk,
10     input resetn,    //后缀"n"代表低电平有效
11
12     //拨码开关, 用于选择输入数和产生 cin
13     input input_sel, //0:输入为加数 1 (add_operand1); 1:为加数 2 (add_operand2)
14     input sw_cin,
15
16     //led 灯, 用于显示 cout
17     output led_cout,
18
19     //触摸屏相关接口, 不需要更改
20     output lcd_rst,
21     output lcd_cs,
22     output lcd_rs,
23     output lcd_wr,
24     output lcd_rd,
25     inout[15:0] lcd_data_io,
26     output lcd_bl_ctr,
27     inout  ct_int,
28     Inout  ct_sda,
29     output ct_scl,
30     output ct_rstn
31 );
32
33 //-----{调用加法模块}begin
34     reg  [31:0] adder_operand1;
35     reg  [31:0] adder_operand2;
36     wire        adder_cin;
37     wire [31:0] adder_result;
38     wire        adder_cout;
39     adder adder_module(
40         .operand1 (adder_operand1),
41         .operand2 (adder_operand2),
42         .cin      (adder_cin      ),
43         .result   (adder_result  ),
44         .cout     (adder_cout     )
45     );
46     assign adder_cin = sw_cin;
47     assign led_cout  = adder_cout;
48 //-----{调用加法模块}end
49
```

```
50 //-----{调用触摸屏模块}begin-----//
51 //-----{实例化触摸屏}begin
52 //此小节不需要更改
53     reg          display_valid;
54     reg [39:0] display_name;
55     reg [31:0] display_value;
56     wire [5 :0] display_number;
57     wire          input_valid;
58     wire [31:0] input_value;
59
60     lcd_module lcd_module(
61         .clk          (clk          ),    //10Mhz
62         .resetn        (resetn        ),
63
64         //调用触摸屏的接口
65         .display_valid (display_valid ),
66         .display_name  (display_name  ),
67         .display_value (display_value ),
68         .display_number (display_number),
69         .input_valid   (input_valid   ),
70         .input_value   (input_value   ),
71
72         //lcd 触摸屏相关接口，不需要更改
73         .lcd_rst        (lcd_rst        ),
74         .lcd_cs         (lcd_cs         ),
75         .lcd_rs         (lcd_rs         ),
76         .lcd_wr         (lcd_wr         ),
77         .lcd_rd         (lcd_rd         ),
78         .lcd_data_io    (lcd_data_io    ),
79         .lcd_bl_ctr     (lcd_bl_ctr     ),
80         .ct_int         (ct_int         ),
81         .ct_sda         (ct_sda         ),
82         .ct_scl         (ct_scl         ),
83         .ct_rstn        (ct_rstn        )
84     );
85 //-----{实例化触摸屏}end
86
87 //-----{从触摸屏获取输入}begin
88 //根据实际需要输入的数修改此小节，
89 //建议对每一个数的输入，编写单独一个 always 块
90     //当 input_sel 为 0 时，表示输入数为加数 1，即 operand1
91     always @(posedge clk)
92     begin
93         if (!resetn)
94             begin
95                 adder_operand1 <= 32'd0;
96             end
97         else if (input_valid && !input_sel)
98             begin
99                 adder_operand1 <= input_value;
```

```
100         end
101     end
102
103     //当 input_sel 为 1 时, 表示输入数为加数 2, 即 operand2
104     always @(posedge clk)
105     begin
106         if (!resetn)
107             begin
108                 adder_operand2 <= 32'd0;
109             end
110         else if (input_valid && input_sel)
111             begin
112                 adder_operand2 <= input_value;
113             end
114         end
115     //-----{从触摸屏获取输入}end
116
117     //-----{输出到触摸屏显示}begin
118     //根据需要显示的数修改此小节,
119     //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
120     //44 块显示区域从 1 开始编号, 编号为 1~44,
121     always @(posedge clk)
122     begin
123         case(display_number)
124             6'd1 :
125             begin
126                 display_valid <= 1'b1;
127                 display_name  <= "ADD_1";
128                 display_value <= adder_operand1;
129             end
130             6'd2 :
131             begin
132                 display_valid <= 1'b1;
133                 display_name  <= "ADD_2";
134                 display_value <= adder_operand2;
135             end
136             6'd3 :
137             begin
138                 display_valid <= 1'b1;
139                 display_name  <= "RESUL";
140                 display_value <= adder_result;
141             end
142             default :
143             begin
144                 display_valid <= 1'b0;
145                 display_name  <= 40'd0;
146                 display_value <= 32'd0;
147             end
148         endcase
149     end
150     //-----{输出到触摸屏显示}end
```

```
151 //-----{调用触摸屏模块}end-----//
152 endmodule
```

可以看到顶层模块中调用了加法模块和 lcd_module 模块，在调用 LCD 触摸屏模块时，别忘了通过“Add Source”添加“lcd_module.dcp”。

6 定点加法实验拓展

根据之前所述，可以画出定点加法实验的顶层模块框图，如下图：

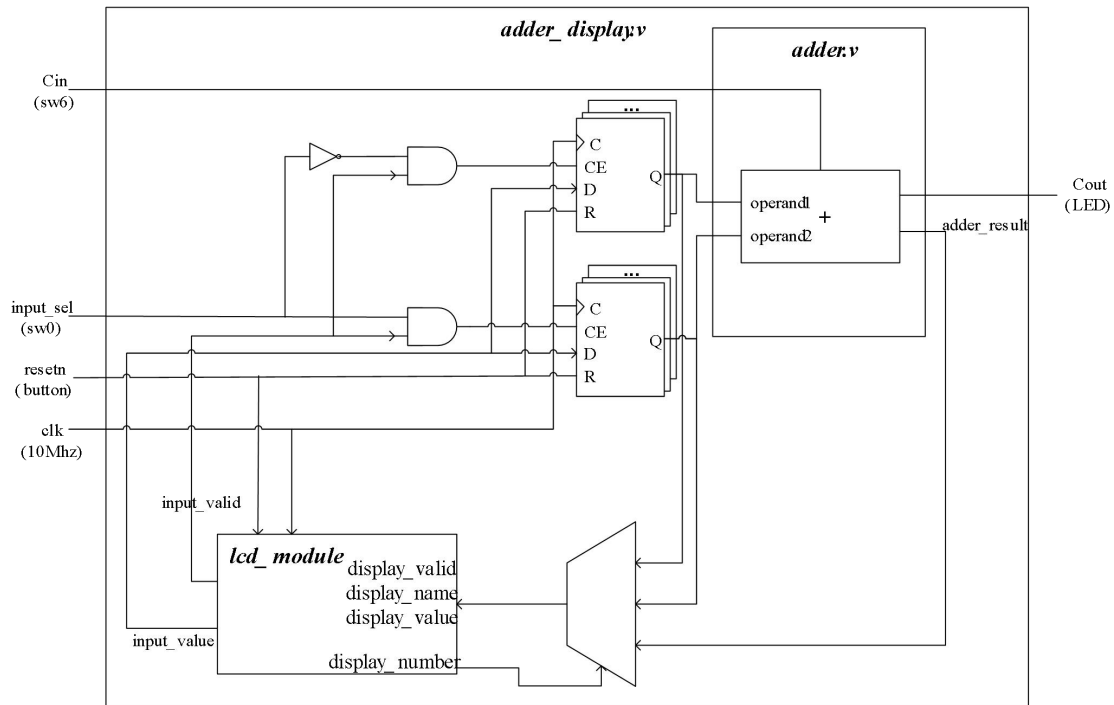


图 2.40 定点加法参考设计的顶层模块框图

从图中可以看到，外围模块 adder_display 内部调用了 adder 和 lcd_module 模块。在外部除了时钟和复位信号外，还有 cin 和 input_sel 通过拨码开关输入，以及 cout 输出到 led 灯上。

从实验二开始，针对每个实验需要画出类似图 2.40 的顶层模块。并且对内部功能模块(此处为 adder.v)需要画出原理图或实现框图，此处由于 adder.v 就用“+”实现，比较简单，故未给出加法模块的原理图。

本例中提供的实验参考设计主用于熟悉软硬件平台的，加法也是直接使用加号实现的。

学生可以在走通软硬件流程后，建议尝试修改加法的实现，可以使用 1 位全加器搭建出 32 位的加法，搭建的方法就有很多了，可以根据已学定点加法实现方法选择性地搭建串行进位加法器或先行进位加法器等，此时需要画出对应的原理图。

三、 实验二 数据运算：定点乘法

1 实验目的

1. 理解定点乘法的不同实现算法的原理，掌握基本实现算法。
2. 熟悉并运用 verilog 语言进行电路设计。
3. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习并理解计算机中定点乘法器的多种实现算法的原理，重点掌握迭代乘法的实现算法。

2. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，本次实验的乘法器建议采用迭代的方式实现，如果能力有余的，也可以采用其他效率更高的算法实现。本次实验要求实现的乘法为有符号乘法，因此需要注意计算机存储的有符号数都是补码的形式，设计方案传递进来的数也需是补码。

3. 根据设计的实验方案，使用 verilog 编写相应代码。

4. 对编写的代码进行仿真，得到正确的波形图。

5. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图

3.1。外围模块中需调用封装好的 LCD 触摸屏模块，显示两个乘数和乘法结果，且需要利用触摸功能输入两个乘数。

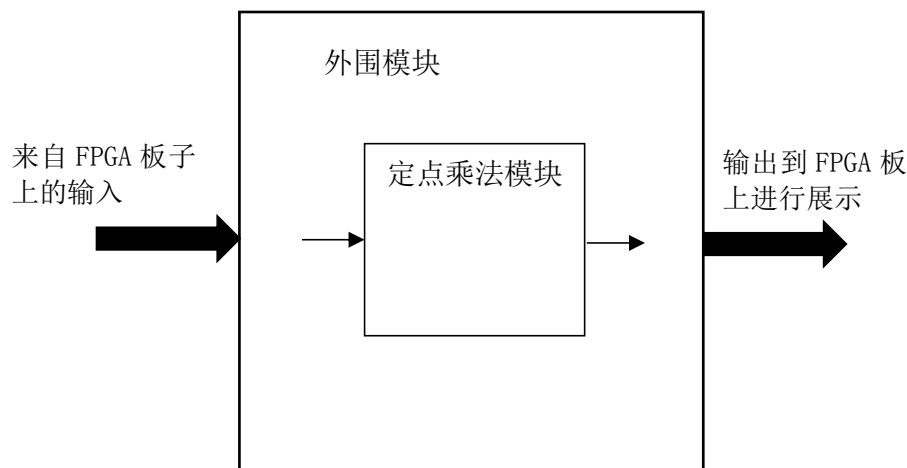


图 3.1 定点乘法设计实验的顶层模块大致框图

6. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

4 实验要求

1. 做好预习：
 - 1) 掌握定点乘法的多种实现算法的原理；
 - 2) 确定定点乘法的输入输出端口设计；
 - 3) 在课前画好设计框图或实验原理图；
 - 4) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 3.1。
2. 实验实施：
 - 1) 确认定点乘法的设计框图的正确性；
 - 2) 编写 verilog 代码；
 - 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
 - 4) 完成调用定点乘法模块的外围模块的设计，并编写代码；
 - 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。
3. 实验检查：
 - 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，可对演示结果进行拍照作为实验报告结果一项的材料。
4. 实验报告的撰写：
 - 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

5.1 实验原理

本例中设计的乘法部件实现的是迭代乘法算法，其原理图如下：

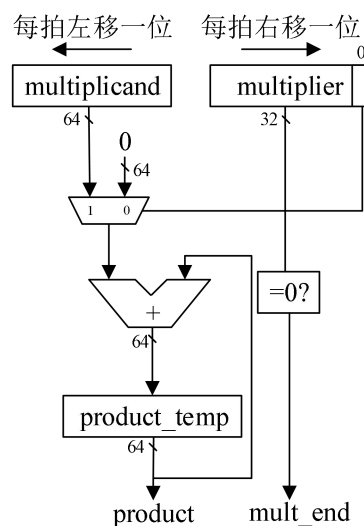


图 3.2 迭代乘法算法的原理图

图 3.2 中参与运算的为两个乘数的绝对值，乘法结果也是绝对值，需要单独判断符号位后校正乘积。

迭代乘法是在模拟我们人算乘法的过程，乘数每次右移一位，根据最低位，判断是加被乘数移位后的值还是加 0，不停地累加最后就得到乘积了。可以看到迭代乘法是用多次加法完成乘法操作的，故需要多拍时间，其结束标志为乘数移位后为 0，故对于 32 位乘法，最多需要 32 拍才能完成一次乘法。

5.2 实验顶层模块框图

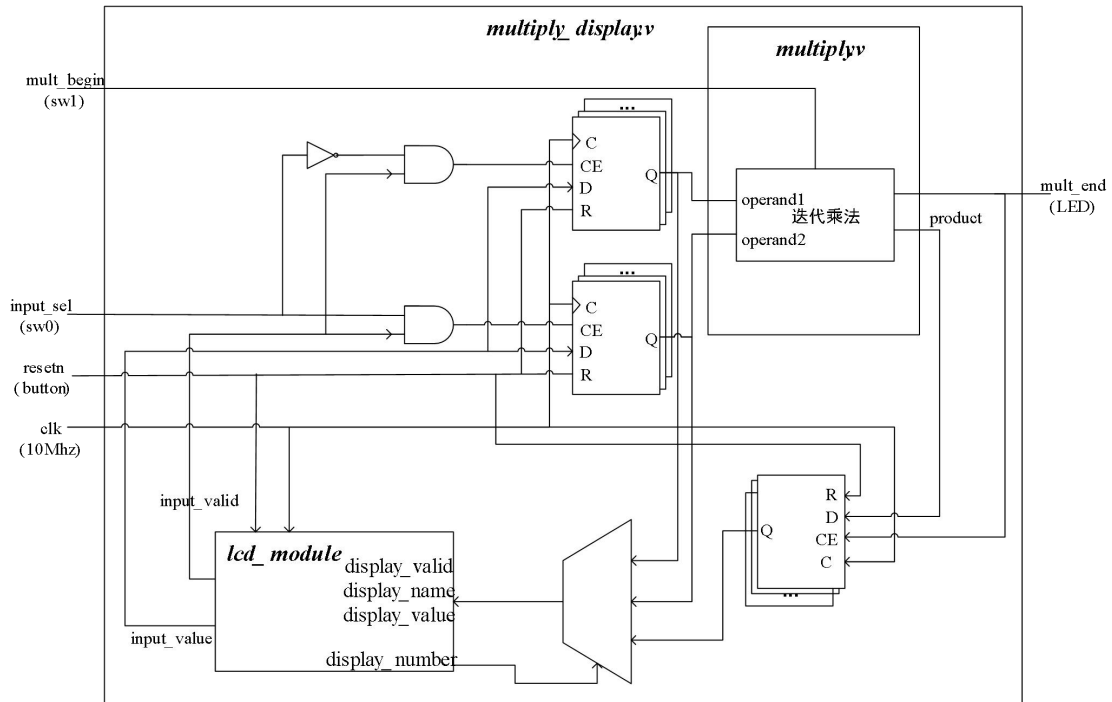


图 3.3 定点乘法参考设计的顶层模块框图

可以看到图 3.3 的框图与定点加法实验的框图类似，关键在于功能部件一个做加法，一个做乘法。由于乘法操作时需要多拍的，故只有当 mult_end 有效时，乘积才真正有效，故将乘积展示到 LCD 触摸屏上显示之前需要使用 mult_end 锁存。

5.3 源代码

multiply.v 源代码：

```
1 `timescale 1ns / 1ps
2 //*****
3 // > 文件名: multiply.v
4 // > 描述 : 乘法器模块，低效率的迭代乘法算法，使用两个乘数绝对值参与运算
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 //*****
8 module multiply( // 乘法器
9     input      clk, // 时钟
10    input      mult_begin, // 乘法开始信号
11    input [31:0] mult_op1, // 乘法源操作数 1
12    input [31:0] mult_op2, // 乘法源操作数 2
13    output [63:0] product, // 乘积
14    output      mult_end // 乘法结束信号
```

```
15 );
16
17 //乘法正在运算信号和结束信号
18 reg mult_valid;
19 assign mult_end = mult_valid & ~(|multiplier); //乘法结束信号: 乘数全
0
20 always @(posedge clk)
21 begin
22     if (!mult_begin || mult_end)
23     begin
24         mult_valid <= 1'b0;
25     end
26     else
27     begin
28         mult_valid <= 1'b1;
29     end
30 end
31
32 //两个源操作取绝对值, 正数的绝对值为其本身, 负数的绝对值为取反加 1
33 wire      op1_sign;      //操作数 1 的符号位
34 wire      op2_sign;      //操作数 2 的符号位
35 wire [31:0] op1_absolute; //操作数 1 的绝对值
36 wire [31:0] op2_absolute; //操作数 2 的绝对值
37 assign op1_sign = mult_op1[31];
38 assign op2_sign = mult_op2[31];
39 assign op1_absolute = op1_sign ? (~mult_op1+1) : mult_op1;
40 assign op2_absolute = op2_sign ? (~mult_op2+1) : mult_op2;
41
42 //加载被乘数, 运算时每次左移一位
43 reg [63:0] multiplicand;
44 always @ (posedge clk)
45 begin
46     if (mult_valid)
47     begin // 如果正在进行乘法, 则被乘数每时钟左移一位
48         multiplicand <= {multiplicand[62:0],1'b0};
49     end
50     else if (mult_begin)
51     begin // 乘法开始, 加载被乘数, 为乘数 1 的绝对值
52         multiplicand <= {32'd0,op1_absolute};
53     end
54 end
55
56 //加载乘数, 运算时每次右移一位
57 reg [31:0] multiplier;
58 always @ (posedge clk)
59 begin
60     if (mult_valid)
61     begin // 如果正在进行乘法, 则乘数每时钟右移一位
62         multiplier <= {1'b0,multiplier[31:1]};
63     end
64     else if (mult_begin)
```

```

65         begin    // 乘法开始，加载乘数，为乘数 2 的绝对值
66             multiplier <= op2_absolute;
67         end
68     end
69
70     // 部分积：乘数末位为 1，由被乘数左移得到；乘数末位为 0，部分积为 0
71     wire [63:0] partial_product;
72     assign partial_product = multiplier[0] ? multiplicand :
64'd0;
73
74     //累加器
75     reg [63:0] product_temp;
76     always @ (posedge clk)
77     begin
78         if (mult_valid)
79         begin
80             product_temp <= product_temp + partial_product;
81         end
82         else if (mult_begin)
83         begin
84             product_temp <= 64'd0;    // 乘法开始，乘积清零
85         end
86     end
87
88     //乘法结果的符号位和乘法结果
89     reg product_sign;
90     always @ (posedge clk)    // 乘积
91     begin
92         if (mult_valid)
93         begin
94             product_sign <= op1_sign ^ op2_sign;
95         end
96     end
97     //若乘法结果为负数，则需要对结果取反+1
98     assign product = product_sign ? (~product_temp+1) : product_temp;
99 endmodule

```

multiply_display.v 源代码:

```

1  //*****
2  //    > 文件名: multiply_display.v
3  //    > 描述   : 乘法器显示模块，调用 FPGA 板上的 IO 接口和触摸屏
4  //    > 作者   : LOONGSON
5  //    > 日期   : 2016-04-14
6  //*****
7  module multiply_display(
8      //时钟与复位信号
9      input clk,
10     input resetn,    //后缀"n"代表低电平有效
11
12     //拨码开关，用于选择输入数
13     input input_sel, //0:输入为乘数 1;1:输入为乘数 2

```

```
14     input sw_begin,
15
16     //乘法结束信号
17     output led_end,
18
19     //触摸屏相关接口，不需要更改
20     output lcd_rst,
21     output lcd_cs,
22     output lcd_rs,
23     output lcd_wr,
24     output lcd_rd,
25     inout[15:0] lcd_data_io,
26     output lcd_bl_ctr,
27     inout ct_int,
28     inout ct_sda,
29     output ct_scl,
30     output ct_rstn
31 );
32 //-----{调用乘法器模块}begin
33     wire      mult_begin;
34     reg  [31:0] mult_op1;
35     reg  [31:0] mult_op2;
36     wire [63:0] product;
37     wire      mult_end;
38     assign mult_begin = sw_begin;
39     assign led_end = mult_end;
40     multiply multiply_module (
41         .clk      (clk      ),
42         .mult_begin(mult_begin),
43         .mult_op1  (mult_op1 ),
44         .mult_op2  (mult_op2 ),
45         .product   (product  ),
46         .mult_end  (mult_end )
47     );
48     reg [63:0] product_r;
49     always @(posedge clk)
50     begin
51         if (!resetn)
52         begin
53             product_r <= 64'd0;
54         end
55         else if (mult_end)
56         begin
57             product_r <= product;
58         end
59     end
60 //-----{调用乘法器模块}end
61
62 //-----{调用触摸屏模块}begin-----//
63 //-----{实例化触摸屏}begin
64 //此小节不需要更改
```



```
65     reg          display_valid;
66     reg  [39:0] display_name;
67     reg  [31:0] display_value;
68     wire [5 :0] display_number;
69     wire          input_valid;
70     wire [31:0] input_value;
71
72     lcd_module lcd_module(
73         .clk          (clk          ),    //10Mhz
74         .resetn       (resetn       ),
75
76         //调用触摸屏的接口
77         .display_valid (display_valid ),
78         .display_name  (display_name  ),
79         .display_value (display_value ),
80         .display_number (display_number),
81         .input_valid   (input_valid   ),
82         .input_value   (input_value   ),
83
84         //lcd 触摸屏相关接口，不需要更改
85         .lcd_rst       (lcd_rst       ),
86         .lcd_cs        (lcd_cs        ),
87         .lcd_rs        (lcd_rs        ),
88         .lcd_wr        (lcd_wr        ),
89         .lcd_rd        (lcd_rd        ),
90         .lcd_data_io   (lcd_data_io   ),
91         .lcd_bl_ctr    (lcd_bl_ctr    ),
92         .ct_int        (ct_int        ),
93         .ct_sda        (ct_sda        ),
94         .ct_scl        (ct_scl        ),
95         .ct_rstn       (ct_rstn       )
96     );
97     //-----{实例化触摸屏}end
98
99     //-----{从触摸屏获取输入}begin
100    //根据实际需要输入的数修改此小节，
101    //建议对每一个数的输入，编写单独一个 always 块
102    //当 input_sel 为 0 时，表示输入数为乘数 1
103    always @(posedge clk)
104    begin
105        if (!resetn)
106        begin
107            mult_op1 <= 32'd0;
108        end
109        else if (input_valid && !input_sel)
110        begin
111            mult_op1 <= input_value;
112        end
113    end
114
115    //当 input_sel 为 1 时，表示输入数为乘数 2
```

```
116     always @(posedge clk)
117     begin
118         if (!resetn)
119             begin
120                 mult_op2 <= 32'd0;
121             end
122         else if (input_valid && input_sel)
123             begin
124                 mult_op2 <= input_value;
125             end
126         end
127 //-----{从触摸屏获取输入}end
128
129 //-----{输出到触摸屏显示}begin
130 //根据需要显示的数修改此小节,
131 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
132 //44 块显示区域从 1 开始编号, 编号为 1~44,
133     always @(posedge clk)
134     begin
135         case(display_number)
136             6'd1 :
137                 begin
138                     display_valid <= 1'b1;
139                     display_name  <= "M_OP1";
140                     display_value <= mult_op1;
141                 end
142             6'd2 :
143                 begin
144                     display_valid <= 1'b1;
145                     display_name  <= "M_OP2";
146                     display_value <= mult_op2;
147                 end
148             6'd3 :
149                 begin
150                     display_valid <= 1'b1;
151                     display_name  <= "PRO_H";
152                     display_value <= product_r[63:32];
153                 end
154             6'd4 :
155                 begin
156                     display_valid <= 1'b1;
157                     display_name  <= "PRO_L";
158                     display_value <= product_r[31: 0];
159                 end
160             default :
161                 begin
162                     display_valid <= 1'b0;
163                     display_name  <= 48'd0;
164                     display_value <= 32'd0;
165                 end
166             endcase
```

```

167         end
168     //-----{输出到触摸屏显示}end
169     //-----{调用触摸屏模块}end-----//
170 endmodule

```

约束文件 multiply.xdc 源代码:

```

1     set_property PACKAGE_PIN AC19 [get_ports clk]
2     set_property PACKAGE_PIN A3 [get_ports led_end]
3     set_property PACKAGE_PIN Y3 [get_ports resetn]
4     set_property PACKAGE_PIN AC21 [get_ports input_sel]
5     set_property PACKAGE_PIN AD24 [get_ports sw_begin]
6
7     set_property IOSTANDARD LVCMOS33 [get_ports clk]
8     set_property IOSTANDARD LVCMOS33 [get_ports led_end]
9     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
10    set_property IOSTANDARD LVCMOS33 [get_ports input_sel]
11    set_property IOSTANDARD LVCMOS33 [get_ports sw_begin]
12
13    #触摸屏引脚连接
14    set_property PACKAGE_PIN J25 [get_ports lcd_rst]
15    set_property PACKAGE_PIN H18 [get_ports lcd_cs]
16    set_property PACKAGE_PIN K16 [get_ports lcd_rs]
17    set_property PACKAGE_PIN L8 [get_ports lcd_wr]
18    set_property PACKAGE_PIN K8 [get_ports lcd_rd]
19    set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
20    set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
21    set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
22    set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
23    set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
24    set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
25    set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
26    set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
27    set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
28    set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
29    set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
30    set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
31    set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
32    set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
33    set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
34    set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
35    set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
36    set_property PACKAGE_PIN L19 [get_ports ct_int]
37    set_property PACKAGE_PIN J24 [get_ports ct_sda]
38    set_property PACKAGE_PIN H21 [get_ports ct_scl]
39    set_property PACKAGE_PIN G24 [get_ports ct_rstn]
40
41    set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
42    set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
43    set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]

```

```
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
47 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

四、 实验三 寄存器堆实现

1 实验目的

1. 熟悉并掌握 MIPS 计算机中寄存器堆的原理和设计方法。
2. 初步了解 MIPS 指令结构和源操作数/目的操作数的概念。
3. 熟悉并运用 verilog 语言进行电路设计。
4. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 计算机中寄存器堆的设计及原理，如：有多少个寄存器，有无特殊设置的寄存器，mips 指令如何去索引寄存器的等。
2. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，本次实验建议设计为异步读同步写的寄存器堆，即读寄存器不需要时钟控制，但写寄存器需时钟控制。
3. 本次实验建议寄存器堆设计为 1 个写端口和 2 个读端口，后续 CPU 实验用到的寄存器堆需要 1 个写端口和 2 个读端口。
4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 4.1。外围模块中需调用封装好的 LCD 触摸屏模块，显示寄存器堆的读写端口地址和数据，最好能扫描出所有寄存器的值显示在 LCD 触摸屏上，并且需要利用触摸功能输入寄存器堆的读写地址和写数据。

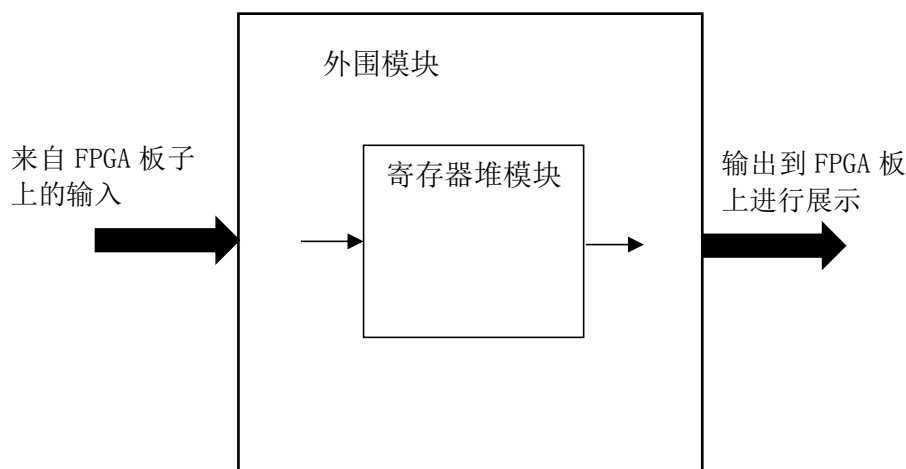


图 4.1 寄存器堆设计实验的顶层模块大致框图

7. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

4 实验要求

1. 做好预习：

- 1) 掌握寄存器堆的工作原理；
- 2) 确定寄存器堆的输入输出端口设计；
- 3) 在课前画好寄存器堆的设计框图或实验原理图；
- 4) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 4.1。

2. 实验实施：

- 1) 确认寄存器堆的设计框图的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用寄存器堆模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，按照检查人员的要求，对特定寄存器读写，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 调用 LCD 触摸屏

在寄存器堆实验中，调用 LCD 触摸屏的显示和输入功能时，有几点需要注意的地方。

5.1 显示功能

在本实验中需要扫描出寄存器堆里的所有寄存器的值显示在 LCD 触摸屏上，比如我们在 LCD 屏上 1~32 号显示块上对应显示 0~31 号寄存器的显示地址。由于 LCD 屏在显示时，是根据显示块号“display_number”来选择要显示的数据的，故可以给寄存器堆再增加一个调试读端口(此时寄存器堆有 3 个读端口)，该读端口的地址为 display_number-1，对应 0~31，读出的数据即为要显示的模块，代码可以如下编写：

```
1 assign test_addr = display_number-5'd1;
2 //-----{输出到触摸屏显示}begin
3 //根据需要显示的数修改此小节，
4 //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
5 //44 块显示区域从 1 开始编号，编号为 1~44，
6     always @(posedge clk)
```



```
7      begin
8          if (display_number > 6'd0 && display_number < 6'd33 )
9              begin //块号 1~32 显示 32 个通用寄存器的值
10                 display_valid <= 1'b1;
11                 display_name[39:16] <= "REG";
12                 display_name[15: 8] <= {4'b0011,3'b000,test_addr[4]};
13                 display_name[7 : 0] <= {4'b0011,test_addr[3:0]};
14                 display_value <= test_data;
15             end
16         else
17             begin
18                 case(display_number)
19                     6'd33: //显示读端口 1 的地址
20                         begin
21                             display_valid <= 1'b1;
22                             display_name <= "RADD1";
23                             display_value <= raddr1;
24                         end
25                     .....
26                     .....
```

test_addr 为寄存器堆新增的调试读端口，而 test_data 为通过该端口读出的数据

可以看到在显示 32 个寄存器时，其“display_name”域有特殊处理，其头 3 个字符显示“REG”，后两个字符则显示 16 进制的寄存器号“00~1F”，由于 0~9 字符的 ASCII 码为 48~57(8'b0011_0000~8'b0011_1001)，故“display_name”第 4 位字符(0 或 1)赋值{7'b0011_000,test_addr[4]}即可。

但对于第 5 位字符(0~F)的赋值则有稍许麻烦，因为 A~F 字符的 ASCII 编码不是随后的 58~63，即与 0~9 的编码不连续。但我们在设计 LCD 屏显示字符时，将内部的 ASCII 编码 58~63 也作为 A~F 字符的编码，即 ASCII 编码 48~ 63(8'b0011_0000 ~ 8'b0011_1111)对应字符 0~F，，故我们可以直接给第 5 位字符赋值{4'b0011,test_addr[3:0]}。

5.2 输入功能

在调用 LCD 触摸屏的输入功能时，其输入的数据为 32 位的数据，但在寄存器堆的实验中，读写地址为 5 位的，但依然可以使用屏输入读写地址，比如，可以取输入的 32 位数据的低 5 位作为读写地址。

6 参考设计

本部分给出的寄存器堆实验的参考设计方案中，共实现 32 个寄存器，其中 0 号寄存器读出的值恒为 0，寄存器堆为异步读同步写，共有 1 个写端口和 3 个读端口，其中一个读端口为调试读端口，用于扫描读出 32 个寄存器，给出了实验原理和设计框图，并提供源代码。

6.1 实验原理

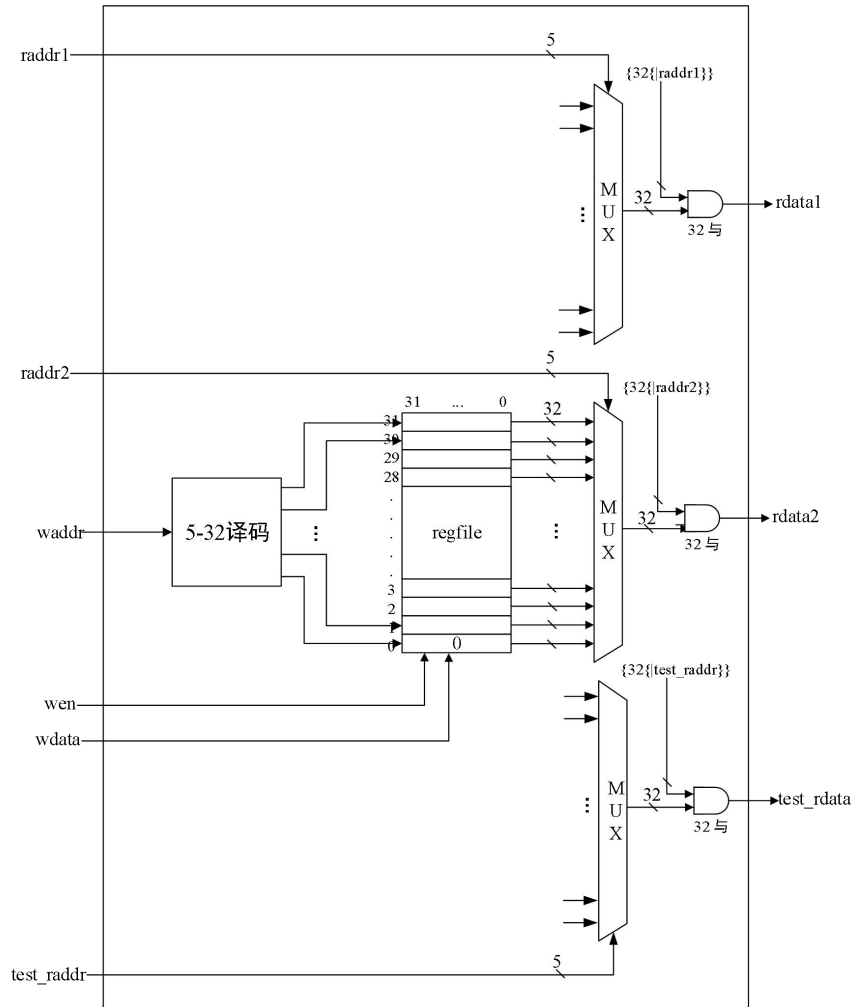


图 4.2 寄存器堆实验的原理图

6.2 实验顶层模块框图

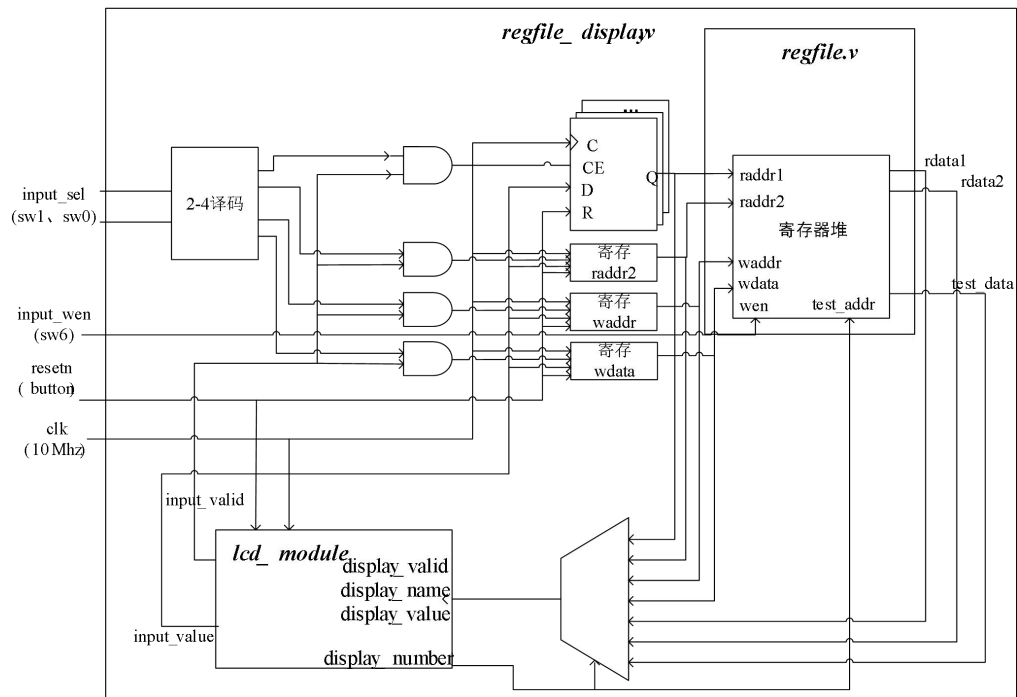


图 4.3 寄存器堆参考设计的顶层模块框图

6.3 源代码

regfile.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
3 //    > 文件名: regfile.v
4 //    > 描述   : 寄存器堆模块, 同步写, 异步读
5 //    > 作者   : LOONGSON
6 //    > 日期   : 2016-04-14
7 //*****
8 module regfile(
9     input          clk,
10    input          wen,
11    input [4 :0]    raddr1,
12    input [4 :0]    raddr2,
13    input [4 :0]    waddr,
14    input [31:0]    wdata,
15    output reg [31:0] rdata1,
16    output reg [31:0] rdata2,
17    input [4 :0]    test_addr,
18    output reg [31:0] test_data
19 );
20 reg [31:0] rf[31:0];
21
22 // three ported register file
23 // read two ports combinationally
24 // write third port on rising edge of clock
25 // register 0 hardwired to 0
26
27 always @(posedge clk)
28 begin
29     if (wen)
30     begin
31         rf[waddr] <= wdata;
32     end
33 end
34
35 //读端口 1
36 always @(*)
37 begin
38     case (raddr1)
39         5'd1 : rdata1 <= rf[1 ];
40         5'd2 : rdata1 <= rf[2 ];
41         5'd3 : rdata1 <= rf[3 ];
42         5'd4 : rdata1 <= rf[4 ];
43         5'd5 : rdata1 <= rf[5 ];
44         5'd6 : rdata1 <= rf[6 ];
45         5'd7 : rdata1 <= rf[7 ];
46         5'd8 : rdata1 <= rf[8 ];
47         5'd9 : rdata1 <= rf[9 ];
```

```
48          5'd10: rdata1 <= rf[10];
49          5'd11: rdata1 <= rf[11];
50          5'd12: rdata1 <= rf[12];
51          5'd13: rdata1 <= rf[13];
52          5'd14: rdata1 <= rf[14];
53          5'd15: rdata1 <= rf[15];
54          5'd16: rdata1 <= rf[16];
55          5'd17: rdata1 <= rf[17];
56          5'd18: rdata1 <= rf[18];
57          5'd19: rdata1 <= rf[19];
58          5'd20: rdata1 <= rf[20];
59          5'd21: rdata1 <= rf[21];
60          5'd22: rdata1 <= rf[22];
61          5'd23: rdata1 <= rf[23];
62          5'd24: rdata1 <= rf[24];
63          5'd25: rdata1 <= rf[25];
64          5'd26: rdata1 <= rf[26];
65          5'd27: rdata1 <= rf[27];
66          5'd28: rdata1 <= rf[28];
67          5'd29: rdata1 <= rf[29];
68          5'd30: rdata1 <= rf[30];
69          5'd31: rdata1 <= rf[31];
70          default : rdata1 <= 32'd0;
71      endcase
72  end
73  //读端口 2
74  always @(*)
75  begin
76      case (raddr2)
77          5'd1 : rdata2 <= rf[1 ];
78          5'd2 : rdata2 <= rf[2 ];
79          5'd3 : rdata2 <= rf[3 ];
80          5'd4 : rdata2 <= rf[4 ];
81          5'd5 : rdata2 <= rf[5 ];
82          5'd6 : rdata2 <= rf[6 ];
83          5'd7 : rdata2 <= rf[7 ];
84          5'd8 : rdata2 <= rf[8 ];
85          5'd9 : rdata2 <= rf[9 ];
86          5'd10: rdata2 <= rf[10];
87          5'd11: rdata2 <= rf[11];
88          5'd12: rdata2 <= rf[12];
89          5'd13: rdata2 <= rf[13];
90          5'd14: rdata2 <= rf[14];
91          5'd15: rdata2 <= rf[15];
92          5'd16: rdata2 <= rf[16];
93          5'd17: rdata2 <= rf[17];
94          5'd18: rdata2 <= rf[18];
95          5'd19: rdata2 <= rf[19];
96          5'd20: rdata2 <= rf[20];
97          5'd21: rdata2 <= rf[21];
98          5'd22: rdata2 <= rf[22];
```

```
99          5'd23: rdata2 <= rf[23];
100         5'd24: rdata2 <= rf[24];
101         5'd25: rdata2 <= rf[25];
102         5'd26: rdata2 <= rf[26];
103         5'd27: rdata2 <= rf[27];
104         5'd28: rdata2 <= rf[28];
105         5'd29: rdata2 <= rf[29];
106         5'd30: rdata2 <= rf[30];
107         5'd31: rdata2 <= rf[31];
108         default : rdata2 <= 32'd0;
109     endcase
110 end
111 //调试端口，读出寄存器值显示在触摸屏上
112 always @(*)
113 begin
114     case (test_addr)
115         5'd1 : test_data <= rf[1 ];
116         5'd2 : test_data <= rf[2 ];
117         5'd3 : test_data <= rf[3 ];
118         5'd4 : test_data <= rf[4 ];
119         5'd5 : test_data <= rf[5 ];
120         5'd6 : test_data <= rf[6 ];
121         5'd7 : test_data <= rf[7 ];
122         5'd8 : test_data <= rf[8 ];
123         5'd9 : test_data <= rf[9 ];
124         5'd10: test_data <= rf[10];
125         5'd11: test_data <= rf[11];
126         5'd12: test_data <= rf[12];
127         5'd13: test_data <= rf[13];
128         5'd14: test_data <= rf[14];
129         5'd15: test_data <= rf[15];
130         5'd16: test_data <= rf[16];
131         5'd17: test_data <= rf[17];
132         5'd18: test_data <= rf[18];
133         5'd19: test_data <= rf[19];
134         5'd20: test_data <= rf[20];
135         5'd21: test_data <= rf[21];
136         5'd22: test_data <= rf[22];
137         5'd23: test_data <= rf[23];
138         5'd24: test_data <= rf[24];
139         5'd25: test_data <= rf[25];
140         5'd26: test_data <= rf[26];
141         5'd27: test_data <= rf[27];
142         5'd28: test_data <= rf[28];
143         5'd29: test_data <= rf[29];
144         5'd30: test_data <= rf[30];
145         5'd31: test_data <= rf[31];
146         default : test_data <= 32'd0;
147     endcase
148 end
149 endmodule
```

regfile_display.v 源代码:

```
1  //*****
2  //  > 文件名: regfile_display.v
3  //  > 描述   : 寄存器堆显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
4  //  > 作者   : LOONGSON
5  //  > 日期   : 2016-04-14
6  //*****
7  module regfile_display(
8      //时钟与复位信号
9      input clk,
10     input resetn,    //后缀"n"代表低电平有效
11
12     //拨码开关, 用于产生写使能和选择输入数
13     input wen,
14     input [1:0] input_sel,
15
16     //led 灯, 用于指示写使能信号, 和正在输入什么数据
17     output led_wen,
18     output led_waddr,    //指示输入写地址
19     output led_wdata,    //指示输入写数据
20     output led_raddr1,    //指示输入读地址 1
21     output led_raddr2,    //指示输入读地址 2
22
23     //触摸屏相关接口, 不需要更改
24     output lcd_rst,
25     output lcd_cs,
26     output lcd_rs,
27     output lcd_wr,
28     output lcd_rd,
29     inout[15:0] lcd_data_io,
30     output lcd_bl_ctr,
31     inout ct_int,
32     inout ct_sda,
33     output ct_scl,
34     output ct_rstn
35 );
36 //----{LED 显示}begin
37     assign led_wen    = wen;
38     assign led_raddr1 = (input_sel==2'd0);
39     assign led_raddr2 = (input_sel==2'd1);
40     assign led_waddr  = (input_sel==2'd2);
41     assign led_wdata  = (input_sel==2'd3);
42 //----{LED 显示}end
43
44 //----{调用寄存器堆模块}begin
45     //寄存器堆多增加一个读端口, 用于在触摸屏上显示 32 个寄存器值
46     wire [31:0] test_data;
47     wire [4 :0] test_addr;
48
```



```
49     reg  [4 :0]  raddr1;
50     reg  [4 :0]  raddr2;
51     reg  [4 :0]  waddr;
52     reg  [31:0]  wdata;
53     wire [31:0]  rdata1;
54     wire [31:0]  rdata2;
55     regfile rf_module(
56         .clk      (clk      ),
57         .wen      (wen      ),
58         .raddr1   (raddr1),
59         .raddr2   (raddr2),
60         .waddr    (waddr   ),
61         .wdata    (wdata   ),
62         .rdata1   (rdata1),
63         .rdata2   (rdata2),
64         .test_addr(test_addr),
65         .test_data(test_data)
66     );
67 //-----{调用寄存器堆模块}end
68
69 //-----{调用触摸屏模块}begin-----//
70 //-----{实例化触摸屏}begin
71 //此小节不需要更改
72     reg      display_valid;
73     reg  [39:0] display_name;
74     reg  [31:0] display_value;
75     wire [5 :0] display_number;
76     wire      input_valid;
77     wire [31:0] input_value;
78
79     lcd_module lcd_module(
80         .clk      (clk      ),    //10Mhz
81         .resetn   (resetn   ),
82
83         //调用触摸屏的接口
84         .display_valid  (display_valid ),
85         .display_name   (display_name  ),
86         .display_value  (display_value ),
87         .display_number (display_number),
88         .input_valid    (input_valid   ),
89         .input_value    (input_value   ),
90
91         //lcd 触摸屏相关接口，不需要更改
92         .lcd_rst        (lcd_rst       ),
93         .lcd_cs         (lcd_cs        ),
94         .lcd_rs         (lcd_rs        ),
95         .lcd_wr         (lcd_wr        ),
96         .lcd_rd         (lcd_rd        ),
97         .lcd_data_io    (lcd_data_io   ),
98         .lcd_bl_ctr     (lcd_bl_ctr    ),
99         .ct_int         (ct_int        ),
```

```
100         .ct_sda          (ct_sda          ),
101         .ct_scl          (ct_scl          ),
102         .ct_rstn         (ct_rstn         )
103     );
104 //-----{实例化触摸屏}end
105
106 //-----{从触摸屏获取输入}begin
107 //根据实际需要输入的数修改此小节,
108 //建议对每一个数的输入, 编写单独一个 always 块
109     //32 个寄存器显示在 7~38 号的显示块, 故读地址为 (display_number-1)
110     assign test_addr = display_number-5'd7;
111     //当 input_sel 为 2'b00 时, 表示输入数为读地址 1, 即 raddr1
112     always @(posedge clk)
113     begin
114         if (!resetn)
115         begin
116             raddr1 <= 5'd0;
117         end
118         else if (input_valid && input_sel==2'd0)
119         begin
120             raddr1 <= input_value[4:0];
121         end
122     end
123
124     //当 input_sel 为 2'b01 时, 表示输入数为读地址 2, 即 raddr2
125     always @(posedge clk)
126     begin
127         if (!resetn)
128         begin
129             raddr2 <= 5'd0;
130         end
131         else if (input_valid && input_sel==2'd1)
132         begin
133             raddr2 <= input_value[4:0];
134         end
135     end
136
137     //当 input_sel 为 2'b10 时, 表示输入数为写地址, 即 waddr
138     always @(posedge clk)
139     begin
140         if (!resetn)
141         begin
142             waddr <= 5'd0;
143         end
144         else if (input_valid && input_sel==2'd2)
145         begin
146             waddr <= input_value[4:0];
147         end
148     end
149
150     //当 input_sel 为 2'b11 时, 表示输入数为写数据, 即 wdata
```

```
151     always @(posedge clk)
152     begin
153         if (!resetn)
154             begin
155                 wdata <= 32'd0;
156             end
157         else if (input_valid && input_sel==2'd3)
158             begin
159                 wdata <= input_value;
160             end
161     end
162 //-----{从触摸屏获取输入}end
163
164 //-----{输出到触摸屏显示}begin
165 //根据需要显示的数修改此小节,
166 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
167 //44 块显示区域从 1 开始编号, 编号为 1~44,
168     always @(posedge clk)
169     begin
170         if (display_number >6'd6 && display_number <6'd39 )
171             begin //块号 7~38 显示 32 个通用寄存器的值
172                 display_valid <= 1'b1;
173                 display_name[39:16] <= "REG";
174                 display_name[15: 8] <=
175                 {4'b0011,3'b000,test_addr[4]};
176                 display_name[7 : 0] <= {4'b0011,test_addr[3:0]};
177                 display_value <= test_data;
178             end
179         else
180             begin
181                 case(display_number)
182                     6'd1 : //显示读端口 1 的地址
183                         begin
184                             display_valid <= 1'b1;
185                             display_name <= "RADD1";
186                             display_value <= raddr1;
187                         end
188                     6'd2 : //显示读端口 1 读出的数据
189                         begin
190                             display_valid <= 1'b1;
191                             display_name <= "RDAT1";
192                             display_value <= rdata1;
193                         end
194                     6'd3 : //显示读端口 2 的地址
195                         begin
196                             display_valid <= 1'b1;
197                             display_name <= "RADD2";
198                             display_value <= raddr2;
199                         end
200                     6'd4 : //显示读端口 2 读出的数据
201                         begin
```

```
201             display_valid <= 1'b1;
202             display_name  <= "RDAT2";
203             display_value <= rdata2;
204         end
205         6'd5 : //显示写端口的地址
206         begin
207             display_valid <= 1'b1;
208             display_name  <= "WADDR";
209             display_value <= waddr;
210         end
211         6'd6 : //显示写端口写入的数据
212         begin
213             display_valid <= 1'b1;
214             display_name  <= "WDATA";
215             display_value <= wdata;
216         end
217         default :
218         begin
219             display_valid <= 1'b0;
220             display_name  <= 40'd0;
221             display_value <= 32'd0;
222         end
223     endcase
224 end
225 end
226 //-----{输出到触摸屏显示}end
227 //-----{调用触摸屏模块}end-----//
228 endmodule
```

约束文件 regfile.xdc 源代码:

```
1      #时钟信号连接
2      set_property PACKAGE_PIN AC19 [get_ports clk]
3
4      #脉冲开关，用于输入作为复位信号，低电平有效
5      set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7      #led 灯连接，用于输出
8      set_property PACKAGE_PIN H7 [get_ports led_wen]
9      set_property PACKAGE_PIN D5 [get_ports led_raddr1]
10     set_property PACKAGE_PIN A3 [get_ports led_raddr2]
11     set_property PACKAGE_PIN A5 [get_ports led_waddr]
12     set_property PACKAGE_PIN A4 [get_ports led_wdata]
13
14     #拨码开关连接，用于输入，依次为 sw0,sw1,sw7
15     set_property PACKAGE_PIN AC21 [get_ports wen]
16     set_property PACKAGE_PIN AD24 [get_ports input_sel[1]]
17     set_property PACKAGE_PIN Y6  [get_ports input_sel[0]]
18
19     set_property IOSTANDARD LVCMOS33 [get_ports clk]
20     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
```

```
21 set_property IOSTANDARD LVCMOS33 [get_ports led_wen]
22 set_property IOSTANDARD LVCMOS33 [get_ports led_raddr1]
23 set_property IOSTANDARD LVCMOS33 [get_ports led_raddr2]
24 set_property IOSTANDARD LVCMOS33 [get_ports led_waddr]
25 set_property IOSTANDARD LVCMOS33 [get_ports led_wdata]
26 set_property IOSTANDARD LVCMOS33 [get_ports wen]
27 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]
28 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]
29
30 #触摸屏引脚连接
31 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
32 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
33 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
34 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
35 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
36 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
37 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
38 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
39 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
40 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
41 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
42 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
43 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
44 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
45 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
46 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
47 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
48 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
49 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
50 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
51 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
52 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
53 set_property PACKAGE_PIN L19 [get_ports ct_int]
54 set_property PACKAGE_PIN J24 [get_ports ct_sda]
55 set_property PACKAGE_PIN H21 [get_ports ct_scl]
56 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
57
58 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
59 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
60 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
61 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
62 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
63 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
64 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
66 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
67 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
```

```
68 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
69 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
70 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
71 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
72 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
73 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
74 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
75 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
76 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
78 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
79 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
80 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
81 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
82 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
83 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

五、 实验四 ALU 模块实现

1 实验目的

1. 熟悉 MIPS 指令集中的运算指令，学会对这些指令进行归纳分类。
2. 了解 MIPS 指令结构。
3. 熟悉并掌握 ALU 的原理、功能和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 指令集，熟知指令类型，了解指令功能和编码，归纳基础的 ALU 运算指令。
2. 归纳确定自己本次实验中准备实现的 ALU 运算，要求不实现定点乘除指令和浮点运算指令，要求至少实现 5 种 ALU 运算，其中要包含加减运算，其中减法在内部要转换为加法，与加法运算共同调用实验一里自己完成的加法模块去做。
3. 自行设计本次实验的方案，画出结构框图，大致结构框图如图 5.1。图 5.1 中的操作码位数和类型请自行设计，可以设计为独热码（一位有效编码）或二进制编码。比如，设计方案中预定实现 7 种 ALU 运算，则操作码采用独热码，则需 7bit 数据，每位单独指示一种运算；若采用二进制编码，则只用 3bit 数据位即可，但在需 ALU 内部先进行解码，才能确定 ALU 作何种运算。

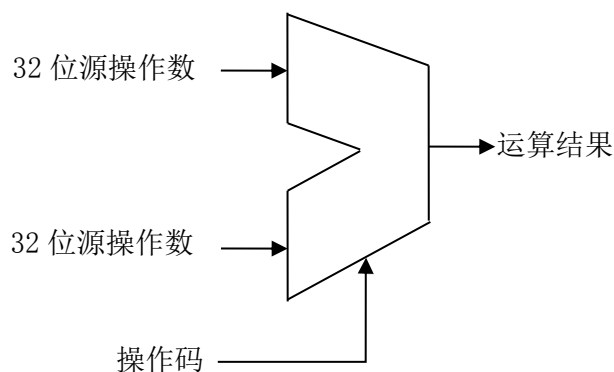


图 5.1 ALU 模块的大致框图

4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 5.2。外围模块中需调用封装好的 LCD 触摸屏模块，显示 ALU 的两个源操作数、操作码

和运算结果，并且需要利用触摸功能输入源操作数。操作码可以考虑用 LCD 触摸屏输入，也可以用拨码开关输入。

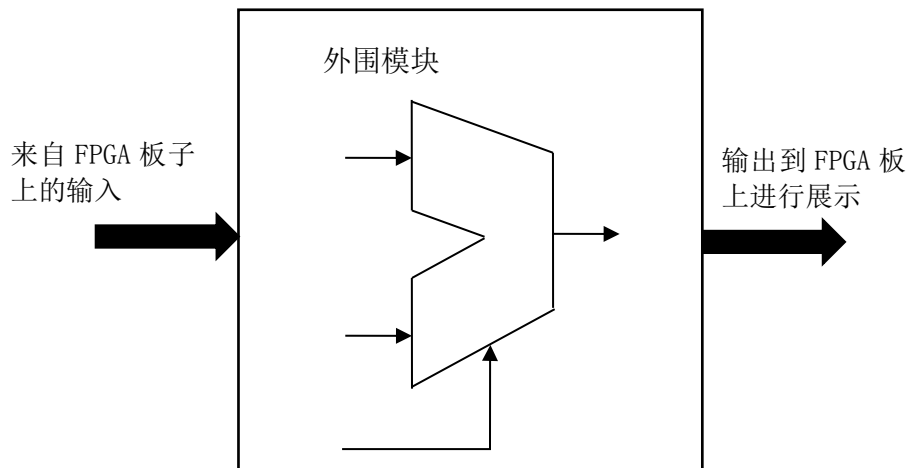


图 5.2 ALU 设计实验的顶层模块大致框图

6. 将编写的代码进行综合布局布线，并下载到试验箱中的 FPGA 板子上进行演示。

4 参考设计

1. 做好预习：

- 1) 熟知指令类型，了解指令功能和编码；
- 2) 归纳基础的 ALU 运算指令，确定自己准备实现的 ALU 运算；
- 3) 设计本次实验的方案，列出准备实现的 ALU 运算和操作码的编码；
- 4) 在课前画好实验方案的设计框图，即补充完善图 5.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 5.2。

2. 实验实施：

- 1) 确认 ALU 模块的设计框图的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用 ALU 模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到试验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先说明自己实现的 ALU 运算类型，按照检查人员的要求，对特定源操作数进行特定运算操作，检查运算结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

本部分给出的 ALU 实验的参考设计方案中，共实现了 12 种运算，采用独热编码操作码(即 ALU 的控制信号)，对应关系如下：

表 5.1 ALU 的控制信号

控制信号												ALU 操作
11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	无
1	0	0	0	0	0	0	0	0	0	0	0	加法
0	1	0	0	0	0	0	0	0	0	0	0	减法
0	0	1	0	0	0	0	0	0	0	0	0	有符号比较，小于置位
0	0	0	1	0	0	0	0	0	0	0	0	无符号比较，小于置位
0	0	0	0	1	0	0	0	0	0	0	0	按位与
0	0	0	0	0	1	0	0	0	0	0	0	按位或非
0	0	0	0	0	0	1	0	0	0	0	0	按位或
0	0	0	0	0	0	0	1	0	0	0	0	按位异或
0	0	0	0	0	0	0	0	1	0	0	0	逻辑左移
0	0	0	0	0	0	0	0	0	1	0	0	逻辑右移
0	0	0	0	0	0	0	0	0	0	1	0	算术右移
0	0	0	0	0	0	0	0	0	0	0	1	高位加载

5.1 实验原理

ALU 模块的原理图如下：

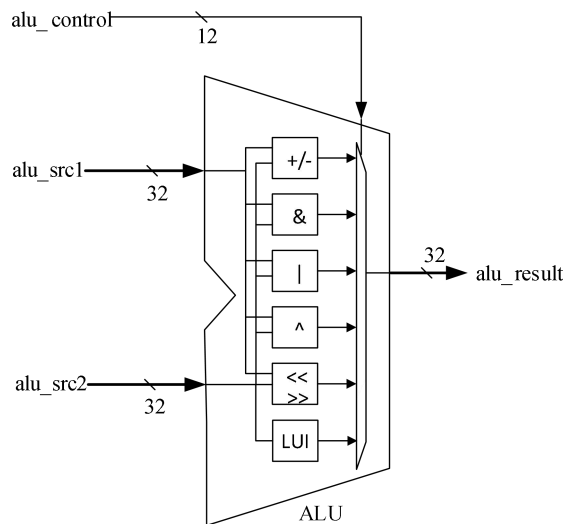


图 5.3 ALU 的原理图

1) 加减运算原理图

加减运算最后都是调用实验一中的加法器做的：

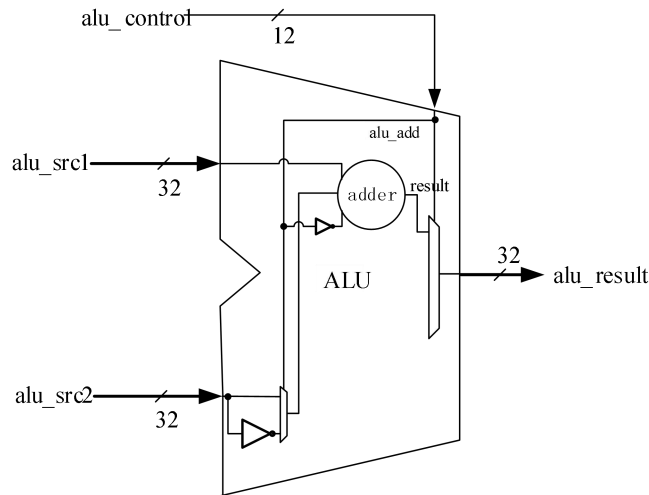


图 5.4 加减法运算的原理图

2) 小于置位运算

对于有符号比较的小于置位，是利用减法结果比较的，其比较的真值表如下：

表 5.2 有符号比较小于置位的真值表

源操作数 1 符号位		源操作数 2 符号位		结果符号位		判断	slt 结果
alu_src1[31]		alu_src2[31]		adder_result[31]			
0	正数	0	正数	0	正数	正>正	0
0	正数	0	正数	1	负数	正<正	1
0	正数	1	负数	X	无关	正>负	0
1	负数	0	正数	X	无关	负<正	1
1	负数	1	负数	0	正数	负>负	0
1	负数	1	负数	1	负数	负<负	1

由表 5.2 可得有符号 32 位比较小于置位运算结果表达式：

$$\text{slt_result} = (\text{alu_src1}[31] \& \sim \text{alu_src2}[31]) \mid (\sim (\text{alu_src1}[31] \wedge \text{alu_src2}[31]) \& \text{adder_result}[31])$$

对于 32 位无符号比较的小于置位，可在其高位前填 0 组合为 33 位正数的比较，即 {1'b0, src1} 和 {1'b0, src2} 的比较，最高位符号位为 0。对比表 5.2 可知，对于正数的比较，只要减法结果的符号位为 1，则表示小于。而 33 位正数相减，其结果的符号位最终可由 32 位加法的 cout+1'b1 得到，如图 5.5。故无符号 32 位比较小于置位运算结果表达式为：sltu_result = ~adder_cout。

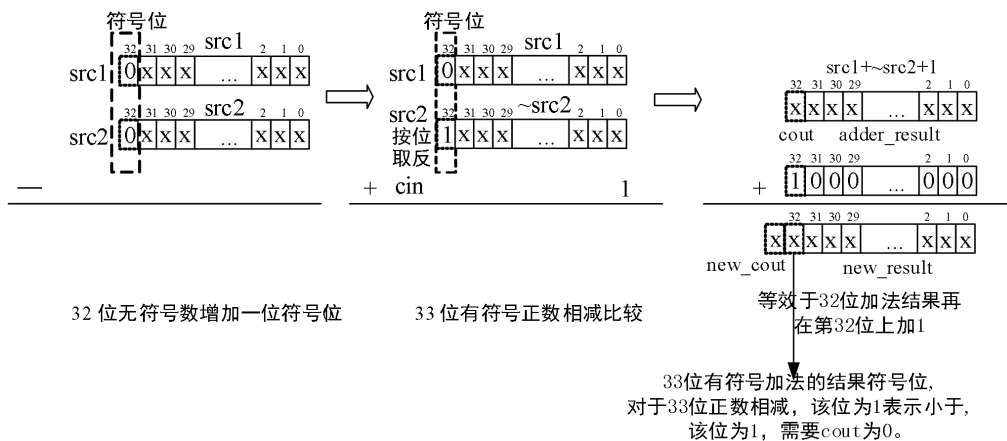


图 5.5 无符号 32 位数比较的原理图

5.2 实验顶层模块框图

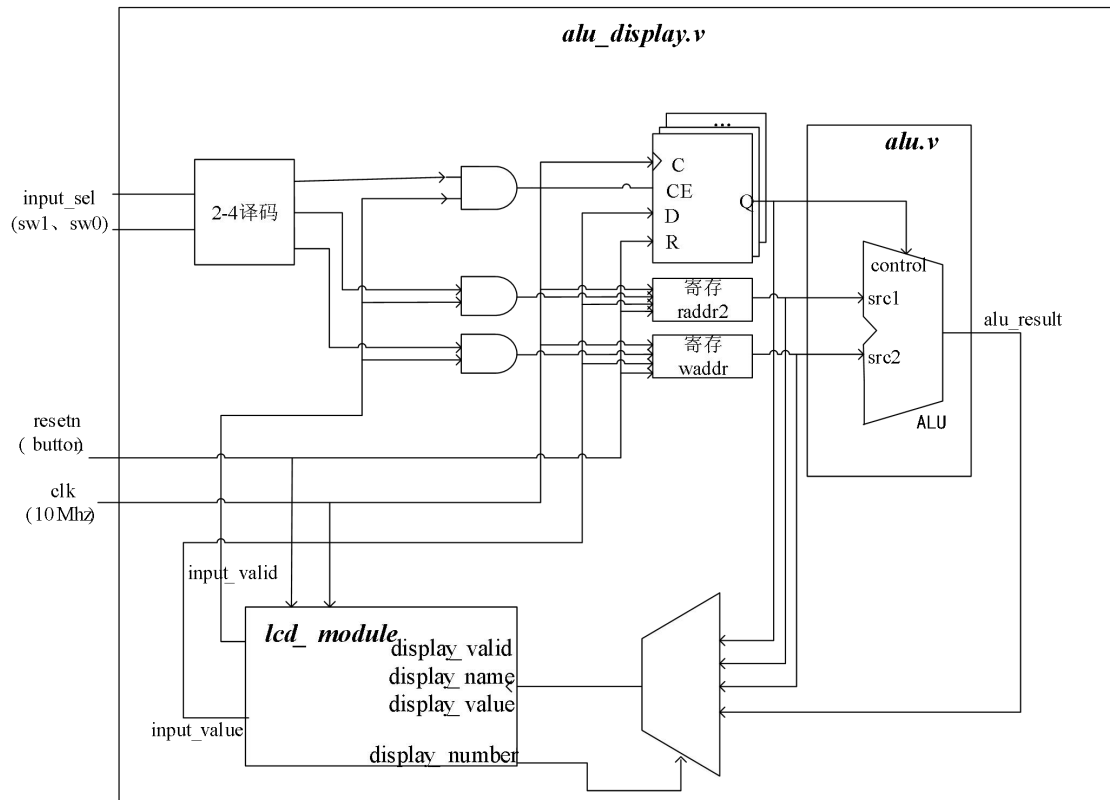


图 5.6 ALU 参考设计的顶层模块框图

5.3 源代码

alu.v 源代码:

```
1 `timescale 1ns / 1ps
2 /*******
3 // > 文件名: alu.v
4 // > 描述 : ALU 模块, 可做 12 种操作
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 /*******
8 module alu(
9     input  [11:0] alu_control, // ALU 控制信号
10    input  [31:0] alu_src1,    // ALU 操作数 1, 为补码
11    input  [31:0] alu_src2,    // ALU 操作数 2, 为补码
12    output [31:0] alu_result   // ALU 结果
13);
14
15    // ALU 控制信号, 独热码
16    wire alu_add; //加法操作
17    wire alu_sub; //减法操作
18    wire alu_slt; //有符号比较, 小于置位, 复用加法器做减法
19    wire alu_sltu; //无符号比较, 小于置位, 复用加法器做减法
20    wire alu_and; //按位与
21    wire alu_nor; //按位或非
22    wire alu_or;  //按位或
23    wire alu_xor; //按位异或
```

```
24    wire alu_sll;    //逻辑左移
25    wire alu_srl;    //逻辑右移
26    wire alu_sra;    //算术右移
27    wire alu_lui;    //高位加载
28
29    assign alu_add  = alu_control[11];
30    assign alu_sub  = alu_control[10];
31    assign alu_slt  = alu_control[ 9];
32    assign alu_sltu = alu_control[ 8];
33    assign alu_and  = alu_control[ 7];
34    assign alu_nor  = alu_control[ 6];
35    assign alu_or   = alu_control[ 5];
36    assign alu_xor  = alu_control[ 4];
37    assign alu_sll  = alu_control[ 3];
38    assign alu_srl  = alu_control[ 2];
39    assign alu_sra  = alu_control[ 1];
40    assign alu_lui  = alu_control[ 0];
41
42    wire [31:0] add_sub_result;
43    wire [31:0] slt_result;
44    wire [31:0] sltu_result;
45    wire [31:0] and_result;
46    wire [31:0] nor_result;
47    wire [31:0] or_result;
48    wire [31:0] xor_result;
49    wire [31:0] sll_result;
50    wire [31:0] srl_result;
51    wire [31:0] sra_result;
52    wire [31:0] lui_result;
53
54    assign and_result = alu_src1 & alu_src2;    //与结果为两数按位与
55    assign or_result  = alu_src1 | alu_src2;    //或结果为两数按位或
56    assign nor_result = ~or_result;            //或非结果为或结果取反
57    assign xor_result = alu_src1 ^ alu_src2;    //异或结果为两数按位异或
58    assign lui_result  = {alu_src2[15:0], 16'd0}; //src2 低 16 位装载至高 16 位
59
60    //-----{加法器}begin
61    //add, sub, slt, sltu 均使用该模块
62    wire [31:0] adder_operand1;
63    wire [31:0] adder_operand2;
64    wire          adder_cin      ;
65    wire [31:0] adder_result    ;
66    wire          adder_cout     ;
67    assign adder_operand1 = alu_src1;
68    assign adder_operand2 = alu_add ? alu_src2 : ~alu_src2;
69    assign adder_cin      = ~alu_add; //减法需要 cin
70    adder adder_module(
71        .operand1(adder_operand1),
72        .operand2(adder_operand2),
73        .cin      (adder_cin      ),
74        .result   (adder_result   ),
```

```

75     .cout      (adder_cout      )
76 );
77
78 //加减结果
79 assign add_sub_result = adder_result;
80
81 //slt 结果
82 //adder_src1[31] adder_src2[31] adder_result[31]
83 //      0      1      x(0 或 1)      "正-负", 显然小于不成立
84 //      0      0      1      相减为负, 说明小于
85 //      0      0      0      相减为正, 说明不小于
86 //      1      1      1      相减为负, 说明小于
87 //      1      1      0      相减为正, 说明不小于
88 //      1      0      x(0 或 1)      "负-正", 显然小于成立
89 assign slt_result[31:1] = 31'd0;
90 assign slt_result[0]=(alu_src1[31] & ~alu_src2[31]) |
91     (~(alu_src1[31]^alu_src2[31]) & adder_result[31]);
92
93 //sltu 结果
94 //对于 32 位无符号数比较, 最高位前填 0 作为符号位, 可转为
95 //33 位有符号数 ({1'b0,src1}和{1'b0,src2}) 的比较
96 //故, 33 位正数相减, 需要对{1'b0,src2}取反,
97 //即需要{1'b0,src1}+{1'b1,~src2}+cin
98 //但此处用的为 32 位加法器, 只做了运算: src1 + ~src2 + cin
99 //32 位加法的结果为{adder_cout,adder_result},
100 //则 33 位加法结果应该为{adder_cout+1'b1,adder_result}
101 //对比 slt 结果注释, 知道, 此时判断大小属于第二三种情况,
102 //即源操作数 1 符号位为 0, 源操作数 2 符号位为 0
103 //结果的符号位为 1, 说明小于, 即 adder_cout+1'b1 为 2'b01,
104 //即 adder_cout 为 0
105 assign sltu_result = {31'd0, ~adder_cout};
106 //-----{加法器}end
107
108 //-----{移位器}begin
109 // 移位分三步进行,
110 // 第一步根据移位量低 2 位即[1:0]位做第一次移位,
111 // 第二步在第一次移位基础上根据移位量[3:2]位做第二次移位,
112 // 第三步在第二次移位基础上根据移位量[4]位做第三次移位。
113 wire [4:0] shf;
114 assign shf = alu_src1[4:0];
115 wire [1:0] shf_1_0;
116 wire [1:0] shf_3_2;
117 assign shf_1_0 = shf[1:0];
118 assign shf_3_2 = shf[3:2];
119
120 // 逻辑左移
121 wire [31:0] sll_step1;
122 wire [31:0] sll_step2;
123 // 依据 shf[1:0],左移 0、1、2、3 位
124 assign sll_step1 = {32{shf_1_0 == 2'b00}} & alu_src2
125     | {32{shf_1_0 == 2'b01}} & {alu_src2[30:0], 1'd0}

```

```

126         | {32{shf_1_0 == 2'b10}} & {alu_src2[29:0], 2'd0}
127         | {32{shf_1_0 == 2'b11}} & {alu_src2[28:0], 3'd0};
128 // 依据 shf[3:2], 将第一次移位结果左移 0、4、8、12 位
129 assign sll_step2 = {32{shf_3_2 == 2'b00}} & sll_step1
130         | {32{shf_3_2 == 2'b01}} & {sll_step1[27:0], 4'd0}
131         | {32{shf_3_2 == 2'b10}} & {sll_step1[23:0], 8'd0}
132         | {32{shf_3_2 == 2'b11}} & {sll_step1[19:0], 12'd0};
133 // 依据 shf[4], 将第二次移位结果左移 0、16 位
134 assign sll_result = shf[4] ? {sll_step2[15:0], 16'd0} : sll_step2;
135
136 // 逻辑右移
137 wire [31:0] srl_step1;
138 wire [31:0] srl_step2;
139 // 依据 shf[1:0], 右移 0、1、2、3 位, 高位补 0
140 assign srl_step1 = {32{shf_1_0 == 2'b00}} & alu_src2
141         | {32{shf_1_0 == 2'b01}} & {1'd0, alu_src2[31:1]}
142         | {32{shf_1_0 == 2'b10}} & {2'd0, alu_src2[31:2]}
143         | {32{shf_1_0 == 2'b11}} & {3'd0, alu_src2[31:3]};
144 // 依据 shf[3:2], 将第一次移位结果右移 0、4、8、12 位, 高位补 0
145 assign srl_step2 = {32{shf_3_2 == 2'b00}} & srl_step1
146         | {32{shf_3_2 == 2'b01}} & {4'd0, srl_step1[31:4]}
147         | {32{shf_3_2 == 2'b10}} & {8'd0, srl_step1[31:8]}
148         | {32{shf_3_2 == 2'b11}} & {12'd0, srl_step1[31:12]};
149 // 依据 shf[4], 将第二次移位结果右移 0、16 位, 高位补 0
150 assign srl_result = shf[4] ? {16'd0, srl_step2[31:16]} : srl_step2;
151
152 // 算术右移
153 wire [31:0] sra_step1;
154 wire [31:0] sra_step2;
155 // 依据 shf[1:0], 右移 0、1、2、3 位, 高位补符号位
156 assign sra_step1 = {32{shf_1_0 == 2'b00}} & alu_src2
157 | {32{shf_1_0 == 2'b01}} & {alu_src2[31], alu_src2[31:1]}
158 | {32{shf_1_0 == 2'b10}} & {{2{alu_src2[31]}}, alu_src2[31:2]}
159 | {32{shf_1_0 == 2'b11}} & {{3{alu_src2[31]}}, alu_src2[31:3]};
160 // 依据 shf[3:2], 将第一次移位结果右移 0、4、8、12 位, 高位补符号位
161 assign sra_step2 = {32{shf_3_2 == 2'b00}} & sra_step1
162 | {32{shf_3_2 == 2'b01}} & {{4{sra_step1[31]}}, sra_step1[31:4]}
163 | {32{shf_3_2 == 2'b10}} & {{8{sra_step1[31]}}, sra_step1[31:8]}
164 | {32{shf_3_2 == 2'b11}} & {{12{sra_step1[31]}}, sra_step1[31:12]};
165 // 依据 shf[4], 将第二次移位结果右移 0、16 位, 高位补符号位
166 assign sra_result = shf[4] ? {{16{sra_step2[31]}}, sra_step2[31:16]} :
167         sra_step2;
168 //-----{移位器}end
169 // 选择相应结果输出
170 assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
171         alu_slt ? slt_result :
172         alu_sltu ? sltu_result :
173         alu_and ? and_result :
174         alu_nor ? nor_result :
175         alu_or ? or_result :
176         alu_xor ? xor_result :

```



```

177             alu_sll             ? sll_result :
178             alu_srl             ? srl_result :
179             alu_sra             ? sra_result :
180             alu_lui             ? lui_result :
181             32'd0;
182 endmodule

```

alu_display.v 源代码:

```

1 //*****
2 // > 文件名: alu_display.v
3 // > 描述 : ALU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
4 // > 作者 : LOONGSON
5 // > 日期 : 2016-04-14
6 //*****
7 module alu_display(
8     //时钟与复位信号
9     input clk,
10    input resetn,    //后缀"n"代表低电平有效
11
12    //拨码开关, 用于选择输入数
13    input [1:0] input_sel, //00:输入为控制信号(alu_control)
14                        //10:输入为源操作数 1(alu_src1)
15                        //11:输入为源操作数 2(alu_src2)
16
17    //触摸屏相关接口, 不需要更改
18    output lcd_rst,
19    output lcd_cs,
20    output lcd_rs,
21    output lcd_wr,
22    output lcd_rd,
23    inout[15:0] lcd_data_io,
24    output lcd_bl_ctr,
25    inout ct_int,
26    inout ct_sda,
27    output ct_scl,
28    output ct_rstn
29);
30 //-----{调用 ALU 模块}begin
31     reg [11:0] alu_control; // ALU 控制信号
32     reg [31:0] alu_src1;    // ALU 操作数 1
33     reg [31:0] alu_src2;    // ALU 操作数 2
34     wire [31:0] alu_result; // ALU 结果
35     alu alu_module(
36         .alu_control(alu_control),
37         .alu_src1 (alu_src1 ),
38         .alu_src2 (alu_src2 ),
39         .alu_result (alu_result )
40     );
41 //-----{调用 ALU 模块}end
42
43 //-----{调用触摸屏模块}begin-----//

```

```
44 //-----{实例化触摸屏}begin
45 //此小节不需要更改
46     reg          display_valid;
47     reg [39:0] display_name;
48     reg [31:0] display_value;
49     wire [5 :0] display_number;
50     wire          input_valid;
51     wire [31:0] input_value;
52
53     lcd_module lcd_module(
54         .clk          (clk          ),    //10Mhz
55         .resetn        (resetn        ),
56
57         //调用触摸屏的接口
58         .display_valid (display_valid ),
59         .display_name  (display_name  ),
60         .display_value (display_value ),
61         .display_number (display_number),
62         .input_valid   (input_valid   ),
63         .input_value   (input_value   ),
64
65         //lcd 触摸屏相关接口，不需要更改
66         .lcd_rst        (lcd_rst        ),
67         .lcd_cs          (lcd_cs          ),
68         .lcd_rs          (lcd_rs          ),
69         .lcd_wr          (lcd_wr          ),
70         .lcd_rd          (lcd_rd          ),
71         .lcd_data_io     (lcd_data_io     ),
72         .lcd_bl_ctr      (lcd_bl_ctr      ),
73         .ct_int          (ct_int          ),
74         .ct_sda          (ct_sda          ),
75         .ct_scl          (ct_scl          ),
76         .ct_rstn         (ct_rstn         )
77     );
78 //-----{实例化触摸屏}end
79
80 //-----{从触摸屏获取输入}begin
81 //根据实际需要输入的数修改此小节，
82 //建议对每一个数的输入，编写单独一个 always 块
83 //当 input_sel 为 00 时，表示输入数控制信号，即 alu_control
84     always @(posedge clk)
85     begin
86         if (!resetn)
87             begin
88                 alu_control <= 12'd0;
89             end
90         else if (input_valid && input_sel==2'b00)
91             begin
92                 alu_control <= input_value[11:0];
93             end
94     end
```

```
95
96    //当 input_sel 为 10 时，表示输入数为源操作数 1，即 alu_src1
97    always @(posedge clk)
98    begin
99        if (!resetn)
100        begin
101            alu_src1 <= 32'd0;
102        end
103        else if (input_valid && input_sel==2'b10)
104        begin
105            alu_src1 <= input_value;
106        end
107    end
108
109    //当 input_sel 为 11 时，表示输入数为源操作数 2，即 alu_src2
110    always @(posedge clk)
111    begin
112        if (!resetn)
113        begin
114            alu_src2 <= 32'd0;
115        end
116        else if (input_valid && input_sel==2'b11)
117        begin
118            alu_src2 <= input_value;
119        end
120    end
121    //-----{从触摸屏获取输入}end
122
123    //-----{输出到触摸屏显示}begin
124    //根据需要显示的数修改此小节，
125    //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
126    //44 块显示区域从 1 开始编号，编号为 1~44，
127    always @(posedge clk)
128    begin
129        case(display_number)
130            6'd1 :
131            begin
132                display_valid <= 1'b1;
133                display_name  <= "SRC_1";
134                display_value <= alu_src1;
135            end
136            6'd2 :
137            begin
138                display_valid <= 1'b1;
139                display_name  <= "SRC_2";
140                display_value <= alu_src2;
141            end
142            6'd3 :
143            begin
144                display_valid <= 1'b1;
145                display_name  <= "CONTR";
```

```

146         display_value <= {20'd0, alu_control};
147     end
148     6'd4 :
149     begin
150         display_valid <= 1'b1;
151         display_name <= "RESUL";
152         display_value <= alu_result;
153     end
154     default :
155     begin
156         display_valid <= 1'b0;
157         display_name <= 40'd0;
158         display_value <= 32'd0;
159     end
160 endcase
161 end
162 //-----{输出到触摸屏显示} end
163 //-----{调用触摸屏模块} end-----//
164 endmodule

```

约束文件 alu.xdc 源代码:

```

1  #时钟信号连接
2  set_property PACKAGE_PIN AC19 [get_ports clk]
3
4  #脉冲开关，用于输入作为复位信号，低电平有效
5  set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7  #拨码开关连接，用于输入，依次为 sw0,sw1
8  set_property PACKAGE_PIN Y6 [get_ports input_sel[0]]
9  set_property PACKAGE_PIN AD24 [get_ports input_sel[1]]
10
11 set_property IOSTANDARD LVCMOS33 [get_ports clk]
12 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
13 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]
14 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]
15
16 #触摸屏引脚连接
17 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
18 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
19 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
20 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
21 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
22 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
23 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
24 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
25 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
26 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
27 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
28 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
29 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]

```

```
30 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
31 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
32 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
33 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
34 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
35 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
36 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
37 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
38 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
39 set_property PACKAGE_PIN L19 [get_ports ct_int]
40 set_property PACKAGE_PIN J24 [get_ports ct_sda]
41 set_property PACKAGE_PIN H21 [get_ports ct_scl]
42 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
43
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
48 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
49 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
68 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
69 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

实验五 存储器实现

1 实验目的

1. 了解只读存储器 ROM 和随机存取存储器 RAM 的原理。
2. 理解 ROM 读取数据及 RAM 读取、写入数据的过程。
3. 理解计算机中存储器地址编址和数据索引方法。
4. 理解同步 RAM 和异步 RAM 的区别。
5. 掌握调用 xilinx 库 IP 实例化 RAM 的设计方法。
6. 熟悉并运用 verilog 语言进行电路设计。
7. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习存储器的设计及原理，如：ROM 读地址索引读取数据过程及时序，RAM 读写时序，同步和异步的区别等。
2. 学习计算机中内存地址编址和数据索引方法。
3. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，确定存储器宽度、深度和写使能位数。
4. 学习 ISE 工具中调用库 IP 的方法。
5. 本次实验要求调用 xilinx 库 IP 实例化一块 RAM。实例化的 RAM 选择为同步 RAM。本次实验的 RAM 建议设置为两个端口，一个端口用来正常的读写，另一个端口作为调试端口只使用读功能用于观察存储器内部数据。
6. 调用 xilinx 库 IP 实例化一块 RAM，并进行仿真，得到正确的波形图。
7. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 6.1。外围模块中需调用封装好的 LCD 触摸屏模块，显示 RAM 的正常端口的地址、待写入的数据和读出的数据，显示调试端口的地址和读出的数据。并且需要利用触摸功能输入正常端口的地址和写数据，以及调试端口的地址。

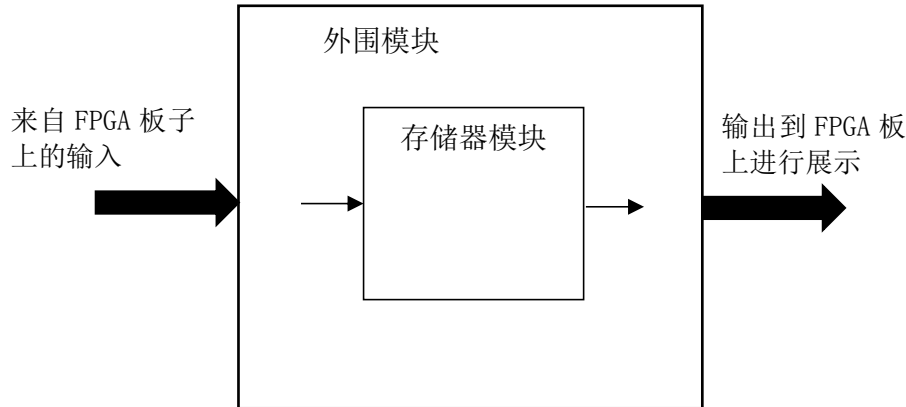


图 6.1 存储器设计实验的顶层模块大致框图

8. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：存储器深度不要过大，避免耗费过多的 FPGA 上的资源。本次实验要求实现同步的存储器。而异步存储器的搭建方法同寄存器堆的搭建，但不同的是，寄存器堆中读写端口是分开的，但对于异步 RAM 要求读写共用一个端口，只是会增加一个写使能信号。可以自行尝试搭建异步的 ROM 和 RAM，在单周期 CPU 实验中会用到异步的 ROM 作为指令存储器，而异步 RAM 作为数据存储器。

4 实验要求

1. 做好预习：

- 1) 掌握存储器的工作原理，明白 ROM 和 RAM，同步和异步的区别；
- 2) 学习并掌握调用 xilinx 库 IP 进行设计的方法；
- 3) 确定存储器的输入输出端口及宽度、深度和写使能设计；
- 4) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 6.1。

2. 实验实施：

- 1) 确认存储器的设计方案的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用存储器模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，按照检查人员的要求，对特定存储器单元读/写，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

本部分给出了调用 xilinx 库 IP 实例化 RAM 和 ROM 的方法。

5.1 生成 IP 核 RAM

1) 新建工程

新建一个工程 data_ram:

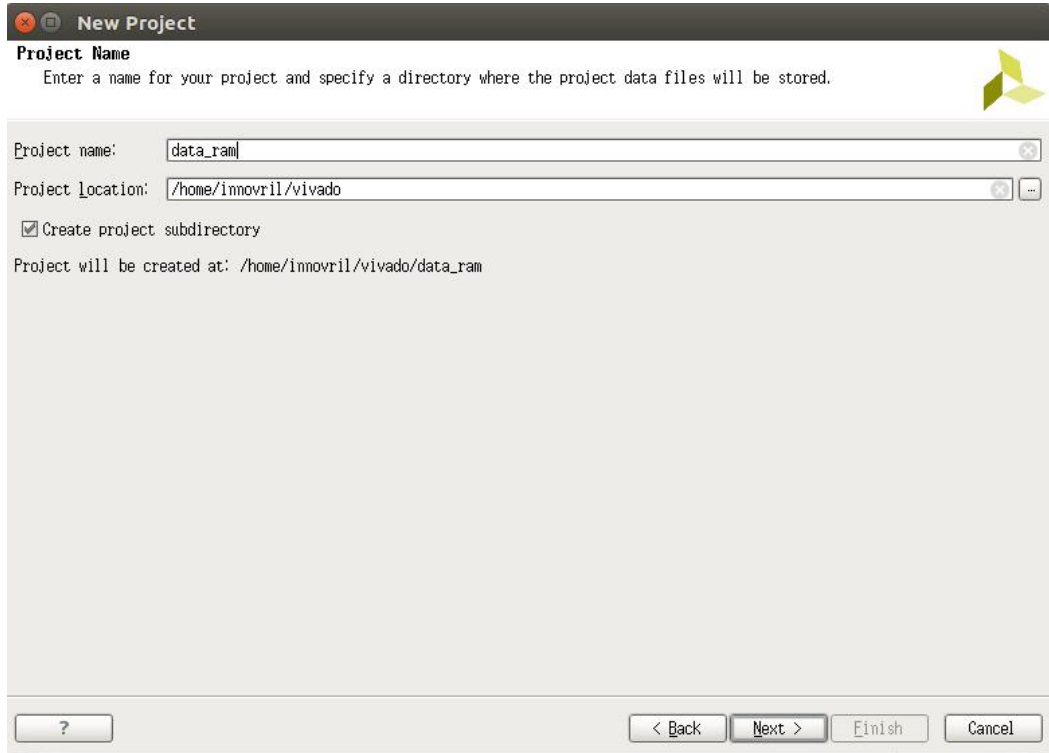


图 6.2 新建工程 data_ram

2) 新建 IP

点击“IP Catalog”：

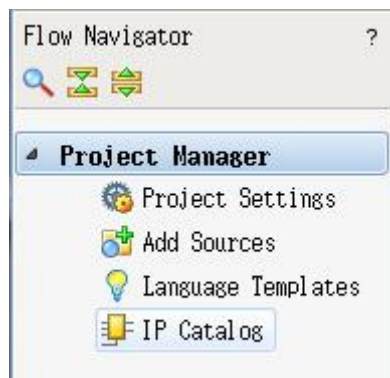


图 6.3 打开 IP 目录

在右侧列表中双击选择“Memories and Storage Elements”->“RAMs & ROMs & BRAM”中的“Block Memory Generator”：

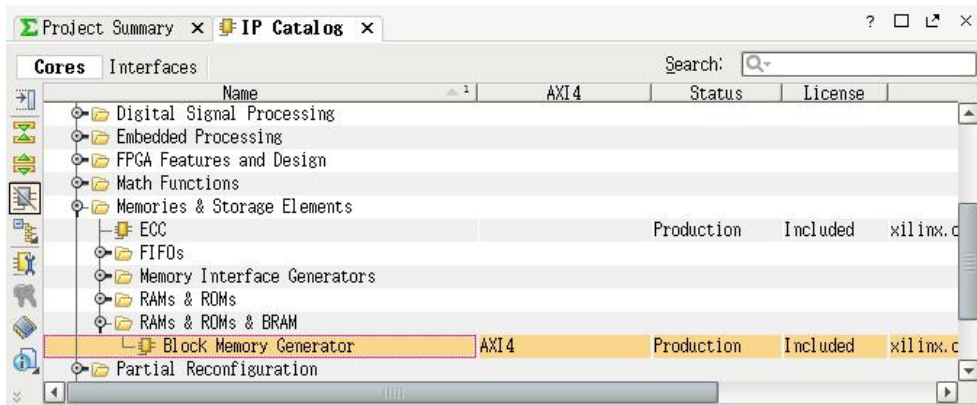


图 6.4 选择 IP 类型

3) 设置 RAM 参数

完成第二步后，会出现如下界面，需要依次选择 Memory 的参数：

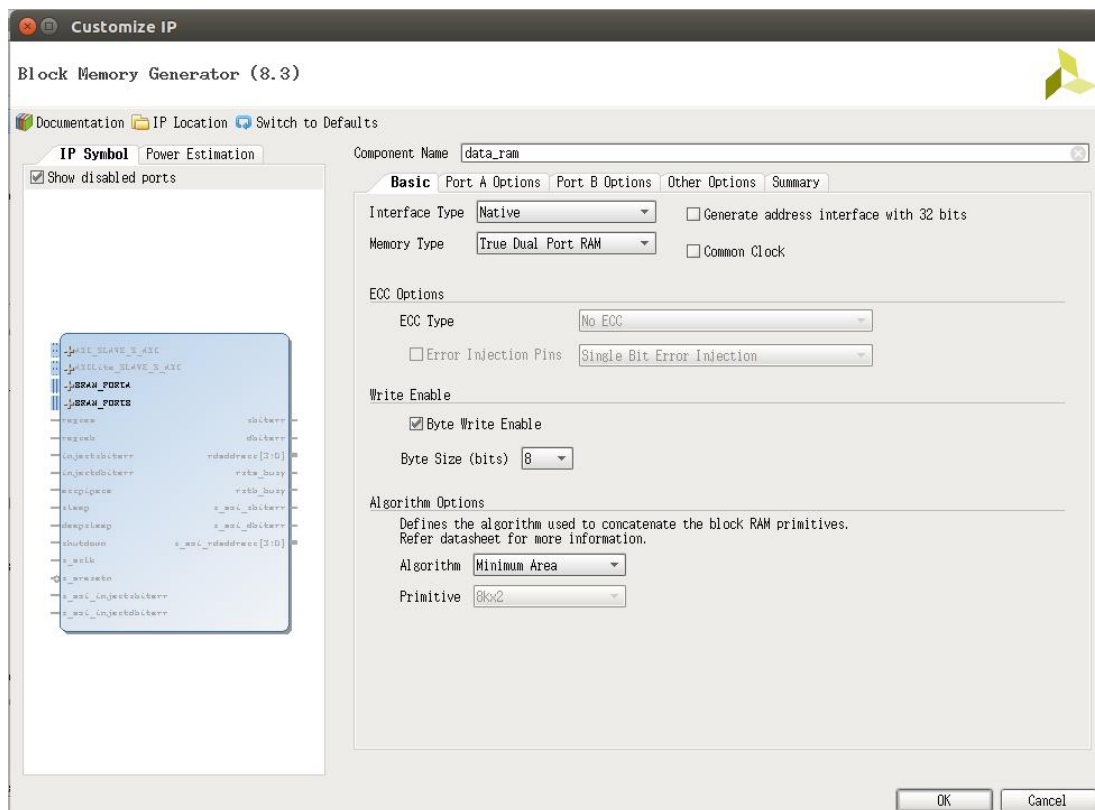


图 6.5 Memory 设定参数界面

在图 6.5 中，Component Name 输入“data_ram”，选择 Memory 类型为“True Dual Port RAM”，一个端口作为正常的读写端口，一个端口作为调试端口。勾选“Write Enable”下的“Byte Write Enable”，“Byte Size”选 8bits，因为后续 CPU 实验中存在写一个字节 store 指令，故需要数据 RAM 为字节写使能。“Basic”部分设置完成后击“Port A options”选项卡：

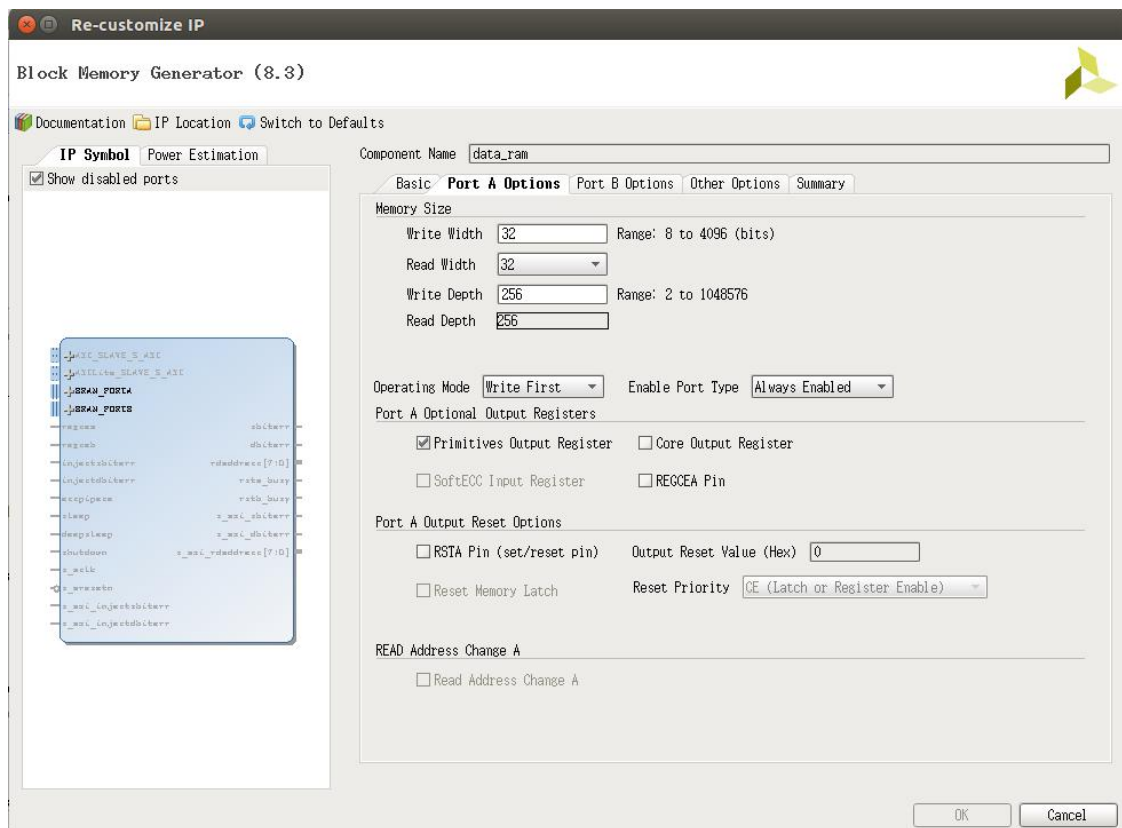


图 6.6 选择 Memory 类型

在图 6.6 中，RAM 宽度设置为 32 位，深度为 256，因为后续 CPU 实验是基于 32 位数据运算的。“Enalbe Port Type”选择“Always Enabled”。在“Port B options”选项卡下进行同样的设置。

对于一般的 RAM 生成，后续的步骤都不需要了，故此处直接点击“OK”，在弹出的窗口中点击“Generate”生成 IP 核即可。

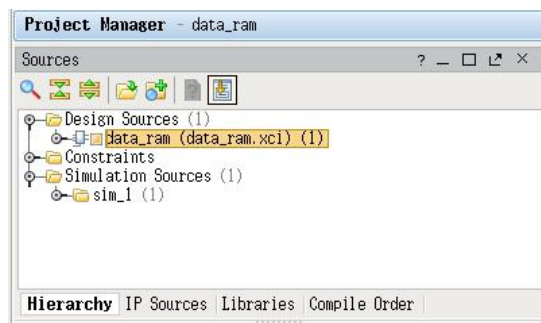


图 6.7 生成 RAM 成功

生成成功的工程管理界面如图 6.7，.xci 文件为存放 IP 核配置信息的文件，以后需要更改直接双击该文件即可重新配置 IP 核。

5.2 生成 IP 核 ROM

ROM 为只读存储器，需要初始化内部数据，可作为指令存储器。生成 ROM 的方法同 RAM 类似，首先新建一个 IP 核，取名为 inst_rom, 后续步骤同 5.1 所述，但在图 6.5 那步时，Memory 类型需要选择为“Single Port ROM”，如下图：

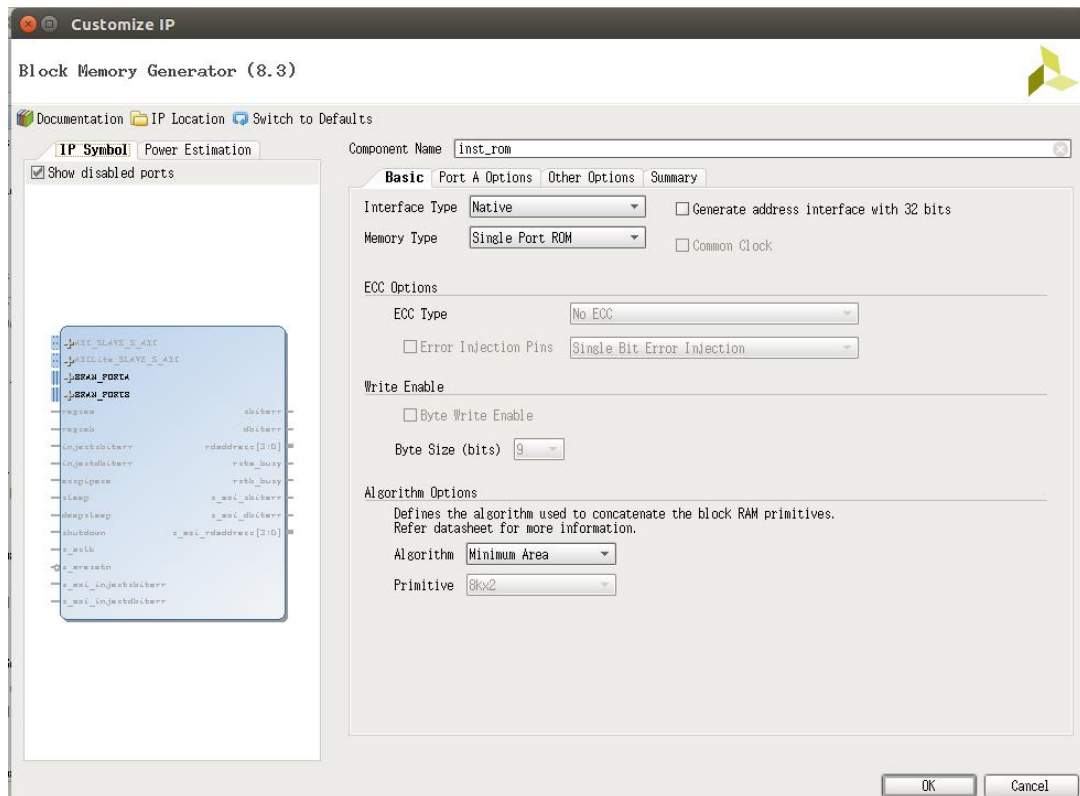


图 6.8 生成 ROM 时选择 Memory 类型

由于 ROM 为只读的，故我们不需要增加调试端口去观察内部数据的变化，故此处选择单端口即可。

点击“Port A Options”设置宽度和深度，“Enalbe Port Type”选择“Always Enabled”，如下图：

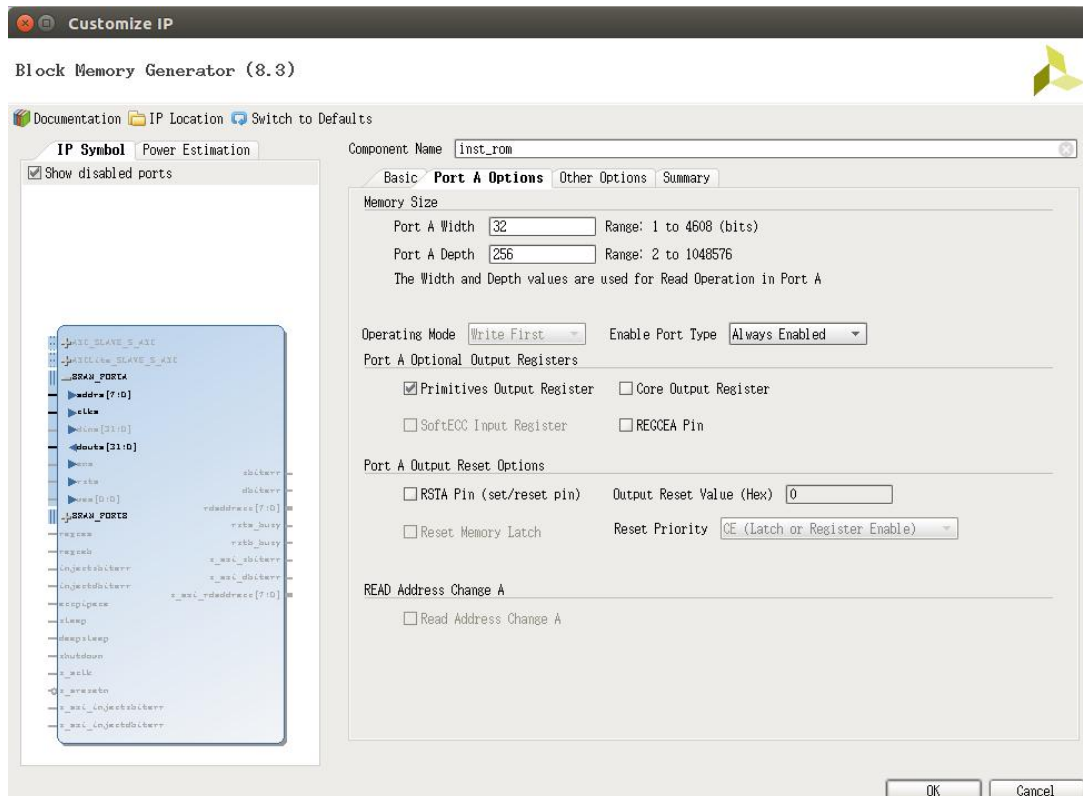


图 6.9 生成 ROM 时宽度和深度

宽度需要设置为 32 位，因为一条指令占用 32 位，深度可以依据自己要执行的指令数设定，此处先设定为 256。

点击“Other Options”选项卡：

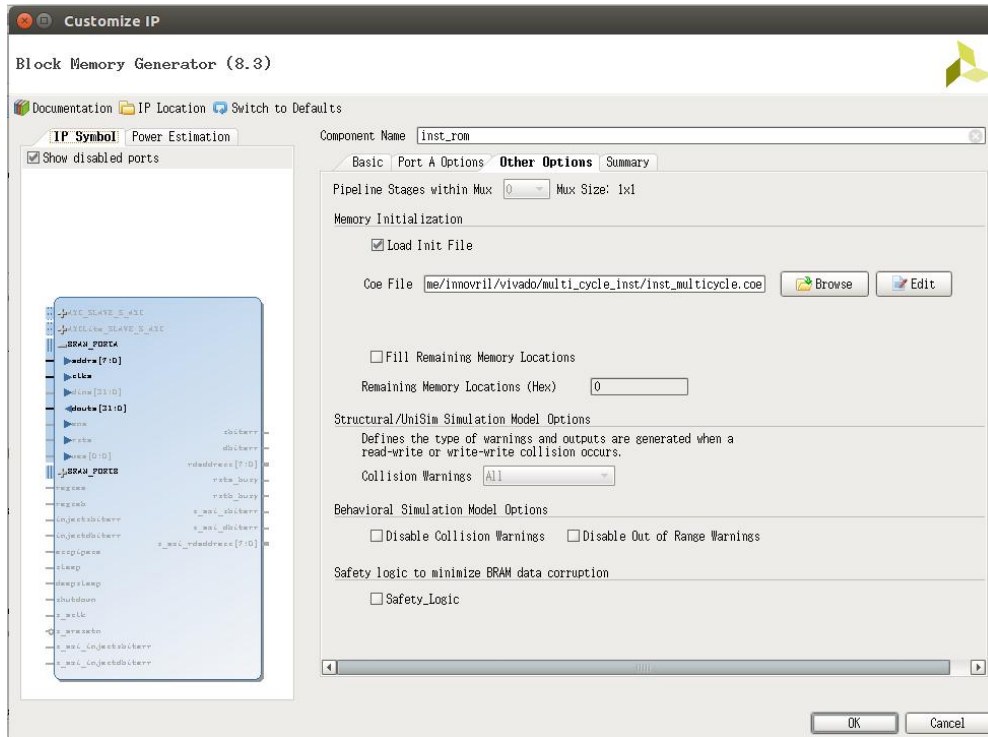


图 6.10 给 ROM 装载初始数据

在图 6.11 中，需要勾选“Load Init File”，并选中需要装载的初始化文件(.coe 文件)。.coe 文件为 Vivado 中 ROM 初始化文件，其格式如下：

```
1 memory_initialization_radix = 16;
2 memory_initialization_vector =
3 24010001
4 00011100
5 .....
```

第一行指定了初始化数据格式，此处为 16 进制，也可以设置为 2 进制。第二行说明从第三行开始为初始化的数据向量，由于 ROM 宽度为 32 位，故一个初始化向量为 32 位数据。初始化向量之间必须用空格或换行符隔开，此处使用换行符，故一行为一个初始化向量。初始化数据会从 ROM 中的 0 地址处开始依次填充。当初始化数据格式设置为 2 进制时，后续的初始化向量需要用二进制编写。

至此，生成 ROM 的所有参数都已设置完成，点击“OK”，“Generate”生成 ROM 即可。

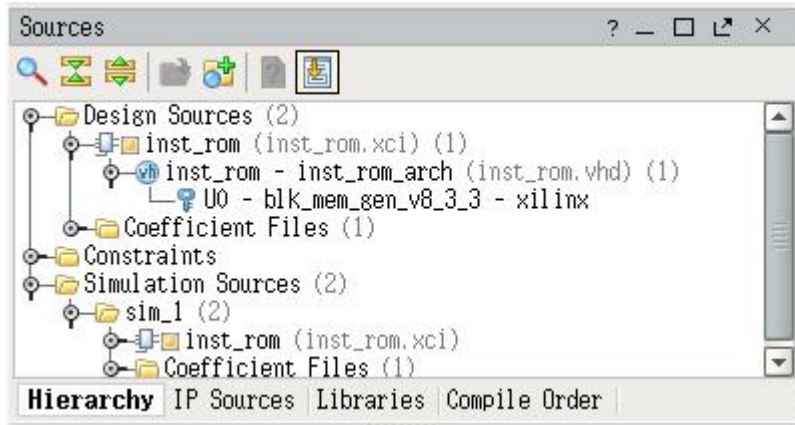


图 6.12 生成 ROM 成功

从图 6.12 中可以看到，生成结果和 RAM 的类似，以后需要更改参数设置或初始化文件时，双击中的 xci 文件即可。

5.3 同步 RAM 的顶层展示模块

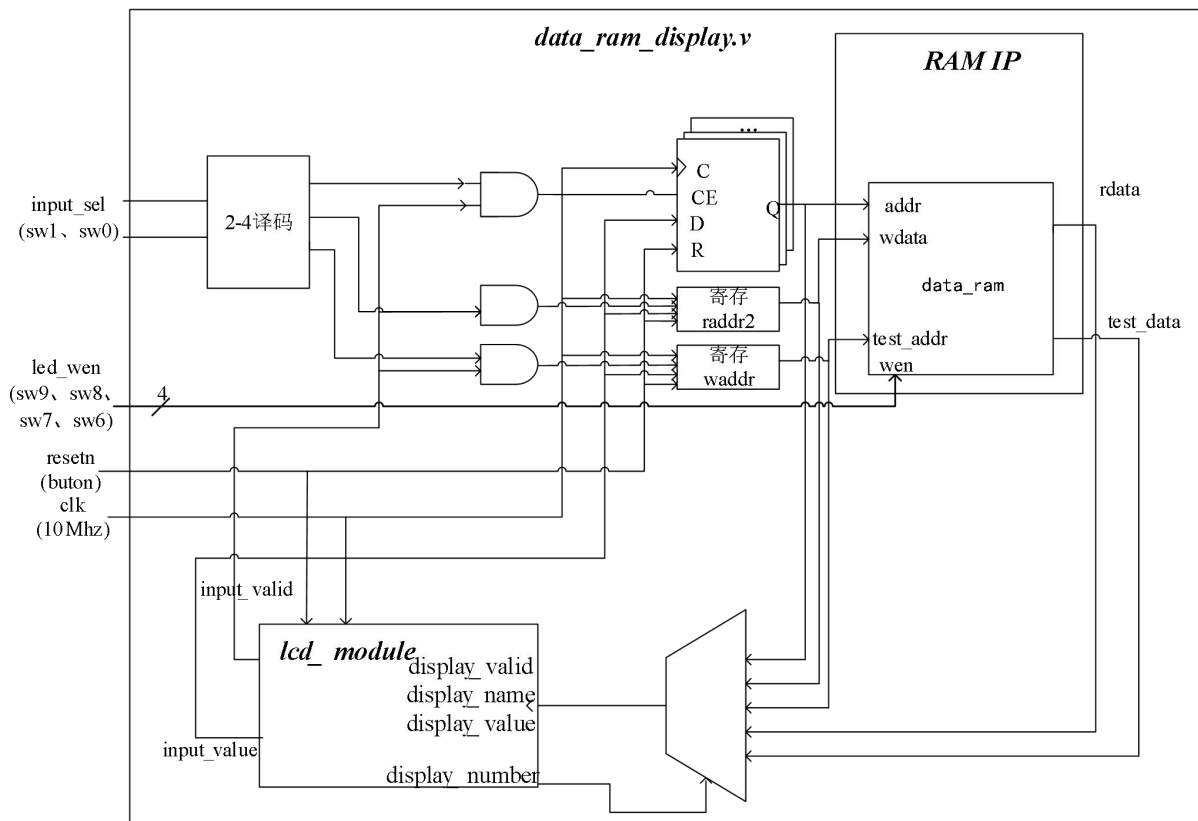


图 6.13 data_ram 参考设计的顶层模块框图

data_ram_display.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: data_ram_display.v
4  //    > 描述   : 数据存储器模块显示模块，调用 FPGA 板上的 IO 接口和触摸屏
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
```



```

    **
8  module data_ram_display(
9      //时钟与复位信号
10     input clk,
11     input resetn,    //后缀"n"代表低电平有效
12
13     //拨码开关，用于产生写使能和选择输入数
14     input [3:0] wen,
15     input [1:0] input_sel,
16
17     //led 灯，用于指示写使能信号，和正在输入什么数据
18     output [3:0] led_wen,
19     output led_addr,    //指示输入读写地址
20     output led_wdata,    //指示输入写数据
21     output led_test_addr, //指示输入 test 地址
22
23     //触摸屏相关接口，不需要更改
24     output lcd_rst,
25     output lcd_cs,
26     output lcd_rs,
27     output lcd_wr,
28     output lcd_rd,
29     inout[15:0] lcd_data_io,
30     output lcd_bl_ctr,
31     inout ct_int,
32     inout ct_sda,
33     output ct_scl,
34     output ct_rstn
35 );
36 //-----{LED 显示}begin
37     assign led_wen      = wen;
38     assign led_addr     = (input_sel==2'd0);
39     assign led_wdata    = (input_sel==2'd1);
40     assign led_test_addr = (input_sel==2'd2);
41 //-----{LED 显示}end
42 //-----{调用数据储存器模块}begin
43     //数据存储器多增加一个读端口，用于读出特定内存地址显示在触摸屏上
44     reg  [31:0] addr;
45     reg  [31:0] wdata;
46     wire [31:0] rdata;
47     reg  [31:0] test_addr;
48     wire [31:0] test_data;
49
50     data_ram data_ram_module(
51         .clka  (clk          ),
52         .wea   (wen          ),
53         .addra (addr[9:2]    ),
54         .dina  (wdata        ),
55         .douta (rdata        ),
56         .clkb  (clk          ),
57         .web   (4'd0         ),

```



```

58         .addrb (test_addr[9:2]),
59         .doutb (test_data      ),
60         .dinb  (32'd0          )
61     );
62 //-----{调用寄存器堆模块}end
63
64 //-----{调用触摸屏模块}begin-----//
65 //-----{实例化触摸屏}begin
66 //此小节不需要更改
67     reg          display_valid;
68     reg  [39:0] display_name;
69     reg  [31:0] display_value;
70     wire [5 :0] display_number;
71     wire          input_valid;
72     wire [31:0] input_value;
73
74     lcd_module lcd_module(
75         .clk          (clk          ),    //10Mhz
76         .resetn       (resetn       ),
77
78         //调用触摸屏的接口
79         .display_valid (display_valid ),
80         .display_name  (display_name  ),
81         .display_value (display_value ),
82         .display_number (display_number),
83         .input_valid   (input_valid   ),
84         .input_value   (input_value   ),
85
86         //lcd 触摸屏相关接口，不需要更改
87         .lcd_rst       (lcd_rst       ),
88         .lcd_cs        (lcd_cs        ),
89         .lcd_rs        (lcd_rs        ),
90         .lcd_wr        (lcd_wr        ),
91         .lcd_rd        (lcd_rd        ),
92         .lcd_data_io   (lcd_data_io   ),
93         .lcd_bl_ctr     (lcd_bl_ctr   ),
94         .ct_int        (ct_int        ),
95         .ct_sda        (ct_sda        ),
96         .ct_scl        (ct_scl        ),
97         .ct_rstn       (ct_rstn       )
98     );
99 //-----{实例化触摸屏}end
100
101 //-----{从触摸屏获取输入}begin
102 //根据实际需要输入的数修改此小节，
103 //建议对每一个数的输入，编写单独一个 always 块
104     //当 input_sel 为 2'b00 时，表示输入数为读写地址，即 addr
105     always @(posedge clk)
106     begin
107         if (!resetn)
108             begin

```

```
109         addr <= 32'd0;
110     end
111     else if (input_valid && input_sel==2'd0)
112     begin
113         addr[31:2] <= input_value[31:2];
114     end
115 end
116
117 //当 input_sel 为 2'b01 时, 表示输入数为写数据, 即 wdata
118 always @(posedge clk)
119 begin
120     if (!resetsn)
121     begin
122         wdata <= 32'd0;
123     end
124     else if (input_valid && input_sel==2'd1)
125     begin
126         wdata <= input_value;
127     end
128 end
129
130 //当 input_sel 为 2'b10 时, 表示输入数为 test 地址, 即 test_addr
131 always @(posedge clk)
132 begin
133     if (!resetsn)
134     begin
135         test_addr <= 32'd0;
136     end
137     else if (input_valid && input_sel==2'd2)
138     begin
139         test_addr[31:2] <= input_value[31:2];
140     end
141 end
142 //-----{从触摸屏获取输入}end
143
144 //-----{输出到触摸屏显示}begin
145 //根据需要显示的数修改此小节,
146 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
147 //44 块显示区域从 1 开始编号, 编号为 1~44,
148 always @(posedge clk)
149 begin
150     case(display_number)
151         6'd1:
152         begin
153             display_valid <= 1'b1;
154             display_name <= "ADDR ";
155             display_value <= addr;
156         end
157         6'd2:
158         begin
159             display_valid <= 1'b1;
```

```
160             display_name  <= "WDATA";
161             display_value <= wdata;
162         end
163         6'd3:
164         begin
165             display_valid <= 1'b1;
166             display_name  <= "RDATA";
167             display_value <= rdata;
168         end
169         6'd5:
170         begin
171             display_valid <= 1'b1;
172             display_name  <= "T_ADD";
173             display_value <= test_addr;
174         end
175         6'd6:
176         begin
177             display_valid <= 1'b1;
178             display_name  <= "T_DAT";
179             display_value <= test_data;
180         end
181         default :
182         begin
183             display_valid <= 1'b0;
184             display_name  <= 40'd0;
185             display_value <= 32'd0;
186         end
187     endcase
188 end
189 //-----{输出到触摸屏显示}end
190 //-----{调用触摸屏模块}end-----//
191 endmodule
```

约束文件 data_ram.ucf 源代码:

```
1  #时钟信号连接
2  set_property PACKAGE_PIN AC19 [get_ports clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports clk]
4
5  #脉冲开关，用于输入作为复位信号，低电平有效
6  set_property PACKAGE_PIN Y3 [get_ports resetn]
7  set_property IOSTANDARD LVCMOS33 [get_ports resetn]
8
9  #led 灯连接，用于输出
10 set_property PACKAGE_PIN H7 [get_ports led_wen[3]]
11 set_property PACKAGE_PIN D5 [get_ports led_wen[1]]
12 set_property PACKAGE_PIN A3 [get_ports led_wen[0]]
13 set_property PACKAGE_PIN A5 [get_ports led_addr]
14 set_property PACKAGE_PIN A4 [get_ports led_wdata]
15 set_property PACKAGE_PIN F7 [get_ports led_test_addr]
16 set_property IOSTANDARD LVCMOS33 [get_ports led_wen[3]]
17 set_property IOSTANDARD LVCMOS33 [get_ports led_wen[2]]
```

```
18 set_property IOSTANDARD LVCMOS33 [get_ports led_wen[1]]
19 set_property IOSTANDARD LVCMOS33 [get_ports led_wen[0]]
20 set_property IOSTANDARD LVCMOS33 [get_ports led_addr]
21 set_property IOSTANDARD LVCMOS33 [get_ports led_wdata]
22 set_property IOSTANDARD LVCMOS33 [get_ports led_test_addr]
23
24 #拨码开关连接，用于输入
25 set_property PACKAGE_PIN AC21 [get_ports input_sel[1]]
26 set_property PACKAGE_PIN AD24 [get_ports input_sel[0]]
27 set_property PACKAGE_PIN AC22 [get_ports wen[3]]
28 set_property PACKAGE_PIN AC23 [get_ports wen[2]]
29 set_property PACKAGE_PIN AB6 [get_ports wen[1]]
30 set_property PACKAGE_PIN W6 [get_ports wen[0]]
31 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]
32 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]
33 set_property IOSTANDARD LVCMOS33 [get_ports wen[3]]
34 set_property IOSTANDARD LVCMOS33 [get_ports wen[2]]
35 set_property IOSTANDARD LVCMOS33 [get_ports wen[1]]
36 set_property IOSTANDARD LVCMOS33 [get_ports wen[0]]
37
38 #触摸屏引脚连接
39 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
40 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
41 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
42 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
43 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
44 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
45 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
46 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
47 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
48 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
49 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
50 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
51 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
52 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
53 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
54 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
55 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
56 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
57 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
58 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
59 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
60 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
61 set_property PACKAGE_PIN L19 [get_ports ct_int]
62 set_property PACKAGE_PIN J24 [get_ports ct_sda]
63 set_property PACKAGE_PIN H21 [get_ports ct_scl]
64 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
65
```

```
66 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
67 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
68 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
69 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
70 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
71 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
72 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
73 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
74 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
75 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
76 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
78 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
79 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
80 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
81 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
82 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
83 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
84 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
85 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
86 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
87 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
88 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
89 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
90 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
91 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

5.4 同步 ROM 的顶层模块

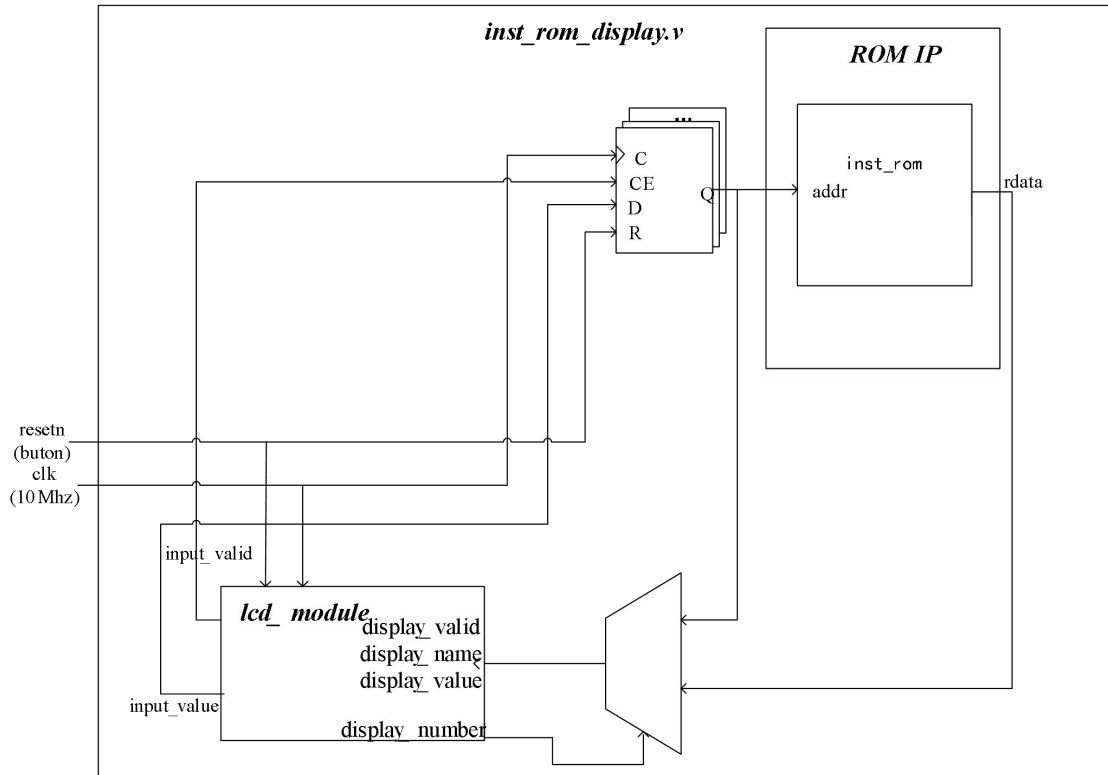


图 6.14 inst_rom 参考设计的顶层模块框图

inst_rom_display.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   *  > 文件名: inst_rom_display.v
4   *  > 描述   : 异步指令存储器显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5   *  > 作者   : LOONGSON
6   *  > 日期   : 2016-04-14
7   */
8  module inst_rom_display(
9      //时钟与复位信号
10     input clk,
11     input resetn,    //后缀"n"代表低电平有效
12
13     //触摸屏相关接口, 不需要更改
14     output lcd_rst,
15     output lcd_cs,
16     output lcd_rs,
17     output lcd_wr,
18     output lcd_rd,
19     inout[15:0] lcd_data_io,
20     output lcd_bl_ctr,
21     inout ct_int,
22     inout ct_sda,
23     output ct_scl,
24     output ct_rstn
25 );

```

```
26 //-----{调用数据储存器模块}begin
27 //数据存储器多增加一个读端口，用于读出特定内存地址显示在触摸屏上
28 reg [31:0] addr;
29 wire [31:0] inst;
30
31 inst_rom inst_rom_module(
32     .clka (clk          ),
33     .addra (addr[9:2]   ),
34     .douta (inst        )
35 );
36 //-----{调用寄存器堆模块}end
37
38 //-----{调用触摸屏模块}begin-----//
39 //-----{实例化触摸屏}begin
40 //此小节不需要更改
41 reg display_valid;
42 reg [39:0] display_name;
43 reg [31:0] display_value;
44 wire [5 :0] display_number;
45 wire input_valid;
46 wire [31:0] input_value;
47
48 lcd_module lcd_module(
49     .clk (clk          ), //10Mhz
50     .resetn (resetn    ),
51
52     //调用触摸屏的接口
53     .display_valid (display_valid ),
54     .display_name (display_name   ),
55     .display_value (display_value ),
56     .display_number (display_number),
57     .input_valid (input_valid     ),
58     .input_value (input_value     ),
59
60     //lcd 触摸屏相关接口，不需要更改
61     .lcd_rst (lcd_rst          ),
62     .lcd_cs (lcd_cs            ),
63     .lcd_rs (lcd_rs            ),
64     .lcd_wr (lcd_wr            ),
65     .lcd_rd (lcd_rd            ),
66     .lcd_data_io (lcd_data_io  ),
67     .lcd_bl_ctr (lcd_bl_ctr    ),
68     .ct_int (ct_int            ),
69     .ct_sda (ct_sda            ),
70     .ct_scl (ct_scl            ),
71     .ct_rstn (ct_rstn          )
72 );
73 //-----{实例化触摸屏}end
74
75 //-----{从触摸屏获取输入}begin
76 //根据实际需要输入的数修改此小节，
```



```
77 //建议对每一个数的输入，编写单独一个 always 块
78     always @(posedge clk)
79     begin
80         if (!resetn)
81         begin
82             addr <= 32'd0;
83         end
84         else if (input_valid)
85         begin
86             addr[31:2] <= input_value[31:2];
87         end
88     end
89 //-----{从触摸屏获取输入}end
90
91 //-----{输出到触摸屏显示}begin
92 //根据需要显示的数修改此小节，
93 //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
94 //44 块显示区域从 1 开始编号，编号为 1~44，
95     always @(posedge clk)
96     begin
97         case(display_number)
98             6'd1:
99             begin
100                 display_valid <= 1'b1;
101                 display_name  <= "ADDR ";
102                 display_value <= addr;
103             end
104             6'd2:
105             begin
106                 display_valid <= 1'b1;
107                 display_name  <= "INST ";
108                 display_value <= inst;
109             end
110             default :
111             begin
112                 display_valid <= 1'b0;
113                 display_name  <= 40'd0;
114                 display_value <= 32'd0;
115             end
116         endcase
117     end
118 //-----{输出到触摸屏显示}end
119 //-----{调用触摸屏模块}end-----//
120 endmodule
```

约束文件 inst_rom.ucf 源代码：

```
1  #时钟信号连接
2  set_property PACKAGE_PIN AC19 [get_ports clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports clk]
4
5  #脉冲开关，用于输入作为复位信号，低电平有效
```

```
6  set_property PACKAGE_PIN Y3 [get_ports resetn]
7  set_property IOSTANDARD LVCMOS33 [get_ports resetn]
8
9  #触摸屏引脚连接
10 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
11 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
12 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
13 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
14 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
15 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
16 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
17 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
18 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
19 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
20 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
21 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
22 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
23 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
24 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
25 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
26 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
27 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
28 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
29 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
30 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
31 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
32 set_property PACKAGE_PIN L19 [get_ports ct_int]
33 set_property PACKAGE_PIN J24 [get_ports ct_sda]
34 set_property PACKAGE_PIN H21 [get_ports ct_scl]
35 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
36
37 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
38 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
39 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
40 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
41 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
43 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
45 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
46 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
```

```

54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
60 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
61 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
62 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]

```

5.5 异步 RAM 实现

本部分给出自行搭建的异步 RAM 源代码。在单周期 CPU 中需要用该模块作为数据存储单元。

data_ram.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: data_mem.v
4  //  > 描述   : 异步数据存储器模块, 采用寄存器搭建而成, 类似寄存器堆
5  //  >          同步写, 异步读
6  //  > 作者   : LOONGSON
7  //  > 日期   : 2016-04-14
8  //*****
9  module data_ram(
10     input      clk,          // 时钟
11     input  [3:0] wen,         // 字节写使能
12     input  [4:0] addr,        // 地址
13     input  [31:0] wdata,      // 写数据
14     output reg [31:0] rdata,   // 读数据
15
16     //调试端口, 用于读出数据显示
17     input  [4 :0] test_addr,
18     output reg [31:0] test_data
19 );
20     reg [31:0] DM[31:0]; //数据存储器, 字节地址 7'b000_0000~7'b111_1111
21
22     //写数据
23     always @(posedge clk)    // 当写控制信号为 1, 数据写入内存
24     begin
25         if (wen[3])
26         begin
27             DM[addr][31:24] <= wdata[31:24];
28         end
29     end
30     always @(posedge clk)
31     begin
32         if (wen[2])
33         begin

```

```
34         DM[addr][23:16] <= wdata[23:16];
35     end
36 end
37 always @(posedge clk)
38 begin
39     if (wen[1])
40     begin
41         DM[addr][15: 8] <= wdata[15: 8];
42     end
43 end
44 always @(posedge clk)
45 begin
46     if (wen[0])
47     begin
48         DM[addr][7 : 0] <= wdata[7 : 0];
49     end
50 end
51
52 //读数据,取4字节
53 always @(*)
54 begin
55     case (addr)
56         5'd0 : rdata <= DM[0 ];
57         5'd1 : rdata <= DM[1 ];
58         5'd2 : rdata <= DM[2 ];
59         5'd3 : rdata <= DM[3 ];
60         5'd4 : rdata <= DM[4 ];
61         5'd5 : rdata <= DM[5 ];
62         5'd6 : rdata <= DM[6 ];
63         5'd7 : rdata <= DM[7 ];
64         5'd8 : rdata <= DM[8 ];
65         5'd9 : rdata <= DM[9 ];
66         5'd10: rdata <= DM[10];
67         5'd11: rdata <= DM[11];
68         5'd12: rdata <= DM[12];
69         5'd13: rdata <= DM[13];
70         5'd14: rdata <= DM[14];
71         5'd15: rdata <= DM[15];
72         5'd16: rdata <= DM[16];
73         5'd17: rdata <= DM[17];
74         5'd18: rdata <= DM[18];
75         5'd19: rdata <= DM[19];
76         5'd20: rdata <= DM[20];
77         5'd21: rdata <= DM[21];
78         5'd22: rdata <= DM[22];
79         5'd23: rdata <= DM[23];
80         5'd24: rdata <= DM[24];
81         5'd25: rdata <= DM[25];
82         5'd26: rdata <= DM[26];
83         5'd27: rdata <= DM[27];
84         5'd28: rdata <= DM[28];
```

```
85             5'd29: rdata <= DM[29];
86             5'd30: rdata <= DM[30];
87             5'd31: rdata <= DM[31];
88         endcase
89     end
90     //调试端口，读出特定内存的数据
91     always @(*)
92     begin
93         case (test_addr)
94             5'd0 : test_data <= DM[0 ];
95             5'd1 : test_data <= DM[1 ];
96             5'd2 : test_data <= DM[2 ];
97             5'd3 : test_data <= DM[3 ];
98             5'd4 : test_data <= DM[4 ];
99             5'd5 : test_data <= DM[5 ];
100            5'd6 : test_data <= DM[6 ];
101            5'd7 : test_data <= DM[7 ];
102            5'd8 : test_data <= DM[8 ];
103            5'd9 : test_data <= DM[9 ];
104            5'd10: test_data <= DM[10];
105            5'd11: test_data <= DM[11];
106            5'd12: test_data <= DM[12];
107            5'd13: test_data <= DM[13];
108            5'd14: test_data <= DM[14];
109            5'd15: test_data <= DM[15];
110            5'd16: test_data <= DM[16];
111            5'd17: test_data <= DM[17];
112            5'd18: test_data <= DM[18];
113            5'd19: test_data <= DM[19];
114            5'd20: test_data <= DM[20];
115            5'd21: test_data <= DM[21];
116            5'd22: test_data <= DM[22];
117            5'd23: test_data <= DM[23];
118            5'd24: test_data <= DM[24];
119            5'd25: test_data <= DM[25];
120            5'd26: test_data <= DM[26];
121            5'd27: test_data <= DM[27];
122            5'd28: test_data <= DM[28];
123            5'd29: test_data <= DM[29];
124            5'd30: test_data <= DM[30];
125            5'd31: test_data <= DM[31];
126        endcase
127    end
128 endmodule
```

data_ram_display.v 源代码:

```
1    `timescale 1ns / 1ps
2    //*****
3    //    > 文件名: data_ram_display.v
4    //    > 描述   : 数据存储器模块显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5    //    > 作者   : LOONGSON
```

```
6 // > 日期 : 2016-04-14
7 //*****
8 module data_ram_display(
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //拨码开关, 用于产生写使能和选择输入数
14     input [3:0] wen,
15     input [1:0] input_sel,
16
17     //led 灯, 用于指示写使能信号, 和正在输入什么数据
18     output [3:0] led_wen,
19     output led_addr, //指示输入读写地址
20     output led_wdata, //指示输入写数据
21     output led_test_addr, //指示输入 test 地址
22
23     //触摸屏相关接口, 不需要更改
24     output lcd_rst,
25     output lcd_cs,
26     output lcd_rs,
27     output lcd_wr,
28     output lcd_rd,
29     inout[15:0] lcd_data_io,
30     output lcd_bl_ctr,
31     inout ct_int,
32     inout ct_sda,
33     output ct_scl,
34     output ct_rstn
35 );
36 //----{LED 显示}begin
37     assign led_wen = wen;
38     assign led_addr = (input_sel==2'd0);
39     assign led_wdata = (input_sel==2'd1);
40     assign led_test_addr = (input_sel==2'd2);
41 //----{LED 显示}end
42 //----{调用数据储存器模块}begin
43     //数据存储器多增加一个读端口, 用于读出特定内存地址显示在触摸屏上
44     reg [31:0] addr;
45     reg [31:0] wdata;
46     wire [31:0] rdata;
47     reg [31:0] test_addr;
48     wire [31:0] test_data;
49
50     data_ram data_ram_module(
51         .clk (clk ),
52         .wen (wen ),
53         .addr (addr[6:2]),
54         .wdata (wdata ),
55         .rdata (rdata ),
```

```

56         .test_addr(test_addr[6:2]),
57         .test_data(test_data)
58     );
59 //-----{调用寄存器堆模块}end
60
61 //-----{调用触摸屏模块}begin-----//
62 //-----{实例化触摸屏}begin
63 //此小节不需要更改
64     reg         display_valid;
65     reg [39:0] display_name;
66     reg [31:0] display_value;
67     wire [5 :0] display_number;
68     wire         input_valid;
69     wire [31:0] input_value;
70
71     lcd_module lcd_module(
72         .clk          (clk          ),    //10Mhz
73         .resetn       (resetn       ),
74
75         //调用触摸屏的接口
76         .display_valid (display_valid ),
77         .display_name  (display_name  ),
78         .display_value (display_value ),
79         .display_number (display_number),
80         .input_valid   (input_valid   ),
81         .input_value   (input_value   ),
82
83         //lcd 触摸屏相关接口，不需要更改
84         .lcd_rst       (lcd_rst       ),
85         .lcd_cs        (lcd_cs        ),
86         .lcd_rs        (lcd_rs        ),
87         .lcd_wr        (lcd_wr        ),
88         .lcd_rd        (lcd_rd        ),
89         .lcd_data_io   (lcd_data_io   ),
90         .lcd_bl_ctr    (lcd_bl_ctr    ),
91         .ct_int        (ct_int        ),
92         .ct_sda        (ct_sda        ),
93         .ct_scl        (ct_scl        ),
94         .ct_rstn       (ct_rstn       )
95     );
96 //-----{实例化触摸屏}end
97
98 //-----{从触摸屏获取输入}begin
99 //根据实际需要输入的数修改此小节，
100 //建议对每一个数的输入，编写单独一个 always 块
101     //当 input_sel 为 2'b00 时，表示输入数为读写地址，即 addr
102     always @(posedge clk)
103     begin
104         if (!resetn)
105         begin
106             addr <= 32'd0;

```



```
107         end
108         else if (input_valid && input_sel==2'd0)
109         begin
110             addr[31:2] <= input_value[31:2];
111         end
112     end
113
114     //当 input_sel 为 2'b01 时, 表示输入数为写数据, 即 wdata
115     always @(posedge clk)
116     begin
117         if (!resetn)
118         begin
119             wdata <= 32'd0;
120         end
121         else if (input_valid && input_sel==2'd1)
122         begin
123             wdata <= input_value;
124         end
125     end
126
127     //当 input_sel 为 2'b10 时, 表示输入数为 test 地址, 即 test_addr
128     always @(posedge clk)
129     begin
130         if (!resetn)
131         begin
132             test_addr <= 32'd0;
133         end
134         else if (input_valid && input_sel==2'd2)
135         begin
136             test_addr[31:2] <= input_value[31:2];
137         end
138     end
139     //-----{从触摸屏获取输入}end
140
141     //-----{输出到触摸屏显示}begin
142     //根据需要显示的数修改此小节,
143     //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
144     //44 块显示区域从 1 开始编号, 编号为 1~44,
145     always @(posedge clk)
146     begin
147         case(display_number)
148             6'd1:
149             begin
150                 display_valid <= 1'b1;
151                 display_name <= "ADDR ";
152                 display_value <= addr;
153             end
154             6'd2:
155             begin
156                 display_valid <= 1'b1;
157                 display_name <= "WDATA";
```

```

158             display_value <= wdata;
159         end
160         6'd3:
161         begin
162             display_valid <= 1'b1;
163             display_name <= "RDATA";
164             display_value <= rdata;
165         end
166         6'd5:
167         begin
168             display_valid <= 1'b1;
169             display_name <= "T_ADD";
170             display_value <= test_addr;
171         end
172         6'd6:
173         begin
174             display_valid <= 1'b1;
175             display_name <= "T_DAT";
176             display_value <= test_data;
177         end
178         default :
179         begin
180             display_valid <= 1'b0;
181             display_name <= 40'd0;
182             display_value <= 32'd0;
183         end
184     endcase
185 end
186 //-----{输出到触摸屏显示}end
187 //-----{调用触摸屏模块}end-----//
188 endmodule

```

异步 data_ram 在烧写到板上展示时需要的约束文件和同步 data_ram 的一样。

5.6 异步 ROM 实现

本部分给出自行搭建的异步 ROM 源代码。在单周期 CPU 中需要用该模块作为指令存储器。

inst_rom.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3  //   > 文件名: inst_rom.v
4  //   > 描述   : 异步指令存储器模块, 采用寄存器搭建而成, 类似寄存器堆
5  //   >         内嵌好指令, 只读, 异步读
6  //   > 作者   : LOONGSON
7  //   > 日期   : 2016-04-14
8  /**
9  module inst_rom(
10     input      [4 :0] addr, // 指令地址
11     output reg [31:0] inst   // 指令

```

```

12     );
13
14     wire [31:0] inst_rom[19:0]; // 指令存储器, 字节地址
15     7'b000_0000~7'b111_1111
16     //----- 指令编码 -----|指令地址|---- 汇编指令 ----|- 指令结果 -----//
17     assign inst_rom[ 0] = 32'h24010001;//00H: addiu $1,$0,#1 | $1 = 0000_0001H
18     assign inst_rom[ 1] = 32'h00011100;//04H: sll   $2,$1,#4 | $2 = 0000_0010H
19     assign inst_rom[ 2] = 32'h00411821;//08H: addu  $3,$2,$1 | $3 = 0000_0011H
20     assign inst_rom[ 3] = 32'h00022082;//0CH: srl   $4,$2,#2 | $4 = 0000_0004H
21     assign inst_rom[ 4] = 32'h00642823;//10H: subu  $5,$3,$4 | $5 = 0000_000DH
22     assign inst_rom[ 5] = 32'hAC250013;//14H: sw    $5,#19($1)| Mem[14H] = DH
23     assign inst_rom[ 6] = 32'h00A23027;//18H: nor   $6,$5,$2 | $6 = FFFF_FFE2H
24     assign inst_rom[ 7] = 32'h00C33825;//1CH: or    $7,$6,$3 | $7 = FFFF_FFF3H
25     assign inst_rom[ 8] = 32'h00E64026;//20H: xor   $8,$7,$6 | $8 = 0000_0011H
26     assign inst_rom[ 9] = 32'hAC08001C;//24H: sw    $8,#28($0)| Mem[1CH] = 11H
27     assign inst_rom[10] = 32'h00C7482A;//28H: slt   $9,$6,$7 | $9 = 0000_0001H
28     assign inst_rom[11] = 32'h11210002;//2CH: beq   $9,$1,#2 | 跳转到指令 34H
29     assign inst_rom[12] = 32'h24010004;//30H: addiu $1,$0,#4 | 不执行
30     assign inst_rom[13] = 32'h8C2A0013;//34H: lw    $10,#19($1)| $10 = 0000_000DH
31     assign inst_rom[14] = 32'h15450003;//38H: bne   $10,$5,#3 | 不跳转
32     assign inst_rom[15] = 32'h00415824;//3CH: and   $11,$2,$1 | $11 = 0000_0000H
33     assign inst_rom[16] = 32'hAC0B001C;//40H: sw    $11,#28($0)| Mem[1CH] = 0H
34     assign inst_rom[17] = 32'hAC040010;//44H: sw    $4,#16($0)| Mem[10H] = 4H
35     assign inst_rom[18] = 32'h3C0C000C;//48H: lui   $12,#12 | [R12] = C_0000H
36     assign inst_rom[19] = 32'h08000000;//4CH: j     00H      | 跳转指令 00H
37
38     //读指令, 取 4 字节
39     always @(*)
40     begin
41         case (addr)
42             5'd0 : inst <= inst_rom[0 ];
43             5'd1 : inst <= inst_rom[1 ];
44             5'd2 : inst <= inst_rom[2 ];
45             5'd3 : inst <= inst_rom[3 ];
46             5'd4 : inst <= inst_rom[4 ];
47             5'd5 : inst <= inst_rom[5 ];
48             5'd6 : inst <= inst_rom[6 ];
49             5'd7 : inst <= inst_rom[7 ];
50             5'd8 : inst <= inst_rom[8 ];
51             5'd9 : inst <= inst_rom[9 ];
52             5'd10: inst <= inst_rom[10];
53             5'd11: inst <= inst_rom[11];
54             5'd12: inst <= inst_rom[12];
55             5'd13: inst <= inst_rom[13];
56             5'd14: inst <= inst_rom[14];
57             5'd15: inst <= inst_rom[15];
58             5'd16: inst <= inst_rom[16];
59             5'd17: inst <= inst_rom[17];
60             5'd18: inst <= inst_rom[18];
61             5'd19: inst <= inst_rom[19];
62             default: inst <= 32'd0;

```

```
62         endcase
63     end
64 endmodule
```

inst_rom_display.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: inst_rom_display.v
4  //  > 描述   : 异步指令存储器显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  //*****
8  module inst_rom_display(
9      //时钟与复位信号
10     input clk,
11     input resetn,    //后缀"n"代表低电平有效
12
13     //触摸屏相关接口, 不需要更改
14     output lcd_rst,
15     output lcd_cs,
16     output lcd_rs,
17     output lcd_wr,
18     output lcd_rd,
19     inout[15:0] lcd_data_io,
20     output lcd_bl_ctr,
21     inout ct_int,
22     inout ct_sda,
23     output ct_scl,
24     output ct_rstn
25 );
26 //-----{调用数据储存器模块}begin
27     //数据存储器多增加一个读端口, 用于读出特定内存地址显示在触摸屏上
28     reg  [31:0] addr;
29     wire [31:0] inst;
30
31     inst_rom inst_rom_module(
32         .addr  (addr[31:2]),
33         .inst  (inst      )
34     );
35 //-----{调用寄存器堆模块}end
36
37 //-----{调用触摸屏模块}begin-----//
38 //-----{实例化触摸屏}begin
39 //此小节不需要更改
40     reg          display_valid;
41     reg  [39:0] display_name;
42     reg  [31:0] display_value;
43     wire [5 :0] display_number;
```

```
44     wire      input_valid;
45     wire [31:0] input_value;
46
47     lcd_module lcd_module(
48         .clk      (clk      ),    //10Mhz
49         .resetn    (resetn    ),
50
51         //调用触摸屏的接口
52         .display_valid (display_valid ),
53         .display_name  (display_name  ),
54         .display_value (display_value ),
55         .display_number (display_number),
56         .input_valid   (input_valid   ),
57         .input_value   (input_value   ),
58
59         //lcd 触摸屏相关接口，不需要更改
60         .lcd_rst      (lcd_rst      ),
61         .lcd_cs       (lcd_cs       ),
62         .lcd_rs       (lcd_rs       ),
63         .lcd_wr       (lcd_wr       ),
64         .lcd_rd       (lcd_rd       ),
65         .lcd_data_io  (lcd_data_io  ),
66         .lcd_bl_ctr   (lcd_bl_ctr   ),
67         .ct_int       (ct_int       ),
68         .ct_sda       (ct_sda       ),
69         .ct_scl       (ct_scl       ),
70         .ct_rstn      (ct_rstn      )
71     );
72     //-----{实例化触摸屏}end
73
74     //-----{从触摸屏获取输入}begin
75     //根据实际需要输入的数修改此小节，
76     //建议对每一个数的输入，编写单独一个 always 块
77     always @(posedge clk)
78     begin
79         if (!resetn)
80         begin
81             addr <= 32'd0;
82         end
83         else if (input_valid)
84         begin
85             addr[31:2] <= input_value[31:2];
86         end
87     end
88     //-----{从触摸屏获取输入}end
89
90     //-----{输出到触摸屏显示}begin
91     //根据需要显示的数修改此小节，
92     //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
93     //44 块显示区域从 1 开始编号，编号为 1~44，
94     always @(posedge clk)
```

```
95     begin
96         case(display_number)
97             6'd1:
98                 begin
99                     display_valid <= 1'b1;
100                     display_name  <= "ADDR ";
101                     display_value <= addr;
102                 end
103             6'd2:
104                 begin
105                     display_valid <= 1'b1;
106                     display_name  <= "INST ";
107                     display_value <= inst;
108                 end
109             default :
110                 begin
111                     display_valid <= 1'b0;
112                     display_name  <= 40'd0;
113                     display_value <= 32'd0;
114                 end
115             endcase
116         end
117     //-----{输出到触摸屏显示}end
118     //-----{调用触摸屏模块}end-----//
119 endmodule
```

异步 inst_rom 在烧写到板上展示时需要的约束文件和同步 inst_rom 的一样。

六、实验六 单周期 CPU 实现

1 实验目的

1. 理解 MIPS 指令结构，理解 MIPS 指令集中常用指令的功能和编码，学会对这些指令进行归纳分类。
2. 了解熟悉 MIPS 体系的处理器结构，如延迟槽，哈佛结构的概念。
3. 熟悉并掌握单周期 CPU 的原理和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计多周期 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 指令集，深入理解常用指令的功能和编码，并进行归纳确定处理器各部件的控制码，比如使用何种 ALU 运算，是否写寄存器堆等。
2. 确定自己本次实验中的准备实现的 MIPS 指令，要求至少实现一条 load 指令、一条 store 指令、10 条基础运算指令、一条跳转指令。其中基础运算指令最好包含多种类型的操作，必须包含一条加法和一条减法指令。不考虑指令可能产生异常的情况。单周期 CPU 的实验重点是搭建出一个 CPU 架构，为避免被繁琐的指令所困惑，建议在单周期 CPU 实验中只实现十几条指令。
3. 对准备实现的指令进行分析，完成表 7.1 的填写。

表 7.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 _{rs} _{rt} rd_000000100001	[rs]	[rt]	rd	GPR[rd]= GPR [rs]+ GPR [rt]
I 型指令	addiu rt,rs,imm	001001 _{rs} _{rt} imm	[rs]	sign_ext(imm)	rt	GPR [rt]= GPR [rs]+sign_ext(imm)
J 型指令	j target	000010 target	PC	target		跳转, PC={PC[31:28],target,2'b00}

注：GPR 表示通用寄存器，[rs]表示寄存器 rs 里存储的值，PC 表示程序计数器；imm 为 16 位立即数，sign_ext(imm)表示对其进行符号扩展；target 为 26 位立即数。

4. 自行设计本次实验的方案，画出结构框图，大致结构框图如图 7.1。图 7.1 中粗线表示接口位数和种类不定，需要在自己的结构框图中详细给出。从图 7.1 中可以看出，本次实验是需要用到之前实验的结果的，比如 ALU 模块、寄存器堆模块、指令 ROM 模块和数据 RAM 模块，其中 ROM 和 RAM 要使用自行搭建的异步存储器。

单周期 CPU 是指一条指令的所有操作在一个时钟周期内执行完。设计中所有寄存器和存储器都是异步读同步写的，即读出数据不需要时钟控制，但写入数据需时钟控制。

故单周期 CPU 的运作即：在一个时钟周期内，根据 PC 值从指令 ROM 中读出相应的指令，将指令译码后从寄存器堆中读出需要的操作数，送往 ALU 模块，ALU 模块运算得到结果。

如果是 store 指令，则 ALU 运算结果为数据存储的地址，就向数据 RAM 发出写请求，在下一个时钟上升沿真正写入到数据存储。

如果是 load 指令，则 ALU 运算结果为数据存储的地址，根据该值从数据存 RAM 中读出数据，送往寄存器堆根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。

如果非 load/store 操作，若有写寄存器堆的操作，则直接将 ALU 运算结果送往寄存器堆根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。

如果是分支跳转指令，则是需要将结果写入到 pc 寄存器中的。

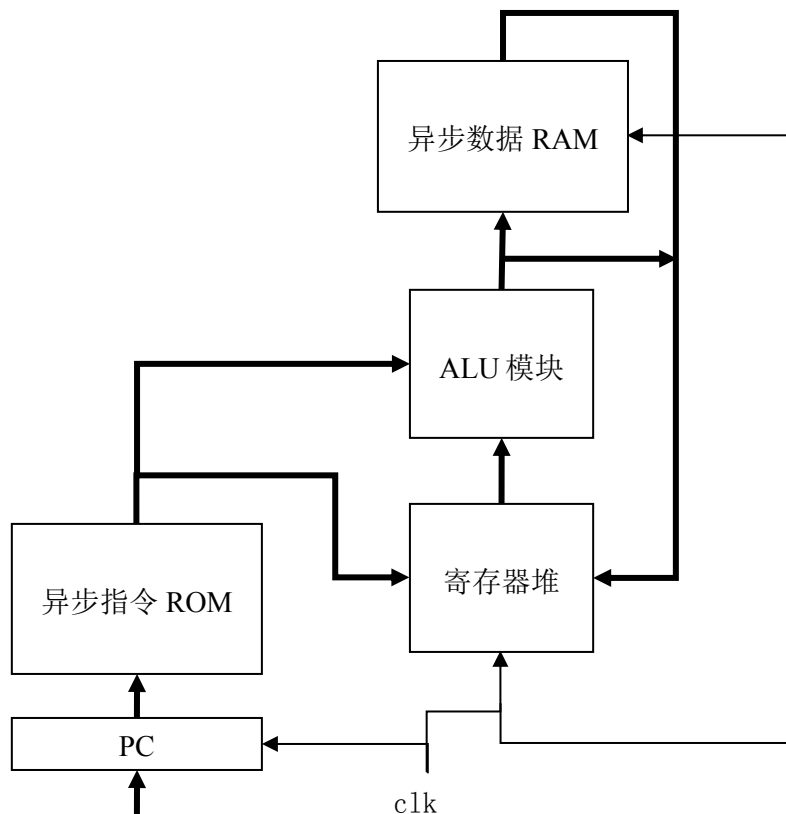


图 7.1 单周期 CPU 的大致框图

5. 根据设计的实验方案，使用 verilog 编写相应代码。
6. 依据自己设计中实现的指令，编写一段不少于 20 行的汇编程序，力求验证到所有实现的指令。该段汇编程序是需要内嵌到自行搭建的异步指令 ROM 中的。完成表 7.2 的填写。

表 7.2 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$t1, \$0, #1	[\$t1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 对编写的代码进行仿真，得到正确的波形图。

8. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 7.2。外围模块中需调用封装好的 LCD 触摸屏模块，观察单周期 CPU 的内部状态，比如 32 个寄存器的值，PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

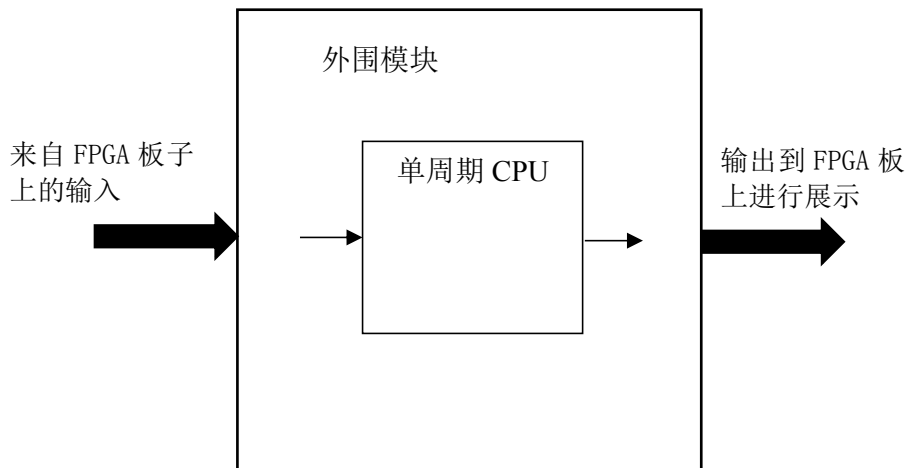


图 7.2 单周期 CPU 设计实验的顶层模块大致框图

9. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：

①：MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度，故在单周期 CPU 中该设定毫无意义，反而带来了 CPU 实现上的麻烦，故建议在单周期 CPU 中不考虑延迟槽技术。

②：MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4)，而由于单周期不考虑延迟槽，故在实验中分支跳转指令参与计算使用本指令的 PC 值，故跳转的偏移量设置需要注意下。比如一条指令

“beq, r0, r0, #2”在不考虑延迟槽的单周期 CPU 中，其跳转的目标地址为 beq 指令后面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。这里需要理解清楚。

4 实验要求

1. 做好预习：

- 1) 熟知 MIPS 指令类型，深入理解常用指令的功能和编码；
- 2) 归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令；
- 3) 对准备实现的指令进行分析，完成表 7.1 的填写；
- 4) 设计本次实验的方案，画出实验方案的设计框图，即补充完善图 7.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 7.2；
- 6) 依据自己设计中实现的指令，编写一段不少于 20 行的汇编程序，要求包含所有实现的指令，完成表 7.2 的填写。

2. 实验实施：

- 1) 确认单周期 CPU 的设计框图的正确性；
- 2) 编写 verilog 代码，将表 7.2 中自己编写的汇编程序翻译为二进制，内嵌到指令 ROM 中；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用单周期 CPU 的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 7.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

单周期 CPU 能够执行 MIPS 指令集系统的一个子集，共 16 条指令，包括存储访问指令、运算指令、跳转指令。选取的指令集将会用到各个数据通路部件，并体现控制部分设计的关键原理，其余暂未支持的指令的实现方式类似，并将在后续多周期和流水 CPU 实验中陆续加入。

5.1 实验原理

表 7.3 为单周期 CPU 实现的指令集。表 7.4 为这些指令的解析。

表 7.3 单周期 CPU 实现的指令

指令名称	汇编指令	功能描述
无符号加法	addu rd,rs,rt	$GPR[rd] = GPR[rs] + GPR[rt]$
无符号减法	subu rd,rs,rt	$GPR[rd] = GPR[rs] - GPR[rt]$
有符号比较, 小于置位	slt rd,rs,rt	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
按位与	and rd,rs,rt	$GPR[rd] = GPR[rs] \& GPR[rt]$
按位或非	nor rd,rs,rt	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
按位或	or rd,rs,rt	$GPR[rd] = GPR[rs] GPR[rt]$
按位异或	xor rd,rs,rt	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
逻辑左移	sll rd,rt,shf	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
逻辑右移	srl rd,rt,shf	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
立即数无符号加法	addiu rt,rs,imm	$GPR[rt] = GPR[rs] + \text{sign_ext}(\text{imm})$
立即数装载高位	lui rt,imm	$GPR[rt] = \{\text{imm}, 16'd0\}$
从内存装载字	lw rt,offset(base)	$GPR[rt] = \text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})]$
向内存存储字	sw rt,offset(base)	$\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})] = GPR[rt]$
判断相等跳转	beq rs,rt,offset	if $GPR[rs] = GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
判断不等跳转	bne rs,rt,offset	if $GPR[rs] \neq GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
直接跳转	j target	$PC = \{PC[31:28], \text{target} \ll 2\}$

表 7.4 单周期 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 0000 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 0000 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 0000 101010	[rs]	[rt]		rd	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
	and rd,rs,rt	000000 rs rt rd 0000 100100	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \& GPR[rt]$
	nor rd,rs,rt	000000 rs rt rd 0000 100111	[rs]	[rt]		rd	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
	or rd,rs,rt	000000 rs rt rd 0000 100101	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] GPR[rt]$
	xor rd,rs,rt	000000 rs rt rd 0000 100110	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
	sll rd,rt,shf	000000 000000 rt rd shf 000000		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
	srl rd,rt,shf	000000 000000 rt rd shf 000010		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = GPR[rs] + \text{sign_ext}(\text{imm})$
	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if $GPR[rs] = GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if $GPR[rs] \neq GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$

表 7.4 单周期 CPU 实现的 mips 指令特性归纳（续）

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext(offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]
I 型指令	lui rt,imm	001111 00000 rt i mm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
J 型指令	j target	000010 target					PC={PC[31:28],target<<2}

单周期 CPU 的实现框图如下：

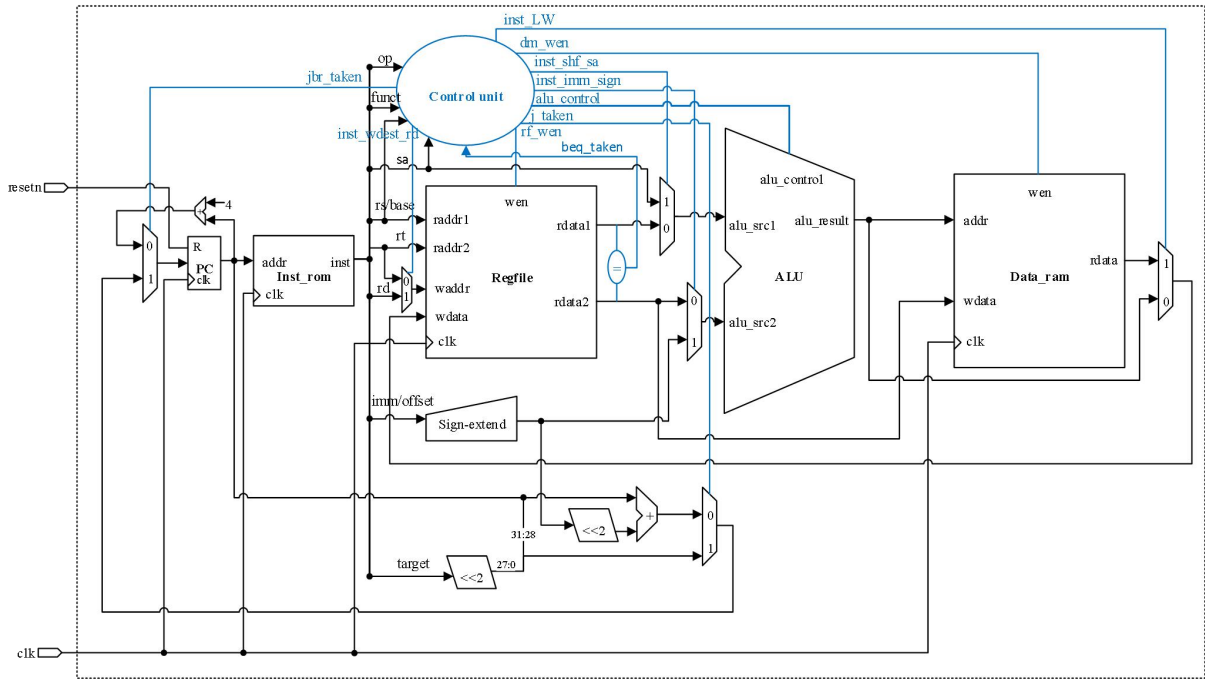


图 7.3 单周期 CPU 的实现框图

利用其实现的 16 条指令，可以编写一段汇编程序，用于测试实现的单周期 CPU 的正确性。表 7.5 给出了一个测试程序。

表 7.5 单周期 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1,\$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
04H	sll \$2,\$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
08H	addu \$3,\$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
0CH	srl \$4,\$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
10H	subu \$5,\$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
14H	sw \$5,#19(\$1)	Mem[0000_0014H] = 0000_000DH	AC250013	1010_1100_0010_0101_0000_0000_0001_0011
18H	nor \$6,\$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
1CH	or \$7,\$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101

表 7.5 单周期 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
20H	xor \$8,\$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
24H	sw \$8,#28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
28H	slt \$9,\$6,\$7	[\$9] = 0000_0001H	00C7482A	0000_0000_1100_0111_0100_1000_0010_1010
2CH	beq \$9,\$1,#2	跳转到指令 34H	11210002	0001_0001_0010_0001_0000_0000_0000_0010
30H	addiu \$1,\$0,#4	不执行	24010004	0010_0100_0000_0001_0000_0000_0000_0100
34H	lw \$10,#19(\$1)	[\$10] = 0000_000DH	8C2A0013	1000_1100_0010_1010_0000_0000_0001_0011
38H	bne \$10,\$5,#3	不跳转	15450003	0001_0101_0100_0101_0000_0000_0000_0011
3CH	and \$11,\$2,\$1	[\$11] = 0000_0000H	00415824	0000_0000_0100_0001_0101_1000_0010_0100
40H	sw \$11,#28(\$0)	Mem[0000_001CH] = 0000_0000H	AC0B001C	1010_1100_0000_1011_0000_0000_0001_1100
44H	sw \$4,#16(\$0)	Mem[0000_0010H] = 0000_0004H	AC040010	1010_1100_0000_0100_0000_0000_0001_0000
48H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
4CH	j 00H	跳转指令 00H	08000000	0000_1000_0000_0000_0000_0000_0000_0000

5.2 实验顶层模块框图

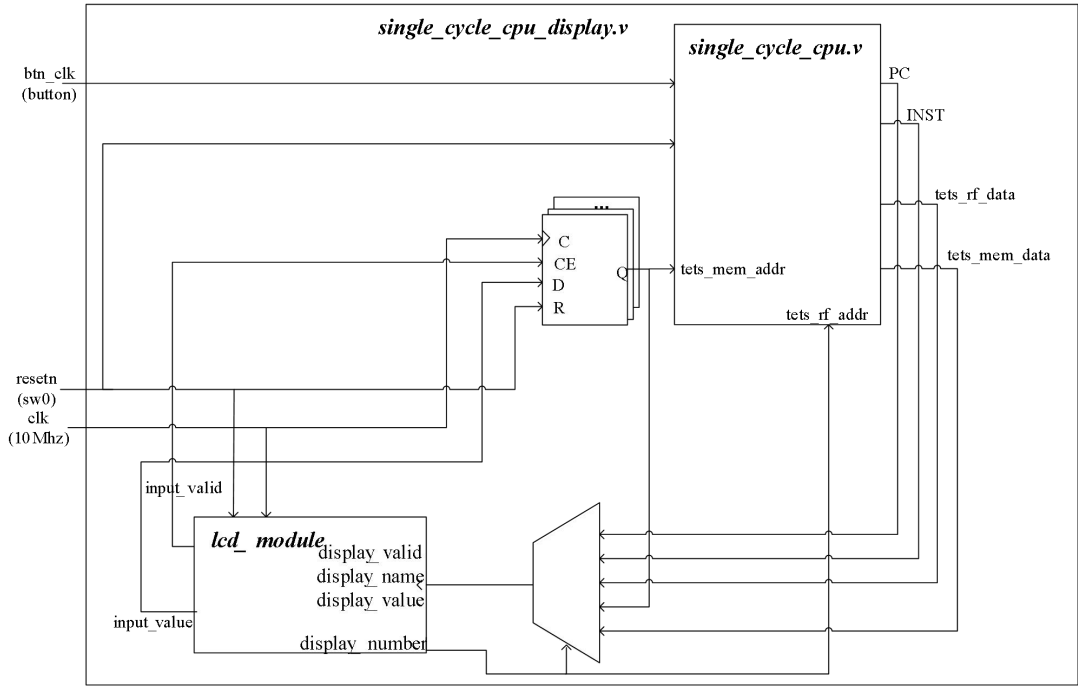


图 7.4 单周期 CPU 参考设计的顶层模块框图

5.3 源代码

single_cycle_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
3 // > 文件名: single_cycle_cpu_display.v
4 // > 描述 : 单周期 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
```

```
6 // > 日期 : 2016-04-14
7 //*****
8 module single_cycle_cpu_display(
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14     input btn_clk,
15
16     //触摸屏相关接口, 不需要更改
17     output lcd_rst,
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改, 用于单步调试
31     wire cpu_clk; //单周期 CPU 里使用脉冲开关作为时钟, 以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37         begin
38             btn_clk_r1<= 1'b0;
39         end
40         else
41         begin
42             btn_clk_r1 <= ~btn_clk;
43         end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk),.CE(clk_en),.O(cpu_clk));
51 //-----{时钟和复位信号}end
52
53 //-----{调用单周期 CPU 模块}begin
54
55     //用于在 FPGA 板上显示结果
```



```

56     wire [31:0] cpu_pc;           //CPU 的 PC
57     wire [31:0] cpu_inst;        //该 PC 取出的指令
58     wire [ 4:0] rf_addr;         //扫描寄存器堆的地址
59     wire [31:0] rf_data;         //寄存器堆从调试端口读出的数据
60     reg  [31:0] mem_addr;        //要观察的内存地址
61     wire [31:0] mem_data;        //内存地址对应的数据
62     single_cycle_cpu cpu(
63         .clk      (cpu_clk      ),
64         .resetn    (resetn      ),
65
66         .rf_addr   (rf_addr    ),
67         .mem_addr  (mem_addr    ),
68         .rf_data   (rf_data    ),
69         .mem_data  (mem_data    ),
70         .cpu_pc    (cpu_pc     ),
71         .cpu_inst  (cpu_inst    )
72     );
73     //-----{调用单周期 CPU 模块}end
74
75     //-----{调用触摸屏模块}begin-----//
76     //-----{实例化触摸屏}begin
77     //此小节不需要更改
78     reg      display_valid;
79     reg  [39:0] display_name;
80     reg  [31:0] display_value;
81     wire [5 :0] display_number;
82     wire      input_valid;
83     wire [31:0] input_value;
84
85     lcd_module lcd_module(
86         .clk      (clk          ),      //10Mhz
87         .resetn    (resetn      ),
88
89         //调用触摸屏的接口
90         .display_valid (display_valid ),
91         .display_name  (display_name  ),
92         .display_value (display_value ),
93         .display_number (display_number),
94         .input_valid   (input_valid   ),
95         .input_value   (input_value   ),
96
97         //lcd 触摸屏相关接口，不需要更改
98         .lcd_rst      (lcd_rst      ),
99         .lcd_cs       (lcd_cs       ),
100        .lcd_rs       (lcd_rs       ),
101        .lcd_wr       (lcd_wr       ),
102        .lcd_rd       (lcd_rd       ),
103        .lcd_data_io  (lcd_data_io  ),
104        .lcd_bl_ctr   (lcd_bl_ctr   ),
105        .ct_int       (ct_int       ),
106        .ct_sda       (ct_sda       ),

```

```
107         .ct_scl          (ct_scl          ),
108         .ct_rstn         (ct_rstn         )
109     );
110 //-----{实例化触摸屏}end
111
112 //-----{从触摸屏获取输入}begin
113 //根据实际需要输入的数修改此小节,
114 //建议对每一个数的输入, 编写单独一个 always 块
115     always @(posedge clk)
116     begin
117         if (!resetn)
118         begin
119             mem_addr <= 32'd0;
120         end
121         else if (input_valid)
122         begin
123             mem_addr <= input_value;
124         end
125     end
126     assign rf_addr = display_number-6'd5;
127 //-----{从触摸屏获取输入}end
128
129 //-----{输出到触摸屏显示}begin
130 //根据需要显示的数修改此小节,
131 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
132 //44 块显示区域从 1 开始编号, 编号为 1~44,
133     always @(posedge clk)
134     begin
135         if (display_number >6'd4 && display_number <6'd37 )
136         begin //块号 5~36 显示 32 个通用寄存器的值
137             display_valid <= 1'b1;
138             display_name[39:16] <= "REG";
139             display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
140             display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
141             display_value      <= rf_data;
142         end
143         else
144         begin
145             case(display_number)
146             6'd1 : //显示 PC 值
147             begin
148                 display_valid <= 1'b1;
149                 display_name  <= "  PC";
150                 display_value <= cpu_pc;
151             end
152             6'd2 : //显示 PC 取出的指令
153             begin
154                 display_valid <= 1'b1;
155                 display_name  <= " INST";
156                 display_value <= cpu_inst;
157             end
```

```

158             6'd3 : //显示要观察的内存地址
159             begin
160                 display_valid <= 1'b1;
161                 display_name  <= "MADDR";
162                 display_value <= mem_addr;
163             end
164             6'd4 : //显示该内存地址对应的数据
165             begin
166                 display_valid <= 1'b1;
167                 display_name  <= "MDATA";
168                 display_value <= mem_data;
169             end
170             default :
171             begin
172                 display_valid <= 1'b0;
173             end
174         endcase
175     end
176 end
177 //-----{输出到触摸屏显示}end
178 //-----{调用触摸屏模块}end-----//
179 endmodule

```

single_cycle_cpu.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: single_cycle_cpu.v
4  //  > 描述   :单周期 CPU 模块,共实现 16 条指令
5  //  >         指令 rom 和数据 ram 均采用异步读数据,以便单周期 CPU 好实现
6  //  > 作者   : LOONGSON
7  //  > 日期   : 2016-04-14
8  //*****
9  `define STARTADDR 32'd0 // 程序起始地址
10 module single_cycle_cpu(
11     input clk,        // 时钟
12     input resetn,     // 复位信号,低电平有效
13
14     //display data
15     input  [4:0] rf_addr,
16     input  [31:0] mem_addr,
17     output [31:0] rf_data,
18     output [31:0] mem_data,
19     output [31:0] cpu_pc,
20     output [31:0] cpu_inst
21 );
22
23 //-----{取指}begin-----
24 //
25     reg  [31:0] pc;
26     wire [31:0] next_pc;

```

```
26    wire [31:0] seq_pc;
27    wire [31:0] jbr_target;
28    wire jbr_taken;
29
30    // 下一指令地址: seq_pc=pc+4
31    assign seq_pc[31:2] = pc[31:2] + 1'b1;
32    assign seq_pc[1:0] = pc[1:0];
33    // 新指令: 若指令跳转, 为跳转地址; 否则为下一指令
34    assign next_pc = jbr_taken ? jbr_target : seq_pc;
35    always @ (posedge clk) // PC 程序计数器
36    begin
37        if (!resetn) begin
38            pc <= `STARTADDR; // 复位, 取程序起始地址
39        end
40        else begin
41            pc <= next_pc; // 不复位, 取新指令
42        end
43    end
44
45    wire [31:0] inst_addr;
46    wire [31:0] inst;
47    assign inst_addr = pc; // 指令地址: 指令长度 32 位
48    inst_rom inst_rom_module( // 指令存储器
49        .addr (inst_addr[6:2]), // I, 5, 指令地址
50        .inst (inst) // O, 32, 指令
51    );
52    assign cpu_pc = pc; //display pc
53    assign cpu_inst = inst;
54    //-----{取指}end-----
55    //
56    //-----{译码}begin-----
57    //
58    wire [5:0] op;
59    wire [4:0] rs;
60    wire [4:0] rt;
61    wire [4:0] rd;
62    wire [4:0] sa;
63    wire [5:0] funct;
64    wire [15:0] imm;
65    wire [15:0] offset;
66    wire [25:0] target;
67
68    assign op = inst[31:26]; // 操作码
69    assign rs = inst[25:21]; // 源操作数 1
70    assign rt = inst[20:16]; // 源操作数 2
71    assign rd = inst[15:11]; // 目标操作数
72    assign sa = inst[10:6]; // 特殊域, 可能存放偏移量
73    assign funct = inst[5:0]; // 功能码
74    assign imm = inst[15:0]; // 立即数
75    assign offset = inst[15:0]; // 地址偏移量
```

```

75     assign target = inst[25:0];    // 目标地址
76
77     wire op_zero;    // 操作码全 0
78     wire sa_zero;    // sa 域全 0
79     assign op_zero = ~(|op);
80     assign sa_zero = ~(|sa);
81
82     // 实现指令列表
83     wire inst_ADDU, inst_SUBU , inst_SLT, inst_AND;
84     wire inst_NOR , inst_OR , inst_XOR, inst_SLL;
85     wire inst_SRL , inst_ADDIU, inst_BEQ, inst_BNE;
86     wire inst_LW , inst_SW , inst_LUI, inst_J;
87
88     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
89     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
90     assign inst_SLT = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
91     assign inst_AND = op_zero & sa_zero & (funct == 6'b100100); // 逻辑与
92     assign inst_NOR = op_zero & sa_zero & (funct == 6'b100111); // 逻辑或非
93     assign inst_OR = op_zero & sa_zero & (funct == 6'b100101); // 逻辑或
94     assign inst_XOR = op_zero & sa_zero & (funct == 6'b100110); // 逻辑异或
95     assign inst_SLL = op_zero & (rs==5'd0) & (funct == 6'b000000); // 逻辑左移
96     assign inst_SRL = op_zero & (rs==5'd0) & (funct == 6'b000010); // 逻辑右移
97     assign inst_ADDIU = (op == 6'b001001); // 立即数无符号加法
98     assign inst_BEQ = (op == 6'b000100); // 判断相等跳转
99     assign inst_BNE = (op == 6'b000101); // 判断不等跳转
100    assign inst_LW = (op == 6'b100011); // 从内存装载
101    assign inst_SW = (op == 6'b101011); // 向内存存储
102    assign inst_LUI = (op == 6'b001111); // 立即数装载高半字节
103    assign inst_J = (op == 6'b000010); // 直接跳转
104
105    // 无条件跳转判断
106    wire j_taken;
107    wire [31:0] j_target;
108    assign j_taken = inst_J;
109
110    assign j_target = {pc[31:28], target, 2'b00};
111
112    //分支跳转
113    wire beq_taken;
114    wire bne_taken;
115    wire [31:0] br_target;
116    assign beq_taken = (rs_value == rt_value); // BEQ 跳转条件: GPR[rs]=GPR[rt]
117    assign bne_taken = ~beq_taken; // BNE 跳转条件: GPR[rs]≠GPR[rt]
118    assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
119    assign br_target[1:0] = pc[1:0]; // 分支跳转目标地址: PC=PC+offset<<2
120
121    //跳转指令的跳转信号和跳转目标地址
122    assign jbr_taken = j_taken // 指令跳转: 无条件跳转 或 满足分支跳转条
件
123    | inst_BEQ & beq_taken
124    | inst_BNE & bne_taken;

```

```

125     assign jbr_target = j_taken ? j_target : br_target;
126
127     // 寄存器堆
128     wire rf_wen;
129     wire [4:0] rf_waddr;
130     wire [31:0] rf_wdata;
131     wire [31:0] rs_value, rt_value;
132
133     regfile rf_module(
134         .clk      (clk      ), // I, 1
135         .wen      (rf_wen   ), // I, 1
136         .raddr1   (rs       ), // I, 5
137         .raddr2   (rt       ), // I, 5
138         .waddr    (rf_waddr ), // I, 5
139         .wdata    (rf_wdata ), // I, 32
140         .rdata1   (rs_value ), // O, 32
141         .rdata2   (rt_value ), // O, 32
142
143         //display rf
144         .test_addr(rf_addr),
145         .test_data(rf_data)
146     );
147
148
149     // 传递到执行模块的 ALU 源操作数和操作码
150     wire inst_add, inst_sub, inst_slt, inst_sltu;
151     wire inst_and, inst_nor, inst_or, inst_xor;
152     wire inst_sll, inst_srl, inst_sra, inst_lui;
153     assign inst_add = inst_ADDU | inst_ADDIU | inst_LW | inst_SW; // 做加法运算
154     assign inst_sub = inst_SUBU; // 减法
155     assign inst_slt = inst_SLT; // 小于置位
156     assign inst_sltu = 1'b0; // 暂未实现
157     assign inst_and = inst_AND; // 逻辑与
158     assign inst_nor = inst_NOR; // 逻辑或非
159     assign inst_or = inst_OR; // 逻辑或
160     assign inst_xor = inst_XOR; // 逻辑或非
161     assign inst_sll = inst_SLL; // 逻辑左移
162     assign inst_srl = inst_SRL; // 逻辑右移
163     assign inst_sra = 1'b0; // 暂未实现
164     assign inst_lui = inst_LUI; // 立即数装载高位
165
166     wire [31:0] sext_imm;
167     wire inst_shf_sa; //使用 sa 域作为偏移量的指令
168     wire inst_imm_sign; //对立即数作符号扩展的指令
169     assign sext_imm = {{16{imm[15]}}}, imm; // 立即数符号扩展
170     assign inst_shf_sa = inst_SLL | inst_SRL;
171     assign inst_imm_sign = inst_ADDIU | inst_LUI | inst_LW | inst_SW;
172
173     wire [31:0] alu_operand1;
174     wire [31:0] alu_operand2;
175     wire [11:0] alu_control;

```

```

176     assign alu_operand1 = inst_shf_sa ? {27'd0,sa} : rs_value;
177     assign alu_operand2 = inst_imm_sign ? sext_imm : rt_value;
178     assign alu_control = {inst_add,          // ALU 操作码, 独热编码
179                          inst_sub,
180                          inst_slt,
181                          inst_sltu,
182                          inst_and,
183                          inst_nor,
184                          inst_or,
185                          inst_xor,
186                          inst_sll,
187                          inst_srl,
188                          inst_sra,
189                          inst_lui};
190 //-----{译码}end-----
191 //
192 //-----{执行}begin-----
193 //
194     wire [31:0] alu_result;
195     alu alu_module(
196         .alu_control (alu_control ), // I, 12, ALU 控制信号
197         .alu_src1     (alu_operand1), // I, 32, ALU 操作数 1
198         .alu_src2     (alu_operand2), // I, 32, ALU 操作数 2
199         .alu_result   (alu_result )  // O, 32, ALU 结果
200     );
201 //-----{执行}end-----
202 //
203 //-----{访存}begin-----
204 //
205     wire [3 :0] dm_wen;
206     wire [31:0] dm_addr;
207     wire [31:0] dm_wdata;
208     wire [31:0] dm_rdata;
209     assign dm_wen  = {4{inst_SW}} & resetn; // 内存写使能,非 resetn 状态下有
210     assign dm_addr = alu_result;             // 内存写地址, 为 ALU 结果
211     assign dm_wdata = rt_value;             // 内存写数据, 为 rt 寄存器值
212     data_ram data_ram_module(
213         .clk      (clk          ), // I, 1, 时钟
214         .wen      (dm_wen       ), // I, 1, 写使能
215         .addr     (dm_addr[6:2]), // I, 32, 读地址
216         .wdata    (dm_wdata     ), // I, 32, 写数据
217         .rdata    (dm_rdata     ), // O, 32, 读数据
218         //display mem
219         .test_addr(mem_addr[6:2]),
220         .test_data(mem_data      )
221     );

```



```

222 //-----{访存}end-----
223 //
224 //-----{写回}begin-----
225 //
225     wire inst_wdest_rt;    // 寄存器堆写入地址为 rt 的指令
226
227     assign inst_wdest_rt = inst_ADDIU | inst_LW | inst_LUI;
228     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_AND | inst_NOR
229                          | inst_OR | inst_XOR | inst_SLL | inst_SRL;
230     // 寄存器堆写使能信号, 非复位状态下有效
231     assign rf_wen = (inst_wdest_rt | inst_wdest_rd) & resetn;
232     assign rf_waddr = inst_wdest_rd ? rd : rt; // 寄存器堆写地址 rd 或 rt
233     assign rf_wdata = inst_LW ? dm_rdata : alu_result; // 写回结果
234 //-----{写回}end-----
235 //
235 endmodule

```

inst_rom.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  // > 文件名: inst_rom.v
4  // > 描述 : 异步指令存储器模块, 采用寄存器搭建而成, 类似寄存器堆
5  // >          内嵌好指令, 只读, 异步读
6  // > 作者 : LOONGSON
7  // > 日期 : 2016-04-14
8  //*****
9  module inst_rom(
10     input      [4 :0] addr, // 指令地址
11     output reg [31:0] inst   // 指令
12 );
13
14     wire [31:0] inst_rom[19:0]; // 指令存储器, 字节地址 7'b000_0000~7'b111_1111
15     //----- 指令编码 -----|指令地址|--- 汇编指令 --|- 指令结果 -----//
16     assign inst_rom[ 0] = 32'h24010001;//00H: addiu $1 , $0, #1 | $1 = 0000_0001H
17     assign inst_rom[ 1] = 32'h00011100;//04H: sll   $2 , $1, #4 | $2 = 0000_0010H
18     assign inst_rom[ 2] = 32'h00411821;//08H: addu  $3 , $2, $1 | $3 = 0000_0011H
19     assign inst_rom[ 3] = 32'h00022082;//0CH: srl   $4 , $2, #2 | $4 = 0000_0004H
20     assign inst_rom[ 4] = 32'h00642823;//10H: subu  $5 , $3, $4 | $5 = 0000_000DH
21     assign inst_rom[ 5] = 32'hAC250013;//14H: sw    $5 , #19($1) | Mem[14H] = DH
22     assign inst_rom[ 6] = 32'h00A23027;//18H: nor   $6 , $5, $2 | $6 = FFFF_FFE2H
23     assign inst_rom[ 7] = 32'h00C33825;//1CH: or    $7 , $6, $3 | $7 = FFFF_FFF3H
24     assign inst_rom[ 8] = 32'h00E64026;//20H: xor   $8 , $7, $6 | $8 = 0000_0011H
25     assign inst_rom[ 9] = 32'hAC08001C;//24H: sw    $8 , #28($0) | Mem[1CH] = 11H
26     assign inst_rom[10] = 32'h00C7482A;//28H: slt   $9 , $6, $7 | $9 = 0000_0001H
27     assign inst_rom[11] = 32'h11210002;//2CH: beq   $9 , $1, #2 | 跳转到指令 34H
28     assign inst_rom[12] = 32'h24010004;//30H: addiu $1 , $0, #4 | 不执行
29     assign inst_rom[13] = 32'h8C2A0013;//34H: lw    $10, #19($1) | $10 = 0000_000DH
30     assign inst_rom[14] = 32'h15450003;//38H: bne   $10, $5, #3 | 不跳转
31     assign inst_rom[15] = 32'h00415824;//3CH: and   $11, $2, $1 | $11 = 0000_0000H

```

```

32     assign inst_rom[16] = 32'hAC0B001C;//40H: sw      $11,#28($0) | Mem[1CH] = 0H
33     assign inst_rom[17] = 32'hAC040010;//44H: sw      $4 ,#16($0) | Mem[10H] = 4H
34     assign inst_rom[18] = 32'h3C0C000C;//48H: lui     $12,#12   | [R12] = C_0000H
35     assign inst_rom[19] = 32'h08000000;//4CH: j       00H       | 跳转指令 00H
36
37     //读指令,取 4 字节
38     always @(*)
39     begin
40         case (addr)
41             5'd0 : inst <= inst_rom[0 ];
42             5'd1 : inst <= inst_rom[1 ];
43             5'd2 : inst <= inst_rom[2 ];
44             5'd3 : inst <= inst_rom[3 ];
45             5'd4 : inst <= inst_rom[4 ];
46             5'd5 : inst <= inst_rom[5 ];
47             5'd6 : inst <= inst_rom[6 ];
48             5'd7 : inst <= inst_rom[7 ];
49             5'd8 : inst <= inst_rom[8 ];
50             5'd9 : inst <= inst_rom[9 ];
51             5'd10: inst <= inst_rom[10];
52             5'd11: inst <= inst_rom[11];
53             5'd12: inst <= inst_rom[12];
54             5'd13: inst <= inst_rom[13];
55             5'd14: inst <= inst_rom[14];
56             5'd15: inst <= inst_rom[15];
57             5'd16: inst <= inst_rom[16];
58             5'd17: inst <= inst_rom[17];
59             5'd18: inst <= inst_rom[18];
60             5'd19: inst <= inst_rom[19];
61             default: inst <= 32'd0;
62         endcase
63     end
64 endmodule

```

data_ram.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: data_mem.v
4  //    > 描述   : 异步数据存储器模块,采用寄存器搭建而成,类似寄存器堆
5  //    >          同步写,异步读
6  //    > 作者   : LOONGSON
7  //    > 日期   : 2016-04-14
8  //*****
9  module data_ram(
10     input          clk,          // 时钟
11     input  [3:0]    wen,          // 字节写使能
12     input  [4:0]    addr,         // 地址
13     input  [31:0]   wdata,        // 写数据
14     output reg [31:0] rdata,       // 读数据

```

```
15
16    //调试端口, 用于读出数据显示
17    input      [4 :0] test_addr,
18    output reg [31:0] test_data
19 );
20    reg [31:0] DM[31:0]; //数据存储器, 字节地址 7'b000_0000~7'b111_1111
21
22    //写数据
23    always @(posedge clk) // 当写控制信号为 1, 数据写入内存
24    begin
25        if (wen[3])
26        begin
27            DM[addr][31:24] <= wdata[31:24];
28        end
29    end
30    always @(posedge clk)
31    begin
32        if (wen[2])
33        begin
34            DM[addr][23:16] <= wdata[23:16];
35        end
36    end
37    always @(posedge clk)
38    begin
39        if (wen[1])
40        begin
41            DM[addr][15: 8] <= wdata[15: 8];
42        end
43    end
44    always @(posedge clk)
45    begin
46        if (wen[0])
47        begin
48            DM[addr][7 : 0] <= wdata[7 : 0];
49        end
50    end
51
52    //读数据, 取 4 字节
53    always @(*)
54    begin
55        case (addr)
56            5'd0 : rdata <= DM[0 ];
57            5'd1 : rdata <= DM[1 ];
58            5'd2 : rdata <= DM[2 ];
59            5'd3 : rdata <= DM[3 ];
60            5'd4 : rdata <= DM[4 ];
61            5'd5 : rdata <= DM[5 ];
62            5'd6 : rdata <= DM[6 ];
63            5'd7 : rdata <= DM[7 ];
64            5'd8 : rdata <= DM[8 ];
65            5'd9 : rdata <= DM[9 ];
```

```
66          5'd10: rdata <= DM[10];
67          5'd11: rdata <= DM[11];
68          5'd12: rdata <= DM[12];
69          5'd13: rdata <= DM[13];
70          5'd14: rdata <= DM[14];
71          5'd15: rdata <= DM[15];
72          5'd16: rdata <= DM[16];
73          5'd17: rdata <= DM[17];
74          5'd18: rdata <= DM[18];
75          5'd19: rdata <= DM[19];
76          5'd20: rdata <= DM[20];
77          5'd21: rdata <= DM[21];
78          5'd22: rdata <= DM[22];
79          5'd23: rdata <= DM[23];
80          5'd24: rdata <= DM[24];
81          5'd25: rdata <= DM[25];
82          5'd26: rdata <= DM[26];
83          5'd27: rdata <= DM[27];
84          5'd28: rdata <= DM[28];
85          5'd29: rdata <= DM[29];
86          5'd30: rdata <= DM[30];
87          5'd31: rdata <= DM[31];
88      endcase
89  end
90  //调试端口，读出特定内存的数据
91  always @(*)
92  begin
93      case (test_addr)
94          5'd0 : test_data <= DM[0 ];
95          5'd1 : test_data <= DM[1 ];
96          5'd2 : test_data <= DM[2 ];
97          5'd3 : test_data <= DM[3 ];
98          5'd4 : test_data <= DM[4 ];
99          5'd5 : test_data <= DM[5 ];
100         5'd6 : test_data <= DM[6 ];
101         5'd7 : test_data <= DM[7 ];
102         5'd8 : test_data <= DM[8 ];
103         5'd9 : test_data <= DM[9 ];
104         5'd10: test_data <= DM[10];
105         5'd11: test_data <= DM[11];
106         5'd12: test_data <= DM[12];
107         5'd13: test_data <= DM[13];
108         5'd14: test_data <= DM[14];
109         5'd15: test_data <= DM[15];
110         5'd16: test_data <= DM[16];
111         5'd17: test_data <= DM[17];
112         5'd18: test_data <= DM[18];
113         5'd19: test_data <= DM[19];
114         5'd20: test_data <= DM[20];
115         5'd21: test_data <= DM[21];
116         5'd22: test_data <= DM[22];
```

```
117         5'd23: test_data <= DM[23];
118         5'd24: test_data <= DM[24];
119         5'd25: test_data <= DM[25];
120         5'd26: test_data <= DM[26];
121         5'd27: test_data <= DM[27];
122         5'd28: test_data <= DM[28];
123         5'd29: test_data <= DM[29];
124         5'd30: test_data <= DM[30];
125         5'd31: test_data <= DM[31];
126     endcase
127 end
128 endmodule
```

单周期 CPU 的 `inst_rom.v` 和 `data_ram.v` 均为自行搭建的异步存储器，也是在实验五中 5.5 和 5.6 小节中给出的源代码。单周期 CPU 的源代码还缺少 `regfile.v` 和 `alu.v`，这两个源文件都已在实验三和实验四中给出，本实验中直接通过“Add Copy of Source”添加到工程中即可。其中 `alu.v` 还调用了实验一中的 `adder.v`。

约束文件 `single_cycle_cpu.xdc` 源代码：

```
1  #时钟信号连接
2  set_property PACKAGE_PIN AC19 [get_ports clk]
3
4  #脉冲开关，用于输入作为复位信号，低电平有效
5  set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7  #脉冲开关，用于输入作为单步执行的 clk
8  set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10 set_property IOSTANDARD LVCMOS33 [get_ports clk]
11 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14 #触摸屏引脚连接
15 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
```

```
31 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37 set_property PACKAGE_PIN L19 [get_ports ct_int]
38 set_property PACKAGE_PIN J24 [get_ports ct_sda]
39 set_property PACKAGE_PIN H21 [get_ports ct_scl]
40 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```

七、 实验七 多周期 CPU 实现

1 实验目的

1. 在单周期 CPU 实验完成的提前下，理解多周期的概念。
2. 熟悉并掌握多周期 CPU 的原理和设计。
3. 进一步提升运用 verilog 语言进行电路设计的能力。
4. 为后续实现流水线 cpu 的课程设计打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 本次实验是对单周期 CPU 实验的拔高，也是为流水线 CPU 打下基础。前期的实验准备同单周期 CPU 的实验，在单周期 CPU 中只要求实现了十几条指令，但此处要求扩展到 30 多条指令。

多周期 CPU 是指，一条指令需要花费多个周期才能完成所有操作，在每个周期内只做一部分操作，比如：取指、译码、执行、访存、写回，此时，一条指令执行完，共需 5 个周期，每个周期只做一部分操作。

将 CPU 划分为多周期的优势在于，每个时钟周期内 CPU 需要做的工作就变少，因此频率可以更高，且每个部件做的事情单一了，比如取指部件只负责从指令存储器中取出指令，因此 CPU 可以进行流水工作，也相当于一个时钟周期完成一条指令。频率更高，依然相当于是一个周期完成一条指令，因此 CPU 可以运行的更快。

本次实验就是将实验六所实现的单周期 CPU 划分为多周期的，并扩展指令到 30 多条。

2. 依据单周期实验的设计框图，将其划分为多个功能块，每个功能块占用一个周期完成，即每个功能块从上一个功能块获取信息，作相关动作，完成后将结果锁存到寄存器中，作为下一个功能块的输入。建议划分为教课书上的 5 个功能块：取指、译码、执行、访存、写回，将理论与实践相结合。

3. 画出划分后的多周期 CPU 的框图，大致框图如图 8.1。从图中可以看出指令每个周期走完一个功能块，进入下一个功能块。标注的 clk 箭头是去往相邻模块的中间锁存器，是因为每个模块的输出需要锁存到寄存器中，下一个模块会从该寄存器中读出数据作为自己的输入，寄存器的锁存是需要时钟控制的。值得注意的是，写回模块所做的就是从访存模块获取要写入寄存器堆的数据和目的寄存器，送往寄存器堆，其所需的 clk 信号是最终发生在寄存器堆的写操作上的。自己设计的框图中要求力求精细，可参考教科书上的 5 级流水的框图。

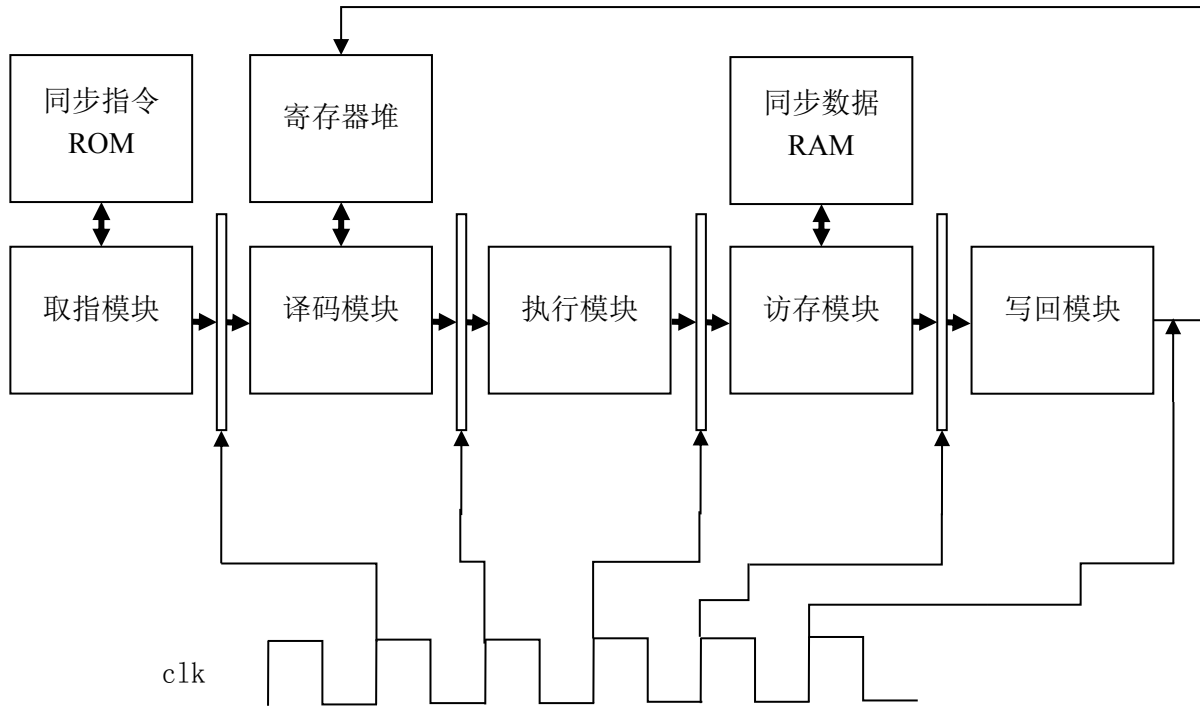


图 8.1 多周期 CPU 的大致框图

4. 本次实验是需要用到之前实验的结果的，比如 ALU 模块、寄存器堆模块、指令 ROM 模块和数据 RAM 模块，其中 ROM 和 RAM 建议使用调用库 IP 实例化的同步存储器，因为存储器在实际应用中基本都是同步读写的，为了更贴近真实情况，此处建议使用同步 RAM 和 ROM。

5. 在实验五中生成的同步 RAM 和 ROM，都是在发送地址后的下一拍才能获得对应数据的。故在在使用同步存储器时，从指令和数据存储器中读取数据就需要等待一拍时钟了，即取指令需要两拍时间，load 操作也需要两拍时间。在真实的处理器系统中，取指令和访存其实都是需要多拍时钟的。

6. 本次实验，同样需要完成表 8.1 和表 8.2 的填写。

表 8.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 rs rt rd 00000 100001	[rs]	[rt]	rd	GPR[rd]=GPR[rs]+GPR[rt]
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)	rt	GPR[rt]=GPR[rs]+sign_ext(imm)
J 型指令	j target	000010 target	PC	target		跳转, PC={PC[31:28],target,2'b00}

表 8.2 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0, #1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 根据设计的实验方案，使用 verilog 编写相应代码。
8. 对编写的代码进行仿真，得到正确的波形图。
9. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 8.2。外围模块中需调用封装好的 LCD 触摸屏模块，观察多周期 CPU 的内部状态，比如 32 个寄存器的值，各模块 PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

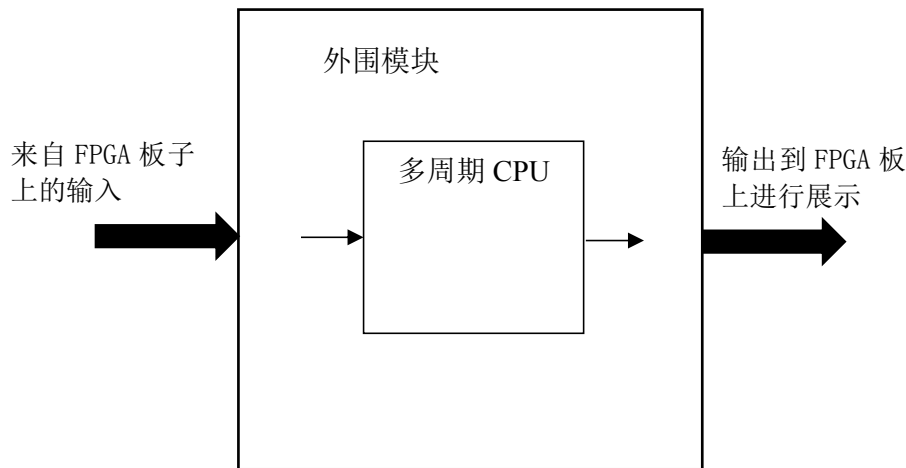


图 8.2 多周期 CPU 设计实验的顶层模块大致框图

10. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：

①：MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度，故在多周期 CPU 中该设定无意义，反而带来了 CPU 实现上的麻烦，故建议在多周期 CPU 中不考虑延迟槽技术。

②：MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4)，而由于多周期不考虑延迟槽，故在实验中分支跳转指令参与计算使用本指令的 PC 值，故跳转的偏移量设置需要注意下。比如一条指令

“beq, r0, r0, #2” 在不考虑延迟槽的多周期 CPU 中，其跳转的目标地址为 beq 指令后

面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。这里需要理解清楚。

③：一般而言控制 CPU 运转的时钟是由 FPGA 板上的时钟输出提供的，但为了方便演示，我们需要在每一个时钟里查看一条指令的运算结果，故演示时理想的时钟是手动输入的，可以使用 FPGA 板上的脉冲开关代替时钟。

4 实验要求

1. 做好预习：

- 1) 复习单周期 CPU 的实验内容，归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令，对其进行分析，完成表 8.1 的填写；
- 2) 依据自己设计中实现的指令，编写一段不少于 40 行的汇编程序，要求包含所有实现的指令，完成表 8.2 的填写；
- 3) 认真学习多周期的概念，了解流水线的概念，明白划分为多周期的意义；
- 4) 认真学习 CPU 各模块的功能，确认模块的划分。设计本次实验的方案，画出实验方案的设计框图，即补充完善图 8.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 8.2。

2. 实验实施：

- 1) 确认多周期 CPU 的设计框图的正确性；
- 2) 编写 verilog 代码，将表 8.2 中自己编写的汇编程序翻译为二进制，以 coe 文件的方式初始化到指令 ROM 中；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料，在仿真时需要将生成指令 ROM 时产生的 .mif 文件拷贝到工程目录下，才能仿真成功；
- 4) 完成调用多周期 CPU 的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 8.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

5.1 实验原理

多周期 CPU 实现的指令集是在单周期设计的基础上增加 20 条算术逻辑运算指令、数据传送指令和控制指令，共实现了 36 条指令。表 8.3 为多周期 CPU 新增的指令。表 8.4 为多周期 CPU 所实现的指令的解析。

表 8.3 多周期 CPU 新增的指令

指令名称	汇编指令	功能描述
无符号小于置位	sltu rd,rs,rt	$GPR[rd] = (zero(GPR[rs]) < zero(GPR[rt]))$
跳转和链接寄存器	jalr rs	$GPR[31] = PC, PC = GPR[rs]$
跳转寄存器	jr rs	$PC = GPR[rs]$
向量逻辑左移字	sllv rd,rt,rs	$GPR[rd] = zero(GPR[rt]) << (GPR[rs] \% 32)$
算术右移字	sra rd,rt,shf	$GPR[rd] = sign(GPR[rt]) >> shf$
向量算术右移字	srav rd,rt,rs	$GPR[rd] = sign(GPR[rt]) >> (GPR[rs] \% 32)$
向量逻辑右移字	srlv rd,rt,rs	$GPR[rd] = zero(GPR[rt]) >> GPR[rs]$
立即数小于置位	slti rt,rs,imm	$GPR[rt] = (sign(GPR[rs]) < sign_ext(imm))$
立即数无符号小于置位	sltiu rt,rs,imm	$GPR[rt] = (zero(GPR[rs]) < sign_ext(imm))$
大于或等于零跳转	bgez rs,offset	if $GPR[rs] \geq 0$ then $PC = PC + sign_ext(offset) << 2$
大于零跳转	bgtz rs,offset	if $GPR[rs] > 0$ then $PC = PC + sign_ext(offset) << 2$
小于或等于零跳转	blez rs,offset	if $GPR[rs] \leq 0$ then $PC = PC + sign_ext(offset) << 2$
小于零跳转	bltz rs,offset	if $GPR[rs] < 0$ then $PC = PC + sign_ext(offset) << 2$
装载字节	lb rt,offset(base)	$GPR[rt] = sign(Mem[GPR[base] + sign_ext(offset)])$
无符号装载字节	lbu rt,offset(base)	$GPR[rt] = zero(Mem[GPR[base] + sign_ext(offset)])$
存储字节	sb rt,offset(base)	$Mem[GPR[base] + sign_ext(offset)] = GPR[rt]$
立即数按位与	andi rt,rs,imm	$GPR[rt] = GPR[rs] \& zero_ext(imm)$
立即数按位或	ori rt,rs,imm	$GPR[rt] = GPR[rs] zero_ext(imm)$
立即数按位异或	xori rt,rs,imm	$GPR[rt] = GPR[rs] \wedge zero_ext(imm)$
跳转和链接	jal target	$GPR[31] = PC, PC = \{PC[31:28], target << 2\}$

表 8.4 多周期 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 0000 0 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 0000 0 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 0000 0 101010	[rs]	[rt]		rd	$GPR[rd] = (sign(GPR[rs]) < sign(GPR[rt]))$
	sltu rd,rs,rt	000000 rs rt rd 0000 0 101011	[rs]	[rt]		rd	$GPR[rd] = (zero(GPR[rs]) < zero(GPR[rt]))$
	jalr rs	000000 rs 00000 111 11 00000 001001	[rs]			31	$GPR[31] = PC, PC = GPR[rs]$
	jr rs	000000 rs 00000000 00 00000 001000	[rs]				$PC = GPR[rs]$

表 8.4 多周期 CPU 实现的 mips 指令特性归纳 (续)

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	and rd,rs,rt	000000 rs rt rd 00000 100100	[rs]	[rt]		rd	GPR[rd]=GPR[rs]&GPR[rt]
	nor rd,rs,rt	000000 rs rt rd 00000 100111	[rs]	[rt]		rd	GPR[rd]=~(GPR[rs] GPR[rt])
	or rd,rs,rt	000000 rs rt rd 00000 100101	[rs]	[rt]		rd	GPR[rd]=GPR[rs] GPR[rt]
	xor rd,rs,rt	000000 rs rt rd 00000 100110	[rs]	[rt]		rd	GPR[rd]=GPR[rs]^GPR[rt]
	sll rd,rt,shf	000000 00000 rt rd shf 000000		[rt]		rd	GPR[rd]=zero(GPR[rt])<<shf
	sllv rd,rt,rs	000000 rs rt rd 00000 000100	[rs]	[rt]		rd	GPR[rd]=zero(GPR[rt])<<(GPR[rs]%32)
	sra rd,rt,shf	000000 00000 rt rd shf 000011		[rt]		rd	GPR[rd]=sign(GPR[rt])>>shf
	srav rd,rt,rs	000000 rs rt rd 00000 000111	[rs]	[rt]		rd	GPR[rd]=sign(GPR[rt])>>(GPR[rs]%32)
	srl rd,rt,shf	000000 00000 rt rd shf 000010		[rt]		rd	GPR[rd]=zero(GPR[rt])>>shf
	srlv rd,rt,rs	000000 rs rt rd 00000 000110	[rs]	[rt]		rd	GPR[rd]=zero(GPR[rt])>>GPR[rs]
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=GPR[rs]+ sign_ext(imm)
	slti rt,rs,imm	001010 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=(sign(GPR[rs])< sign_ext(imm))
	sltiu rt,rs,imm	001011 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=(zero(GPR[rs])< sign_ext(imm))
	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if GPR[rs]=GPR[rt] then PC=PC+sign_ext(offset)<<2
	bgez rs,offset	000001 rs 00001 offset	[rs]				if GPR[rs]≥0 then PC=PC+sign_ext(offset)<<2
	bgtz rs,offset	000111 rs 00000 offset	[rs]				if GPR[rs]>0 then PC=PC+sign_ext(offset)<<2
	blez rs,offset	000110 rs 00000 offset	[rs]				if GPR[rs]≤0 then PC=PC+sign_ext(offset)<<2
	bltz rs,offset	000001 rs 00000 offset	[rs]				if GPR[rs]<0 then PC=PC+sign_ext(offset)<<2
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if GPR[rs]≠GPR[rt] then PC=PC+sign_ext(offset)<<2
	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext(offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext(offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]
	lb rt,offset(b)	100000 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=sign(Mem[GPR[b]+sign_ext(offset)])
	lbu rt,offset(b)	100100 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=zero(Mem[GPR[b]+sign_ext(offset)])
	sb rt,offset(b)	101000 b rt offset	[b]	sign_ext(offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]

andi rt,rs,imm	001100 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]&zero_ext (imm)
lui rt,imm	001111 00000 rt imm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
ori rt,rs,imm	001101 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs] zero_ext (imm)

表 8.4 多周期 CPU 实现的 mips 指令特性归纳 (续)

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	xori rt,rs,imm	001110 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]^zero_ext (imm)
J 型指令	j target	000010 target					PC={PC[31:28],target<<2}
	jal target	000011 target					GPR[31]=PC,PC={PC[31:28],target<<2}

多周期 CPU 设计在单周期 CPU 基础上，主要做两部分改进。第一部分是控制单元，增加控制电路使每一个时钟只有一个阶段的电路产生的结果有效，并锁存上一阶段的结果用于后续阶段的运行；第二部分是数据通路，增加实现新增指令的电路。

第一部分的改进主要是增加状态机控制及增加各阶段之间的用于锁存的寄存器。由于有 5 个阶段，状态机共有 6 个状态：空闲 (IDLE)、取指 (FETCH)、译码 (DECODE)、执行 (EXE)、访存 (MEM)、写回 (WB)，如下图：

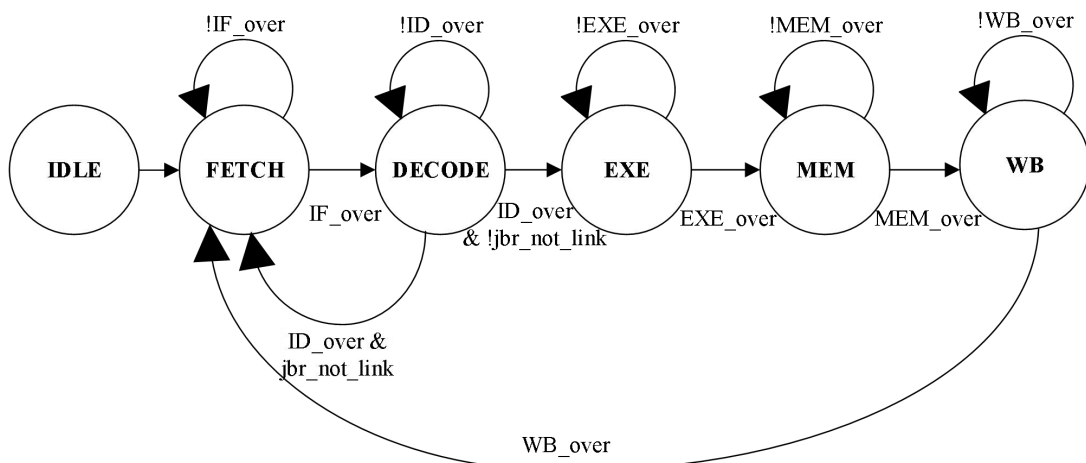


图 8.3 多周期 CPU 的状态机

空闲 (IDLE) 状态：CPU 在复位时，所有阶段电路都无效，CPU 等待复位结束开始下一状态——取指。

取指 (FETCH) 状态：在状态机进入取指状态的同时，PC 更新为下一 PC 值。故取指状态下，将 PC 值作为指令存储器的地址去取指令。由于同步指令存储器在下一时钟周期返回指令，因此取指需要两个时钟周期的时间。当取指结束后锁存取指阶段产生的结果——当前 PC 值和指令。

译码 (DECODE) 状态：类似于单周期 CPU 的译码阶段，主要完成指令译码、读寄存器、判断跳转等。控制单元区分各条指令并产生用于译码、执行、访存、写回的控制信号。当译码结束后锁存译码阶段产生的结果用于下一状态执行：分别用于执行、

访存、写回的控制信号、用于执行阶段的两个源操作数、用于访存阶段的写入内存数据、用于写回阶段的写寄存器地址。

执行（EXE）状态：ALU 模块完成操作。当执行结束后锁存执行阶段产生的结果及前级传递的结果：用于访存、写回的控制信号、ALU 结果、内存写入数据、寄存器写地址。

访存（MEM）状态：完成对数据存储器的读或写，并选择出将要写回寄存器的值。当访存结束后锁存访存阶段产生的结果及前级传递的结果：用于写回的控制信号、写回数据、写回地址。

写回（WB）状态：完成寄存器写入。

CPU 复位结束，状态机由 IDLE 进入取指状态，其后在每次上一级结束信号有效的时候进入下一状态，写回级结束后返回取指级。当然也有例外，当指令是跳转而非链接跳转指令时，在译码状态后直接返回取下一指令不需要经过执行等后续阶段。

多周期 CPU 的实现框图如下：

利用多周期 CPU 实现的 36 条指令，可以编写一段汇编程序，用于测试实现的多周期 CPU 的正确性。表 8.5 给出了一个测试程序。

表 8.5 多周期 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
04H	sll \$2, \$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
08H	addu \$3, \$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
0CH	srl \$4, \$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
10H	slti \$25,\$4,#5	[\$25] = 0000_0001H	28990005	0010_1000_1001_1001_0000_0000_0000_0101
14H	bgez \$25,#16	跳转到 54H	07210010	0000_0111_0010_0001_0000_0000_0001_0000
18H	subu \$5, \$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
1CH	sw \$5, #20(\$0)	Mem[0000_0014H] = 0000_000DH	AC050014	1010_1100_0000_0101_0000_0000_0001_0100
20H	nor \$6, \$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
24H	or \$7, \$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101
28H	xor \$8, \$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
2CH	sw \$8, #28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
30H	beq \$8, \$3,#2	跳转到 38H	11030002	0001_0001_0000_0011_0000_0000_0000_0010
34H	slt \$9, \$6,\$7	不执行	00C7482A	0000_0000_1100_0111_0100_1000_0010_1010
38H	addiu \$1, \$0,#8	[\$1] = 0000_0008H	24010008	0010_0100_0000_0001_0000_0000_0000_1000
3CH	lw \$10,#20(\$1)	[\$10] = 0000_0011H	8C2A0014	1000_1100_0010_1010_0000_0000_0001_0100
40H	bne \$10,\$5,#4	跳转到 50H	15450004	0001_0101_0100_0101_0000_0000_0000_0100
44H	and \$11,\$2,\$1	不执行	00415824	0000_0000_0100_0001_0101_1000_0010_0100
48H	sw \$11,#28(\$1)	不执行	AC2B001C	1010_1100_0010_1011_0000_0000_0001_1100
4CH	sw \$4, #16(\$1)	不执行	AC240010	1010_1100_0010_0100_0000_0000_0001_0000
50H	jal #25	跳转到 64H, [\$31] = 0000_0054H	0C000019	0000_1100_0000_0000_0000_0000_0001_1001
54H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
58H	sra \$26,\$12,\$2	[\$26] = 0000_000CH	004CD007	0000_0000_0100_1100_1101_0000_0000_0111
5CH	sllv \$27,\$26,\$1	[\$27] = 0000_0018H	003AD804	0000_0000_0011_1010_1101_1000_0000_0100
60H	jalr \$27	跳转到 18H, [\$31] = 0000_0064H	0360F809	0000_0011_0110_0000_1111_1000_0000_1001
64H	sb \$26,#5(\$3)	MEM[0000_0016H] = 000C_000DH	A07A0005	1010_0000_0111_1010_0000_0000_0000_0101
68H	sltu \$13,\$3,\$3	[\$13] = 0000_0000H	0063682B	0000_0000_0110_0011_0110_1000_0010_1011
6CH	bgtz \$13,#3	不跳转	1DA00003	0001_1101_1010_0000_0000_0000_0000_0011
70H	sllv \$14,\$6,\$4	[\$14] = FFFF_FE20H	00867004	0000_0000_1000_0110_0111_0000_0000_0100
74H	sra \$15,\$14,#2	[\$15] = FFFF_FF88H	000E7883	0000_0000_0000_1110_0111_1000_1000_0011
78H	srlv \$16,\$15,\$1	[\$16] = 00FF_FFFFH	002F8006	0000_0000_0010_1111_1000_0000_0000_0110
7CH	blez \$16,#8	不跳转	1A000008	0001_1010_0000_0000_0000_0000_0000_1000
80H	sra \$16,\$15,\$1	[\$16] = FFFF_FFFFH	002F8007	0000_0000_0010_1111_1000_0000_0000_0111

表 8.5 多周期 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
84H	addiu \$11,\$0,#140	[\$11] = 0000_008CH	240B008C	0010_0100_0000_1011_0000_0000_1000_1100
88H	bltz \$16,#6	跳转到 A0H	06000006	0000_0110_0000_0000_0000_0000_0000_0110
8CH	lw \$28,#3(\$10)	[\$28] = 000C_000DH / 000C_880DH	8D5C0003	1000_1101_0101_1100_0000_0000_0000_0011
90H	bne \$28,\$29,#7	不跳转/跳转 ACH	179D0007	0001_0111_1001_1101_0000_0000_0000_0111
94H	sb \$15,#8(\$5)	Mem[0000_0015H] = 0000_0088H	A0AF0008	1010_0000_1010_1111_0000_0000_0000_1000
98H	lb \$18,#8(\$5)	[\$18] = FFFF_FF88H	80B20008	1000_0000_1011_0010_0000_0000_0000_1000
9CH	lbu \$19,#8(\$5)	[\$19] = 0000_0088H	90B30008	1001_0000_1011_0011_0000_0000_0000_1000
A0H	sltiu \$24,\$15,#0xFFFF	[\$24] = 0000_0001H	2DF8FFFF	0010_1101_1111_1000_1111_1111_1111_1111
A4H	or \$29,\$12,\$5	[\$29] = 000C000DH	0185E825	0000_0001_1000_0101_1110_1000_0010_0101
A8H	jr \$11	跳转指令 8CH	01600008	0000_0001_0110_0000_0000_0000_0000_1000
ACH	andi \$20,\$15,#0xFFFF	[\$20] = 0000_FF88H	31F4FFFF	0011_0001_1111_0100_1111_1111_1111_1111
B0H	ori \$21,\$15,#0xFFFF	[\$21] = FFFF_FFFFH	35F5FFFF	0011_0101_1111_0101_1111_1111_1111_1111
B4H	xori \$22,\$15,#0xFFFF	[\$22] = FFFF_0077H	39F6FFFF	0011_1001_1111_0110_1111_1111_1111_1111
B8H	j #00H	跳转指令 00H	08000000	0000_1000_0000_0000_0000_0000_0000_0000

5.2 实验顶层模块框图

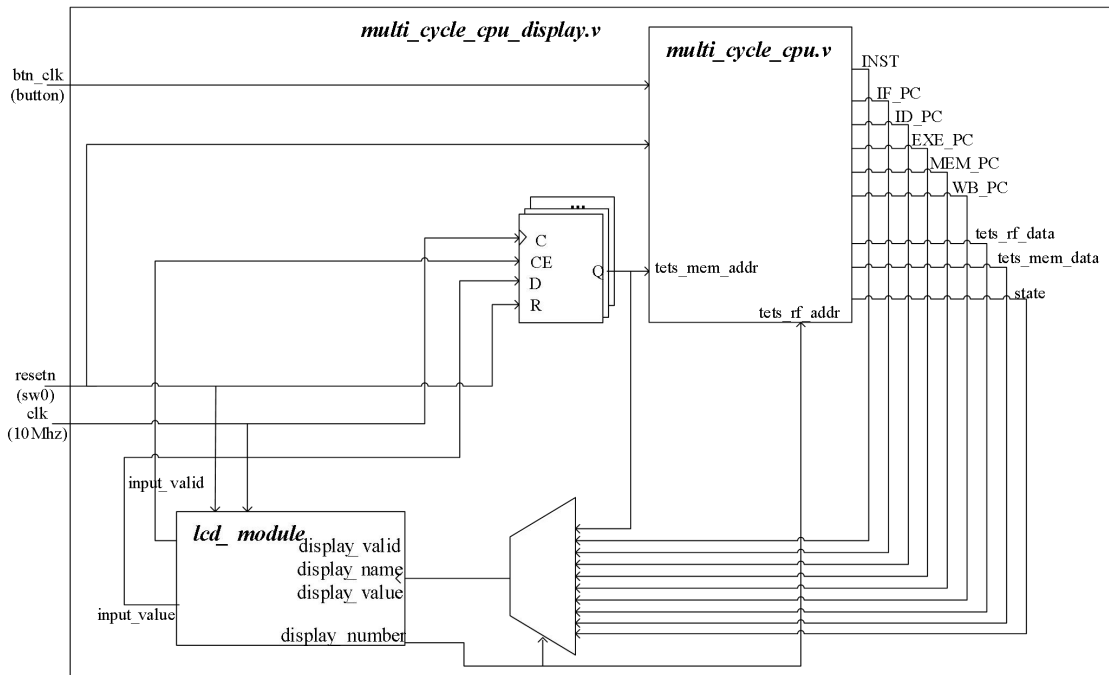


图 8.5 多周期 CPU 参考设计的顶层模块框图

5.3 源代码

multi_cycle_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
  *
```

```
3 // > 文件名: multi_cycle_cpu_display.v
4 // > 描述 :多周期 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 //*****
8 module multi_cycle_cpu_display( // 多周期 cpu
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14     input btn_clk,
15
16     //触摸屏相关接口, 不需要更改
17     output lcd_rst,
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改, 用于单步调试
31     wire cpu_clk; //单周期 CPU 里使用脉冲开关作为时钟, 以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37             begin
38                 btn_clk_r1<= 1'b0;
39             end
40         else
41             begin
42                 btn_clk_r1 <= ~btn_clk;
43             end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk), .CE(clk_en), .O(cpu_clk));
51 //-----{时钟和复位信号}end
52 //-----{调用多周期 CPU 模块}begin
```

```
53 //用于在 FPGA 板上显示结果
54 wire [ 4:0] rf_addr; //扫描寄存器堆的地址
55 wire [31:0] rf_data; //寄存器堆从调试端口读出的数据
56 reg [31:0] mem_addr; //要观察的内存地址
57 wire [31:0] mem_data; //内存地址对应的数据
58 wire [31:0] IF_pc; //IF 模块的 PC
59 wire [31:0] IF_inst; //IF 模块取出的指令
60 wire [31:0] ID_pc; //ID 模块的 PC
61 wire [31:0] EXE_pc; //EXE 模块的 PC
62 wire [31:0] MEM_pc; //MEM 模块的 PC
63 wire [31:0] WB_pc; //WB 模块的 PC
64 wire [31:0] display_state; //展示 CPU 当前状态
65 multi_cycle_cpu cpu(
66     .clk (cpu_clk ),
67     .resetn (resetn ),
68
69     .rf_addr (rf_addr ),
70     .mem_addr(mem_addr),
71     .rf_data (rf_data ),
72     .mem_data(mem_data),
73     .IF_pc (IF_pc ),
74     .IF_inst (IF_inst ),
75     .ID_pc (ID_pc ),
76     .EXE_pc (EXE_pc ),
77     .MEM_pc (MEM_pc ),
78     .WB_pc (WB_pc ),
79     .display_state (display_state)
80 );
81 //-----{调用单周期 CPU 模块}end
82
83 //-----{调用触摸屏模块}begin-----//
84 //-----{实例化触摸屏}begin
85 //此小节不需要更改
86 reg display_valid;
87 reg [39:0] display_name;
88 reg [31:0] display_value;
89 wire [5 :0] display_number;
90 wire input_valid;
91 wire [31:0] input_value;
92
93 lcd_module lcd_module(
94     .clk (clk ), //10Mhz
95     .resetn (resetn ),
96
97     //调用触摸屏的接口
98     .display_valid (display_valid ),
99     .display_name (display_name ),
100     .display_value (display_value ),
101     .display_number (display_number),
102     .input_valid (input_valid ),
103     .input_value (input_value ),
```

```
104
105      //lcd 触摸屏相关接口，不需要更改
106      .lcd_rst      (lcd_rst      ),
107      .lcd_cs       (lcd_cs       ),
108      .lcd_rs        (lcd_rs        ),
109      .lcd_wr        (lcd_wr        ),
110      .lcd_rd        (lcd_rd        ),
111      .lcd_data_io   (lcd_data_io   ),
112      .lcd_bl_ctr    (lcd_bl_ctr    ),
113      .ct_int        (ct_int        ),
114      .ct_sda        (ct_sda        ),
115      .ct_scl        (ct_scl        ),
116      .ct_rstn       (ct_rstn       )
117  );
118  //-----{实例化触摸屏}end
119
120  //-----{从触摸屏获取输入}begin
121  //根据实际需要输入的数修改此小节，
122  //建议对每一个数的输入，编写单独一个 always 块
123      always @(posedge clk)
124      begin
125          if (!resetn)
126          begin
127              mem_addr <= 32'd0;
128          end
129          else if (input_valid)
130          begin
131              mem_addr <= input_value;
132          end
133      end
134      assign rf_addr = display_number-6'd11;
135  //-----{从触摸屏获取输入}end
136
137  //-----{输出到触摸屏显示}begin
138  //根据需要显示的数修改此小节，
139  //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
140  //44 块显示区域从 1 开始编号，编号为 1~44，
141      always @(posedge clk)
142      begin
143          if (display_number >6'd10 && display_number <6'd43 )
144          begin //块号 5~36 显示 32 个通用寄存器的值
145              display_valid <= 1'b1;
146              display_name[39:16] <= "REG";
147              display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
148              display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
149              display_value      <= rf_data;
150          end
151          else
152          begin
153              case(display_number)
154                  6'd1 : //显示 IF 模块的 PC
```

```
155         begin
156             display_valid <= 1'b1;
157             display_name  <= "IF_PC";
158             display_value <= IF_pc;
159         end
160         6'd2 : //显示 IF 模块的指令
161         begin
162             display_valid <= 1'b1;
163             display_name  <= "IF_IN";
164             display_value <= IF_inst;
165         end
166         6'd3 : //显示 ID 模块的 PC
167         begin
168             display_valid <= 1'b1;
169             display_name  <= "ID_PC";
170             display_value <= ID_pc;
171         end
172         6'd4 : //显示 EXE 模块的 PC
173         begin
174             display_valid <= 1'b1;
175             display_name  <= "EXEPC";
176             display_value <= EXE_pc;
177         end
178         6'd5 : //显示 MEM 模块的 PC
179         begin
180             display_valid <= 1'b1;
181             display_name  <= "MEMPC";
182             display_value <= MEM_pc;
183         end
184         6'd6 : //显示 WB 模块的 PC
185         begin
186             display_valid <= 1'b1;
187             display_name  <= "WB_PC";
188             display_value <= WB_pc;
189         end
190         6'd7 : //显示要观察的内存地址
191         begin
192             display_valid <= 1'b1;
193             display_name  <= "MADDR";
194             display_value <= mem_addr;
195         end
196         6'd8 : //显示该内存地址对应的数据
197         begin
198             display_valid <= 1'b1;
199             display_name  <= "MDATA";
200             display_value <= mem_data;
201         end
202         6'd9 : //显示 CPU 当前状态
203         begin
204             display_valid <= 1'b1;
205             display_name  <= "STATE";
```



```
206             display_value <= display_state;
207         end
208         default :
209             begin
210                 display_valid <= 1'b0;
211                 display_name  <= 40'd0;
212                 display_value <= 32'd0;
213             end
214         endcase
215     end
216 end
217 //-----{输出到触摸屏显示}end
218 //-----{调用触摸屏模块}end-----//
219 endmodule
```

multi_cycle_cpu.v 源代码:

```
1  `timescale 1ns / 1ps
2  /**
3   > 文件名: multi_cycle_cpu.v
4   > 描述   :多周期 CPU 模块, 共实现 36 条指令
5   >         指令 rom 和数据 ram 均实例化 xilinx IP 得到, 为同步读写
6   > 作者    : LOONGSON
7   > 日期     : 2016-04-14
8   /**
9  module multi_cycle_cpu( // 多周期 cpu
10     input clk,           // 时钟
11     input resetn,        // 复位信号, 低电平有效
12
13     //display data
14     input  [ 4:0] rf_addr,
15     input  [31:0] mem_addr,
16     output [31:0] rf_data,
17     output [31:0] mem_data,
18     output [31:0] IF_pc,
19     output [31:0] IF_inst,
20     output [31:0] ID_pc,
21     output [31:0] EXE_pc,
22     output [31:0] MEM_pc,
23     output [31:0] WB_pc,
24     output [31:0] display_state
25 );
26 //-----{控制多周期的状态机}begin-----
27 //
28     reg [2:0] state;           // 当前状态
29     reg [2:0] next_state;     // 下一状态
30
31     //展示当前处理器正在执行哪个模块
32     assign display_state = {29'd0,state};
33     // 状态机状态
34     parameter IDLE = 3'd0;    // 开始
```

```
34     parameter FETCH  = 3'd1;  // 取指
35     parameter DECODE = 3'd2;  // 译码
36     parameter EXE    = 3'd3;  // 执行
37     parameter MEM     = 3'd4;  // 访存
38     parameter WB      = 3'd5;  // 写回
39
40     always @ (posedge clk)      // 当前状态
41     begin
42         if (!resetn) begin      // 如果复位信号有效
43             state <= IDLE;      // 当前状态为 开始
44         end
45         else begin              // 否则
46             state <= next_state; // 为下一状态
47         end
48     end
49
50     wire IF_over;      // IF 模块已执行完
51     wire ID_over;      // ID 模块已执行完
52     wire EXE_over;     // EXE 模块已执行完
53     wire MEM_over;     // MEM 模块已执行完
54     wire WB_over;      // WB 模块已执行完
55     wire jbr_not_link; //分支指令(非 link 类), 只走 IF 和 ID 级
56     always @ (*)      // 下一状态
57     begin
58         case (state)
59             IDLE :
60                 begin
61                     next_state = FETCH;    // 开始->取指
62                 end
63             FETCH:
64                 begin
65                     if (IF_over)
66                     begin
67                         next_state = DECODE;    // 取指->译码
68                     end
69                     else
70                     begin
71                         next_state = FETCH;    // 取指->译码
72                     end
73                 end
74             DECODE:
75                 begin
76                     if (ID_over)
77                     begin
78                         // 译码->执行或写回
79                         next_state = jbr_not_link ? FETCH : EXE;
80                     end
81                     else
82                     begin
83                         next_state = DECODE;    // 取指->译码
84                     end
85                 end
86             EXE :
87                 begin
88                     next_state = WB;
89                 end
90             MEM :
91                 begin
92                     next_state = WB;
93                 end
94             WB :
95                 begin
96                     next_state = WB;
97                 end
98         endcase
99     end
```

```

85         EXE:
86         begin
87             if (EXE_over)
88                 begin
89                     next_state = MEM;          // 执行->访存
90                 end
91             else
92                 begin
93                     next_state = EXE;          // 取指->译码
94                 end
95             end
96         MEM:
97         begin
98             if (MEM_over)
99                 begin
100                     next_state = WB;          // 访存->写回
101                 end
102             else
103                 begin
104                     next_state = MEM;          // 取指->译码
105                 end
106             end
107         WB:
108         begin
109             if (WB_over)
110                 begin
111                     next_state = FETCH;        // 写回->取指
112                 end
113             else
114                 begin
115                     next_state = WB;          // 取指->译码
116                 end
117             end
118         default : next_state = IDLE;
119     endcase
120 end
121 //5 模块的 valid 信号
122 wire IF_valid;
123 wire ID_valid;
124 wire EXE_valid;
125 wire MEM_valid;
126 wire WB_valid;
127 assign IF_valid = (state == FETCH ); //当前状态为取指时, IF 级有效
128 assign ID_valid = (state == DECODE); //当前状态为译码时, ID 级有效
129 assign EXE_valid = (state == EXE ); //当前状态为执行时, EXE 级有效
130 assign MEM_valid = (state == MEM ); //当前状态为访存时, MEM 级有效
131 assign WB_valid = (state == WB ); //当前状态为写回时, WB 级有效
132 //-----{控制多周期的状态机}end-----

```

```
//
133
134 //-----{5 级间的总线}begin-----
//
135     wire [ 63:0] IF_ID_bus;    // IF->ID 级总线
136     wire [149:0] ID_EXE_bus;  // ID->EXE 级总线
137     wire [105:0] EXE_MEM_bus; // EXE->MEM 级总线
138     wire [ 69:0] MEM_WB_bus;  // MEM->WB 级总线
139
140     //锁存以上总线信号
141     reg [ 63:0] IF_ID_bus_r;
142     reg [149:0] ID_EXE_bus_r;
143     reg [105:0] EXE_MEM_bus_r;
144     reg [ 69:0] MEM_WB_bus_r;
145
146     //IF 到 ID 的锁存信号
147     always @(posedge clk)
148     begin
149         if(IF_over)
150         begin
151             IF_ID_bus_r <= IF_ID_bus;
152         end
153     end
154     //ID 到 EXE 的锁存信号
155     always @(posedge clk)
156     begin
157         if(ID_over)
158         begin
159             ID_EXE_bus_r <= ID_EXE_bus;
160         end
161     end
162     //EXE 到 MEM 的锁存信号
163     always @(posedge clk)
164     begin
165         if(EXE_over)
166         begin
167             EXE_MEM_bus_r <= EXE_MEM_bus;
168         end
169     end
170     //MEM 到 WB 的锁存信号
171     always @(posedge clk)
172     begin
173         if(MEM_over)
174         begin
175             MEM_WB_bus_r <= MEM_WB_bus;
176         end
177     end
178 //-----{5 级间的总线}end-----
//
179
180 //-----{其他交互信号}begin-----
```

```

//
181 //跳转总线
182 wire [ 32:0] jbr_bus;
183
184 //IF 与 inst_rom 交互
185 wire [31:0] inst_addr;
186 wire [31:0] inst;
187
188 //MEM 与 data_ram 交互
189 wire [ 3:0] dm_wen;
190 wire [31:0] dm_addr;
191 wire [31:0] dm_wdata;
192 wire [31:0] dm_rdata;
193
194 //ID 与 regfile 交互
195 wire [ 4:0] rs;
196 wire [ 4:0] rt;
197 wire [31:0] rs_value;
198 wire [31:0] rt_value;
199
200 //WB 与 regfile 交互
201 wire rf_wen;
202 wire [ 4:0] rf_wdest;
203 wire [31:0] rf_wdata;
204 //-----{其他交互信号}end-----
205 //
206 //-----{各模块实例化}begin-----
207 //
208 wire next_fetch; //即将运行取指模块, 需要先锁存 PC 值
209 //当前状态为 decode, 且指令为跳转分支指令(非 link 类), 且 decode 执行完
210 成
211 //或者, 当前状态为 wb, 且 wb 执行完成, 则即将进入 fetch 状态
212 assign next_fetch = (state==DECODE & ID_over &
213 jbr_not_link)
214 | (state==WB & WB_over);
215
216 fetch IF_module( // 取指级
217 .clk (clk ), // I, 1
218 .resetn (resetn ), // I, 1
219 .IF_valid (IF_valid ), // I, 1
220 .next_fetch(next_fetch), // I, 1
221 .inst (inst ), // I, 32
222 .jbr_bus (jbr_bus ), // I, 33
223 .inst_addr (inst_addr ), // O, 32
224 .IF_over (IF_over ), // O, 1
225 .IF_ID_bus (IF_ID_bus ), // O, 64
226
227 //展示 PC 和取出的指令
228 .IF_pc (IF_pc ),
229 .IF_inst (IF_inst )
230 );

```

```

227
228     decode ID_module(                                // 译码级
229         .ID_valid      (ID_valid      ), // I, 1
230         .IF_ID_bus_r    (IF_ID_bus_r    ), // I, 64
231         .rs_value       (rs_value       ), // I, 32
232         .rt_value       (rt_value       ), // I, 32
233         .rs              (rs              ), // O, 5
234         .rt              (rt              ), // O, 5
235         .jbr_bus        (jbr_bus        ), // O, 33
236         .jbr_not_link   (jbr_not_link   ), // O, 1
237         .ID_over        (ID_over        ), // O, 1
238         .ID_EXE_bus     (ID_EXE_bus     ), // O, 150
239
240         //展示 PC
241         .ID_pc          (ID_pc          )
242     );
243
244     exe EXE_module(                                    // 执行级
245         .EXE_valid      (EXE_valid      ), // I, 1
246         .ID_EXE_bus_r   (ID_EXE_bus_r   ), // I, 150
247         .EXE_over       (EXE_over       ), // O, 1
248         .EXE_MEM_bus    (EXE_MEM_bus    ), // O, 106
249
250         //展示 PC
251         .EXE_pc         (EXE_pc         )
252     );
253
254     mem MEM_module(                                    // 访存级
255         .clk             (clk             ), // I, 1
256         .MEM_valid       (MEM_valid       ), // I, 1
257         .EXE_MEM_bus_r   (EXE_MEM_bus_r   ), // I, 106
258         .dm_rdata        (dm_rdata        ), // I, 32
259         .dm_addr         (dm_addr         ), // O, 32
260         .dm_wen          (dm_wen          ), // O, 4
261         .dm_wdata        (dm_wdata        ), // O, 32
262         .MEM_over        (MEM_over        ), // O, 1
263         .MEM_WB_bus      (MEM_WB_bus      ), // O, 70
264
265         //展示 PC
266         .MEM_pc         (MEM_pc         )
267     );
268
269     wb WB_module(                                       // 写回级
270         .WB_valid       (WB_valid       ), // I, 1
271         .MEM_WB_bus_r   (MEM_WB_bus_r   ), // I, 70
272         .rf_wen         (rf_wen         ), // O, 1
273         .rf_wdest       (rf_wdest       ), // O, 5
274         .rf_wdata       (rf_wdata       ), // O, 32
275         .WB_over        (WB_over        ), // O, 1
276
277         //展示 PC

```

```

278         .WB_pc      (WB_pc      )
279     );
280
281     inst_rom inst_rom_module(          // 指令存储器
282         .clk         (clk         ), // I, 1 ,时钟
283         .addra       (inst_addr[9:2]), // I, 8 ,指令地址
284         .douta       (inst        )  // O, 32,指令
285     );
286
287     regfile rf_module(          // 寄存器堆模块
288         .clk         (clk         ), // I, 1
289         .wen         (rf_wen      ), // I, 1
290         .raddr1      (rs          ), // I, 5
291         .raddr2      (rt          ), // I, 5
292         .waddr       (rf_wdest   ), // I, 5
293         .wdata       (rf_wdata   ), // I, 32
294         .rdata1      (rs_value   ), // O, 32
295         .rdata2      (rt_value   ), // O, 32
296
297         //display rf
298         .test_addr(rf_addr),
299         .test_data(rf_data)
300     );
301
302     data_ram data_ram_module(      // 数据存储模块
303         .clk         (clk         ), // I, 1, 时钟
304         .wea         (dm_wen      ), // I, 1, 写使能
305         .addra       (dm_addr[9:2]), // I, 8, 读地址
306         .dina        (dm_wdata    ), // I, 32, 写数据
307         .douta       (dm_rdata    ), // O, 32, 读数据
308
309         //display mem
310         .clkb        (clk         ),
311         .web         (4'd0        ),
312         .addrb       (mem_addr[9:2]),
313         .doutb       (mem_data    ),
314         .dinb        (32'd0       )
315     );
316     //-----{各模块实例化}end-----
317     endmodule

```

fetch.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3  //  > 文件名: fetch.v
4  //  > 描述   :多周期 CPU 的取指模块
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  /**

```



```
8 `define STARTADDR 32'd0 // 程序起始地址为 0
9 module fetch( // 取指级
10     input          clk, // 时钟
11     input          resetn, // 复位信号, 低电平有效
12     input          IF_valid, // 取指级有效信号
13     input          next_fetch, // 取下一条指令, 用来锁存 PC 值
14     input [31:0] inst, // inst_rom 取出的指令
15     input [32:0] jbr_bus, // 跳转总线
16     output [31:0] inst_addr, // 发往 inst_rom 的取指地址
17     output reg IF_over, // IF 模块执行完成
18     output [63:0] IF_ID_bus, // IF->ID 总线
19
20     //展示 PC 和取出的指令
21     output [31:0] IF_pc,
22     output [31:0] IF_inst
23 );
24
25 //-----{程序计数器 PC}begin
26     wire [31:0] next_pc;
27     wire [31:0] seq_pc;
28     reg [31:0] pc;
29     //跳转 pc
30     wire jbr_taken;
31     wire [31:0] jbr_target;
32     assign {jbr_taken, jbr_target} = jbr_bus; //跳转总线
33
34     assign seq_pc[31:2] = pc[31:2] + 1'b1; //下一指令地址:
PC=PC+4
35     assign seq_pc[1 :0] = pc[1:0];
36
37     // 新指令: 若指令跳转, 为跳转地址; 否则为下一指令
38     assign next_pc = jbr_taken ? jbr_target : seq_pc;
39
40     always @(posedge clk) // PC 程序计数器
41     begin
42         if (!resetn)
43         begin
44             pc <= `STARTADDR; // 复位, 取程序起始地址
45         end
46         else if (next_fetch)
47         begin
48             pc <= next_pc; // 不复位, 取新指令
49         end
50     end
51 //-----{程序计数器 PC}end
52
53 //-----{发往 inst_rom 的取指地址}begin
54     assign inst_addr = pc;
55 //-----{发往 inst_rom 的取指地址}end
56
57 //-----{IF 执行完成}begin
```

```

58      //由于指令 rom 为同步读写的,
59      //取数据时, 有一拍延时
60      //即发地址的下一拍时钟才能得到对应的指令
61      //故取指模块需要两拍时间
62      //将 IF_valid 锁存一拍即是 IF_over 信号
63      always @(posedge clk)
64      begin
65          IF_over <= IF_valid;
66      end
67      //如果指令 rom 为异步读的, 则 IF_valid 即是 IF_over 信号,
68      //即取指一拍完成
69      //-----{IF 执行完成}end
70
71      //-----{IF->ID 总线}begin
72      assign IF_ID_bus = {pc, inst};
73      //-----{IF->ID 总线}end
74
75      //-----{展示 IF 模块的 PC 值和指令}begin
76      assign IF_pc      = pc;
77      assign IF_inst = inst;
78      //-----{展示 IF 模块的 PC 值和指令}end
79      endmodule

```

decode.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: decode.v
4   * > 描述   :多周期 CPU 的译码模块
5   * > 作者   : LOONGSON
6   * > 日期   : 2016-04-14
7   */
8  module decode(
9      input          ID_valid,      // 译码级
10     input          [ 63:0] IF_ID_bus_r, // IF->ID 总线
11     input          [ 31:0] rs_value,   // 第一源操作数值
12     input          [ 31:0] rt_value,   // 第二源操作数值
13     output         [ 4:0] rs,          // 第一源操作数地址
14     output         [ 4:0] rt,          // 第二源操作数地址
15     output         [ 32:0] jbr_bus,    // 跳转总线
16     output         jbr_not_link, // 指令为跳转分支指令, 且非 link 类指令
17     output         ID_over,          // ID 模块执行完成
18     output         [149:0] ID_EXE_bus, // ID->EXE 总线
19
20     //展示 PC
21     output         [ 31:0] ID_pc
22 );
23 //-----{IF->ID 总线}begin
24     wire [31:0] pc;
25     wire [31:0] inst;
26     assign {pc, inst} = IF_ID_bus_r; // IF->ID 总线传 PC 和指令

```

```
27 //-----{IF->ID 总线}end
28
29 //-----{指令译码}begin
30     wire [5:0] op;
31     wire [4:0] rd;
32     wire [4:0] sa;
33     wire [5:0] funct;
34     wire [15:0] imm;
35     wire [15:0] offset;
36     wire [25:0] target;
37
38     assign op      = inst[31:26]; // 操作码
39     assign rs      = inst[25:21]; // 源操作数 1
40     assign rt      = inst[20:16]; // 源操作数 2
41     assign rd      = inst[15:11]; // 目标操作数
42     assign sa      = inst[10:6];  // 特殊域, 可能存放偏移量
43     assign funct   = inst[5:0];   // 功能码
44     assign imm     = inst[15:0];  // 立即数
45     assign offset  = inst[15:0];  // 地址偏移量
46     assign target  = inst[25:0];  // 目标地址
47
48     // 实现指令列表
49     wire inst_ADDU, inst_SUBU, inst_SLT, inst_AND;
50     wire inst_NOR, inst_OR, inst_XOR, inst_SLL;
51     wire inst_SRL, inst_ADDIU, inst_BEQ, inst_BNE;
52     wire inst_LW, inst_SW, inst_LUI, inst_J;
53     wire inst_SLTU, inst_JALR, inst_JR, inst_SLLV;
54     wire inst_SRA, inst_SRAV, inst_SRLV, inst_SLTIU;
55     wire inst_SLTI, inst_BGEZ, inst_BGTZ, inst_BLEZ;
56     wire inst_BLTZ, inst_LB, inst_LBU, inst_SB;
57     wire inst_ANDI, inst_ORI, inst_XORI, inst_JAL;
58     wire op_zero; // 操作码全 0
59     wire sa_zero; // sa 域全 0
60     assign op_zero = ~(|op);
61     assign sa_zero = ~(|sa);
62     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
63     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
64     assign inst_SLT = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
65     assign inst_SLTU = op_zero & sa_zero & (funct == 6'b101011); // 无符号小则置
66     assign inst_JALR = op_zero & (rt==5'd0) & (rd==5'd31)
67         & sa_zero & (funct == 6'b001001); // 跳转寄存器并链接
68     assign inst_JR = op_zero & (rt==5'd0) & (rd==5'd0)
69         & sa_zero & (funct == 6'b001000); // 跳转寄存器
70     assign inst_AND = op_zero & sa_zero & (funct == 6'b100100); // 与运算
71     assign inst_NOR = op_zero & sa_zero & (funct == 6'b100111); // 或非运算
72     assign inst_OR = op_zero & sa_zero & (funct == 6'b100101); // 或运算
73     assign inst_XOR = op_zero & sa_zero & (funct == 6'b100110); // 异或运算
74     assign inst_SLL = op_zero & (rs==5'd0) & (funct == 6'b000000); // 逻辑左移
75     assign inst_SLLV = op_zero & sa_zero & (funct == 6'b000100); // 变量逻辑左移
76     assign inst_SRA = op_zero & (rs==5'd0) & (funct == 6'b000011); // 算术右移
77     assign inst_SRAV = op_zero & sa_zero & (funct == 6'b000111); // 变量算术右移
```

```

78      assign inst_SRL   = op_zero & (rs==5'd0) & (funct == 6'b000010); //逻辑右移
79      assign inst_SRLV  = op_zero & sa_zero   & (funct == 6'b000110); //变量逻辑右移
80      assign inst_ADDIU = (op == 6'b001001);           //立即数无符号加法
81      assign inst_SLTI  = (op == 6'b001010);           //小于立即数则置位
82      assign inst_SLTIU = (op == 6'b001011);           //无符号小于立即数则置位
83      assign inst_BEQ   = (op == 6'b000100);           //判断相等跳转
84      assign inst_BGEZ  = (op == 6'b000001) & (rt==5'd1); //大于等于 0 跳转
85      assign inst_BGTZ  = (op == 6'b000111) & (rt==5'd0); //大于 0 跳转
86      assign inst_BLEZ  = (op == 6'b000110) & (rt==5'd0); //小于等于 0 跳转
87      assign inst_BLTZ  = (op == 6'b000001) & (rt==5'd0); //小于 0 跳转
88      assign inst_BNE   = (op == 6'b000101);           //判断不等跳转
89      assign inst_LW    = (op == 6'b100011);           //从内存装载字
90      assign inst_SW    = (op == 6'b101011);           //向内存存储字
91      assign inst_LB    = (op == 6'b100000);           //load 字节 (符号扩展)
92      assign inst_LBU   = (op == 6'b100100);           //load 字节 (无符号扩展)
93      assign inst_SB    = (op == 6'b101000);           //向内存存储字节
94      assign inst_ANDI  = (op == 6'b001100);           //立即数与
95      assign inst_LUI   = (op == 6'b001111) & (rs==5'd0); //立即数装载高半字节
96      assign inst_ORI   = (op == 6'b001101);           //立即数或
97      assign inst_XORI  = (op == 6'b001110);           //立即数异或
98      assign inst_J     = (op == 6'b000010);           //跳转
99      assign inst_JAL   = (op == 6'b000011);           //跳转和链接
100
101      //跳转分支指令
102      wire inst_jr;      //寄存器跳转指令
103      wire inst_j_link;  //链接跳转指令
104      assign inst_jr     = inst_JALR | inst_JR;
105      assign inst_j_link = inst_JAL  | inst_JALR;
106      assign jbr_not_link= inst_J    | inst_JR      //全部非 link 类跳转指令
107                          | inst_BEQ  | inst_BNE   | inst_BGEZ
108                          | inst_BGTZ | inst_BLEZ | inst_BLTZ;
109
110      //load store
111      wire inst_load;
112      wire inst_store;
113      assign inst_load  = inst_LW | inst_LB | inst_LBU; // load 指
114      assign inst_store = inst_SW | inst_SB;           // store 指
115
116      //alu 操作分类
117      wire inst_add, inst_sub, inst_slt, inst_sltu;
118      wire inst_and, inst_nor, inst_or, inst_xor;
119      wire inst_sll, inst_srl, inst_sra, inst_lui;
120      assign inst_add = inst_ADDU | inst_ADDIU | inst_load
121                      | inst_store | inst_j_link; // 做加法
122      assign inst_sub = inst_SUBU;                // 减法
123      assign inst_slt = inst_SLT | inst_SLTI;      // 有符号小于置
124      assign inst_sltu= inst_SLTIU | inst_SLTU;    // 无符号小于置

```

```

125     assign inst_and = inst_AND | inst_ANDI;           // 逻辑与
126     assign inst_nor = inst_NOR;                       // 逻辑或非
127     assign inst_or  = inst_OR  | inst_ORI;           // 逻辑或
128     assign inst_xor = inst_XOR | inst_XORI;          // 逻辑或非
129     assign inst_sll = inst_SLL | inst_SLLV;          // 逻辑左移
130     assign inst_srl = inst_SRL | inst_SRLV;          // 逻辑右移
131     assign inst_sra = inst_SRA | inst_SRAV;          // 算术右移
132     assign inst_lui = inst_LUI;                       // 立即数装载高
133 位
134     //使用 sa 域作为偏移量的移位指令
135     wire inst_shf_sa;
136     assign inst_shf_sa = inst_SLL | inst_SRL | inst_SRA;
137
138     //依据立即数扩展方式分类
139     wire inst_imm_zero; //立即数 0 扩展
140     wire inst_imm_sign; //立即数符号扩展
141     assign inst_imm_zero = inst_ANDI | inst_LUI | inst_ORI | inst_XORI;
142     assign inst_imm_sign = inst_ADDIU | inst_SLTI | inst_SLTIU
143                          | inst_load | inst_store;
144
145     //依据目的寄存器号分类
146     wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
147     wire inst_wdest_31; // 寄存器堆写入地址为 31 的指令
148     wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令
149     assign inst_wdest_rt = inst_imm_zero | inst_ADDIU |
inst_SLTI
150                          | inst_SLTIU | inst_load;
151     assign inst_wdest_31 = inst_JAL;
152     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU
153                          | inst_JALR | inst_AND | inst_NOR | inst_OR
154                          | inst_XOR | inst_SLL | inst_SLLV | inst_SRA
155                          | inst_SRAV | inst_SRL | inst_SRLV;
156 //-----{指令译码}end
157
158 //-----{分支指令执行}begin
159     //无条件跳转
160     wire j_taken;
161     wire [31:0] j_target;
162     assign j_taken = inst_J | inst_JAL | inst_jr;
163     //寄存器跳转地址为 rs_value, 其他跳转为{pc[31:28],target,2'b00}
164     assign j_target = inst_jr ? rs_value :
{pc[31:28],target,2'b00};
165
166     //branch 指令
167     wire rs_equql_rt;
168     wire rs_ez;
169     wire rs_ltz;
170     assign rs_equql_rt = (rs_value == rt_value); // GPR[rs]==GPR[rt]
171     assign rs_ez = ~(|rs_value); // rs 寄存器值为 0
172     assign rs_ltz = rs_value[31]; // rs 寄存器值小于 0

```

```

173     wire br_taken;
174     wire [31:0] br_target;
175     assign br_taken = inst_BEQ  & rs_equql_rt      // 相等跳转
176                      | inst_BNE  & ~rs_equql_rt    // 不等跳转
177                      | inst_BGEZ & ~rs_ltz         // 大于等于 0 跳
转
178                      | inst_BGTZ & ~rs_ltz & ~rs_ez // 大于 0 跳转
179                      | inst_BLEZ & (rs_ltz | rs_ez) // 小于等于 0 跳
转
180                      | inst_BLTZ & rs_ltz;         // 小于 0 跳转
181     // 分支跳转目标地址: PC=PC+offset<<2
182     assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
183     assign br_target[1:0] = pc[1:0];
184
185     //jump and branch 指令
186     wire jbr_taken;
187     wire [31:0] jbr_target;
188     assign jbr_taken = j_taken | br_taken;
189     assign jbr_target = j_target ? j_target : br_target;
190
191     //ID 到 IF 的跳转总线
192     assign jbr_bus = {jbr_taken, jbr_target};
193 //-----{分支指令执行}end
194
195 //-----{ID 执行完成}begin
196     //由于是多周期的, 不存在数据相关
197     //故 ID 模块一拍就能完成所有操作
198     //故 ID_valid 即是 ID_over 信号
199     assign ID_over = ID_valid;
200 //-----{ID 执行完成}end
201
202 //-----{ID->EXE 总线}begin
203     //EXE 需要用到的信息
204     //ALU 两个源操作数和控制信号
205     wire [11:0] alu_control;
206     wire [31:0] alu_operand1;
207     wire [31:0] alu_operand2;
208
209     //所谓链接跳转是将跳转返回的 PC 值存放到 31 号寄存器里
210     //在多周期 CPU 里, 不考虑延迟槽, 故链接跳转需要计算 PC+4, 存放到 31 号寄存器
里
211     assign alu_operand1 = inst_j_link ? pc :
212                          inst_shf_sa ? {27'd0,sa} : rs_value;
213     assign alu_operand2 = inst_j_link ? 32'd4 :
214                          inst_imm_zero ? {16'd0, imm} :
215                          inst_imm_sign ? {{16{imm[15]}}, imm} : rt_value;
216     assign alu_control = {inst_add,          // ALU 操作码, 独热编码
217                          inst_sub,
218                          inst_slt,
219                          inst_sltu,
220                          inst_and,

```

```

221             inst_nor,
222             inst_or,
223             inst_xor,
224             inst_sll,
225             inst_srl,
226             inst_sra,
227             inst_lui};
228     //访存需要用到的 load/store 信息
229     wire lb_sign; //load 一字节为有符号 load
230     wire ls_word; //load/store 为字节还是字,0:byte;1:word
231     wire [3:0] mem_control; //MEM 需要使用的控制信号
232     wire [31:0] store_data; //store 操作的存的数据
233     assign lb_sign = inst_LB;
234     assign ls_word = inst_LW | inst_SW;
235     assign mem_control = {inst_load,
236                          inst_store,
237                          ls_word,
238                          lb_sign };
239
240     //写回需要用到的信息
241     wire rf_wen; //写回的寄存器写使能
242     wire [4:0] rf_wdest; //写回的目的寄存器
243     assign rf_wen = inst_wdest_rt | inst_wdest_31 |
inst_wdest_rd;
244     assign rf_wdest = inst_wdest_rt ? rt ://在不写寄存器堆时,设置为
0
245                      inst_wdest_31 ? 5'd31 :
246                      inst_wdest_rd ? rd : 5'd0;
247     assign store_data = rt_value;
248     assign ID_EXE_bus = {alu_control,alu_operand1,alu_operand2,//EXE 需要使用的信息
249                        mem_control,store_data, //MEM 需要使用的信号
250                        rf_wen, rf_wdest, //WB 需要使用的信号
251                        pc}; //PC 值
252 //-----{ID->EXE 总线}end
253
254 //-----{展示 ID 模块的 PC 值}begin
255     assign ID_pc = pc;
256 //-----{展示 ID 模块的 PC 值}end
257 endmodule

```

exe.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: exe.v
4  //    > 描述   :多周期 CPU 的执行模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module exe( // 执行级
9      input   EXE_valid, // 执行级有效信号

```



```
10     input      [149:0] ID_EXE_bus_r, // ID->EXE 总线
11     output          EXE_over,      // EXE 模块执行完成
12     output      [105:0] EXE_MEM_bus, // EXE->MEM 总线
13
14     //展示 PC
15     output      [ 31:0] EXE_pc
16 );
17 //-----{ID->EXE 总线}begin
18     //EXE 需要用到的信息
19     //ALU 两个源操作数和控制信号
20     wire [11:0] alu_control;
21     wire [31:0] alu_operand1;
22     wire [31:0] alu_operand2;
23
24     //访存需要用到的 load/store 信息
25     wire [3:0] mem_control; //MEM 需要使用的控制信号
26     wire [31:0] store_data; //store 操作的存的数据
27
28     //写回需要用到的信息
29     wire      rf_wen;      //写回的寄存器写使能
30     wire [4:0] rf_wdest;   //写回的目的寄存器
31
32     //pc
33     wire [31:0] pc;
34     assign {alu_control,
35             alu_operand1,
36             alu_operand2,
37             mem_control,
38             store_data,
39             rf_wen,
40             rf_wdest,
41             pc          } = ID_EXE_bus_r;
42 //-----{ID->EXE 总线}end
43
44 //-----{ALU}begin
45     wire [31:0] alu_result;
46
47     alu alu_module(
48         .alu_control  (alu_control ), // I, 12, ALU 控制信号
49         .alu_src1      (alu_operand1), // I, 32, ALU 操作数 1
50         .alu_src2      (alu_operand2), // I, 32, ALU 操作数 2
51         .alu_result    (alu_result  )  // O, 32, ALU 结果
52     );
53 //-----{ALU}end
54
55 //-----{EXE 执行完成}begin
56     //由于是多周期的, 不存在数据相关
57     //且所有 ALU 运算都可在一拍内完成
58     //故 EXE 模块一拍就能完成所有操作
59     //故 EXE_valid 即是 EXE_over 信号
60     assign EXE_over = EXE_valid;
```

```

61 //-----{EXE 执行完成}end
62
63 //-----{EXE->MEM 总线}begin
64     assign EXE_MEM_bus = {mem_control,store_data,    //load/store 信息和 store 数据
65                           alu_result,                //alu 运算结果
66                           rf_wen,rf_wdest,           // WB 需要使用的信号
67                           pc};                      // PC
68 //-----{EXE->MEM 总线}end
69
70 //-----{展示 EXE 模块的 PC 值}begin
71     assign EXE_pc = pc;
72 //-----{展示 EXE 模块的 PC 值}end
73 endmodule

```

mem.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: mem.v
4   * > 描述 :多周期 CPU 的访存模块
5   * > 作者 : LOONGSON
6   * > 日期 : 2016-04-14
7   */
8  module mem(                                // 访存级
9      input          clk,                    // 时钟
10     input          MEM_valid,              // 访存级有效信号
11     input [105:0] EXE_MEM_bus_r,          // EXE->MEM 总线
12     input [ 31:0] dm_rdata,                // 访存读数据
13     output [ 31:0] dm_addr,                // 访存读写地址
14     output reg [ 3:0] dm_wen,               // 访存写使能
15     output reg [ 31:0] dm_wdata,           // 访存写数据
16     output          MEM_over,              // MEM 模块执行完成
17     output [ 69:0] MEM_WB_bus,             // MEM->WB 总线
18
19     //展示 PC
20     output [ 31:0] MEM_pc
21 );
22 //-----{EXE->MEM 总线}begin
23     //访存需要用到的 load/store 信息
24     wire [3 :0] mem_control; //MEM 需要使用的控制信号
25     wire [31:0] store_data; //store 操作的存的数据
26
27     //alu 运算结果
28     wire [31:0] alu_result;
29
30     //写回需要用到的信息
31     wire rf_wen; //写回的寄存器写使能
32     wire [4:0] rf_wdest; //写回的目的寄存器
33
34     //pc
35     wire [31:0] pc;

```

```
36     assign {mem_control,
37             store_data,
38             alu_result,
39             rf_wen,
40             rf_wdest,
41             pc          } = EXE_MEM_bus_r;
42 //-----{EXE->MEM 总线}end
43
44 //-----{load/store 访存}begin
45     wire inst_load; //load 操作
46     wire inst_store; //store 操作
47     wire ls_word;    //load/store 为字节还是字,0:byte;1:word
48     wire lb_sign;    //load 一字节为有符号 load
49     assign {inst_load,inst_store,ls_word,lb_sign} = mem_control;
50
51     //访存读写地址
52     assign dm_addr = alu_result;
53
54     //store 操作的写使能
55     always @ (*) // 内存写使能信号
56     begin
57         if (MEM_valid && inst_store) // 访存级有效时,且为 store 操作
58         begin
59             if (ls_word)
60             begin
61                 dm_wen <= 4'b1111; // 存储字指令,写使能全 1
62             end
63             else
64             begin // SB 指令,需要依据地址底两位,确定对应的写使能
65                 case (dm_addr[1:0])
66                     2'b00 : dm_wen <= 4'b0001;
67                     2'b01 : dm_wen <= 4'b0010;
68                     2'b10 : dm_wen <= 4'b0100;
69                     2'b11 : dm_wen <= 4'b1000;
70                     default : dm_wen <= 4'b0000;
71                 endcase
72             end
73         end
74         else
75         begin
76             dm_wen <= 4'b0000;
77         end
78     end
79
80     //store 操作的写数据
81     always @ (*) // 对于 SB 指令,需要依据地址低两位,移动 store 的字节至对应位置
82     begin
83         case (dm_addr[1:0])
84             2'b00 : dm_wdata <= store_data;
85             2'b01 : dm_wdata <= {16'd0, store_data[7:0],
86                                     8'd0};
```

```

86      2'b10    : dm_wdata <= {8'd0, store_data[7:0]},
16'd0};
87      2'b11    : dm_wdata <= {store_data[7:0], 24'd0};
88      default : dm_wdata <= store_data;
89      endcase
90  end
91
92      //load 读出的数据
93      wire      load_sign;
94      wire [31:0] load_result;
95      assign load_sign = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7] :
96                          (dm_addr[1:0]==2'd1) ? dm_rdata[15] :
97                          (dm_addr[1:0]==2'd2) ? dm_rdata[23] : dm_rdata[31] ;
98      assign load_result[7:0] = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7:0 ] :
99                          (dm_addr[1:0]==2'd1) ? dm_rdata[15:8 ] :
100                         (dm_addr[1:0]==2'd2) ? dm_rdata[23:16] :
101                         dm_rdata[31:24] ;
102      assign load_result[31:8]= ls_word ? dm_rdata[31:8] :
103                          {24{lb_sign & load_sign}};
104  //-----{load/store 访存}end
105
106  //-----{MEM 执行完成}begin
107      //由于数据 RAM 为同步读写的,
108      //故对 load 指令, 取数据时, 有一拍延时
109      //即发地址的下一拍时钟才能得到 load 的数据
110      //故 mem 在进行 load 操作时有需要两拍时间才能取到数据
111      //而对其他操作, 则只需要一拍时间
112      reg MEM_valid_r;
113      always @(posedge clk)
114      begin
115          MEM_valid_r <= MEM_valid;
116      end
117      assign MEM_over = inst_load ? MEM_valid_r :MEM_valid;
118      //如果数据 ram 为异步读的, 则 MEM_valid 即是 MEM_over 信号,
119      //即 load 一拍完成
120  //-----{MEM 执行完成}end
121
122  //-----{MEM->WB 总线}begin
123      wire [31:0] mem_result; //MEM 传到 WB 的 result 为 load 结果或 ALU 结
果
124      assign mem_result = inst_load ? load_result : alu_result;
125
126      assign MEM_WB_bus = {rf_wen,rf_wdest, // WB 需要使用的信号
127                          mem_result,      // 最终要写回寄存器的数据
128                          pc};             // PC 值
129  //-----{MEM->WB 总线}begin
130
131  //-----{展示 MEM 模块的 PC 值}begin
132      assign MEM_pc = pc;
133  //-----{展示 MEM 模块的 PC 值}end
134  endmodule

```

wb.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: wb.v
4  //    > 描述   :多周期 CPU 的写回模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module wb(                                // 写回级
9      input      WB_valid,                  // 写回级有效
10     input  [69:0] MEM_WB_bus_r,           // MEM->WB 总线
11     output      rf_wen,                    // 寄存器写使能
12     output  [ 4:0] rf_wdest,               // 寄存器写地址
13     output  [31:0] rf_wdata,               // 寄存器写数据
14     output      WB_over,                  // WB 模块执行完成
15
16     //展示 PC
17     output  [ 31:0] WB_pc
18 );
19 //-----{MEM->WB 总线}begin
20     //寄存器堆写使能和写地址
21     wire wen;
22     wire [4:0] wdest;
23
24     //MEM 传来的 result
25     wire [31:0] mem_result;
26
27     //pc
28     wire [31:0] pc;
29     assign {wen,wdest,mem_result,pc} = MEM_WB_bus_r;
30 //-----{MEM->WB 总线}end
31
32 //-----{WB 执行完成}begin
33     //WB 模块只是传递寄存器堆的写使能/写地址和写数据
34     //可在一拍内完成
35     //故 WB_valid 即是 WB_over 信号
36     assign WB_over = WB_valid;
37 //-----{WB 执行完成}end
38
39 //-----{WB->regfile 信号}begin
40     assign rf_wen   = wen & WB_valid;
41     assign rf_wdest = wdest;
42     assign rf_wdata = mem_result;
43 //-----{WB->regfile 信号}end
44
45 //-----{展示 WB 模块的 PC 值}begin
46     assign WB_pc = pc;
47 //-----{展示 WB 模块的 PC 值}end
48 endmodule
```

多周期 CPU 参考设计使用的 inst_rom 和 data_ram 均为实例化的同步存储器 IP。
多周期 CPU 参考设计使用的 regfile.v 和 alu.v，依然是实验三和实验四中给出的代码，
本实验中直接通过“Add Copy of Source”添加到工程中即可，此处就不在附上源码了。
其中 alu.v 还调用了实验一中的 adder.v。

约束文件 multi_cycle_cpu.xdc 源代码：

```
1      #时钟信号连接
2      set_property PACKAGE_PIN AC19 [get_ports clk]
3
4      #脉冲开关，用于输入作为复位信号，低电平有效
5      set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7      #脉冲开关，用于输入作为单步执行的 clk
8      set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10     set_property IOSTANDARD LVCMOS33 [get_ports clk]
11     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12     set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14     #触摸屏引脚连接
15     set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16     set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17     set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18     set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19     set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20     set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21     set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22     set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23     set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24     set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25     set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26     set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27     set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28     set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29     set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30     set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
31     set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32     set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33     set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34     set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35     set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36     set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37     set_property PACKAGE_PIN L19 [get_ports ct_int]
38     set_property PACKAGE_PIN J24 [get_ports ct_sda]
39     set_property PACKAGE_PIN H21 [get_ports ct_scl]
40     set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
```

```
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```


八、 课程设计 静态 5 级流水线 CPU 实现

1 课程设计目的

1. 在多周期 CPU 实验完成的提前下，深入理解 CPU 流水线的概念。
2. 熟悉并掌握流水线 CPU 的原理和设计。
3. 最终检验运用 verilog 语言进行电路设计的能力。
4. 通过亲自设计实现静态 5 级流水线 CPU，加深对计算机组成原理和体系结构理论知识的理解。
5. 培养对 CPU 设计的兴趣，加深对 CPU 现有架构的理解和深思。

2 实验设备

1. 装有 xilinx ISE 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 课程设计任务

1. 本课程设计是对之前课程实验的拔高。前期的课程设计准备同多周期 CPU 的实验，主体部分可以直接使用多周期 CPU 实验的设计方案，但在多周期 CPU 中只要求实现了 30 多条指令，此处要求扩展到 40 多条指令。

多周期 CPU 在单周期基础上提高了时钟频率，但并没有改善执行一条指令的时间，且存在资源闲置的问题，例如当指令在执行级有效时，译码级实际上在空转。若每一级都在执行有效的指令，将解决资源闲置的问题。最理想的情况是，当第一条指令从取指级转换到下一级译码时，第二条指令进入取指级，当第一条指令完成译码进入执行级时，第二条指令进入译码级，第三条指令进入取指级……静态 5 级流水 CPU 就是基于这样的设计思路。

在流水 CPU 中，当在一时钟周期内完成了某一条指令的全部执行时（写回级完成），则有望在下一时钟周期内完成下条指令的执行，因此依然相当于是一个周期完成一条指令，而时钟频率更高，因此 CPU 可以运行的更快。

本次课程就是将上次所实现的多周期 CPU 更改结构为静态 5 级流水线 CPU。

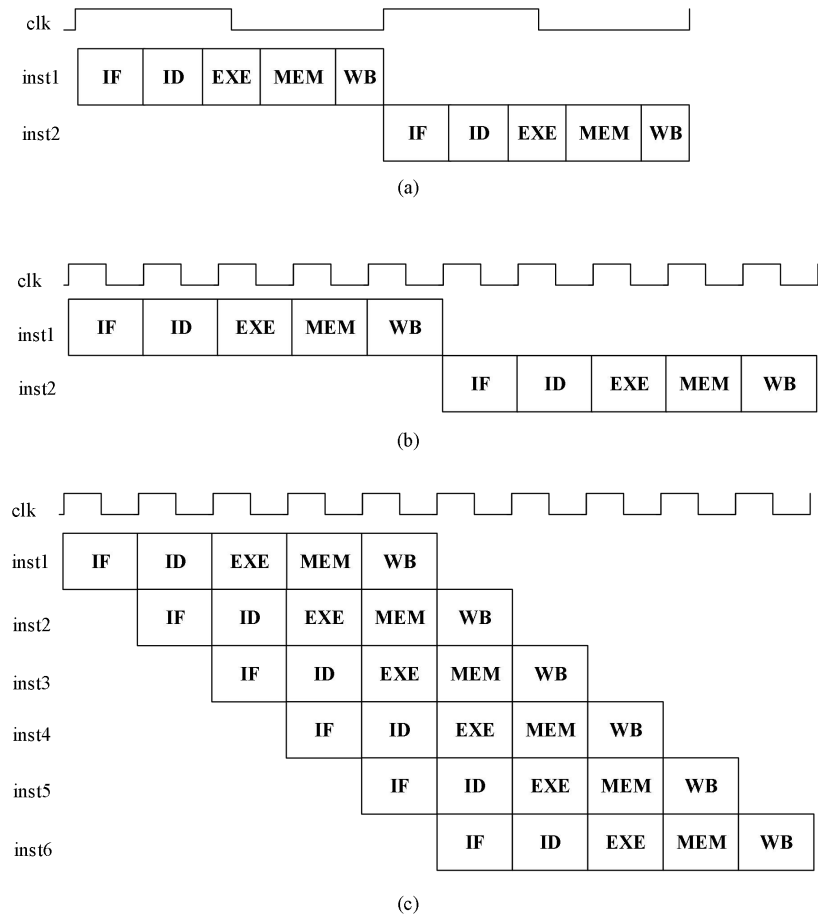


图 9.1 (a)单周期; (b)多周期; (c)5 级流水 CPU 时空图

2. 本次课程设计的设计框图基本同多周期 CPU 实验的设计框图。需要注意 5 个部件都是同时运转的，但对每条指令而言，依然是依次工作的，见图 9.2。

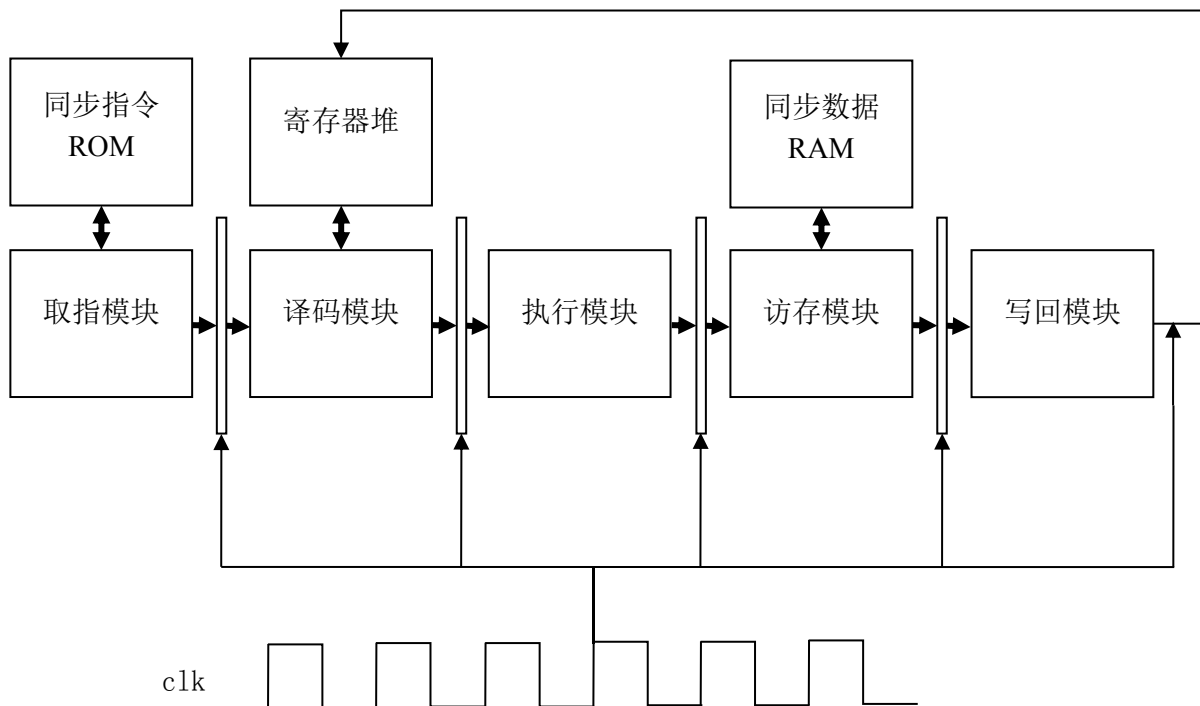


图 9.2 静态 5 级流水 CPU 的大致框图

3. 本次课程设计的关键和难点在于流水线的控制，比如一条指令何时可以从译码级进入执行级，一条指令何时需要堵在流水线中。本次课程设计暂不考虑前递技术，因此有数据相关时就需要堵在流水线中。

4. MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度。在之前单周期和多周期 CPU 实验中未支持延迟槽，但在流水 CPU 中需要支持该设定，因为只有硬件支持了延迟槽技术，用通用编译器编译出来的 MIPS 二进制执行文件才能在自己设计的 CPU 中运行正确。

5. MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4), 在本课程设计中尤其需要注意这一点。比如一条指令

“beq, r0, r0, #2” 在不考虑延迟槽的多周期 CPU 中，其跳转的目标地址为 beq 指令后面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。在编写测试程序时就需要注意分支跳转指令的偏移量。

6. 本次实验，同样需要完成表 9.1 和表 9.2 的填写。

表 9.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 _{rs} _{rt} _{rd} 000001	[rs]	[rt]	rd	GPR[rd]=GPR[rs]+GPR[rt]
I 型指令	addiu rt,rs,imm	001001 _{rs} _{rt} _{imm}	[rs]	sign_ext(imm)	rt	GPR[rt]=GPR[rs]+sign_ext(imm)
J 型指令	j target	000010 target	next_pc	target		跳转, PC={next_pc[31:28],target,2'b00}

表 9.2 测试所用汇编程序描述表

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0, #1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 根据设计的实验方案，使用 verilog 编写相应代码。

8. 对编写的代码进行仿真，得到正确的波形图。

9. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 9.3。外围模块中需调用封装好的 LCD 触摸屏模块，观察 CPU 的内部状态，比如 32 个寄存器的值，各级 PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

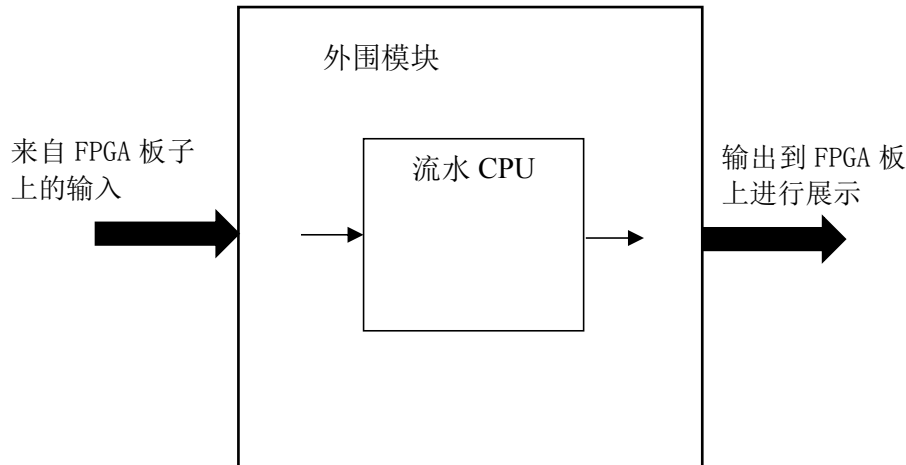


图 9.3 静态 5 级流水 CPU 设计实验的顶层模块大致框图

10. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：一般而言控制 CPU 运转的时钟是由 FPGA 板上的时钟输出提供的，但为了方便演示，我们需要在每一个时钟里查看一条指令的运算结果，故演示时理想的时钟是手动输入的，可以使用 FPGA 板上的脉冲开关代替时钟。

4 课程设计要求

1. 做好预习：

1) 复习好上一次多周期 CPU 的实验，归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令，对其进行分析，完成表 9.1 的填写；

2) 认真学习流水线的概念，明白流水线的意义和架构，理解数据相关、控制相关和结构相关，尤其需要注意分支跳转指令及其延迟槽指令的处理；

3) 依据自己设计中实现的指令，编写一段不少于 50 行的汇编程序，要求包含所有实现的指令，完成表 9.2 的填写。要求标注出指令间存在的相关，指出 CPU 可能存在的阻塞；

4) 在多周期 CPU 实验的设计框图的基础上，完善课程设计的框图，即补充完善图 9.2；

5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 9.3。

2. 实验实施：

1) 确认流水 CPU 的设计框图的正确性；

2) 编写 verilog 代码，将表 9.2 中自己编写的汇编程序翻译为二进制，以 coe 文件的方式初始化到指令 ROM 中；

3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料，在仿真时需要将生成指令 ROM 时产生的.mif 文件拷贝到工程目录下，才能仿真成功；

4) 完成调用流水 CPU 的外围模块的设计，并编写代码；

5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 9.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

5.1 实验原理

流水 CPU 实现的指令集是在多周期 CPU 的指令集上增加 9 条指令，主要是乘法指令，与 HI、LO、协处理器 0 (Coprocessor 0, cp0) 寄存器传送数据的指令以及特权指令，共实现了 45 条指令。表 9.3 为新增指令列表。表 9.4 为流水 CPU 所实现的指令的解析。

表 9.3 5 级流水线 CPU 新增的指令

指令名称	汇编指令	功能描述
有符号乘法	mult rs,rt	(HI,LO)=sign(GPR[rs])*sign(GPR[rt])
从 LO 寄存器取值	mflo rd	GPR[rd]=[LO]
从 HI 寄存器取值	mfhi rd	GPR[rd]=[HI]
向 LO 寄存器存值	mtlo rs	[LO]=GPR[rs]
向 HI 寄存器存值	mtthi rs	[HI]=GPR[rs]
从 cp0 寄存器取值	mfc0 rt,cs	GPR[rt]=CPR[cs]
向 cp0 寄存器存值	mtc0 rt,cd	CPR[cd]=GPR[rt]
系统调用	syscall	CPR[14.0]=PC, CPR[13.0][6:2]=01000, CPR[12.0][1]=1, PC=CPR[15.1]+0x180
异常返回	eret	CPR[12.0][1]=0, PC=CPR[14.0]

注：CPR 表示协处理器 0 寄存器，CPR[12.0]表示 STATUS 寄存器，CPR[13.0]表示 CAUSE 寄存器，CPR[14.0]表示 EPC 寄存器。CPR[15.1]为 EBASE 寄存器，syscall 指令会使用该寄存器确认 Exception 的入口地址，但在本参考设计中未实现该寄存器，并设置 Exception 的入口地址为 0 地址。

表 9.4 5 级流水线 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 00000 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 00000 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 00000 101010	[rs]	[rt]		rd	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
	sltu rd,rs,rt	000000 rs rt rd 00000 101011	[rs]	[rt]		rd	$GPR[rd] = (\text{zero}(GPR[rs]) < \text{zero}(GPR[rt]))$
	jlr rs	000000 rs 00000 1111 00000 001001	[rs]			31	$GPR[31] = PC, PC = GPR[rs]$
	jr rs	000000 rs 000000 0000 00000 001000	[rs]				$PC = GPR[rs]$
	and rd,rs,rt	000000 rs rt rd 00000 100100	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \& GPR[rt]$
	nor rd,rs,rt	000000 rs rt rd 00000 100111	[rs]	[rt]		rd	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
	or rd,rs,rt	000000 rs rt rd 00000 100101	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] GPR[rt]$
	xor rd,rs,rt	000000 rs rt rd 00000 100110	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
	sll rd,rt,shf	000000 00000 rt rd shf 000000		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
	sllv rd,rt,rs	000000 rs rt rd 00000 000100	[rs]	[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll (GPR[rs] \% 32)$
	sra rd,rt,shf	000000 00000 rt rd shf 000011		[rt]		rd	$GPR[rd] = \text{sign}(GPR[rt]) \gg \text{shf}$
	srav rd,rt,rs	000000 rs rt rd 00000 000111	[rs]	[rt]		rd	$GPR[rd] = \text{sign}(GPR[rt]) \gg (GPR[rs] \% 32)$
	srl rd,rt,shf	000000 00000 rt rd shf 000010		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
	srlv rd,rt,rs	000000 rs rt rd 00000 000110	[rs]	[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg GPR[rs]$
	mult rs,rt	000000 rs rt 000000 00000 011000	[rs]	[rt]		(HI,LO)	$(HI,LO) = \text{sign}(GPR[rs]) * \text{sign}(GPR[rt])$
	mflo rd	000000 00000000 00 rd 00000 010010	[LO]			rd	$GPR[rd] = [LO]$
	mfhi rd	000000 00000000 00 rd 00000 010000	[HI]			rd	$GPR[rd] = [HI]$
	mtlo rs	000000 rs 000000 00000000 010011	[rs]			LO	$[LO] = GPR[rs]$
	mthi rs	000000 rs 000000 00000000 010001	[rs]			HI	$[HI] = GPR[rs]$
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = GPR[rs] + \text{sign_ext}(imm)$
	slti rt,rs,imm	001010 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = (\text{sign}(GPR[rs]) < \text{sign_ext}(imm))$
	sltiu rt,rs,imm	001011 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = (\text{zero}(GPR[rs]) < \text{sign_ext}(imm))$

表 9.4 5 级流水线 CPU 实现的 mips 指令特性归纳（续）

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if GPR[rs]=GPR[rt] then PC= next_pc+ sign_ext (offset)<<2
	bgez rs,offset	000001 rs 00001 offset	[rs]				if GPR[rs]≥0 then PC= next_pc + sign_ext (offset)<<2
	bgtz rs,offset	000111 rs 00000 offset	[rs]				if GPR[rs]>0 then PC= next_pc + sign_ext (offset)<<2
	blez rs,offset	000110 rs 00000 offset	[rs]				if GPR[rs]≤0 then PC= next_pc + sign_ext (offset)<<2
	bltz rs,offset	000001 rs 00000 offset	[rs]				if GPR[rs]<0 then PC= next_pc + sign_ext (offset)<<2
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if GPR[rs]≠GPR[rt] then PC= next_pc + sign_ext (offset)<<2
	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext (offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext (offset)]=GPR[rt]
	lb rt,offset(b)	100000 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=sign(Mem[GPR[b]+sign_ext (offset)])
	lbu rt,offset(b)	100100 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=zero(Mem[GPR[b]+sign_ext (offset)])
	sb rt,offset(b)	101000 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext (offset)]=GPR[rt]
	andi rt,rs,imm	001100 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]&zero_ext (imm)
	lui rt,imm	001111 00000 rt imm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
	ori rt,rs,imm	001101 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs] zero_ext (imm)
	xori rt,rs,imm	001110 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]^zero_ext (imm)
J 型指令	j target	000010 target					[PC]={next_pc[31:28],target<<2}
	jal target	000011 target					GPR[31]=PC,PC={next_pc[31:28],target<<2}
cp0 指令	mfc0 rt,cs	010000 00000 rt cs 00000000 sel	CPR[cs.sel]			rt	GPR[rt]=CPR[cs.sel]
	mtc0 rt,cd	010000 00100 rt cd 00000000 sel		GPR[rt]		CPR[cd.sel]	CPR[cd.sel]=GPR[rt]
	syscall	000000 code 001100					CPR[14.0]=PC,CPR[13.0][6:2]=01000,CPR[12.0][1]=1,PC=CPR[15.1]+0x180 并跳转
	Eret	010000 1 0000000000000000 011000					CPR[12.0][1]=0,PC=CPR[14.0]并跳转

注：分支跳转指令参与运算的不在是当前 PC 值，而是 next_pc, 即当前 PC+4，即延迟槽指令的 PC。

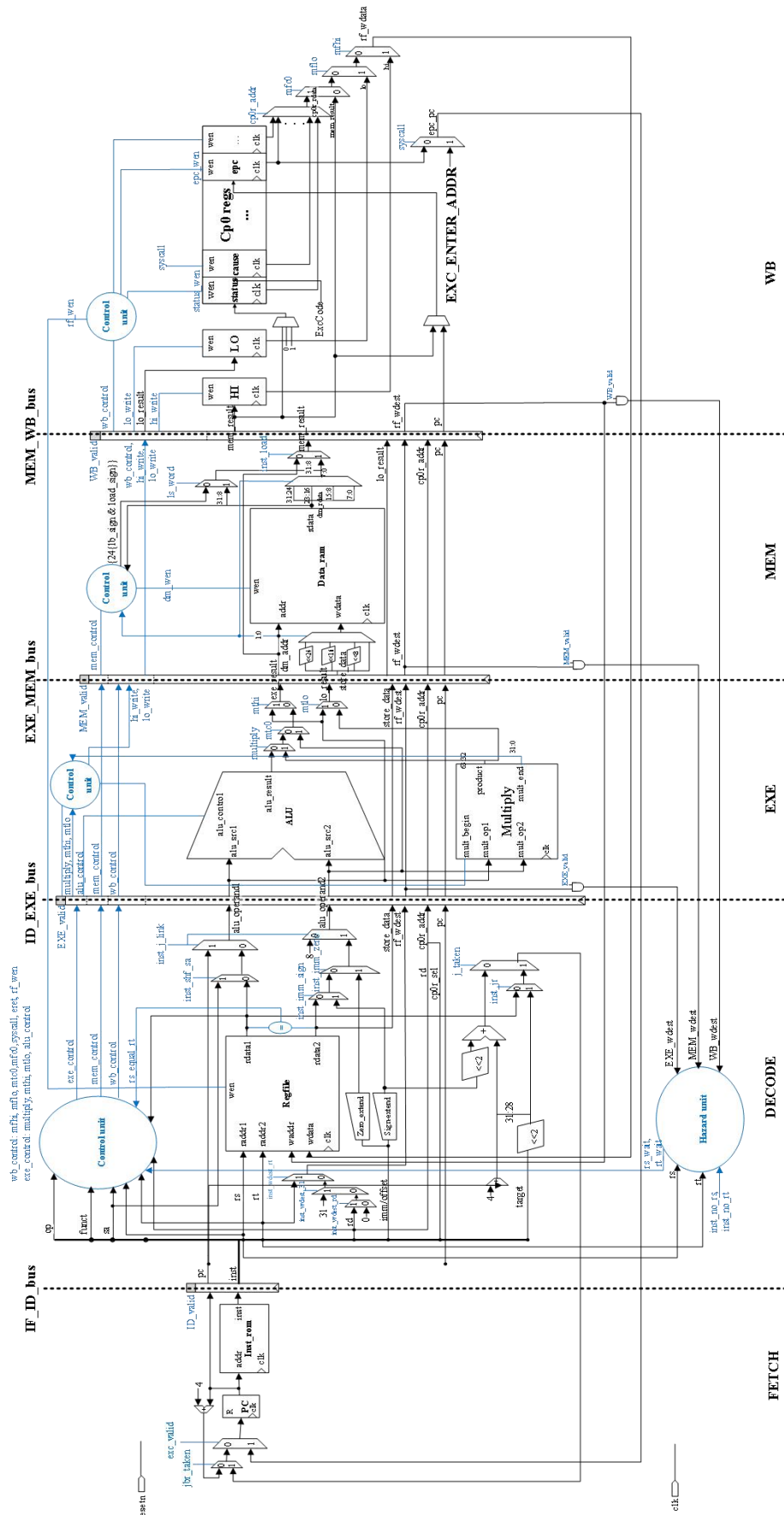


图 9.4 5 级流水线 CPU 的实现框图

利用 5 级流水 CPU 实现的 45 条指令，可以编写一段汇编程序，用于测试实现的静态 5 级流水 CPU 的正确性。表 9.5 给出了一个测试程序。

表 9.5 5 级流水线 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
Exception 入口地址，在 SYSCALL 指令执行后进入此处执行				
00H	sw \$1,#0(\$0)	Mem[0000_0000H] = 0000_0008H	AC010000	1010_1100_0000_0001_0000_0000_0000_0000
04H	sw \$2,#4(\$0)	Mem[0000_0004H] = 0000_0010H	AC020004	1010_1100_0000_0010_0000_0000_0000_0100
08H	sw \$3,#8(\$0)	Mem[0000_0008H] = 0000_0011H	AC030008	1010_1100_0000_0011_0000_0000_0000_1000
0CH	sw \$4,#12(\$0)	Mem[0000_000CH] = 0000_0004H	AC04000C	1010_1100_0000_0100_0000_0000_0000_1100
10H	sw \$5,#16(\$0)	Mem[0000_0010H] = 0000_000DH	AC050010	1010_1100_0000_0101_0000_0000_0001_0000
14H	sw \$6,#24(\$0)	Mem[0000_0018H] = FFFF_FFE2H	AC060018	1010_1100_0000_0110_0000_0000_0001_1000
18H	sw \$7,#112(\$0)	Mem[0000_0070H] = FFFF_FFF3H	AC070070	1010_1100_0000_0111_0000_0000_0111_0000
1CH	sw \$25,#116(\$0)	Mem[0000_0074H] = 0000_0001H	AC190074	1010_1100_0001_1001_0000_0000_0111_0100
20H	sw \$13,#24(\$0)	Mem[0000_0078H] = 0000_0000H	AC0D0078	1010_1100_0000_1101_0000_0000_0111_1000
24H	mfc0 \$1, cp0(14.0)	[\$1] = 0000_0104H	40017000	0100_0000_0000_0001_0111_0000_0000_0000
28H	addiu \$1,\$1,#4	[\$1] = 0000_0108H	24210004	0010_0100_0010_0001_0000_0000_0000_0100
2CH	mtc0 \$1, cp0(14.0)	cp0(14.0) = 0000_0108H	40817000	0100_0000_1000_0001_0111_0000_0000_0000
30H	eret	返回 108H	42000018	0100_0010_0000_0000_0000_0000_0001_1000
CPU 复位地址 0000_0034H				
34H	addiu \$1, \$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
38H	sll \$2, \$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
3CH	addu \$3, \$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
40H	srl \$4, \$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
44H	slti \$25,\$4,#5	[\$25] = 0000_0001H	28990005	0010_1000_1001_1001_0000_0000_0000_0101
48H	bgez \$25,#14	跳转到 84H	0721000E	0000_0111_0010_0001_0000_0000_0000_1110
4CH	subu \$5, \$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
50H	sw \$5, #20(\$0)	Mem[0000_0014H] = 0000_000DH	AC050014	1010_1100_0000_0101_0000_0000_0001_0100
54H	nor \$6, \$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
58H	or \$7, \$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101
5CH	xor \$8, \$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
60H	beq \$8, \$3,#2	跳转到 6CH	11030002	0001_0001_0000_0011_0000_0000_0000_0010
64H	sw \$8, #28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
68H	slt \$9, \$1,\$2	不执行	0022482A	0000_0000_0010_0010_0100_1000_0010_1010
6CH	lw \$10,#28(\$0)	[\$10] = 0000_0011H	8C0A001C	1000_1100_0000_1010_0000_0000_0001_1100
70H	bne \$10,\$5,#2	跳转到 7CH	15450002	0001_0101_0100_0101_0000_0000_0000_0010

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
74H	and \$11,\$2,\$1	[\$11] = 0000_0000H	00415824	0000_0000_0100_0001_0101_1000_0010_0100
78H	sw \$11,#28(\$0)	不执行	AC0B001C	1010_1100_0000_1011_0000_0000_0001_1100
7CH	jal #38	跳转到 98H, [\$31] = 0000_0084H	0C000026	0000_1100_0000_0000_0000_0000_0010_0110
80H	sw \$4,#16(\$0)	Mem[0000_0010H] = 0000_0004H	AC040010	1010_1100_0000_0100_0000_0000_0001_0000
84H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
88H	sra \$26,\$12,\$2	[\$26] = 0000_000CH	004CD007	0000_0000_0100_1100_1101_0000_0000_0111
8CH	addiu \$27,\$26,#68	[\$27] = 0000_0050H	275B0044	0010_0111_0101_1011_0000_0000_0100_0100
90H	jalr \$27	跳转到 50H, [\$31] = 0000_0098H	0360F809	0000_0011_0110_0000_1111_1000_0000_1001
94H	addiu \$1,\$0,#8	[\$1] = 0000_0008H	24010008	0010_0100_0000_0001_0000_0000_0000_1000
98H	sb \$26,#5(\$3)	MEM[0000_0014H] = 000C_000DH	A07A0005	1010_0000_0111_1010_0000_0000_0000_0101
9CH	sltu \$13,\$10,\$3	[\$13] = 0000_0000H	0143682B	0000_0001_0100_0011_0110_1000_0010_1011
A0H	bgtz \$13,#2	不跳转	1DA00002	0001_1101_1010_0000_0000_0000_0000_0010
A4H	sllv \$14,\$6,\$4	[\$14] = FFFF_FE20H	00867004	0000_0000_1000_0110_0111_0000_0000_0100
A8H	sra \$15,\$14,#2	[\$15] = FFFF_FF88H	000E7883	0000_0000_0000_1110_0111_1000_1000_0011
ACH	srlv \$16,\$15,\$1	[\$16] = 00FF_FFFFH	002F8006	0000_0000_0010_1111_1000_0000_0000_0110
B0H	blez \$16,#7	不跳转	1A000007	0001_1010_0000_0000_0000_0000_0000_0111
B4H	sra \$16,\$15,\$1	[\$16] = FFFF_FFFFH	002F8007	0000_0000_0010_1111_1000_0000_0000_0111
B8H	bltz \$16,#6	跳转到 D4H	06000006	0000_0110_0000_0000_0000_0000_0000_0110
BCH	sll \$11,\$26,#4	[\$11] = 0000_00C0H	001A5900	0000_0000_0001_1010_0101_1001_0000_0000
C0H	lw \$28,#3(\$10)	[\$28]=000C_000DH / 000C_880DH	8D5C0003	1000_1101_0101_1100_0000_0000_0000_0011
C4H	bne \$28,\$29,#7	不跳转/跳转 E4H	179D0007	0001_0111_1001_1101_0000_0000_0000_0111
C8H	sb \$15,#8(\$5)	Mem[0000_0014H] = 000C_8800H	A0AF0008	1010_0000_1010_1111_0000_0000_0000_1000
CCH	lb \$18,#8(\$5)	[\$18] = FFFF_FF88H	80B20008	1000_0000_1011_0010_0000_0000_0000_1000
D0H	lbu \$19,#8(\$5)	[\$19] = 0000_0088H	90B30008	1001_0000_1011_0011_0000_0000_0000_1000
D4H	sltiu \$24,\$15,#0xFFFF	[\$24] = 0000_0001H	2DF8FFFF	0010_1101_1111_1000_1111_1111_1111_1111
D8H	or \$29,\$12,\$5	[\$29] = 000C_000DH	0185E825	0000_0001_1000_0101_1110_1000_0010_0101
DCH	jr \$11	跳转到 C0H	01600008	0000_0001_0110_0000_0000_0000_0000_1000
E0H	andi \$20,\$15,#0xFFFF	[\$20] = 0000_FF88H	31F4FFFF	0011_0001_1111_0100_1111_1111_1111_1111
E4H	ori \$21,\$15,#0xFFFF	[\$21] = FFFF_FFFFH	35F5FFFF	0011_0101_1111_0101_1111_1111_1111_1111
E8H	xori \$22,\$15,#0xFFFF	[\$22] = FFFF_0077H	39F6FFFF	0011_1001_1111_0110_1111_1111_1111_1111
ECH	mult \$12,\$29	[HI] = 0000_0090H, [LO] = 009C_0000H	019D0018	0000_0001_1001_1101_0000_0000_0001_1000
F0H	mflo \$23	[\$23] = 009C_0000H	0000B812	0000_0000_0000_0000_1011_1000_0001_0010
F4H	mfhi \$30	[\$30] = 0000_0090H	0000F010	0000_0000_0000_0000_1111_0000_0001_0000
F8H	mtlo \$26	[LO] = 0000_000CH	03400013	0000_0011_0100_0000_0000_0000_0001_0011

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
FCH	mthi \$27	[HI] = 0000_0050H	03600011	0000_0011_0110_0000_0000_0001_0001
100H	mtc0 \$0,c14	cp0[14.0] = 0000_0000H	40807000	0100_0000_1000_0000_0111_0000_0000_0000
104H	syscall	cp0[14.0] =0000_0104H, cp0[13.0][6..2] =01000B, cp0[12.0][1]=1, 跳转 Exception 入口 地址, 00H	0000000C	0000_0000_0000_0000_0000_0000_0000_1100
108H	mfc0 \$2, cp0(14.0)	[\$2] = 0000_0108H	40027000	0100_0000_0000_0010_0111_0000_0000_0000
10CH	mfc0 \$3, cp0(13.0)	[\$3] = 0000_0020H	40036800	0100_0000_0000_0011_0110_1000_0000_0000
110H	mfc0 \$4, cp0(12.0)	[\$4] = 0000_0000H	40046000	0100_0000_0000_0100_0110_0000_0000_0000
114H	addiu \$1, \$0,#32	[\$1] = 0000_0020H	24010020	0010_0100_0000_0001_0000_0000_0010_0000
118H	slt \$17,\$15,\$14	[\$17] = 0000_0000H	01EE882A	0000_0001_1110_1110_1000_1000_0010_1010
11CH	lui \$17,#1234H	[\$17] = 1234_0000H	3C111234	0011_1100_0001_0001_0001_0010_0011_0100
120H	addiu \$17,\$17,#5678	[\$17] = 1234_5678H	26315678	0010_0110_0011_0001_0101_0110_0111_1000
124H	sw \$17,#0(\$1)	Mem[0000_0020H] =1234_5678H / Mem[0000_0024H] =2345_6780H / Mem[0000_0028H] =3456_7800H / Mem[0000_002CH] =4567_8000H / Mem[0000_0030H] =5678_0000H / Mem[0000_0034H] =6780_0000H / Mem[0000_0038H] =7800_0000H	AC310000	1010_1100_0011_0001_0000_0000_0000_0000
128H	sll \$17,\$17,#4	[\$17] = 2345_6780H / 3456_7800H / 4567_8000H / 5678_0000H / 6780_0000H / 7800_0000H / 8000_0000H	00118900	0000_0000_0001_0001_1000_1001_0000_0000
12CH	bgtz \$17,#-3	跳转到 124H /跳转/ 跳转/跳转/跳转/跳转 124H/不跳转	1E20FFFD	0001_1110_0010_0000_1111_1111_1111_1101
130H	addiu \$1,\$1,#4	[\$1] = 0000_0024H / 0000_0028H / 0000_002CH / 0000_0030H / 0000_0034H / 0000_0038H / 0000_003CH	24210004	0010_0100_0010_0001_0000_0000_0000_0100
134H	addiu \$2,\$0,#60	[\$2] = 0000_003CH	2402003C	0010_0100_0000_0010_0000_0000_0011_1100
138H	lw \$17,#-28(\$1)	[\$17] = 1234_5678H	8C31FFE4	1000_1100_0011_0001_1111_1111_1110_0100

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
13CH	srl \$17,\$17,#4	[\$17] = 0123_4567H / 0012_3456H / 0001_2345H / 0000_1234H / 0000_0123H / 0000_0012H / 0000_0001H / 0000_0000H	00118902	0000_0000_0001_0001_1000_1001_0000_0010
140H	sw \$17,#0(\$2)	Mem[0000_003CH] =0123_4567H / Mem[0000_0040H] =0012_3456H / Mem[0000_0044H] =0001_2345H / Mem[0000_0048H] =0000_1234H / Mem[0000_004CH] =0000_0123H / Mem[0000_0050H] =0000_0012H / Mem[0000_0054H] =0000_0001H / Mem[0000_0058H] =0000_0000H	AC510000	1010_1100_0101_0001_0000_0000_0000_0000
144H	bne \$17,\$0,#-3	跳转到 13CH /跳转/ 跳转/跳转/跳转/跳转/ /跳转 13CH/不跳转	1620FFFD	0001_0110_0010_0000_1111_1111_1111_1101
148H	addiu \$2,\$2,#4	[\$2] = 0000_0040H / 0000_0044H / 0000_0048H / 0000_004CH / 0000_0050H / 0000_0054H / 0000_0058H / 0000_005CH	24420004	0010_0100_0100_0010_0000_0000_0000_0100
14CH	addiu \$6,\$0,#68	[\$6] = 0000_0044H	24060044	0010_0100_0000_0110_0000_0000_0100_0100
150H	addiu \$7,\$0,#100	[\$7] = 0000_0064H	24070064	0010_0100_0000_0111_0000_0000_0110_0100
154H	lw \$3,#-28(\$1)	[\$3] = 1234_5678H / 2345_6780H / 3456_7800H / 4567_8000H / 5678_0000H / 6780_0000H / 7800_0000H	8C23FFE4	1000_1100_0010_0011_1111_1111_1110_0100
158H	lw \$4,#-4(\$2)	[\$4] = 0000_0000H / 0000_0001H / 0000_0012H / 0000_0123H / 0000_1234H / 0001_2345H / 0012_3456H	8C44FFFC	1000_1100_0100_0100_1111_1111_1111_1100
15CH	or \$5,\$3,\$4	[\$5] = 1234_5678H / 2345_6781H / 3456_7812H / 4567_8123H / 5678_1234H / 6781_2345H / 7812_3456H	00642825	0000_0000_0110_0100_0010_1000_0010_0101

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
160H	sb \$5,#0(\$7)	Mem[0000_0064H] =0000_0078H / Mem[0000_0064H] =0000_8178H / Mem[0000_0064H] =0012_8178H / Mem[0000_0064H] =2312_8178H / Mem[0000_0068H] =0000_0034H / Mem[0000_0068H] =0000_4534H / Mem[0000_0068H] =0056_4534H	A0E50000	1010_0000_1110_0101_0000_0000_0000_0000
164H	addiu \$7,\$7,#1	[\$7] = 0000_0065H / 0000_0066H / 0000_0067H / 0000_0068H / 0000_0069H / 0000_006AH / 0000_006BH	24E70001	0010_0100_1110_0111_0000_0000_0000_0001
168H	addiu \$1,\$1,#4	[\$1] = 0000_0040H / 0000_0044H / 0000_0048H / 0000_004CH / 0000_0050H / 0000_0054H / 0000_0058H	24210004	0010_0100_0010_0001_0000_0000_0000_0100
16CH	bne \$2,\$6,#-7	跳转到 154H/跳转/跳 转/跳转/跳转/跳转 154H/不跳转	1446FFF9	0001_0100_0100_0110_1111_1111_1111_1001
170H	ADDIU \$2,\$2,#-4	[\$2] = 0000_0058H / 0000_0054H / 0000_0050H / 0000_004CH / 0000_0048H / 0000_0044H / 0000_0040H	2442FFFC	0010_0100_0100_0010_1111_1111_1111_1100
174H	addiu \$9,\$0,#100	[\$9] = 0000_0064H	24090064	0010_0100_0000_1001_0000_0000_0110_0100
178H	lbu \$9,#3(\$9)	[\$9] = 0000_0023H	91290003	1001_0001_0010_1001_0000_0000_0000_0011
17CH	addiu \$13,\$0,#104	[\$13] = 0000_0068H	240D0068	0010_0100_0000_1101_0000_0000_0110_1000
180H	lw \$13,#0(\$13)	[\$13] = 0056_4534H	8DAD0000	1000_1101_1010_1101_0000_0000_0000_0000
184H	ll \$9,\$9,#24	[\$9] = 2300_0000H	00094E00	0000_0000_0000_1001_0100_1110_0000_0000
188H	xori \$13,\$13,#9	[\$13] = 0056_453DH	39AD0009	0011_1001_1010_1101_0000_0000_0000_1001
18CH	sw \$13,#1(\$7)	MEM[0000_006CH]= 0056_453DH	ACED0001	1010_1100_1110_1101_0000_0000_0000_0001
190H	lw \$1,#0(\$0)	[\$1] = 0000_0008H	8C010000	1000_1100_0000_0001_0000_0000_0000_0000
194H	lw \$2,#4(\$0)	[\$2] = 0000_0010H	8C020004	1000_1100_0000_0010_0000_0000_0000_0100
198H	lw \$3,#8(\$0)	[\$3] = 0000_0011H	8C030008	1000_1100_0000_0011_0000_0000_0000_1000
19CH	lw \$4,#12(\$0)	[\$4] = 0000_0004H	8C04000C	1000_1100_0000_0100_0000_0000_0000_1100
1A0H	lw \$5,#16(\$0)	[\$5] = 0000_000DH	8C050010	1000_1100_0000_0101_0000_0000_0001_0000
1A4H	lw \$6,#24(\$0)	[\$6] = FFFF_FFE2H	8C060018	1000_1100_0000_0110_0000_0000_0001_1000

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
1A8H	lw \$7,#112(\$0)	[\$7] = FFFF_FFF3H	8C070070	1000_1100_0000_0111_0000_0000_0111_0000
1ACH	lw \$25,#116(\$0)	[\$25] = 0000_0001H	8C190074	1000_1100_0001_1001_0000_0000_0111_0100
1B0H	lw \$13,#120(\$0)	[\$13] = 0000_0000H	8C0D0078	1000_1100_0000_1101_0000_0000_0111_1000
1B4H	j #34H	跳转到 34H	0800000D	0000_1000_0000_0000_0000_0000_0000_1101

5.2 实验顶层模块框图

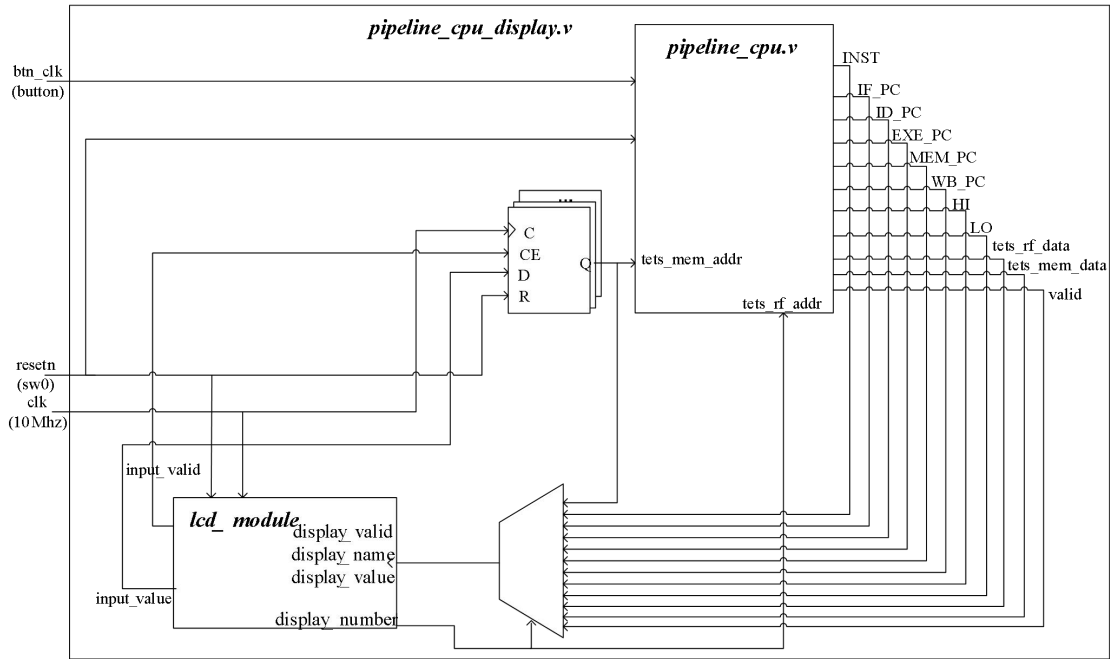


图 9.5 5 级流水线 CPU 参考设计的顶层模块框图

5.3 源代码

pipeline_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
3 // > 文件名: pipeline_cpu_display.v
4 // > 描述 : 五级流水 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 //*****
8 module pipeline_cpu_display( // 多周期 cpu
9     //时钟与复位信号
10    input clk,
11    input resetn, //后缀"n"代表低电平有效
12
13    //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14    input btn_clk,
15
16    //触摸屏相关接口, 不需要更改
17    output lcd_rst,
```



```
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改，用于单步调试
31     wire cpu_clk;      //单周期 CPU 里使用脉冲开关作为时钟，以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37         begin
38             btn_clk_r1<= 1'b0;
39         end
40         else
41         begin
42             btn_clk_r1 <= ~btn_clk;
43         end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk),.CE(clk_en),.O(cpu_clk));
51 //-----{时钟和复位信号}end
52
53 //-----{调用多周期 CPU 模块}begin
54     //用于在 FPGA 板上显示结果
55     wire [ 4:0] rf_addr;    //扫描寄存器堆的地址
56     wire [31:0] rf_data;    //寄存器堆从调试端口读出的数据
57     reg  [31:0] mem_addr;   //要观察的内存地址
58     wire [31:0] mem_data;   //内存地址对应的数据
59     wire [31:0] IF_pc;      //IF 模块的 PC
60     wire [31:0] IF_inst;    //IF 模块取出的指令
61     wire [31:0] ID_pc;      //ID 模块的 PC
62     wire [31:0] EXE_pc;     //EXE 模块的 PC
63     wire [31:0] MEM_pc;     //MEM 模块的 PC
64     wire [31:0] WB_pc;      //WB 模块的 PC
65     wire [31:0] cpu_5_valid; //展示 CPU5 级的 valid 信号
66     wire [31:0] HI_data;    //展示 HI 寄存器的值
67     wire [31:0] LO_data;    //展示 LO 寄存器的值
68     pipeline_cpu cpu(
```

```

69         .clk      (cpu_clk ),
70         .resetn   (resetn  ),
71
72         .rf_addr  (rf_addr ),
73         .mem_addr (mem_addr),
74         .rf_data  (rf_data ),
75         .mem_data (mem_data),
76         .IF_pc    (IF_pc   ),
77         .IF_inst  (IF_inst ),
78         .ID_pc    (ID_pc   ),
79         .EXE_pc   (EXE_pc  ),
80         .MEM_pc   (MEM_pc  ),
81         .WB_pc    (WB_pc   ),
82         .cpu_5_valid (cpu_5_valid),
83         .HI_data  (HI_data ),
84         .LO_data  (LO_data ),
85     );
86 //-----{调用单周期 CPU 模块}end
87
88 //-----{调用触摸屏模块}begin-----//
89 //-----{实例化触摸屏}begin
90 //此小节不需要更改
91     reg          display_valid;
92     reg [39:0] display_name;
93     reg [31:0] display_value;
94     wire [5 :0] display_number;
95     wire          input_valid;
96     wire [31:0] input_value;
97
98     lcd_module lcd_module(
99         .clk      (clk      ),    //10Mhz
100        .resetn   (resetn   ),
101
102        //调用触摸屏的接口
103        .display_valid (display_valid ),
104        .display_name  (display_name  ),
105        .display_value (display_value ),
106        .display_number (display_number),
107        .input_valid   (input_valid   ),
108        .input_value   (input_value   ),
109
110        //lcd 触摸屏相关接口, 不需要更改
111        .lcd_rst      (lcd_rst      ),
112        .lcd_cs       (lcd_cs       ),
113        .lcd_rs       (lcd_rs       ),
114        .lcd_wr       (lcd_wr       ),
115        .lcd_rd       (lcd_rd       ),
116        .lcd_data_io  (lcd_data_io  ),
117        .lcd_bl_ctr   (lcd_bl_ctr   ),
118        .ct_int       (ct_int       ),
119        .ct_sda       (ct_sda       ),

```

```
120         .ct_scl          (ct_scl          ),
121         .ct_rstn         (ct_rstn         )
122     );
123 //-----{实例化触摸屏}end
124
125 //-----{从触摸屏获取输入}begin
126 //根据实际需要输入的数修改此小节,
127 //建议对每一个数的输入, 编写单独一个 always 块
128     always @(posedge clk)
129     begin
130         if (!resetn)
131         begin
132             mem_addr <= 32'd0;
133         end
134         else if (input_valid)
135         begin
136             mem_addr <= input_value;
137         end
138     end
139     assign rf_addr = display_number-6'd13;
140 //-----{从触摸屏获取输入}end
141
142 //-----{输出到触摸屏显示}begin
143 //根据需要显示的数修改此小节,
144 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
145 //44 块显示区域从 1 开始编号, 编号为 1~44,
146     always @(posedge clk)
147     begin
148         if (display_number >6'd12 && display_number <6'd45 )
149         begin //块号 5~36 显示 32 个通用寄存器的值
150             display_valid <= 1'b1;
151             display_name[39:16] <= "REG";
152             display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
153             display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
154             display_value      <= rf_data;
155         end
156         else
157         begin
158             case(display_number)
159             6'd1 : //显示 IF 模块的 PC
160             begin
161                 display_valid <= 1'b1;
162                 display_name  <= "IF_PC";
163                 display_value <= IF_pc;
164             end
165             6'd2 : //显示 IF 模块的指令
166             begin
167                 display_valid <= 1'b1;
168                 display_name  <= "IF_IN";
169                 display_value <= IF_inst;
170             end
```

```
171      6'd3 : //显示 ID 模块的 PC
172      begin
173          display_valid <= 1'b1;
174          display_name  <= "ID_PC";
175          display_value <= ID_pc;
176      end
177      6'd4 : //显示 EXE 模块的 PC
178      begin
179          display_valid <= 1'b1;
180          display_name  <= "EXEPC";
181          display_value <= EXE_pc;
182      end
183      6'd5 : //显示 MEM 模块的 PC
184      begin
185          display_valid <= 1'b1;
186          display_name  <= "MEMPC";
187          display_value <= MEM_pc;
188      end
189      6'd6 : //显示 WB 模块的 PC
190      begin
191          display_valid <= 1'b1;
192          display_name  <= "WB_PC";
193          display_value <= WB_pc;
194      end
195      6'd7 : //显示要观察的内存地址
196      begin
197          display_valid <= 1'b1;
198          display_name  <= "MADDR";
199          display_value <= mem_addr;
200      end
201      6'd8 : //显示该内存地址对应的数据
202      begin
203          display_valid <= 1'b1;
204          display_name  <= "MDATA";
205          display_value <= mem_data;
206      end
207      6'd9 : //显示 CPU 当前状态
208      begin
209          display_valid <= 1'b1;
210          display_name  <= "VALID";
211          display_value <= cpu_5_valid;
212      end
213      6'd11: //显示 HI 寄存器的值
214      begin
215          display_valid <= 1'b1;
216          display_name  <= "    HI";
217          display_value <= HI_data;
218      end
219      6'd12: //显示 LO 寄存器的值
220      begin
221          display_valid <= 1'b1;
```

```
222             display_name <= "    LO";
223             display_value <= LO_data;
224         end
225         default :
226         begin
227             display_valid <= 1'b0;
228             display_name <= 40'd0;
229             display_value <= 32'd0;
230         end
231     endcase
232 end
233 end
234 //-----{输出到触摸屏显示}end
235 //-----{调用触摸屏模块}end-----//
236 endmodule
```

pipeline_cpu.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: pipeline_cpu.v
4  //    > 描述   : 五级流水 CPU 模块, 共实现 XX 条指令
5  //    >        指令 rom 和数据 ram 均实例化 xilinx IP 得到, 为同步读写
6  //    > 作者   : LOONGSON
7  //    > 日期   : 2016-04-14
8  //*****
9  module pipeline_cpu( // 多周期 cpu
10     input clk,          // 时钟
11     input resetn,       // 复位信号, 低电平有效
12
13     //display data
14     input  [ 4:0] rf_addr,
15     input  [31:0] mem_addr,
16     output [31:0] rf_data,
17     output [31:0] mem_data,
18     output [31:0] IF_pc,
19     output [31:0] IF_inst,
20     output [31:0] ID_pc,
21     output [31:0] EXE_pc,
22     output [31:0] MEM_pc,
23     output [31:0] WB_pc,
24
25     //5 级流水新增
26     output [31:0] cpu_5_valid,
27     output [31:0] HI_data,
28     output [31:0] LO_data
29 );
30 //-----{5 级流水控制信号}begin-----//
31 //5 模块的 valid 信号
32 reg IF_valid;
33 reg ID_valid;
```

```
34     reg EXE_valid;
35     reg MEM_valid;
36     reg WB_valid;
37     //5 模块执行完成信号,来自各模块的输出
38     wire IF_over;
39     wire ID_over;
40     wire EXE_over;
41     wire MEM_over;
42     wire WB_over;
43     //5 模块允许下一级指令进入
44     wire IF_allow_in;
45     wire ID_allow_in;
46     wire EXE_allow_in;
47     wire MEM_allow_in;
48     wire WB_allow_in;
49
50     // syscall 和 eret 到达写回级时会发出 cancel 信号,
51     wire cancel;    // 取消已经取出的正在其他流水级执行的指令
52
53     //各级允许进入信号:本级无效,或本级执行完成且下级允许进入
54     assign IF_allow_in  = (IF_over & ID_allow_in) | cancel;
55     assign ID_allow_in  = ~ID_valid  | (ID_over  &
EXE_allow_in);
56     assign EXE_allow_in = ~EXE_valid | (EXE_over &
MEM_allow_in);
57     assign MEM_allow_in = ~MEM_valid | (MEM_over &
WB_allow_in );
58     assign WB_allow_in  = ~WB_valid  | WB_over;
59
60     //IF_valid, 在复位后,一直有效
61     always @(posedge clk)
62     begin
63         if (!resetn)
64             begin
65                 IF_valid <= 1'b0;
66             end
67         else
68             begin
69                 IF_valid <= 1'b1;
70             end
71     end
72
73     //ID_valid
74     always @(posedge clk)
75     begin
76         if (!resetn || cancel)
77             begin
78                 ID_valid <= 1'b0;
79             end
80         else if (ID_allow_in)
81             begin
82                 ID_valid <= IF_over;
```

```
83         end
84     end
85
86     //EXE_valid
87     always @(posedge clk)
88     begin
89         if (!resetsn || cancel)
90         begin
91             EXE_valid <= 1'b0;
92         end
93         else if (EXE_allow_in)
94         begin
95             EXE_valid <= ID_over;
96         end
97     end
98
99     //MEM_valid
100    always @(posedge clk)
101    begin
102        if (!resetsn || cancel)
103        begin
104            MEM_valid <= 1'b0;
105        end
106        else if (MEM_allow_in)
107        begin
108            MEM_valid <= EXE_over;
109        end
110    end
111
112    //WB_valid
113    always @(posedge clk)
114    begin
115        if (!resetsn || cancel)
116        begin
117            WB_valid <= 1'b0;
118        end
119        else if (WB_allow_in)
120        begin
121            WB_valid <= MEM_over;
122        end
123    end
124
125    //展示 5 级的 valid 信号
126    assign cpu_5_valid = {12'd0 , {4{IF_valid }}, {4{ID_valid}},
127    {4{EXE_valid}}, {4{MEM_valid}}, {4{WB_valid}}};
128    //-----{5 级流水控制信号}end-----//
129
130    //-----{5 级间的总线}begin-----
131    //
131    wire [ 63:0] IF_ID_bus;    // IF->ID 级总线
```



```
132    wire [166:0] ID_EXE_bus; // ID->EXE 级总线
133    wire [153:0] EXE_MEM_bus; // EXE->MEM 级总线
134    wire [117:0] MEM_WB_bus; // MEM->WB 级总线
135
136    //锁存以上总线信号
137    reg [ 63:0] IF_ID_bus_r;
138    reg [166:0] ID_EXE_bus_r;
139    reg [153:0] EXE_MEM_bus_r;
140    reg [117:0] MEM_WB_bus_r;
141
142    //IF 到 ID 的锁存信号
143    always @(posedge clk)
144    begin
145        if(IF_over && ID_allow_in)
146        begin
147            IF_ID_bus_r <= IF_ID_bus;
148        end
149    end
150    //ID 到 EXE 的锁存信号
151    always @(posedge clk)
152    begin
153        if(ID_over && EXE_allow_in)
154        begin
155            ID_EXE_bus_r <= ID_EXE_bus;
156        end
157    end
158    //EXE 到 MEM 的锁存信号
159    always @(posedge clk)
160    begin
161        if(EXE_over && MEM_allow_in)
162        begin
163            EXE_MEM_bus_r <= EXE_MEM_bus;
164        end
165    end
166    //MEM 到 WB 的锁存信号
167    always @(posedge clk)
168    begin
169        if(MEM_over && WB_allow_in)
170        begin
171            MEM_WB_bus_r <= MEM_WB_bus;
172        end
173    end
174    //-----{5 级间的总线}end-----
175    //
176    //-----{其他交互信号}begin-----
177    //
178    //跳转总线
179    wire [ 32:0] jbr_bus;
180
181    //IF 与 inst_rom 交互
```

```

181     wire [31:0] inst_addr;
182     wire [31:0] inst;
183
184     //ID 与 EXE、MEM、WB 交互
185     wire [ 4:0] EXE_wdest;
186     wire [ 4:0] MEM_wdest;
187     wire [ 4:0] WB_wdest;
188
189     //MEM 与 data_ram 交互
190     wire [ 3:0] dm_wen;
191     wire [31:0] dm_addr;
192     wire [31:0] dm_wdata;
193     wire [31:0] dm_rdata;
194
195     //ID 与 regfile 交互
196     wire [ 4:0] rs;
197     wire [ 4:0] rt;
198     wire [31:0] rs_value;
199     wire [31:0] rt_value;
200
201     //WB 与 regfile 交互
202     wire          rf_wen;
203     wire [ 4:0] rf_wdest;
204     wire [31:0] rf_wdata;
205
206     //WB 与 IF 间的交互信号
207     wire [32:0] exc_bus;
208     //-----{其他交互信号}end-----
209     //
210     //-----{各模块实例化}begin-----
211     //
212     wire next_fetch; //即将运行取指模块，需要先锁存 PC 值
213     //IF 允许进入时，即锁存 PC 值，取下一条指令
214     assign next_fetch = IF_allow_in;
215     fetch IF_module( // 取指级
216         .clk      (clk      ), // I, 1
217         .resetn    (resetn    ), // I, 1
218         .IF_valid  (IF_valid  ), // I, 1
219         .next_fetch(next_fetch), // I, 1
220         .inst      (inst      ), // I, 32
221         .jbr_bus   (jbr_bus   ), // I, 33
222         .inst_addr (inst_addr ), // O, 32
223         .IF_over   (IF_over   ), // O, 1
224         .IF_ID_bus (IF_ID_bus ), // O, 64
225
226         //5 级流水新增接口
227         .exc_bus   (exc_bus   ), // I, 32
228
229         //展示 PC 和取出的指令
230         .IF_pc     (IF_pc     ), // O, 32

```

```

230         .IF_inst      (IF_inst      )    // O, 32
231     );
232
233     decode ID_module(                      // 译码级
234         .ID_valid      (ID_valid      ),    // I, 1
235         .IF_ID_bus_r   (IF_ID_bus_r),    // I, 64
236         .rs_value      (rs_value      ),    // I, 32
237         .rt_value      (rt_value      ),    // I, 32
238         .rs             (rs            ),    // O, 5
239         .rt             (rt            ),    // O, 5
240         .jbr_bus       (jbr_bus       ),    // O, 32
241     //     .inst_jbr     (inst_jbr     ),    // O, 1
242         .ID_over       (ID_over       ),    // O, 1
243         .ID_EXE_bus    (ID_EXE_bus    ),    // O, 167
244
245         //5 级流水新增
246         .IF_over       (IF_over       ),    // I, 1
247         .EXE_wdest     (EXE_wdest     ),    // I, 5
248         .MEM_wdest     (MEM_wdest     ),    // I, 5
249         .WB_wdest      (WB_wdest      ),    // I, 5
250
251         //展示 PC
252         .ID_pc         (ID_pc         )    // O, 32
253     );
254
255     exe EXE_module(                        // 执行级
256         .EXE_valid      (EXE_valid      ),    // I, 1
257         .ID_EXE_bus_r   (ID_EXE_bus_r),    // I, 167
258         .EXE_over       (EXE_over       ),    // O, 1
259         .EXE_MEM_bus    (EXE_MEM_bus    ),    // O, 154
260
261         //5 级流水新增
262         .clk            (clk            ),    // I, 1
263         .EXE_wdest      (EXE_wdest      ),    // O, 5
264
265         //展示 PC
266         .EXE_pc         (EXE_pc         )    // O, 32
267     );
268
269     mem MEM_module(                      // 访存级
270         .clk            (clk            ),    // I, 1
271         .MEM_valid      (MEM_valid      ),    // I, 1
272         .EXE_MEM_bus_r   (EXE_MEM_bus_r),    // I, 154
273         .dm_rdata       (dm_rdata       ),    // I, 32
274         .dm_addr        (dm_addr        ),    // O, 32
275         .dm_wen         (dm_wen         ),    // O, 4
276         .dm_wdata       (dm_wdata       ),    // O, 32
277         .MEM_over       (MEM_over       ),    // O, 1
278         .MEM_WB_bus     (MEM_WB_bus     ),    // O, 118
279
280         //5 级流水新增接口

```

```

281     .MEM_allow_in (MEM_allow_in ), // I, 1
282     .MEM_wdest    (MEM_wdest    ), // O, 5
283
284     //展示 PC
285     .MEM_pc        (MEM_pc        ) // O, 32
286 );
287
288 wb WB_module( // 写回级
289     .WB_valid      (WB_valid      ), // I, 1
290     .MEM_WB_bus_r (MEM_WB_bus_r), // I, 118
291     .rf_wen        (rf_wen        ), // O, 1
292     .rf_wdest      (rf_wdest      ), // O, 5
293     .rf_wdata      (rf_wdata      ), // O, 32
294     .WB_over       (WB_over       ), // O, 1
295
296     //5 级流水新增接口
297     .clk           (clk           ), // I, 1
298     .resetn        (resetn        ), // I, 1
299     .exc_bus       (exc_bus       ), // O, 32
300     .WB_wdest      (WB_wdest      ), // O, 5
301     .cancel        (cancel        ), // O, 1
302
303     //展示 PC 和 HI/LO 值
304     .WB_pc         (WB_pc         ), // O, 32
305     .HI_data       (HI_data       ), // O, 32
306     .LO_data       (LO_data       ) // O, 32
307 );
308
309 inst_rom inst_rom_module( // 指令存储器
310     .clka          (clk          ), // I, 1 ,时钟
311     .addra         (inst_addr[9:2]), // I, 8 ,指令地址
312     .douta         (inst         ) // O, 32,指令
313 );
314
315 regfile rf_module( // 寄存器堆模块
316     .clk           (clk           ), // I, 1
317     .wen           (rf_wen        ), // I, 1
318     .raddr1        (rs            ), // I, 5
319     .raddr2        (rt            ), // I, 5
320     .waddr         (rf_wdest      ), // I, 5
321     .wdata         (rf_wdata      ), // I, 32
322     .rdata1        (rs_value      ), // O, 32
323     .rdata2        (rt_value      ), // O, 32
324
325     //display rf
326     .test_addr     (rf_addr), // I, 5
327     .test_data     (rf_data) // O, 32
328 );
329
330 data_ram data_ram_module( // 数据存储模块
331     .clka          (clk          ), // I, 1, 时钟

```

```

332         .wea      (dm_wen      ), // I, 1, 写使能
333         .addra    (dm_addr[9:2]), // I, 8, 读地址
334         .dina      (dm_wdata    ), // I, 32, 写数据
335         .douta     (dm_rdata    ), // O, 32, 读数据
336
337         //display mem
338         .clkb      (clk          ), // I, 1, 时钟
339         .web        (4'd0        ), // 不使用端口 2 的写功能
340         .addrb      (mem_addr[9:2]), // I, 8, 读地址
341         .doutb      (mem_data     ), // I, 32, 写数据
342         .dinb       (32'd0       ) // 不使用端口 2 的写功能
343     );
344     //-----{各模块实例化}end-----
345     endmodule

```

fetch.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  // > 文件名: fetch.v
4  // > 描述 : 五级流水 CPU 的取指模块
5  // > 作者 : LOONGSON
6  // > 日期 : 2016-04-14
7  //*****
8  `define STARTADDR 32'H00000034 // 程序起始地址为 34H
9  module fetch( // 取指级
10     input          clk, // 时钟
11     input          resetn, // 复位信号, 低电平有效
12     input          IF_valid, // 取指级有效信号
13     input          next_fetch, // 取下一条指令, 用来锁存 PC 值
14     input          [31:0] inst, // inst_rom 取出的指令
15     input          [32:0] jbr_bus, // 跳转总线
16     output         [31:0] inst_addr, // 发往 inst_rom 的取指地址
17     output reg      IF_over, // IF 模块执行完成
18     output         [63:0] IF_ID_bus, // IF->ID 总线
19
20     //5 级流水新增接口
21     input          [32:0] exc_bus, // Exception pc 总线
22
23     //展示 PC 和取出的指令
24     output         [31:0] IF_pc,
25     output         [31:0] IF_inst
26 );
27
28 //-----{程序计数器 PC}begin
29     wire [31:0] next_pc;
30     wire [31:0] seq_pc;
31     reg [31:0] pc;
32
33     //跳转 pc

```

```
34     wire          jbr_taken;
35     wire [31:0] jbr_target;
36     assign {jbr_taken, jbr_target} = jbr_bus; // 跳转总线传是否跳转和目标地址
37
38     //Exception PC
39     wire          exc_valid;
40     wire [31:0] exc_pc;
41     assign {exc_valid,exc_pc} = exc_bus;
42
43     //pc+4
44     assign seq_pc[31:2]      = pc[31:2] + 1'b1; // 下一指令地址: PC=PC+4
45     assign seq_pc[1:0]      = pc[1:0];
46
47     // 新指令: 若有 Exception,则 PC 为 Exceptio 入口地址
48     //          若指令跳转, 则 PC 为跳转地址; 否则为 pc+4
49     assign next_pc = exc_valid ? exc_pc :
50                    jbr_taken ? jbr_target : seq_pc;
51     always @(posedge clk) // PC 程序计数器
52     begin
53         if (!resetsn)
54             begin
55                 pc <= `STARTADDR; // 复位, 取程序起始地址
56             end
57         else if (next_fetch)
58             begin
59                 pc <= next_pc; // 不复位, 取新指令
60             end
61     end
62 //-----{程序计数器 PC}end
63
64 //-----{发往 inst_rom 的取指地址}begin
65     assign inst_addr = pc;
66 //-----{发往 inst_rom 的取指地址}end
67
68 //-----{IF 执行完成}begin
69     //由于指令 rom 为同步读写的,
70     //取数据时, 有一拍延时
71     //即发地址的下一拍时钟才能得到对应的指令
72     //故取指模块需要两拍时间
73     //故每次 PC 刷新, IF_over 都要置 0
74     //然后将 IF_valid 锁存一拍即是 IF_over 信号
75     always @(posedge clk)
76     begin
77         if (!resetsn || next_fetch)
78             begin
79                 IF_over <= 1'b0;
80             end
81         else
82             begin
83                 IF_over <= IF_valid;
84             end
85     end
```

```

85     end
86     //如果指令 rom 为异步读的, 则 IF_valid 即是 IF_over 信号,
87     //即取指一拍完成
88 //-----{IF 执行完成}end
89
90 //-----{IF->ID 总线}begin
91     assign IF_ID_bus = {pc, inst}; // 取指级有效时, 锁存 PC 和指令
92 //-----{IF->ID 总线}end
93
94 //-----{展示 IF 模块的 PC 值和指令}begin
95     assign IF_pc     = pc;
96     assign IF_inst   = inst;
97 //-----{展示 IF 模块的 PC 值和指令}end
98 endmodule

```

decode.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: decode.v
4  //  > 描述   : 五级流水 CPU 的译码模块
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  //*****
8  module decode(                                // 译码级
9      input                                ID_valid, // 译码级有效信号
10     input      [ 63:0] IF_ID_bus_r, // IF->ID 总线
11     input      [ 31:0] rs_value, // 第一源操作数值
12     input      [ 31:0] rt_value, // 第二源操作数值
13     output     [  4:0] rs, // 第一源操作数地址
14     output     [  4:0] rt, // 第二源操作数地址
15     output     [ 32:0] jbr_bus, // 跳转总线
16 // 要 output     inst_jbr, // 指令为跳转分支指令, 五级流水不需
17     output     ID_over, // ID 模块执行完成
18     output     [166:0] ID_EXE_bus, // ID->EXE 总线
19
20     //5 级流水新增
21     input     IF_over, //对于分支指令, 需要该信号
22     input     [  4:0] EXE_wdest, // EXE 级要写回寄存器堆的目标地址
23     input     [  4:0] MEM_wdest, // MEM 级要写回寄存器堆的目标地址
24     input     [  4:0] WB_wdest, // WB 级要写回寄存器堆的目标地址号
25
26     //展示 PC
27     output     [ 31:0] ID_pc
28 );
29 //-----{IF->ID 总线}begin
30     wire [31:0] pc;
31     wire [31:0] inst;

```



```

32     assign {pc, inst} = IF_ID_bus_r; // IF->ID 总线传 PC 和指令
33 //-----{IF->ID 总线}end
34
35 //-----{指令译码}begin
36     wire [5:0] op;
37     wire [4:0] rd;
38     wire [4:0] sa;
39     wire [5:0] funct;
40     wire [15:0] imm;
41     wire [15:0] offset;
42     wire [25:0] target;
43     wire [2:0] cp0r_sel;
44
45     assign op      = inst[31:26]; // 操作码
46     assign rs      = inst[25:21]; // 源操作数 1
47     assign rt      = inst[20:16]; // 源操作数 2
48     assign rd      = inst[15:11]; // 目标操作数
49     assign sa      = inst[10:6];  // 特殊域, 可能存放偏移量
50     assign funct   = inst[5:0];   // 功能码
51     assign imm     = inst[15:0];  // 立即数
52     assign offset  = inst[15:0];  // 地址偏移量
53     assign target  = inst[25:0];  // 目标地址
54     assign cp0r_sel= inst[2:0];   // cp0 寄存器的 select 域
55
56     // 实现指令列表
57     wire inst_ADDU, inst_SUBU , inst_SLT , inst_AND;
58     wire inst_NOR , inst_OR   , inst_XOR , inst_SLL;
59     wire inst_SRL , inst_ADDIU, inst_BEQ , inst_BNE;
60     wire inst_LW  , inst_SW   , inst_LUI , inst_J;
61     wire inst_SLTU, inst_JALR , inst_JR  , inst_SLLV;
62     wire inst_SRA , inst_SRAV , inst_SRLV, inst_SLTIU;
63     wire inst_SLTI, inst_BGEZ , inst_BGTZ, inst_BLEZ;
64     wire inst_BLTZ, inst_LB   , inst_LBU , inst_SB;
65     wire inst_ANDI, inst_ORI  , inst_XORI, inst_JAL;
66     wire inst_MULT, inst_MFLO , inst_MFHI, inst_MTLO;
67     wire inst_MTHI, inst_MFC0 , inst_MTC0;
68     wire inst_ERET, inst_SYSCALL;
69     wire op_zero; // 操作码全 0
70     wire sa_zero; // sa 域全 0
71     assign op_zero = ~(|op);
72     assign sa_zero = ~(|sa);
73     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
74     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
75     assign inst_SLT  = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
76     assign inst_SLTU = op_zero & sa_zero & (funct == 6'b101011); // 无符号小则置
77     assign inst_JALR = op_zero & (rt==5'd0) & (rd==5'd31)
78                     & sa_zero & (funct == 6'b001001); // 跳转寄存器并链接
79     assign inst_JR   = op_zero & (rt==5'd0) & (rd==5'd0 )
80                     & sa_zero & (funct == 6'b001000); // 跳转寄存器
81     assign inst_AND  = op_zero & sa_zero & (funct == 6'b100100); // 与运算
82     assign inst_NOR  = op_zero & sa_zero & (funct == 6'b100111); // 或非运算

```

```

83    assign inst_OR      = op_zero & sa_zero      & (funct == 6'b100101); //或运算
84    assign inst_XOR     = op_zero & sa_zero      & (funct == 6'b100110); //异或运算
85    assign inst_SLL     = op_zero & (rs==5'd0) & (funct == 6'b000000); //逻辑左移
86    assign inst_SLLV    = op_zero & sa_zero      & (funct == 6'b000100); //变量逻辑左移
87    assign inst_SRA     = op_zero & (rs==5'd0) & (funct == 6'b000011); //算术右移
88    assign inst_SRAV    = op_zero & sa_zero      & (funct == 6'b000111); //变量算术右移
89    assign inst_SRL     = op_zero & (rs==5'd0) & (funct == 6'b000010); //逻辑右移
90    assign inst_SRLV    = op_zero & sa_zero      & (funct == 6'b000110); //变量逻辑右移
91    assign inst_MULT    = op_zero & (rd==5'd0)
92                                & sa_zero & (funct == 6'b011000); //乘法
93    assign inst_MFLO    = op_zero & (rs==5'd0) & (rt==5'd0)
94                                & sa_zero & (funct == 6'b010010); //从 LO 读取
95    assign inst_MFHI    = op_zero & (rs==5'd0) & (rt==5'd0)
96                                & sa_zero & (funct == 6'b010000); //从 HI 读取
97    assign inst_MTLO    = op_zero & (rt==5'd0) & (rd==5'd0)
98                                & sa_zero & (funct == 6'b010011); //向 LO 写数据
99    assign inst_MTHI    = op_zero & (rt==5'd0) & (rd==5'd0)
100                               & sa_zero & (funct == 6'b010001); //向 HI 写数据
101    assign inst_ADDIU   = (op == 6'b001001); //立即数无符号加法
102    assign inst_SLTI    = (op == 6'b001010); //小于立即数则置位
103    assign inst_SLTIU   = (op == 6'b001011); //无符号小于立即数则置位
104    assign inst_BEQ     = (op == 6'b000100); //判断相等跳转
105    assign inst_BGEZ    = (op == 6'b000001) & (rt==5'd1); //大于等于 0 跳转
106    assign inst_BGTZ    = (op == 6'b000111) & (rt==5'd0); //大于 0 跳转
107    assign inst_BLEZ    = (op == 6'b000110) & (rt==5'd0); //小于等于 0 跳转
108    assign inst_BLTZ    = (op == 6'b000001) & (rt==5'd0); //小于 0 跳转
109    assign inst_BNE     = (op == 6'b000101); //判断不等跳转
110    assign inst_LW      = (op == 6'b100011); //从内存装载字
111    assign inst_SW      = (op == 6'b101011); //向内存存储字
112    assign inst_LB      = (op == 6'b100000); //load 字节 (符号扩展)
113    assign inst_LBU     = (op == 6'b100100); //load 字节 (无符号扩展)
114    assign inst_SB      = (op == 6'b101000); //向内存存储字节
115    assign inst_ANDI    = (op == 6'b001100); //立即数与
116    assign inst_LUI     = (op == 6'b001111) & (rs==5'd0); //立即数装载高半字节
117    assign inst_ORI     = (op == 6'b001101); //立即数或
118    assign inst_XORI    = (op == 6'b001110); //立即数异或
119    assign inst_J       = (op == 6'b000010); //跳转
120    assign inst_JAL     = (op == 6'b000011); //跳转和链接
121    assign inst_MFC0    = (op == 6'b010000) & (rs==5'd0)
122                                & sa_zero & (funct[5:3] == 3'b000); // 从 cp0 寄存器装载
123    assign inst_MTC0    = (op == 6'b010000) & (rs==5'd4)
124                                & sa_zero & (funct[5:3] == 3'b000); // 向 cp0 寄存器存储
125    assign inst_SYSCALL = (op == 6'b000000) & (funct == 6'b001100); // 系统调用
126    assign inst_ERET    = (op == 6'b010000) & (rs==5'd16) & (rt==5'd0)
127                                & (rd==5'd0) & sa_zero & (funct == 6'b011000); //异常返回
128
129    //跳转分支指令
130    wire inst_jr; //寄存器跳转指令
131    wire inst_j_link; //链接跳转指令
132    wire inst_jbr; //所有分支跳转指令
133    assign inst_jr      = inst_JALR | inst_JR;

```

```

134     assign inst_j_link = inst_JAL | inst_JALR;
135     assign inst_jbr = inst_J      | inst_JAL  | inst_jr
136                   | inst_BEQ  | inst_BNE  | inst_BGEZ
137                   | inst_BGTZ | inst_BLEZ | inst_BLTZ;
138
139     //load store
140     wire inst_load;
141     wire inst_store;
142     assign inst_load  = inst_LW | inst_LB | inst_LBU;  // load 指
143     assign inst_store = inst_SW | inst_SB;             // store 指
144
145     //alu 操作分类
146     wire inst_add, inst_sub, inst_slt,inst_sltu;
147     wire inst_and, inst_nor, inst_or, inst_xor;
148     wire inst_sll, inst_srl, inst_sra,inst_lui;
149     assign inst_add = inst_ADDU | inst_ADDIU | inst_load
150                   | inst_store | inst_j_link;          // 做加法
151     assign inst_sub = inst_SUBU;                       // 减
152     assign inst_slt = inst_SLT | inst_SLTI;            // 有符号小于置
153     assign inst_sltu= inst_SLTIU | inst_SLTU;          // 无符号小于置
154     assign inst_and = inst_AND | inst_ANDI;           // 逻辑与
155     assign inst_nor = inst_NOR;                       // 逻辑或非
156     assign inst_or  = inst_OR  | inst_ORI;            // 逻辑或
157     assign inst_xor = inst_XOR | inst_XORI;           // 逻辑异或
158     assign inst_sll = inst_SLL | inst_SLLV;           // 逻辑左移
159     assign inst_srl = inst_SRL | inst_SRLV;           // 逻辑右移
160     assign inst_sra = inst_SRA | inst_SRAV;           // 算术右移
161     assign inst_lui = inst_LUI;                       // 立即数装载高
162
163     //使用 sa 域作为偏移量的移位指令
164     wire inst_shf_sa;
165     assign inst_shf_sa =  inst_SLL | inst_SRL | inst_SRA;
166
167     //依据立即数扩展方式分类
168     wire inst_imm_zero; //立即数 0 扩展
169     wire inst_imm_sign; //立即数符号扩展
170     assign inst_imm_zero = inst_ANDI | inst_LUI | inst_ORI |
inst_XORI;
171     assign inst_imm_sign = inst_ADDIU | inst_SLTI | inst_SLTIU
172                   | inst_load | inst_store;
173
174     //依据目的寄存器号分类
175     wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
176     wire inst_wdest_31; // 寄存器堆写入地址为 31 的指令
177     wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令

```

```

178     assign inst_wdest_rt = inst_imm_zero | inst_ADDIU | inst_SLTI
179                               | inst_SLTIU | inst_load | inst_MFC0;
180     assign inst_wdest_31 = inst_JAL;
181     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT |
inst_SLTU
182                               | inst_JALR | inst_AND | inst_NOR |
inst_OR
183                               | inst_XOR | inst_SLL | inst_SLLV |
inst_SRA
184                               | inst_SRAV | inst_SRL | inst_SRLV
185                               | inst_MFHI | inst_MFLO;
186
187     //依据源寄存器号分类
188     wire inst_no_rs; //指令 rs 域非 0, 且不是从寄存器堆读 rs 的数据
189     wire inst_no_rt; //指令 rt 域非 0, 且不是从寄存器堆读 rt 的数据
190     assign inst_no_rs = inst_MTC0 | inst_SYSCALL | inst_ERET;
191     assign inst_no_rt = inst_ADDIU | inst_SLTI | inst_SLTIU
192                               | inst_BGEZ | inst_load | inst_imm_zero
193                               | inst_J | inst_JAL | inst_MFC0
194                               | inst_SYSCALL;
195     //-----{指令译码}end
196
197     //-----{分支指令执行}begin
198     //bd_pc, 分支跳转指令参与计算的为延迟槽指令的 PC 值, 即当前分支指令的 PC+4
199     wire [31:0] bd_pc; //延迟槽指令 PC 值
200     assign bd_pc = pc + 3'b100;
201
202     //无条件跳转
203     wire j_taken;
204     wire [31:0] j_target;
205     assign j_taken = inst_J | inst_JAL | inst_jr;
206     //寄存器跳转地址为 rs_value, 其他跳转为{bd_pc[31:28], target, 2'b00}
207     assign j_target = inst_jr ? rs_value :
{bd_pc[31:28], target, 2'b00};
208
209     //branch 指令
210     wire rs_equql_rt;
211     wire rs_ez;
212     wire rs_ltz;
213     assign rs_equql_rt = (rs_value == rt_value); //
GPR[rs]==GPR[rt]
214     assign rs_ez = ~(|rs_value); // rs 寄存器值为
0
215     assign rs_ltz = rs_value[31]; // rs 寄存器值小
于 0
216     wire br_taken;
217     wire [31:0] br_target;
218     assign br_taken = inst_BEQ & rs_equql_rt // 相等跳转
219                               | inst_BNE & ~rs_equql_rt // 不等跳转
220                               | inst_BGEZ & ~rs_ltz // 大于等于 0
跳转
221                               | inst_BGTZ & ~rs_ltz & ~rs_ez // 大于 0 跳转

```

```

222                                     | inst_BLEZ & (rs_ltz | rs_ez) // 小于等于 0
    跳转
223                                     | inst_BLTZ & rs_ltz; // 小于 0 跳转
224 // 分支跳转目标地址: PC=PC+offset<<2
225     assign br_target[31:2] = bd_pc[31:2] + {{14{offset[15]}}},
    offset};
226     assign br_target[1:0] = bd_pc[1:0];
227
228 //jump and branch 指令
229     wire jbr_taken;
230     wire [31:0] jbr_target;
231     assign jbr_taken = (j_taken | br_taken) & ID_over;
232     assign jbr_target = j_taken ? j_target : br_target;
233
234 //ID 到 IF 的跳转总线
235     assign jbr_bus = {jbr_taken, jbr_target};
236 //-----{分支指令执行}end
237
238 //-----{ID 执行完成}begin
239 //由于是流水的, 存在数据相关
240     wire rs_wait;
241     wire rt_wait;
242     assign rs_wait = ~inst_no_rs & (rs!=5'd0)
243                                     & ( (rs==EXE_wdest) | (rs==MEM_wdest) |
    (rs==WB_wdest) );
244     assign rt_wait = ~inst_no_rt & (rt!=5'd0)
245                                     & ( (rt==EXE_wdest) | (rt==MEM_wdest) |
    (rt==WB_wdest) );
246
247 //对于分支跳转指令, 只有在 IF 执行完成后, 才可以算 ID 完成;
248 //否则, ID 级先完成了, 而 IF 还在取指令, 则 next_pc 不能锁存到 PC 里去,
249 //那么等 IF 完成, next_pc 能锁存到 PC 里去时, jbr_bus 上的数据已变成无
    效,
250 //导致分支跳转失败
251 //(~inst_jbr | IF_over)即是(~inst_jbr | (inst_jbr & IF_over))
252     assign ID_over = ID_valid & ~rs_wait & ~rt_wait & (~inst_jbr
    | IF_over);
253 //-----{ID 执行完成}end
254
255 //-----{ID->EXE 总线}begin
256 //EXE 需要用到的信息
257     wire multiply; //乘法 MULT
258     wire mthi; //MTHI
259     wire mtlo; //MTLO
260     assign multiply = inst_MULT;
261     assign mthi = inst_MTHI;
262     assign mtlo = inst_MTLO;
263 //ALU 两个源操作数和控制信号
264     wire [11:0] alu_control;
265     wire [31:0] alu_operand1;
266     wire [31:0] alu_operand2;
267

```

```

268 //所谓链接跳转是将跳转返回的 PC 值存放到 31 号寄存器里
269 //在流水 CPU 里，考虑延迟槽，故链接跳转需要计算 PC+8，存放到 31 号寄存器
    里
270 assign alu_operand1 = inst_j_link ? pc :
271                     inst_shf_sa ? {27'd0,sa} : rs_value;
272 assign alu_operand2 = inst_j_link ? 32'd8 :
273                     inst_imm_zero ? {16'd0, imm} :
274                     inst_imm_sign ? {{16{imm[15]}}}, imm} :
    rt_value;
275 assign alu_control = {inst_add,          // ALU 操作码，独热编码
276                     inst_sub,
277                     inst_slt,
278                     inst_sltu,
279                     inst_and,
280                     inst_nor,
281                     inst_or,
282                     inst_xor,
283                     inst_sll,
284                     inst_srl,
285                     inst_sra,
286                     inst_lui};
287 //访存需要用到的 load/store 信息
288 wire lb_sign; //load 一字节为有符号 load
289 wire ls_word; //load/store 为字节还是字,0:byte;1:word
290 wire [3:0] mem_control; //MEM 需要使用的控制信号
291 wire [31:0] store_data; //store 操作的存的数据
292 assign lb_sign = inst_LB;
293 assign ls_word = inst_LW | inst_SW;
294 assign mem_control = {inst_load,
295                     inst_store,
296                     ls_word,
297                     lb_sign };
298
299 //写回需要用到的信息
300 wire mfhi;
301 wire mflo;
302 wire mtc0;
303 wire mfc0;
304 wire [7 :0] cp0r_addr;
305 wire syscall; //syscall 和 eret 在写回级有特殊的操作
306 wire eret;
307 wire rf_wen; //写回的寄存器写使能
308 wire [4:0] rf_wdest; //写回的目的寄存器
309 assign syscall = inst_SYSCALL;
310 assign eret = inst_ERET;
311 assign mfhi = inst_MFHI;
312 assign mflo = inst_MFLO;
313 assign mtc0 = inst_MTC0;
314 assign mfc0 = inst_MFC0;
315 assign cp0r_addr= {rd,cp0r_sel};
316 assign rf_wen = inst_wdest_rt | inst_wdest_31 |

```

```

    inst_wdest_rd;
317     assign rf_wdest = inst_wdest_rt ? rt :      //在不写寄存器堆时设
        置为 0
318                                     inst_wdest_31 ? 5'd31 :  //以便能准确判断数据
        相关
319                                     inst_wdest_rd ? rd : 5'd0;
320     assign store_data = rt_value;
321     assign ID_EXE_bus = {multiply,mthi,mtlo,
        //EXE 需用信息,新增
322     alu_control,alu_operand1,alu_operand2,//EXE 需用的信息
        mem_control,store_data,
323     //MEM 需用的信号
        mfhi,mflo,
324     //WB 需用的信号,新增
        mtc0,mfc0,cp0r_addr,syscall,eret,
325     //WB 需用的信号,新增
        rf_wen, rf_wdest,
326     //WB 需用的信号
        pc};
327     //PC 值
328     //-----{ID->EXE 总线}end
329
330     //-----{展示 ID 模块的 PC 值}begin
331     assign ID_pc = pc;
332     //-----{展示 ID 模块的 PC 值}end
333 endmodule

```

exe.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: exe.v
4  //    > 描述   :五级流水 CPU 的执行模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module exe(                                // 执行级
9      input                                EXE_valid,    // 执行级有效信号
10     input      [166:0] ID_EXE_bus_r, // ID->EXE 总线
11     output      EXE_over,    // EXE 模块执行完成
12     output      [153:0] EXE_MEM_bus, // EXE->MEM 总线
13
14     //5 级流水新增
15     input      clk,          // 时钟
16     output      [ 4:0] EXE_wdest, // EXE 级要写回寄存器堆的目标地址号
17
18     //展示 PC
19     output      [ 31:0] EXE_pc
20 );
21 //-----{ID->EXE 总线}begin
22     //EXE 需要用到的信息

```



```
23    wire multiply;                //乘法
24    wire mthi;                    //MTHI
25    wire mtlo;                    //MTLO
26    wire [11:0] alu_control;
27    wire [31:0] alu_operand1;
28    wire [31:0] alu_operand2;
29
30    //访存需要用到的 load/store 信息
31    wire [3:0] mem_control; //MEM 需要使用的控制信号
32    wire [31:0] store_data; //store 操作的存的数据
33
34    //写回需要用到的信息
35    wire mfhi;
36    wire mflo;
37    wire mtc0;
38    wire mfc0;
39    wire [7 :0] cp0r_addr;
40    wire        syscall; //syscall 和 eret 在写回级有特殊的操作
41    wire        eret;
42    wire        rf_wen; //写回的寄存器写使能
43    wire [4:0] rf_wdest; //写回的目的寄存器
44
45    //pc
46    wire [31:0] pc;
47    assign {multiply,
48            mthi,
49            mtlo,
50            alu_control,
51            alu_operand1,
52            alu_operand2,
53            mem_control,
54            store_data,
55            mfhi,
56            mflo,
57            mtc0,
58            mfc0,
59            cp0r_addr,
60            syscall,
61            eret,
62            rf_wen,
63            rf_wdest,
64            pc          } = ID_EXE_bus_r;
65    //-----{ID->EXE 总线}end
66
67    //-----{ALU}begin
68    wire [31:0] alu_result;
69
70    alu alu_module(
71        .alu_control  (alu_control ), // I, 12, ALU 控制信号
72        .alu_src1     (alu_operand1), // I, 32, ALU 操作数 1
73        .alu_src2     (alu_operand2), // I, 32, ALU 操作数 2
```

```

74         .alu_result    (alu_result )    // 0, 32, ALU 结果
75     );
76 //-----{ALU}end
77
78 //-----{乘法器}begin
79     wire            mult_begin;
80     wire [63:0] product;
81     wire            mult_end;
82
83     assign mult_begin = multiply & EXE_valid;
84     multiply multiply_module (
85         .clk            (clk            ),
86         .mult_begin(mult_begin ),
87         .mult_op1    (alu_operand1),
88         .mult_op2    (alu_operand2),
89         .product      (product      ),
90         .mult_end     (mult_end     )
91     );
92 //-----{乘法器}end
93
94 //-----{EXE 执行完成}begin
95     //对于 ALU 操作，都是 1 拍可完成，
96     //但对于乘法操作，需要多拍完成
97     assign EXE_over = EXE_valid & (~multiply | mult_end);
98 //-----{EXE 执行完成}end
99
100 //-----{EXE 模块的 dest 值}begin
101     //只有在 EXE 模块有效时，其写回目的寄存器号才有意义
102     assign EXE_wdest = rf_wdest & {5{EXE_valid}};
103 //-----{EXE 模块的 dest 值}end
104
105 //-----{EXE->MEM 总线}begin
106     wire [31:0] exe_result;    //在 exe 级能确定的最终写回结果
107     wire [31:0] lo_result;
108     wire        hi_write;
109     wire        lo_write;
110     //要写入 HI 的值放在 exe_result 里，包括 MULT 和 MTHI 指令，
111     //要写入 LO 的值放在 lo_result 里，包括 MULT 和 MTLO 指令，
112     assign exe_result = mthi      ? alu_operand1 :
113                        mtc0      ? alu_operand2 :
114                        multiply ? product[63:32] : alu_result;
115     assign lo_result  = mtlo ? alu_operand1 : product[31:0];
116     assign hi_write   = multiply | mthi;
117     assign lo_write   = multiply | mtlo;
118
119     assign EXE_MEM_bus = {mem_control,store_data, //load/store 信息和 store 数
据
120                        exe_result,                //exe 运算结果
121                        lo_result,                //乘法低 32 位结果，新增
122                        hi_write,lo_write,        //HI/LO 写使能，新增
123                        mfhi,mflo,                //WB 需用的信号，新增

```

```

124          mtc0,mfc0,cp0r_addr,syscall,eret, //WB 需用的信号,新
增
125          rf_wen,rf_wdest,          //WB 需用的信号
126          pc};                      //PC
127 //-----{EXE->MEM 总线}end
128
129 //-----{展示 EXE 模块的 PC 值}begin
130     assign EXE_pc = pc;
131 //-----{展示 EXE 模块的 PC 值}end
132 endmodule

```

mem.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: mem.v
4   * > 描述 : 五级流水 CPU 的访存模块
5   * > 作者 : LOONGSON
6   * > 日期 : 2016-04-14
7   */
8  module mem(                                // 访存级
9      input          clk,                    // 时钟
10     input          MEM_valid,              // 访存级有效信号
11     input          [153:0] EXE_MEM_bus_r, // EXE->MEM 总线
12     input          [ 31:0] dm_rdata,       // 访存读数据
13     output         [ 31:0] dm_addr,        // 访存读写地址
14     output reg     [ 3:0] dm_wen,          // 访存写使能
15     output reg     [31:0] dm_wdata,        // 访存写数据
16     output         MEM_over,               // MEM 模块执行完成
17     output         [117:0] MEM_WB_bus,    // MEM->WB 总线
18
19     //5 级流水新增接口
20     input          MEM_allow_in, // MEM 级允许下级进入
21     output         [ 4:0] MEM_wdest, // MEM 级要写回寄存器堆的目标地址
22
23     //展示 PC
24     output         [ 31:0] MEM_pc
25 );
26 //-----{EXE->MEM 总线}begin
27 //访存需要用到的 load/store 信息
28 wire [3 :0] mem_control; //MEM 需要使用的控制信号
29 wire [31:0] store_data;  //store 操作的存的数据
30
31 //EXE 结果和 HI/LO 数据
32 wire [31:0] exe_result;
33 wire [31:0] lo_result;
34 wire      hi_write;
35 wire      lo_write;
36
37 //写回需要用到的信息

```

```
38     wire mfhi;
39     wire mflo;
40     wire mtc0;
41     wire mfc0;
42     wire [7 :0] cp0r_addr;
43     wire      syscall;    //syscall 和 eret 在写回级有特殊的操作
44     wire      eret;
45     wire      rf_wen;     //写回的寄存器写使能
46     wire [4:0] rf_wdest;  //写回的目的寄存器
47
48     //pc
49     wire [31:0] pc;
50     assign {mem_control,
51            store_data,
52            exe_result,
53            lo_result,
54            hi_write,
55            lo_write,
56            mfhi,
57            mflo,
58            mtc0,
59            mfc0,
60            cp0r_addr,
61            syscall,
62            eret,
63            rf_wen,
64            rf_wdest,
65            pc      } = EXE_MEM_bus_r;
66 //-----{EXE->MEM 总线}end
67
68 //-----{load/store 访存}begin
69     wire inst_load;  //load 操作
70     wire inst_store; //store 操作
71     wire ls_word;    //load/store 为字节还是字,0:byte;1:word
72     wire lb_sign;    //load 一字节为有符号 load
73     assign {inst_load,inst_store,ls_word,lb_sign} = mem_control;
74
75     //访存读写地址
76     assign dm_addr = exe_result;
77
78     //store 操作的写使能
79     always @ (*)    // 内存写使能信号
80     begin
81         if (MEM_valid && inst_store) // 访存级有效时,且为 store 操作
82         begin
83             if (ls_word)
84             begin
85                 dm_wen <= 4'b1111; // 存储字指令,写使能全 1
86             end
87             else
88             begin // SB 指令,需要依据地址底两位,确定对应的写使能
```

```

89             case (dm_addr[1:0])
90                 2'b00 : dm_wen <= 4'b0001;
91                 2'b01 : dm_wen <= 4'b0010;
92                 2'b10 : dm_wen <= 4'b0100;
93                 2'b11 : dm_wen <= 4'b1000;
94                 default : dm_wen <= 4'b0000;
95             endcase
96         end
97     end
98     else
99     begin
100         dm_wen <= 4'b0000;
101     end
102 end
103
104 //store 操作的写数据
105 always @ (*) // 对于 SB 指令，需要依据地址底两位，移动 store 的字节至对应位置
106 begin
107     case (dm_addr[1:0])
108         2'b00 : dm_wdata <= store_data;
109         2'b01 : dm_wdata <= {16'd0, store_data[7:0],
110 8'd0};
111         2'b10 : dm_wdata <= {8'd0, store_data[7:0],
112 16'd0};
113         2'b11 : dm_wdata <= {store_data[7:0], 24'd0};
114         default : dm_wdata <= store_data;
115     endcase
116 end
117
118 //load 读出的数据
119 wire load_sign;
120 wire [31:0] load_result;
121 assign load_sign = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7] :
122 (dm_addr[1:0]==2'd1) ? dm_rdata[15] :
123 (dm_addr[1:0]==2'd2) ? dm_rdata[23] :
124 dm_rdata[31] ;
125 assign load_result[7:0] = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7:0 ] :
126 (dm_addr[1:0]==2'd1) ? dm_rdata[15:8 ] :
127 (dm_addr[1:0]==2'd2) ? dm_rdata[23:16] :
128 dm_rdata[31:24] ;
129 assign load_result[31:8]= ls_word ? dm_rdata[31:8] :{24{lb_sign&load_sign}};
130 //-----{load/store 访存}end
131
132 //-----{MEM 执行完成}begin
133 //由于数据 RAM 为同步读写的，
134 //故对 load 指令，取数据时，有一拍延时
135 //即发地址的下一拍时钟才能得到 load 的数据
136 //故 mem 在进行 load 操作时有需要两拍时间才能取到数据
137 //而对其他操作，则只需要一拍时间
138 reg MEM_valid_r;
139 always @(posedge clk)
140 begin

```

```

138         if (MEM_allow_in)
139         begin
140             MEM_valid_r <= 1'b0;
141         end
142         else
143         begin
144             MEM_valid_r <= MEM_valid;
145         end
146     end
147     assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
148     //如果数据 ram 为异步读的, 则 MEM_valid 即是 MEM_over 信号,
149     //即 load 一拍完成
150 //-----{MEM 执行完成}end
151
152 //-----{MEM 模块的 dest 值}begin
153     //只有在 MEM 模块有效时, 其写回目的寄存器号才有意义
154     assign MEM_wdest = rf_wdest & {5{MEM_valid}};
155 //-----{MEM 模块的 dest 值}end
156
157 //-----{MEM->WB 总线}begin
158     wire [31:0] mem_result; //MEM 传到 WB 的 result 为 load 结果或 EXE 结果
159     assign mem_result = inst_load ? load_result : exe_result;
160
161     assign MEM_WB_bus = {rf_wen,rf_wdest,                //WB 需要使用的信号
162                         mem_result,                    //最终要写回寄存器的数据
163                         lo_result,                    //乘法低 32 位结果, 新增
164                         hi_write,lo_write,            //HI/LO 写使能, 新增
165                         mfhi,mflo,                    //WB 需要使用的信号, 新增
166                         mtc0,mfc0,cp0r_addr,syscall,eret, //WB 需要使用的信号, 新增
167                         pc};                          // PC 值
168 //-----{MEM->WB 总线}begin
169
170 //-----{展示 MEM 模块的 PC 值}begin
171     assign MEM_pc = pc;
172 //-----{展示 MEM 模块的 PC 值}end
173 endmodule

```

wb.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: wb.v
4  //    > 描述   :五级流水 CPU 的写回模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  `define EXC_ENTER_ADDR 32'd0    // Exception 入口地址,
9                                   // 此处实现的 Exception 只有 SYSCALL
10 module wb(                       // 写回级
11     input          WB_valid,      // 写回级有效
12     input  [117:0] MEM_WB_bus_r, // MEM->WB 总线

```

```

13     output          rf_wen,          // 寄存器写使能
14     output [ 4:0] rf_wdest,         // 寄存器写地址
15     output [ 31:0] rf_wdata,        // 寄存器写数据
16     output          WB_over,        // WB 模块执行完成
17
18     //5 级流水新增接口
19     input           clk,            // 时钟
20     input           resetn,         // 复位信号，低电平有效
21     output [ 32:0] exc_bus,         // Exception pc 总线
22     output [ 4:0] WB_wdest,         // WB 级要写回寄存器堆的目标地址号
23     output          cancel,         // syscall 和 eret 到达写回级时会发出 cancel 信
号,
24                                     // 取消已经取出的正在其他流水级执行的指令
25
26     //展示 PC 和 HI/LO 值
27     output [ 31:0] WB_pc,
28     output [ 31:0] HI_data,
29     output [ 31:0] LO_data
30 );
31 //-----{MEM->WB 总线}begin
32     //MEM 传来的 result
33     wire [31:0] mem_result;
34     //HI/LO 数据
35     wire [31:0] lo_result;
36     wire          hi_write;
37     wire          lo_write;
38
39     //寄存器堆写使能和写地址
40     wire wen;
41     wire [4:0] wdest;
42
43     //写回需要用到的信息
44     wire mfhi;
45     wire mflo;
46     wire mtc0;
47     wire mfc0;
48     wire [7 :0] cp0r_addr;
49     wire          syscall;         //syscall 和 eret 在写回级有特殊的操作
50     wire          eret;
51
52     //pc
53     wire [31:0] pc;
54     assign {wen,
55             wdest,
56             mem_result,
57             lo_result,
58             hi_write,
59             lo_write,
60             mfhi,
61             mflo,
62             mtc0,

```



```
63         mfc0,
64         cp0r_addr,
65         syscall,
66         eret,
67         pc} = MEM_WB_bus_r;
68 //-----{MEM->WB 总线}end
69
70 //-----{HI/LO 寄存器}begin
71     //HI 用于存放乘法结果的高 32 位
72     //LO 用于存放乘法结果的低 32 位
73     reg [31:0] hi;
74     reg [31:0] lo;
75
76     //要写入 HI 的数据存放在 mem_result 里
77     always @(posedge clk)
78     begin
79         if (hi_write)
80         begin
81             hi <= mem_result;
82         end
83     end
84     //要写入 LO 的数据存放在 lo_result 里
85     always @(posedge clk)
86     begin
87         if (lo_write)
88         begin
89             lo <= lo_result;
90         end
91     end
92 //-----{HI/LO 寄存器}end
93
94 //-----{cp0 寄存器}begin
95 // cp0 寄存器即是协处理器 0 寄存器
96 // 由于目前设计的 CPU 并不完备，所用到的 cp0 寄存器也很少
97 // 故暂时只实现 STATUS(12.0), CAUSE(13.0), EPC(14.0) 这三个
98 // 每个 CP0 寄存器都是使用 5 位的 cp0 号
99     wire [31:0] cp0r_status;
100     wire [31:0] cp0r_cause;
101     wire [31:0] cp0r_epc;
102
103     //写使能
104     wire status_wen;
105     //wire cause_wen;
106     wire epc_wen;
107     assign status_wen = mtc0 & (cp0r_addr=={5'd12,3'd0});
108     assign epc_wen     = mtc0 & (cp0r_addr=={5'd14,3'd0});
109
110     //cp0 寄存器读
111     wire [31:0] cp0r_rdata;
112     assign cp0r_rdata = (cp0r_addr=={5'd12,3'd0}) ? cp0r_status :
113                        (cp0r_addr=={5'd13,3'd0}) ? cp0r_cause :
```

```
114         (cp0r_addr=={5'd14,3'd0}) ? cp0r_epc :
115     32'd0;
116     //STATUS 寄存器
117     //目前只实现 STATUS[1]位,即 EXL 域
118     //EXL 域为软件可读写,故需要 statu_wen
119     reg status_exl_r;
120     assign cp0r_status = {30'd0,status_exl_r,1'b0};
121     always @(posedge clk)
122     begin
123         if (!resetn || eret)
124         begin
125             status_exl_r <= 1'b0;
126         end
127         else if (syscall)
128         begin
129             status_exl_r <= 1'b1;
130         end
131         else if (status_wen)
132         begin
133             status_exl_r <= mem_result[1];
134         end
135     end
136
137     //CAUSE 寄存器
138     //目前只实现 CAUSE[6:2]位,即 ExcCode 域,存放 Exception 编码
139     //ExcCode 域为软件只读,不可写,故不需要 cause_wen
140     reg [4:0] cause_exc_code_r;
141     assign cp0r_cause = {25'd0,cause_exc_code_r,2'd0};
142     always @(posedge clk)
143     begin
144         if (syscall)
145         begin
146             cause_exc_code_r <= 5'd8;
147         end
148     end
149
150     //EPC 寄存器
151     //存放产生例外的地址
152     //EPC 整个域为软件可读写的,故需要 epc_wen
153     reg [31:0] epc_r;
154     assign cp0r_epc = epc_r;
155     always @(posedge clk)
156     begin
157         if (syscall)
158         begin
159             epc_r <= pc;
160         end
161         else if (epc_wen)
162         begin
163             epc_r <= mem_result;
```

```
164         end
165     end
166
167     //syscall 和 eret 发出的 cancel 信号
168     assign cancel = (syscall | eret) & WB_over;
169     //-----{cp0 寄存器}begin
170
171     //-----{WB 执行完成}begin
172         //WB 模块所有操作都可在一拍内完成
173         //故 WB_valid 即是 WB_over 信号
174         assign WB_over = WB_valid;
175     //-----{WB 执行完成}end
176
177     //-----{WB->regfile 信号}begin
178         assign rf_wen    = wen & WB_over;
179         assign rf_wdest  = wdest;
180         assign rf_wdata  = mfhi ? hi :
181                             mflo ? lo :
182                             mfc0 ? cp0r_rdata : mem_result;
183     //-----{WB->regfile 信号}end
184
185     //-----{Exception pc 信号}begin
186         wire          exc_valid;
187         wire [31:0] exc_pc;
188         assign exc_valid = (syscall | eret) & WB_valid;
189         //eret 返回地址为 EPC 寄存器的值
190         //SYSCALL 的 excPC 应该为{EBASE[31:10],10'h180},
191         //但作为实验,先设置 EXC_ENTER_ADDR 为 0,方便测试程序的编写
192         assign exc_pc = syscall ? `EXC_ENTER_ADDR : cp0r_epc;
193
194         assign exc_bus = {exc_valid,exc_pc};
195     //-----{Exception pc 信号}end
196
197     //-----{WB 模块的 dest 值}begin
198         //只有在 WB 模块有效时,其写回目的寄存器号才有意义
199         assign WB_wdest = rf_wdest & {5{WB_valid}};
200     //-----{WB 模块的 dest 值}end
201
202     //-----{展示 WB 模块的 PC 值和 HI/LO 寄存器的值}begin
203         assign WB_pc = pc;
204         assign HI_data = hi;
205         assign LO_data = lo;
206     //-----{展示 WB 模块的 PC 值和 HI/LO 寄存器的值}end
207 endmodule
```

静态 5 级流水线 CPU 参考设计使用的 inst_rom 和 data_ram 均为实例化的同步存储器 IP。另外,参考设计中还使用的实验二、三、四里实现的 multiply.v、regfile.v 和 alu.v,源代码见对应实验的参考设计说明,本实验中直接通过“Add Copy of Source”添加到工程中即可。其中 alu.v 还调用了实验一中的 adder.v。

约束文件 pipeline_cpu.xdc 源代码:

```
1      #时钟信号连接
2      set_property PACKAGE_PIN AC19 [get_ports clk]
3
4      #脉冲开关，用于输入作为复位信号，低电平有效
5      set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7      #脉冲开关，用于输入作为单步执行的 clk
8      set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10     set_property IOSTANDARD LVCMOS33 [get_ports clk]
11     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12     set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14     #触摸屏引脚连接
15     set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16     set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17     set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18     set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19     set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20     set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21     set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22     set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23     set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24     set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25     set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26     set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27     set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28     set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29     set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30     set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
31     set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32     set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33     set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34     set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35     set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36     set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37     set_property PACKAGE_PIN L19 [get_ports ct_int]
38     set_property PACKAGE_PIN J24 [get_ports ct_sda]
39     set_property PACKAGE_PIN H21 [get_ports ct_scl]
40     set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
42     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43     set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45     set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47     set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48     set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
```

```
49      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63      set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64      set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65      set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66      set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67      set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69      set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```

九、 课程设计拓展 优化 CPU 系统

1 课程设计拓展目的

1. 加深对计算机组成原理和体系结构理论知识的理解。
2. 培养对 CPU 设计的兴趣，在理解现有 CPU 架构的基础上，引发对体系结构的思考和创新。
3. 培养创新思维能力，并通过实践验证新想法。

2 实验设备

1. 装有 xilinx ISE 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 课程设计拓展题说明

本拓展是对之前课程设计的拔高，是为大家在学有余力的时候设定的，旨在发挥大家的创新思维能力，为大家提供一个学习反思验证的环境，可以尽情地提出自己关于计算机组成原理和体系结构的想法，大胆地对书中的知识提出疑问，通过设计实验方案验证自己的想法，从而获得对课本知识的升华。

因此本拓展会提出几个拓展题，供大家思考，大家可以选择性地实现，也可以全部实现，当然更鼓励大家地实现自己的新想法。为充分发挥大家的主观能动性，也为了不限制大家的思维，各拓展题只会给出现有问题，并不会像以往的实验说明给出初步的实验方案。

4 课程设计拓展题

1. 分析静态 5 级流水 CPU 中的流水线阻塞情况，包括数据相关、控制相关、结构相关等，优化流水线设计，尽可能减少流水线阻塞情况，比如前递技术等。
2. 对于分支跳转指令，mips 架构中有延迟槽指令的设定，利用这一点，在静态 5 级流水 CPU 中，可实现分支指令永远不阻塞后续指令，大家可以检查自己的流水线设定，进行优化实现这一点。
3. 针对第 2 点，此时，分支跳转指令就不需要进行转移猜测了，但大家可以将流水线结构改为 x86 中无延迟槽技术的设定，此时分支跳转指令与后续真正需要执行的指令至少会堵塞一拍，此时可以考虑实现转移预测技术，提升流水线结构。
4. 在学习的过程中，大家一定会有很多自己的想法，比如，为什么取指、译码、执行、访存、写回称为经典的 5 级流水结构，可以实现 3 级、4 级、6 级流水结构的 CPU 吗？答案肯定是可以的，甚至在各类产品的 CPU 中采用经典 5 级流水结构的都很少。所以希望大家尽情地发挥自己的想法，大改流水结构，验证自己关于流水结构的想法，您可以实现 3 级（比如：取指、译码、执行）、4 级、6 级等等各类流水结构的 CPU。

5. 课程设计要求大家实现的 mips 指令有限，大家可以分析其余 mips 指令，加以实现。

6. 目前课程设计实现的指令存储器和数据存储器是同步读的机制的，故在当前拍数（时钟周期）发出读数据的地址请求时，在下一拍才能获得读的数据，因此取值级和访存级的 load 都需要两拍时间。其实发地址请求在下一拍获得读的数据，明显也是一个可以流水做的工作，故可以考虑对流水线设计方案作稍微修改，使得取指级和访存级的 load 不需要多等一拍。

5 课程设计拓展要求

拓展题无特别要求，如果想做的话，需要自己花较多心思，在充分理解的基础上加以思考。在确定自己准备实现的想法后，请依然按照以往实验要求，画出设计框图，给出相应设计方案和验证方案。实现完后向老师充分阐述自己的改动，并进行演示。建议最后也形成一份实验报告的文档。文档还是很重要的，要养成一个好习惯。

附录 A 实现的 MIPS 指令集

附 A1 单周期 CPU 实现指令

(1) 无符号加法

ADDU rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100001						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] + GPR[rt]$

(2) 无符号减法

SUBU rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100011						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] - GPR[rt]$

(3) 有符号比较, 小于置位

SLT rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	101010						
6	5	5	5	5	6						

$GPR[rd] \leftarrow (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$

(4) 按位与

AND rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100100						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] \& GPR[rt]$

(5) 按位或非

NOR rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100111						
6	5	5	5	5	6						

$GPR[rd] \leftarrow \sim(GPR[rs] | GPR[rt])$

(6) 按位或

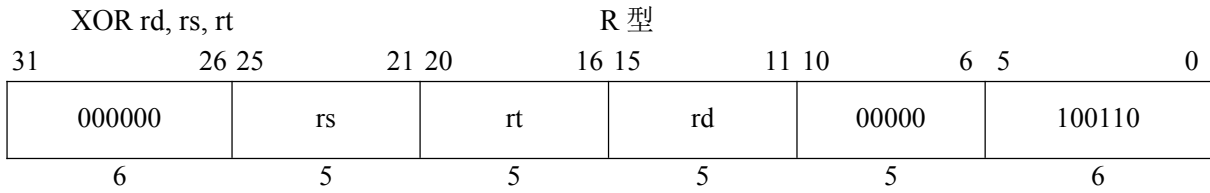
OR rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100101						
6	5	5	5	5	6						

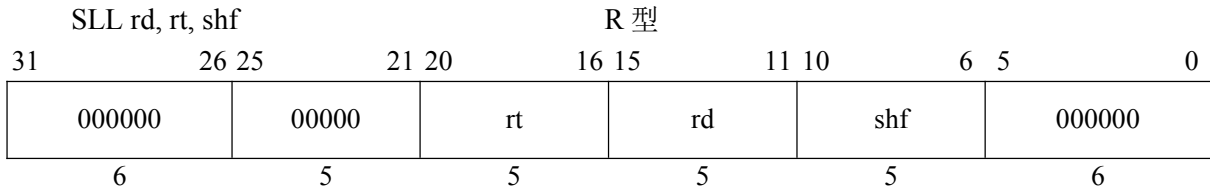
$GPR[rd] \leftarrow GPR[rs] | GPR[rt]$

(7) 按位异或



$GPR[rd] \leftarrow GPR[rs] \wedge GPR[rt]$

(8) 逻辑左移



$GPR[rd] \leftarrow \text{zero}(GPR[rt]) \ll \text{shf}$

(9) 逻辑右移



$GPR[rd] \leftarrow \text{zero}(GPR[rt]) \gg \text{shf}$

(10) 立即数、无符号加法



$GPR[rt] \leftarrow GPR[rs] + \text{sign_ext}(\text{imm})$

(11) 相等跳转



if $GPR[rs] = GPR[rt]$ then $PC \leftarrow B_PC + \text{sign_ext}(\text{offset}) \ll 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(12) 不等跳转



if $GPR[rs] \neq GPR[rt]$ then $PC \leftarrow B_PC + \text{sign_ext}(\text{offset}) \ll 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(13) 装载字

LW rt, offset(base) I 型

31	26 25	21 20	16 15	0
100011	base	rt	offset	
6	5	5	16	

$GPR[rt] \leftarrow Mem[GPR[base] + sign_ext(offset)]$

(14) 存储字

SW rt, offset(base) I 型

31	26 25	21 20	16 15	0
101011	base	rt	offset	
6	5	5	16	

$Mem[GPR[base] + sign_ext(offset)] \leftarrow GPR[rt]$

(15) 立即数装载高位

LUI rt, imm I 型

31	26 25	21 20	16 15	0
001111	00000	rt	imm	
6	5	5	16	

$GPR[rt] \leftarrow \{imm, 16'd0\}$

(16) 直接跳转

J target J 型

31	26 25	0
000010	target	
6	26	

$PC \leftarrow \{B_PC[31:28], target \ll 2\}$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

附 A2 多周期 CPU 新增实现指令

(17) 无符号小于置位

SLTU rd, rs, rt R 型

31	26 25	21 20	16 15	11 10	6 5	0
000000	rs	rt	rd	00000	101011	
6	5	5	5	5	6	

$GPR[rd] \leftarrow (zero(GPR[rs]) < zero(GPR[rt]))$

(18) 跳转寄存器并链接

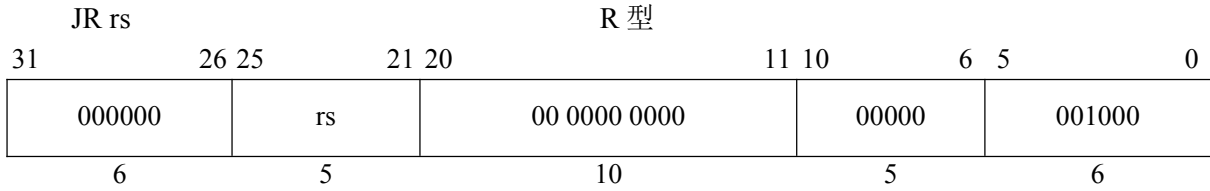
JALR rs R 型

31	26 25	21 20	16 15	11 10	6 5	0
000000	rs	00000	11111	00000	001001	
6	5	5	5	5	6	

$GPR[31] \leftarrow B_PC + 4, PC \leftarrow GPR[rs]$

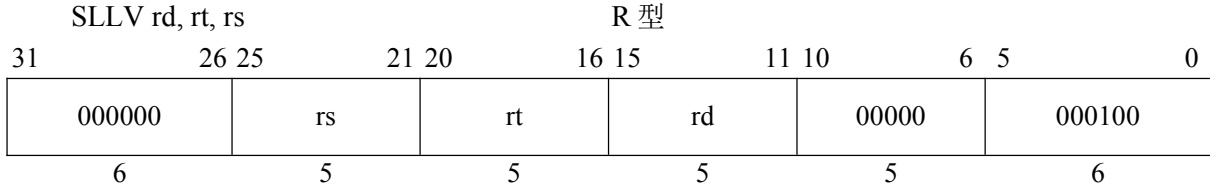
B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(19) 跳转寄存器



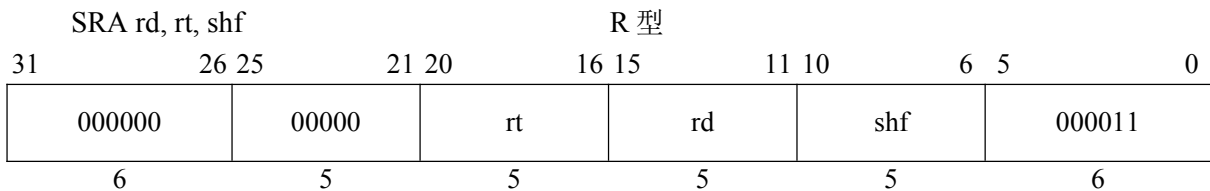
$PC \leftarrow GPR[rs]$

(20) 变量逻辑左移



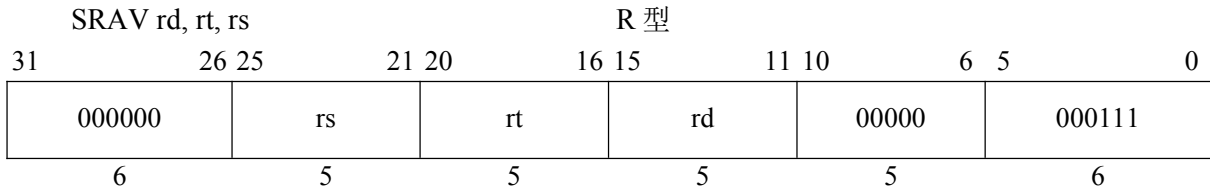
$GPR[rd] \leftarrow zero(GPR[rt]) \ll GPR[rs]$

(21) 算术右移



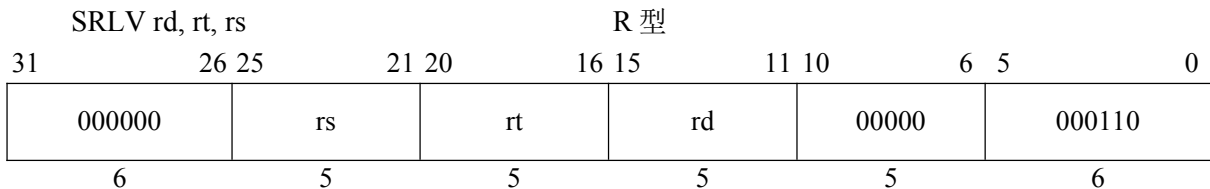
$GPR[rd] \leftarrow sign(GPR[rt]) \gg shf$

(22) 变量算术右移



$GPR[rd] \leftarrow sign(GPR[rt]) \gg GPR[rs]$

(23) 变量逻辑右移



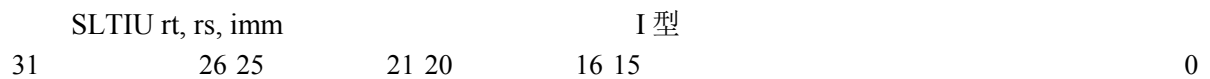
$GPR[rd] \leftarrow zero(GPR[rt]) \gg GPR[rs]$

(24) 立即数有符号比较, 小于置位



$GPR[rt] \leftarrow (sign(GPR[rs]) < sign_ext(imm))$

(25) 立即数无符号比较, 小于置位



001011	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow (zero(GPR[rs]) < sign_ext(imm))$

(26) 大于或等于零跳转

BGEZ rs, offset I 型

31	26	25	21	20	16	15	0
000001	rs	00001	offset				
6	5	5	16				

if $GPR[rs] \geq 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(27) 大于零跳转

BGTZ rs, offset I 型

31	26	25	21	20	16	15	0
000111	rs	00000	offset				
6	5	5	16				

if $GPR[rs] > 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(28) 小于或等于零跳转

BLEZ rs, offset I 型

31	26	25	21	20	16	15	0
000110	rs	00000	offset				
6	5	5	16				

if $GPR[rs] \leq 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(29) 小于零跳转

BLTZ rs, offset I 型

31	26	25	21	20	16	15	0
000001	rs	00000	offset				
6	5	5	16				

if $GPR[rs] < 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(30) 装载字节, 并作符号扩展

LB rt, offset(base) I 型

31	26	25	21	20	16	15	0
----	----	----	----	----	----	----	---

100000	base	rt	offset
6	5	5	16

$GPR[rt] \leftarrow \text{sign}(\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})])$

(31) 装载字节，并作无符号扩展

LBU rt, offset(base) I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

100100	base	rt	offset
6	5	5	16

$GPR[rt] \leftarrow \text{zero}(\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})])$

(32) 存储字节

SB rt, offset(base) I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

101000	base	rt	offset
6	5	5	16

$\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})] \leftarrow GPR[rt]$

(33) 立即数按位与

ANDI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001100	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] \& \text{zero_ext}(\text{imm})$

(34) 立即数按位或

ORI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001101	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] | \text{zero_ext}(\text{imm})$

(35) 立即数按位异或

XORI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001110	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] \wedge \text{zero_ext}(\text{imm})$

(36) 跳转和链接

JAL target J 型

31	26 25			0
----	-------	--	--	---

000011	target
6	26

$GPR[31] \leftarrow B_PC + 4, PC \leftarrow \{B_PC[31:28], target \ll 2\}$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

附 A3 静态 5 级流水 CPU 新增实现指令

(37) 有符号字乘法

MULT rs, rt			R 型		
31	26 25	21 20	16 15	6 5	0
000000	rs	rt	00 0000 0000	011000	
6	5	5	10	6	

$(HI, LO) \leftarrow sign(GPR[rs]) * sign(GPR[rt])$

(38) 从 LO 寄存器取值

MFLO rd			R 型			
31	26 25	21 20	16 15	11 10	6 5	0
000000		00 0000 0000		rd	00000	010010
6		10		5	5	6

$GPR[rd] \leftarrow [LO]$

(39) 从 HI 寄存器取值

MFHI rd		R 型				
31	26 25	21 20	16 15	11 10	6 5	0
000000		00 0000 0000		rd	00000	010000
6		10		5	5	6

$GPR[rd] \leftarrow [HI]$

(40) 向 LO 寄存器存值

MTLO rs		R 型			
31	26 25	21 20	16 15	6 5	0
000000		rs	000 0000 0000 0000		010011
6		5	15		6

$[LO] \leftarrow GPR[rs]$

(41) 向 HI 寄存器存值

MTHI rs		R 型			
31	26 25	21 20	16 15	6 5	0
000000		rs	000 0000 0000 0000		010001
6		5	15		6

$[HI] \leftarrow GPR[rs]$

(42) 从协处理器 0 寄存器取值

MFC0 rt, cs.sel			R 型			
31	26 25	21 20	16 15	11 10	3 2	0

010000	00000	rt	cs	00000000	sel
6	5	5	5	8	3

$GPR[rt] \leftarrow CPR[cs.sel]$

(43) 向协处理器 0 寄存器存值

MTC0 rt, cd.sel

R 型

31	26	25	21	20	16	15	11	10	3	2	0
010000	00100	rt	cd	00000000	sel						
6	5	5	5	8	3						

$CPR[cd.sel] \leftarrow GPR[rt]$

(44) 系统调用

SYSCALL

31	26	25	6	5	0
000000	code	001100			
6	20	6			

$CPR[14.0] \leftarrow PC$, $CPR[13.0][6:2] \leftarrow 01000$, $CPR[12.0][1] \leftarrow 1$, $PC \leftarrow EXC_ENTER_ADDR$

EXC_ENTER_ADDR 为例外入口地址，原本应为 $CPR[15.1] + 0x180$ ，但在课程设计中为方便编写测试程序，将 EXC_ENTER_ADDR 设置为 0。

(45) 异常返回

ERET

31	26	25	21	20	16	15	11	10	6	5	0
010000	1	000 0000 0000 0000 0000	011000								
6	1	19	6								

$CPR[12.0][1] \leftarrow 0$, $PC \leftarrow CPR[14.0]$

附 B3 例外程序计数器 EPC（CP0 寄存器 14，选择 0）

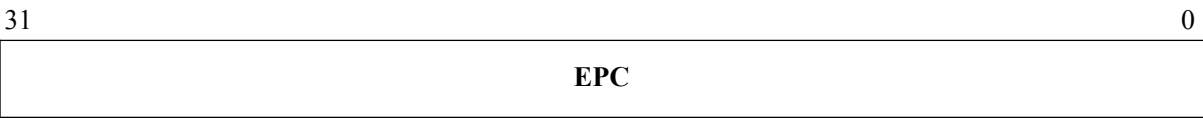


表 B-5 EPC（Exception Program Counter）域

域		描述	读/写	复位状态	规则
Name	Bits				
EPC	31..0	例外程序计数器	可读/写	Undefined	Required